Prácticas Profesionalizantes III - 3ro 2da
ISFT 172

Proyecto: Gestorix – Sistema de Control de Inventario para Maxikioscos
3°2°

Integrantes:
Cristian Janza.
Emiliano Rodriguez Lemiña.
Emanuel Rivas.
Tobias Calafatello.
Marcelo Ferreyra.

Índice

Palabras Clave, Definiciones, Acrónimos, y Abreviaciones.....6
3.1 Descripción de la Propuesta.....12
3.1.1 Alcance.....12
3.1.2 Límites.....13
3.1.4 Presentación del Negocio.....14
3.1.4.1 Propósito del Negocio.....14
3.1.4.2 Propósito de Gestorix.....14
3.1.4.3 Alcance del Negocio.....15
3.1.5 Idea de Negocio.....15
3.1.5.1 Misión y Visión.....16
Misión.....16
Visión.....16
3.1.5.2 Análisis Estratégico.....17
3.1.5.3 Análisis de mercado.....17
3.1.5.4 Segmentación del mercado.....19
3.1.5.5 FODA.....21
Fortalezas.....21
Oportunidades.....21
Debilidades.....21
Amenazas.....22
3.1.5.6 Análisis de factibilidad.....22
Factibilidad técnica.....22
Factibilidad económica.....22
Factibilidad operativa.....23
3.1.6 Descripción general de Stakeholders y Usuarios.....24
3.1.6.1 Resumen de Stakeholders.....24
Usuarios clave.....24
Resumen de Usuarios.....25
3.1.6.2 Entorno de Usuario.....26

3.1.6.3 Perfil de los Stakeholders 27
3.1.6.3.1 Dueño 27
3.1.6.3.2 Empleado 27
3.1.6.3.3 Equipo de desarrollo 28
3.1.6.3.4 Cliente del Kiosco 28
3.2 Descripción Global del Producto 30
3.2.1 Perspectiva del Producto 30
3.2.2 Resumen de Características 31
3.2.3 Suposiciones y Dependencias 32
3.3 Descripción Detallada del Producto 34
3.3.1 Módulo de Gestión de Usuarios 34
3.3.2 Módulo de Productos 35
3.3.3 Módulo de Ventas 36
3.3.4 Módulo de Compras 36
3.3.5 Módulo de Inventario 37
3.3.6 Módulo de Reportes y Balances 38
3.3.7 Módulo de Autenticación y Seguridad 38
3.4 Restricciones 40
3.4.1 Otros Requisitos del Producto 41
3.4.2 Estándares Aplicables 41
3.4.3 Requisitos de Sistema 42
3.4.4 Requisitos de Desempeño 43
3.4.5 Requisitos de Entorno 43
3.5 Casos de Uso 45
Casos de Uso de Negocio 45
CU01 - Registrar Producto 45
CU02 - Registrar Venta 48
CU03 - Carga Empleado 51
CU04 - Crear Rol 55
CU05 - Control de Caja Diaria 59
CU06 - Registrar Proveedor 62
CU07 - Devolución 66

CU08 - Buscar Artículo.....70
CU09 - Productos Próximos a Vencer.....73
CU10 - Modificar Alerta Stock Mínimo.....76
CU11 - Modificar / dar de baja producto.....81
CU12 - Cargar stock a artículo existente.....86
CU13 - Gestionar rubros.....91
CU14 - Gestionar marcas.....96
CU15 - Importar artículos desde Excel.....101
CU16 - Exportar artículos a Excel.....105
CU17 - Consultar historial de ventas.....109
CU18 - Consultar historial de devoluciones.....112
CU19 - Crear promoción.....116
CU20 - Dar de baja promoción.....121
CU21 - Registrar compra a proveedor.....124
CU22 - Consultar / anular compra.....128
CU23 - Generar pedido a proveedor.....131
CU24 - Modificar / dar de baja proveedor.....134
CU25 - Registrar pago a proveedor (cuenta corriente).....139
CU26 - Consultar listado de precios.....143
CU27 - Consultar listado de reposición.....146
CU28 - Generar informe de ventas.....149
CU29 - Generar balance de ingresos y egresos.....152
CU30 - Consultar movimientos de stock.....155
CU31 - Abrir caja.....158
CU32 - Registrar ingreso manual de caja.....162
CU33 - Registrar egreso manual de caja.....166
CU34 - Consultar alertas de stock bajo / crítico.....170
CU35 - Dar de baja alarma.....173
Casos de Uso de Seguridad.....176
CUS01 - Cambiar Idioma.....176
CUS02 - Consultar Bitácora.....179
CUS03 - Realizar Backup.....183

CUS04 - Gestión de Dígitos Verificadores 187
CUS05 - Gestión de Encriptado 192
3.5.9 Especificaciones Adicionales 197
3.5.10 Mapa de Navegación 200
3.6 Prototipos de Interfaces del Usuario 201
3.7 Diagrama Global de Clases Del Sistema 208
Capa Datos 208
Capa Entidades 208
Capa Lógica 209
Capa Vista 210
3.9 DER del Sistema 211
4.0 Estructura principal del software con interfaces 212
4.1 Estructura de capas del Software 212
4.2 Estructura principal del Login del software con interfaces 215
4.3 Estructura principal del menú principal del software con interfaces 217
4.4 Estructura principal de un caso de uso de negocio del software con interfaces y código: ABM Marca 218
Capa Vista // FrmBuscarMarca 220
Capa Lógica // ClsGestorLogicaProducto 224
Capa Entidades // ClsGestorProductoEntidad 225
Capa Datos // ClsGestorProducto 226
4.5 Estructura principal de un caso de uso de negocio del software con interfaces y código: Venta 230
Capa Vista // frmmenuPres 233
Capa Vista // frmInformeCobro 247
Capa Lógica // ClsGestorLogicaProducto 254
Capa Datos // ClsGestorVentaDatos 256
4.6 Estructura principal de un caso de uso de negocio del software con interfaces y código: Importar y Exportar Excel 260
Capa Vista // frmExportarArchivos 260
Capa Vista // frmImportarArticulos 265
4.7 Estructura principal de un caso de uso de negocio del software con interfaces y código: Compras 285

Capa Vista // frmCompras..... 287
Capa Vista // frmNuevaCompra.....290
Capa Vista // frmDetalleCompra.....298
Capa Lógica // ClsCompraLogica.....299
Capa Datos // ClsCompraDatos..... 300
5.0 Roles y Permisos..... 303
Roles Base..... 306
Administrador.....306
Empleado General.....307
Empleado Compras..... 307
Empleado Ventas..... 307

Palabras Clave, Definiciones, Acrónimos, y Abreviaciones

Término	Descripción
ABM	Alta, Baja y Modificación. Operaciones básicas de gestión de datos para crear, eliminar o actualizar registros en un sistema.
Arquitectura en capas	Patrón de diseño estructural que organiza el código en niveles independientes con responsabilidades delimitadas (ej. Presentación, Negocio, Datos).
ARS	Peso Argentino. Código ISO para la moneda oficial de la República Argentina.
Backup	Copia de seguridad de datos que permite restaurar la información en caso de pérdidas o fallos.
Bitácora	Registro cronológico detallado de los eventos, acciones o errores ocurridos dentro de un sistema.
bool	Tipo de dato booleano que solo puede almacenar dos valores lógicos: True o False.
Buenas prácticas OWASP	Guías y estándares de la Open Web Application Security Project para identificar y mitigar vulnerabilidades críticas en el software.
btn / Boton	Control de interfaz gráfica de usuario que desencadena una acción al ser presionado.
C#	Lenguaje de programación orientado a objetos desarrollado por Microsoft para la plataforma .NET.
cls / clase	Plantilla o modelo en programación orientada a objetos que define los atributos y métodos de un tipo de objeto.
cmb / Combo Box	Control de interfaz gráfica que despliega una lista desplegable para seleccionar una o más opciones.
Control de Caja	Proceso administrativo y contable de verificación de las entradas y salidas de dinero en efectivo u otros medios de pago.
CU / Caso de Uso	Descripción de una secuencia de interacciones entre un actor y el sistema para alcanzar un objetivo específico.
CUIL	Código Único de Identificación Laboral. Clave utilizada en Argentina para identificar a las personas físicas en la seguridad social y el sistema tributario.
CUS / Caso de Uso de Seguridad	Caso de uso enfocado en los requisitos, restricciones o flujos de seguridad del sistema.
Cta. Cte. (Cuenta Corriente)	Si el proveedor te deja la mercadería y se la pagas la semana que viene.

.bak	Archivo que contiene una Base de datos de SQL Server.
DataSet / DataTable	Objeto en memoria que representa una estructura de datos tabular compuesta por filas y columnas.
dateTime	Tipo de dato o estructura utilizado para almacenar información sobre una fecha y una hora simultáneamente.
dgv / datagridview	Control de interfaz de usuario diseñado para mostrar y editar datos representados en forma de tabla.
Digito Verificador	Mecanismo de control de integridad de datos consistente en uno o más dígitos derivados matemáticamente de un conjunto numérico.
DNI	Documento Nacional de Identidad. Documento legal para la identificación de los ciudadanos en Argentina y otros países.
dtp / dataTimePicker	Control de interfaz gráfica que permite al usuario seleccionar una fecha y una hora de un calendario desplegable.
DVH / Digito Verificador Horizontal	Algoritmo de integridad que calcula un código validador sumando o procesando los valores de una sola fila o registro.
DVV / Digito Verificador Vertical	Algoritmo de integridad que calcula un código validador procesando los valores de una columna completa entre múltiples registros de una tabla.
Egreso	Salida o flujo negativo de dinero o recursos de una entidad o sistema contable.
Encriptación	Proceso de transformación de datos de texto plano a un formato cifrado e ilegible mediante algoritmos criptográficos para garantizar la confidencialidad.
Exclude	Relación entre casos de uso en la que un flujo secundario u opcional no es indispensable para la ejecución del caso de uso base.
Faltante	Productos anotados como presentes pero que finalmente no están físicamente.
False	Valor booleano que representa la condición lógica de falso.
float	Tipo de dato numérico que representa números reales con coma flotante de precisión simple.
Flujo Alternativo	Secuencia de pasos en un caso de uso que describe desviaciones, excepciones o caminos secundarios respecto al proceso habitual.
Flujo Normal	Secuencia principal de pasos dentro de un caso de uso que se ejecuta exitosamente sin errores ni desviaciones.
Franco	Día de descanso del empleado.
frm / Formulario	Ventana o contenedor de interfaz gráfica sobre el cual se disponen otros controles e interactúa el usuario.
Fondo	Plata dejada al iniciar al día con billetes chicos.

Ganancia	Diferencia entre el costo de compra y el precio de venta de los productos.
IEEE 830	Estándar internacional que especifica las mejores prácticas para la redacción y organización de las Especificaciones de Requisitos de Software (SRS).
Include	Relación entre casos de uso donde un caso de uso base incorpora obligatoriamente el comportamiento de otro caso de uso secundario.
Ingreso	Entrada o flujo positivo de dinero o recursos en una entidad o sistema contable.
Insert	Sentencia de lenguaje SQL utilizada para añadir uno o varios registros nuevos dentro de una tabla de base de datos.
int	Tipo de dato entero que representa números sin parte decimal.
Inventario	Conjunto total de productos almacenados en el kiosco.
ISO 9001	Norma internacional sobre Sistemas de Gestión de Calidad (SGC) enfocada en la mejora continua y satisfacción del cliente.
ISO/IEC 12207	Estándar que establece el marco de trabajo general para los procesos del ciclo de vida del software.
ISO/IEC 25000	Familia de normas (SQuaRE) dedicada a la evaluación y especificación de la calidad del producto de software y datos.
ISO/IEC 27001	Estándar internacional para la gestión de la seguridad de la información mediante un SGSI (Sistema de Gestión de la Seguridad de la Información).
IVA (Impuesto al Valor Agregado)	Impuesto indirecto sobre el consumo derivado de la compra de bienes o de servicios profesionales.
lbl / label	Control de interfaz gráfica pasivo que muestra texto no editable por el usuario en pantalla.
list	Colección ordenada de elementos a los que se puede acceder por índice o recorrer secuencialmente.
Lote	Conjunto agrupado de artículos, datos o transacciones que se procesan o producen como una unidad individual.
lnk / LinkLabel	Control de interfaz gráfica similar a un Label que se comporta como un hipervínculo navegable.
Mayorista	Persona, empresa u organización que vende productos en grandes cantidades a negocios.
MensageBox	Ventana emergente predefinida para mostrar mensajes informativos, de advertencia o confirmación al usuario.
Módulo	Unidad lógica o componente independiente dentro de una aplicación que agrupa funcionalidades específicas.
Monto	Suma final o cantidad total de dinero acumulada en una transacción.

Pack	Se usa para compras cerradas.
Patrón Observer	Patrón de diseño de software de comportamiento donde un objeto (sujeto) notifica automáticamente sus cambios de estado a sus dependientes (observadores).
Pedido	La lista de los productos que se van a comprar al distribuidor.
Post-Condición	Estado o conjunto de garantías que el sistema debe cumplir obligatoriamente al finalizar con éxito un caso de uso.
Pre-Condición	Requisitos o estado previo necesario que debe cumplirse en el sistema antes de que se pueda iniciar la ejecución de un caso de uso.
Precio Unitario	Costo o valor monetario asignado a una sola unidad de un producto o servicio.
PP (Precio público)	El precio sugerido de venta al cliente.
Proveedor	Persona, empresa u organización que suministra productos, materias primas o servicios a otra empresa o negocio.
Razón Social	Nombre legal e identificatorio con el que una empresa o persona jurídica se encuentra registrada formalmente ante el estado.
Reparto	El camión que viene a traer la mercadería.
Retiro	Cuando sacas plata de la caja durante el día.
Rotación de stock	Frecuencia con la que los productos son vendidos y reemplazados.
Rubro	Categoría, sector o actividad comercial a la que pertenece una empresa o negocio.
SHA256	Función hash criptográfica que genera una huella digital única e irreversible de 256 bits para cualquier conjunto de datos.
SKU	Stock Keeping Unit. Código alfanumérico único asignado a un producto para realizar el seguimiento e inventario.
sqlconection	Objeto encargado de establecer y gestionar la comunicación física y la autenticación entre la aplicación y el servidor de SQL Server.
SQL Server	Sistema de gestión de bases de datos relacionales (RDBMS) desarrollado por Microsoft.
Stakeholder	Cualquier persona, grupo u organización que tiene un interés directo o indirecto en el desarrollo o resultado de
Stock	Cantidad de productos disponibles para la venta.
Stock Crítico	Nivel mínimo de unidades de un producto en el inventario, a partir del cual se considera urgente su reposición para evitar quiebres de stock y posibles pérdidas de ventas.

string	Cadena de caracteres o secuencia de texto utilizada como tipo de dato en programación.
Subtotal	Suma de los importes de una venta o factura antes de aplicar impuestos, recargos o descuentos.
txt / textbox	Control de interfaz gráfica de usuario que permite la entrada y edición de texto libre.
Turno	La división del tiempo durante la cual se trabaja. Ej: Mañana/Tarde.
USD	Dólar estadounidense. Código ISO de la moneda oficial de los Estados Unidos.
Vale	Papel que firma el empleado si pide plata adelantada o si saca algún producto del Kiosco.
Vencimiento	Fecha límite de consumo de un producto.
void	Palabra clave en programación que indica que un método o función no retorna ningún valor al finalizar su ejecución.
Vuelto	Cantidad de dinero devuelta al comprador cuando el monto entregado supera el total a pagar.
Windows Forms	Biblioteca de clases de la plataforma .NET para construir interfaces gráficas de usuario en aplicaciones de escritorio para Windows.
.bak	Extensión de archivo estándar utilizada para los archivos de copia de seguridad (backup) de bases de datos como SQL Server.
.NET 8	Versión LTS (soporte a largo plazo) del entorno de ejecución y desarrollo multiplataforma de software desarrollado por Microsoft.
.NET Runtime	Entorno de ejecución que proporciona las herramientas, memoria y servicios necesarios para correr aplicaciones .NET.

3.1 Descripción de la Propuesta

El proyecto consiste en el desarrollo de un sistema informático para la gestión de stock de un maxikiosco, un comercio minorista que maneja una gran variedad de productos con alta rotación diaria, como alimentos, bebidas y artículos de higiene. Actualmente, muchos de estos negocios llevan el control de mercadería de forma manual o con herramientas poco eficientes, lo que genera errores en el inventario, pérdidas por productos vencidos y faltantes de stock.

La propuesta busca informatizar la administración del inventario mediante un software sencillo, intuitivo y accesible para usuarios sin conocimientos técnicos. El sistema permitirá registrar movimientos de mercadería, controlar existencias en tiempo real y generar alertas para facilitar la reposición de productos y reducir pérdidas.

De esta manera, el maxikiosco podrá optimizar su gestión interna, mejorar la organización del stock y tomar decisiones comerciales basadas en información precisa y actualizada.

3.1.1 Alcance

Tareas/Acciones Incluidas en el Desarrollo del Sistema

- Desarrollo de módulo de inicio de sesión con autenticación de usuarios.
- Gestión de roles y permisos de usuario, permitiendo asignar permisos fijos en diferentes roles según se requiera, dando libertad de creación de roles personalizables.
- Registro, modificación, consulta y baja lógica de usuarios (ABM).
- Registro, modificación, consulta y baja de Proveedores, Marcas y Rubros.
- Registro de productos disponibles para la venta.
- Actualización de datos de productos y modificación de precios.
- Registro de compras a proveedores con ingreso automático al stock.
- Registro de ventas con descuento automático de stock.
- Visualización de historial de ventas por fecha o producto.
- Control de inventario en tiempo real.
- Organización del inventario por categorías de productos.
- Configuración de stock ideal por producto.
- Configuración de punto de reposición.
- Configuración de stock crítico.
- Generación de alertas visuales por bajo stock.
- Generación de alertas por productos próximos a vencer.
- Envío opcional de notificaciones por correo electrónico.

- Recuperación y cambio de contraseñas de usuarios con envío de email y preguntas de seguridad integradas en el programa.
- Seguimiento básico de vencimientos por fecha de ingreso al sistema utilizando una lógica mixta entre la aproximación base de Primero en Entrar, Primero en Salir y el control manual de los artículos.
- Incorporación de filtros de búsqueda para facilitar la localización de productos, proveedores, compras y ventas dentro del sistema.
- Generación de informes de venta para el control del rendimiento. (Por fecha, rubro, artículo, marca o medio de pago.
- Registro y visualización de un historial completo de movimientos de mercadería, contemplando entradas y salidas de stock.
- Creación y gestión de promociones de productos, permitiendo definir ofertas y condiciones especiales de venta.
- Gestión de cobros y medios de pago, permitiendo cobros mixtos, con múltiples métodos de pago.
- Control de caja diaria.
- Integración con lectores de código de barras externos.
- Gestión de devoluciones o notas de crédito con los respectivos detalles e historial para el control de las mismas.
- Gestión de Backup, respaldo programable y restauración por sistema.
- Gestión de Bitácora, guardando datos de modificaciones críticas que puedan ser realizadas junto a la fecha, detalles y usuario que realizó la acción.
- Importación y Exportacion de Articulos a Excel.
- Ingresos y Egresos manuales de caja pidiendo justificación y notificando al administrador por mail.

3.1.2 Límites

Tareas/Acciones Excluidas del Desarrollo del Sistema

- Reportes contables o financieros avanzados.
- Gestión de clientes o programas de fidelización.
- Trazabilidad física por número de lote de productos.
- Ubicación individual de cada unidad de mercadería.
- Gestión logística de distribución o envíos.
- El sistema funcionará solo en Windows 10 en adelante en computadoras de escritorio.
- No habrá conexión con plataformas de entrega/delivery.
- No multi-sucursal.
- Sin integración automática con terminales de cobro.

- No permite realizar facturas oficiales relacionadas con la AFIP.
- No calcula ni almacena sueldos.
- Sin conexión con balanzas.
- Permite cambiar el idioma de las interfaces, pero esto no cambia el idioma de los datos que están dentro de la Base de Datos.

3.1.4 Presentación del Negocio

A continuación se presenta la información obtenida en el relevamiento realizado sobre el MaxiKiosco “MTK (MiniTiendaKiosco)”, ubicado en AV. Colombres 746 de la localidad de Lomas de Zamora.
Dicho establecimiento se dedica a la venta de productos de consumo cotidiano, tales como Snacks y galletitas, con el fin de satisfacer a los clientes de la zona, conformados en su mayoría por vecinos o gente que se moviliza a diario por la avenida.
Su operativa se basa en la adquisición de mercadería a mayoristas y proveedores y su posterior comercialización al consumidor final. No incluye actividades de producción ni distribución a gran escala.

3.1.4.1 Propósito del Negocio

El propósito de MTK es ofrecer a los clientes productos de consumo cotidiano de manera rápida, accesible y eficiente, satisfaciendo las necesidades inmediatas de los mismos.

Asimismo, el kiosco tiene como objetivo brindar una atención cercana y constante, fomentando la fidelización de clientes habituales y adaptándose a la demanda del entorno en el que se encuentra.

3.1.4.2 Propósito de Gestorix

El propósito del sistema es mejorar la gestión interna del kiosco mediante la digitalización del control de stock y el registro de productos.

El sistema permitirá conocer en todo momento la cantidad disponible de cada producto, reducir errores asociados al manejo manual de la información y facilitar la actualización de datos de manera rápida y precisa.

Asimismo, contribuirá a optimizar la toma de decisiones del negocio, evitando faltantes de mercadería y mejorando la organización general de las operaciones diarias.

3.1.4.3 Alcance del Negocio

El alcance de MTK comprende la adquisición de mercadería a distintos proveedores y mayoristas, la gestión básica de dichos productos y su comercialización a consumidores dentro del local. Asimismo, incluye la atención al cliente, la reposición de productos y el control general de la disponibilidad de stock.
El kiosco interactúa principalmente con dos actores externos: los proveedores, encargados del suministro de mercadería, y los clientes, quienes adquieren los productos ofrecidos. No se contemplan actividades de producción ni distribución a gran escala, centrándose exclusivamente en la venta minorista.

3.1.5 Idea de Negocio

La idea de negocio del maxikiosco se basa en la comercialización de productos de consumo masivo (snacks, galletitas, bebidas, tabaquería, artículos de almacén, perfumería y librería) y prestaciones de servicio (carga de transporte y celular). Todo esto orientado a satisfacer las necesidades inmediatas de los clientes. El comercio se caracteriza por contar con amplios horarios de atención, adaptándose a la demanda del público y funcionando durante gran parte del día, e incluso en horarios extendidos.

El funcionamiento del kiosco se apoya en dos empleados, quienes se encargan de la atención al cliente, la reposición de mercadería, el control de stock y la verificación de productos. Estas tareas suelen realizarse de manera manual, mediante anotaciones o controles informales, lo que puede generar desorganización, errores en el registro de ventas o compras y dificultades para mantener un control preciso del inventario.

Asimismo, la gestión del negocio incluye la relación con proveedores y mayoristas, la compra de productos, la organización del stock y la implementación de promociones para incentivar las ventas.

3.1.5.1 Misión y Visión

Misión

Brindar a nuestros clientes una atención rápida, cercana y confiable, ofreciendo una amplia variedad de productos de calidad, mientras impulsamos la digitalización de nuestra gestión mediante herramientas tecnológicas que mejoren la organización, el control de ventas y la eficiencia operativa del negocio.

Visión

Expandir su negocio mediante la apertura de una cadena de sucursales, consolidándose como un kiosco moderno y organizado, apoyado en sistemas de gestión eficientes que nos permitan crecer de manera sostenida y ofrecer una mejor experiencia a nuestros clientes.

3.1.5.2 Análisis Estratégico

A partir del relevamiento realizado en distintos comercios ubicados en la zona de influencia del maxikiosco MTK, se identificaron patrones comunes en la forma de operación del rubro.

La mayoría de los negocios relevados presentan una gestión tradicional, con bajo nivel de digitalización, donde el control de stock y ventas no se encuentra sistematizado. Esto evidencia una fuerte dependencia de métodos manuales y de la experiencia de los empleados.

Asimismo, se observa que muchos comercios combinan el kiosco con otros rubros (como almacén o carnicería), ampliando la oferta de productos para competir en el mercado.

En cuanto a la operación diaria, varios locales no cuentan con un flujo constante superior a 100 clientes por día, lo que indica diferencias en el nivel de actividad comercial dentro de la zona.

También se identificó que no todos los negocios poseen horario corrido, lo que puede representar una ventaja frente a nosotros que mantenemos disponibilidad continua.

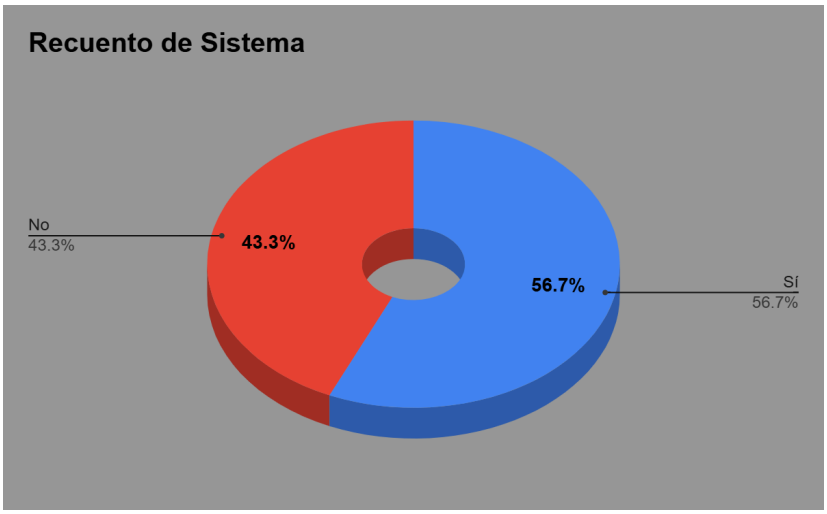
Este contexto refleja un mercado competitivo pero con oportunidades de mejora en la organización y eficiencia operativa.

3.1.5.3 Análisis de mercado

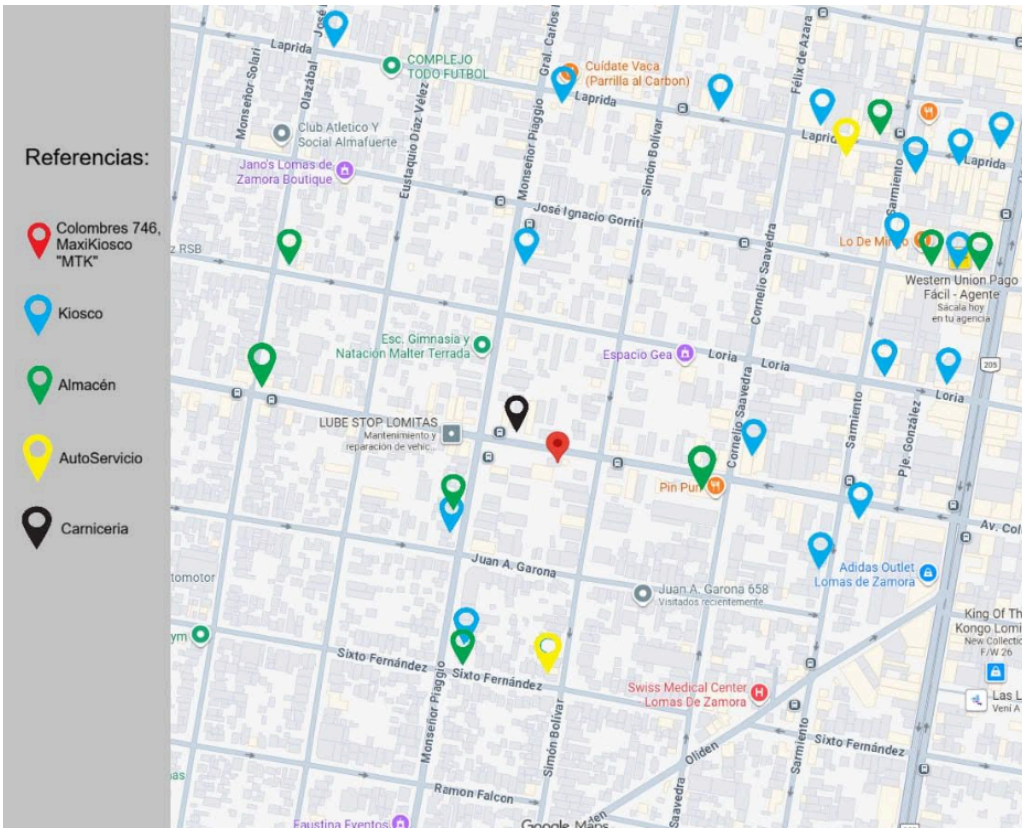
El análisis de los comercios cercanos al maxikiosco MTK permitió identificar aproximadamente 30 locales dentro de un radio de cuatro cuadras, lo que confirma un alto nivel de competencia.

A partir de los datos relevados, se pueden destacar las siguientes características:

- Una parte significativa de los competidores utiliza sistemas de gestión básicos, lo que limita el control del stock y de las ventas.



- La mayoría ofrece variedad de productos, lo que incrementa la competencia directa.
- Solo algunos comercios alcanzan un flujo superior a 100 clientes diarios, lo que evidencia diferencias en el posicionamiento.
- No todos los locales trabajan con horario corrido, lo que puede afectar la disponibilidad para el cliente.
- Ubicación y posicionamiento: se identifican competidores con ubicaciones estratégicas, como cercanía directa a puntos de alto tránsito o zonas comerciales consolidadas. Sin embargo, el maxikiosco MTK se encuentra en una ubicación intermedia con características favorables, ya que está próximo a instituciones escolares y a un edificio residencial de aproximadamente 90 departamentos, lo que representa una fuente constante de potenciales clientes tanto en horarios escolares como en la vida cotidiana de los residentes.



En este contexto, la competencia no solo se da en precios o productos, sino también en aspectos como la organización interna, la disponibilidad horaria y la capacidad de atención.

3.1.5.4 Segmentación del mercado

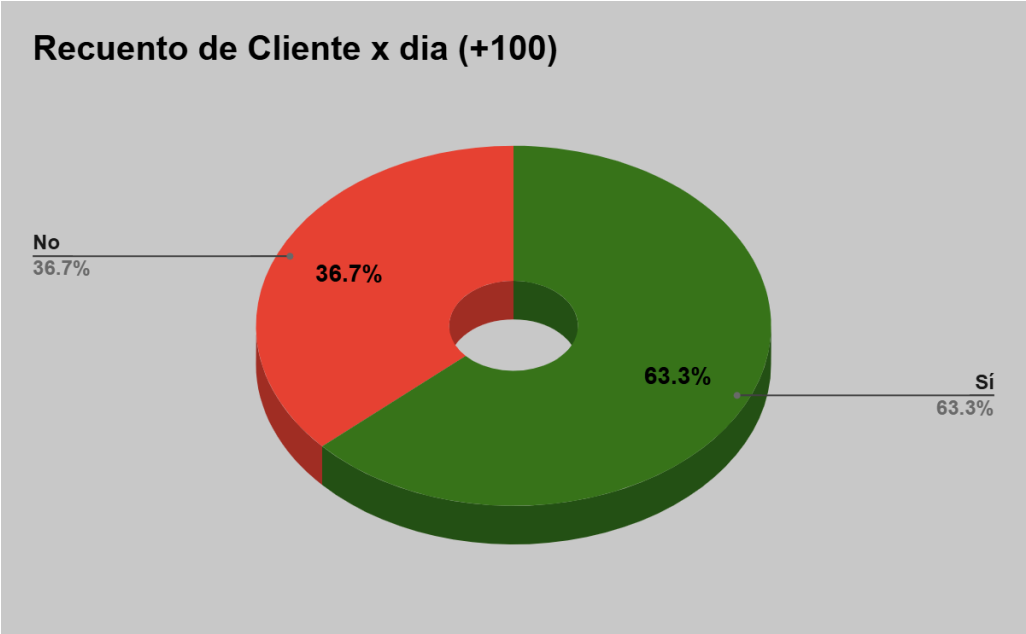
A partir del análisis del entorno y del comportamiento de los comercios relevados, se identifican distintos tipos de clientes:

- Clientes de alta frecuencia: realizan compras diarias o recurrentes, generalmente vecinos de la zona.
- Clientes de paso: personas que circulan por la avenida y realizan compras rápidas.
- Clientes ocasionales: compran de manera esporádica según necesidad.

Dirección	Numeración	Rubro	Sistema	Cliente x dia (+100)	Horario de corrido
Colombres	494	Kiosco	Sí	Sí	Si
Colombres	759	Carniceria/Almacen	Sí	No	Sí
Colombres	617	Carniceria/Almacen	No	No	No
Colombres	999	Carniceria/Almacen	No	No	No
Saavedra	290	Kiosco	Sí	Sí	Sí
Loria	488	Kiosco/Almacen	No	Sí	Sí
Loria	423	Kiosco	Sí	Sí	Sí
Gorriti	401	Almacen/Fiambreria	No	Sí	Sí
Gorriti	417	Kiosco	Sí	Sí	Sí
Gorriti	435	Almacen/Fiambreria	No	No	Sí
Sarmiento	322	Kiosco	Sí	No	Sí
Sarmiento	99	Almacen/Fiambreria	No	No	Sí
Sarmiento	28	Kiosco	Sí	No	Sí
Laprida	180	AutoServicio	Sí	Sí	No
Laprida	417	Kiosco	Sí	Sí	Sí
Laprida	593	Kiosco	No	Sí	Sí
Laprida	440	Kiosco	No	No	Sí
Laprida	519	Almacen/Fiambreria	Sí	Sí	Sí

Dirección	Numeración	Rubro	Sistema	Cliente x dia (+100)	Horario de corrido
Laprida	548	AutoServicio	Sí	Sí	Sí
Colombres	652	Fiambreria/Almacen	No	No	No
Laprida	780	AutoServicio	Sí	Sí	Sí
Laprida	796	Kiosco	No	Sí	Sí
Laprida	999	Kiosco	Sí	Sí	Sí
Loria	997	Almacen	No	No	No
Piaggio	383	Kiosco	No	Sí	Sí
Piaggio	377	Almacen	No	No	No
Piaggio	150	Kiosco	Sí	Sí	No
Piaggio	460	Kiosco	Sí	Sí	Sí
Piaggio	494	Almacen/Kiosco	Sí	Sí	Sí
Bolivar	489	AutoServicio	Sí	Sí	Sí

Además, el flujo de clientes varía según el comercio, observándose que no todos alcanzan altos niveles de demanda diaria, lo que refleja diferencias en la competitividad de cada negocio.



3.1.5.5 FODA

Fortalezas

- Ubicación en zona de alto tránsito.
- Variedad de productos similar o competitiva respecto a otros comercios.
- Posibilidad de captar clientes de comercios con menor organización.

Oportunidades

- Bajo nivel de digitalización en la competencia.
- Posibilidad de diferenciarse mediante mejor organización y control.
- Comercios sin horario corrido generan oportunidades de captar más clientes.

Debilidades

- Gestión manual similar a la de muchos competidores.
- Falta de control preciso del stock.
- Dependencia de procesos informales.

Amenazas

- Alta densidad de competidores (30 locales).
- Comercios con mayor variedad (almacén + kiosco).
- Diferencias en flujo de clientes entre negocios.
- Competidores con mejor ubicación puntual o mayor visibilidad.

3.1.5.6 Análisis de factibilidad

Factibilidad técnica

El maxikiosco MTK cuenta con los recursos tecnológicos básicos necesarios para implementar mejoras en su gestión, ya que dispone de al menos una computadora operativa dentro del local. La misma cuenta con sistema operativo Windows 10, 8 GB de memoria RAM para el correcto funcionamiento del Sistema Gestorix y el software Excel instalado, lo que permite la migración y administración de la información del negocio de manera adecuada.

Por otro lado, el equipo desarrollador dispone de los recursos necesarios para llevar adelante el proyecto, contando con cinco computadoras para el desarrollo, lo que garantiza la viabilidad técnica del mismo.

Factibilidad económica

Desde el punto de vista económico, la propuesta resulta viable debido a que el costo de implementación será accesible, adaptado a la realidad del comercio y experiencia del equipo.

El kiosco se encuentra en una situación estable, con actividad constante y proyección a futuro, lo que permite afrontar la incorporación de mejoras sin comprometer su funcionamiento.

Además, la inversión se justifica por los beneficios esperados, como la reducción de pérdidas, un mejor control del stock y una mayor eficiencia en la gestión.

Equipo:

- 5 personas
- 7 meses
- 25 usd por hora

Costo:

Por día 2hs × 25 usd = USD 50

Por semana 5 días x 50 usd= USD 250

Por mes 4 Semanas x 250 = USD 1000

En 7 meses USD 7000 (Total x 5 empleados)

En pesos (aprox):

- Si 1 USD ≈ 1300 ARS
  >> **9.100.000,00 ARS**

Factibilidad operativa

Las tareas del kiosco son repetitivas y estructuradas (ventas, reposición, control de productos), lo que facilita la incorporación de mejoras en la gestión.

El personal ya se encuentra habituado a la operatoria del negocio, por lo que podrá adaptarse progresivamente a nuevas herramientas sin generar un impacto negativo en la atención al cliente ni en el funcionamiento diario.

Asimismo, el equipo de trabajo está compuesto por empleados que pueden utilizar estas herramientas en sus tareas diarias, lo que facilita la incorporación de nuevas formas de gestión sin requerir conocimientos técnicos avanzados.

3.1.6 Descripción general de Stakeholders y Usuarios

Los principales stakeholders son el dueño del comercio, quien además cumple funciones operativas dentro del local, dos empleados encargados de las tareas diarias, el equipo desarrollador compuesto por cinco integrantes responsables del análisis, diseño e implementación del sistema y todos los clientes del kiosco a los que el uso del sistema les beneficiara con una mayor velocidad de atención.

3.1.6.1 Resumen de Stakeholders

Stakeholder	Rol en el proyecto	Interés principal	Nivel de Acceso

Dueño del kiosco	Cliente y usuario	Mejorar la gestión y control del negocio.	Completo
Empleados	Usuario operativo	Registrar ventas; facilitar el control de stock, reposición y verificación de vencimientos.	Operativo limitado
Equipo desarrollador	Desarrollo del sistema	Diseñar e implementar el sistema.	Especializado
Clientes finales	Consumidor del kiosco	Mejor experiencia de la atención recibida.	Nulo

Usuarios clave

Nombre	Descripción	Responsabilidad
Alberto Suarez	Dueño del Kiosco y trabajador en el mismo.	Define intereses, necesidades, alcances y aporta datos e información.

Resumen de Usuarios

Rol	Descripción	Nombre	StakeHolder
Dueño del kiosco	Responsable de la administración general del sistema.	Alberto Suarez	Dueño

	Gestiona ventas, compras, stock, reportes, configuración y permisos de usuarios.		
Empleado	Responsable de registrar ventas, cobros y movimientos de caja. Consulta stock y controla vencimientos de productos.	Matias	Ventas
Empleada	Responsable de registrar ventas, cobros y movimientos de caja. Consulta stock y controla vencimientos de productos.	Silvina	Ventas

3.1.6.2 Entorno de Usuario

El sistema será utilizado en el maxikiosco MTK, ubicado en Lomas de Zamora, mediante computadoras de escritorio conectadas a una red local interna.

Los usuarios accederán al sistema desde equipos con sistema operativo Windows 10, utilizando una interfaz gráfica intuitiva diseñada para facilitar la operación diaria del negocio. El acceso al sistema requerirá autenticación mediante nombre de usuario y contraseña, permitiendo identificar a cada operador y habilitando las funciones correspondientes según el perfil asignado.

La aplicación trabajará con una base de datos SQL Server conectada en forma local, garantizando rapidez de acceso y disponibilidad de la información durante la jornada comercial.

El entorno de uso se caracteriza por una dinámica de atención simultánea al cliente y una operatoria continua entre las 8:00 y las 22:00 hs, por lo que el sistema deberá responder de manera rápida, simple y confiable, minimizando tiempos de carga y errores operativos.

Los usuarios poseen conocimientos informáticos básicos, principalmente relacionados con el uso cotidiano de sistemas de facturación y herramientas de oficina. Debido a ello, la interfaz deberá priorizar:

- Simplicidad de uso.
- Facilidad de aprendizaje.
- Rapidez en las operaciones.
- Claridad visual.
- Seguridad en el manejo de la información.

No se prevé una capacitación técnica extensa, aunque será necesaria una breve inducción sobre el uso específico del sistema y la gestión de usuarios.

Los reportes generados por el sistema podrán exportarse en formatos digitales como Excel para su almacenamiento.

3.1.6.3 Perfil de los Stakeholders

3.1.6.3.1 Dueño

Representante	Administración general del kiosco MTK
Descripción	Responsable de administrar el sistema y supervisar el funcionamiento general del kiosco.
Tipo	Usuario Clave
Responsabilidad	Gestionar usuarios, controlar permisos, supervisar ventas, compras, stock y reportes generales del sistema.
Criterio de éxito	Que el sistema permita administrar correctamente todas las operaciones del kiosco de forma rápida y segura.
Grado de participación	Alta.
Comentarios	Cualquier modificación importante del sistema deberá ser aprobada por el administrador.

3.1.6.3.2 Empleado

Representante	Personal operativo del kiosco MTK
Descripción	Usuarios encargados de las operaciones diarias del kiosco, incluyendo ventas, atención al cliente, control de stock y compras.
Tipo	Usuario Operativo
Responsabilidad	Registrar ventas, controlar caja, consultar stock, registrar compras, gestionar reposición de productos y verificar vencimientos.
Criterio de éxito	Realizar las operaciones diarias de manera rápida, ordenada y sin errores, manteniendo actualizado el inventario y garantizando una correcta atención al cliente.
Grado de participación	Alta
Comentarios	Utilizan el sistema diariamente durante toda la jornada laboral y dependen directamente de su correcto funcionamiento.

3.1.6.3.3 Equipo de desarrollo

Representante	Grupo de desarrollo del sistema
Descripción	Equipo responsable del análisis, diseño, desarrollo, implementación y mantenimiento del sistema del kiosco.
Tipo	Stakeholder Técnico
Responsabilidad	Desarrollar el sistema según los requerimientos establecidos, corregir errores, implementar mejoras y garantizar el correcto funcionamiento del software.
Criterio de éxito	Entregar un sistema funcional, seguro, estable y adaptado a las necesidades del kiosco.
Grado de participación	Alta
Comentarios	El equipo participa durante todas las etapas del proyecto: análisis, desarrollo, pruebas e implementación.

3.1.6.3.4 Cliente del Kiosco

Representante	Consumidores finales
Descripción	Personas que realizan compras en el kiosco.
Tipo	Usuario Externo
Responsabilidad	Realizar compras y recibir atención rápida y eficiente.
Criterio de éxito	Ser atendido de manera rápida y obtener disponibilidad de productos.
Grado de participación	Baja
Comentarios	No interactúa directamente con el sistema, pero es beneficiado por su funcionamiento.

3.2 Descripción Global del Producto

3.2.1 Perspectiva del Producto

Gestorix es un sistema de escritorio orientado a la gestión operativa de maxikioscos y pequeños comercios minoristas.

Su objetivo principal es digitalizar y optimizar procesos que actualmente se realizan de manera manual, como el control de stock, registro de ventas, compras, usuarios y alertas de productos.

El sistema busca brindar una solución simple, accesible y segura para negocios con bajo nivel de digitalización.

Gestorix es un producto independiente desarrollado para ejecutarse en computadoras de escritorio con sistema operativo Windows.

El sistema interactúa con:

- Base de datos SQL Server para almacenamiento persistente
- Usuarios internos (administrador y empleados)
- Sistema local del kiosco

Se integra conceptualmente con la operatoria habitual del negocio:

- Ingreso de mercadería
- Ventas diarias
- Control de inventario
- Administración de usuarios

3.2.2 Resumen de Características

El sistema contará con los siguientes módulos funcionales:

Característica	Descripción
Gestión de Usuarios	Permite administrar a los empleados junto a sus permisos. ● Alta de usuarios. ● Modificación. ● Baja lógica. ● Asignación de roles.
Gestión de Productos	Permite administrar el stock, flujo, precios, fechas y categorías de la mercadería. ● Registrar productos. ● Editar información. ● Categorizar productos. ● Visualizar stock.
Gestión de Ventas	Permite realizar, registrar y consultar las ventas. ● Registrar ventas. ● Descontar stock automáticamente. ● Registrar medio de pago. ● Consultar historial.
Gestión de Compras	Permite visualizar las compras realizadas, información de proveedores y el ingreso del inventario. ● Registrar compras a proveedores. ● Actualizar el inventario. ● Historial de compras.
Gestión de Inventario	Permite visualizar el stock en tiempo real, junto a las respectivas alertas de vencimiento o bajo stock. ● Visualizar el stock. ● Visualizar alertas por stock crítico.

	• Visualizar alertas por productos próximos a vencer.
Reportes	Permite acceder a los informes de ventas. • Ganancias diarias. • Ganancias mensuales. • Movimientos de stock.
Seguridad	Permite controlar el acceso a información sensible y el acceso a datos de cada empleado. • Login de usuarios. • Primer inicio obligatorio. • Cambio de contraseña. • Recuperación de contraseña. • Preguntas de seguridad.

3.2.3 Suposiciones y Dependencias

Para el correcto funcionamiento del sistema se asume que el maxikiosco dispone de al menos una computadora operativa con sistema operativo Windows 10 y acceso a la red local interna.

Se supone además que los usuarios poseen conocimientos básicos de informática para operar el sistema, registrar ventas y consultar información de stock.

El sistema dependerá de una base de datos SQL Server correctamente instalada y configurada, utilizada para el almacenamiento y recuperación de la información comercial.

La aplicación requerirá conexión local entre el sistema y la base de datos para garantizar el acceso a los datos de ventas, productos y stock en tiempo real.

El sistema podrá interactuar con herramientas externas para la exportación de reportes e informes, especialmente mediante archivos compatibles con Microsoft Excel.

Para agilizar la operatoria diaria se considera el uso de un lector de código de barras compatible con el sistema.

La recuperación de credenciales por correo electrónico dependerá de la disponibilidad de conexión a internet y de un servicio de correo configurado correctamente.

El correcto funcionamiento del sistema dependerá de la disponibilidad del suministro eléctrico y del correcto estado del equipamiento informático utilizado en el kiosco.

3.3 Descripción Detallada del Producto

El sistema estará compuesto por diferentes módulos funcionales destinados a optimizar y digitalizar la administración integral del kiosco.

3.3.1 Módulo de Gestión de Usuarios

Este módulo permitirá administrar los usuarios que tendrán acceso al sistema. El administrador podrá crear, modificar, desactivar y controlar usuarios mediante una interfaz simple e intuitiva.

Cada usuario contará con:

- Nombre de usuario.
- Datos Personales.
- Contraseña cifrada mediante SHA256.
- Correo electrónico.
- Rol asignado dentro del sistema.

El sistema manejará distintos niveles de acceso según el rol y los permisos asignados de cada uno.

Funcionamiento del módulo

El administrador registrará un nuevo usuario indicando sus datos principales y el rol correspondiente. El sistema generará una contraseña temporal que deberá ser modificada obligatoriamente en el primer inicio de sesión.

Cada vez que un usuario intente acceder:

1. El sistema validará credenciales.
2. Verificará que el usuario se encuentre activo.
3. Comparará la contraseña cifrada almacenada en la base de datos.
4. Habilitará únicamente los módulos permitidos según el rol.

Controles implementados:

- Validación de campos obligatorios.
- Restricción de acceso según permisos.
- Contraseñas cifradas.

- Recuperación de contraseña mediante correo electrónico.
- Control de usuarios activos/inactivos.

3.3.2 Módulo de Productos

Este módulo permitirá administrar todos los productos comercializados por el kiosco.

Cada producto contará con:

- Código.
- Nombre.
- Categoría.
- Precio.
- Stock disponible.
- Fecha de vencimiento.
- Estado.
- Lote.

Funcionamiento del módulo:

El usuario autorizado podrá registrar nuevos productos, modificar información existente o desactivar productos que ya no se comercialicen.

El sistema permitirá:

- Buscar productos rápidamente.
- Filtrar por categoría.
- Consultar stock disponible.
- Detectar productos próximos a vencer.

Controles implementados:

- Validación de precios positivos.
- Control de stock mínimo.
- Restricción de eliminación física de productos.
- Verificación de fechas de vencimiento.

3.3.3 Módulo de Ventas

Este módulo estará orientado a agilizar el proceso de atención al cliente y registro de ventas.

Funcionamiento del módulo:

El empleado seleccionará productos mediante búsqueda o lectura de código de barras. El sistema calculará automáticamente:

- Subtotal.
- Cantidad de productos.
- Total final.

Al confirmar la venta:

1. Se registrará la operación.
2. Se actualizará automáticamente el stock.
3. Se almacenará el movimiento en la base de datos.
4. Se emitirá un comprobante simple.

Controles implementados:

- Validación de stock disponible.
- Restricción de ventas con stock insuficiente.
- Registro automático de movimientos.
- Cálculo automático de importes.

3.3.4 Módulo de Compras

Permitirá registrar compras realizadas a proveedores y actualizar el inventario del sistema.

Funcionamiento del módulo:

El empleado encargado podrá:

- Registrar proveedores.
- Generar órdenes de compra.

- Registrar ingreso de mercadería.

Cuando una compra sea confirmada:

1. El sistema actualizará el stock automáticamente.
2. Registrará costos de compra.
3. Asociará productos al proveedor correspondiente.

Controles implementados:

- Validación de proveedores existentes.
- Control de cantidades ingresadas.
- Registro histórico de compras.
- Actualización automática del inventario.

3.3.5 Módulo de Inventario

Este módulo permitirá controlar permanentemente el estado del stock del kiosco.

Funcionamiento del módulo:

El sistema realizará controles automáticos para:

- Detectar faltantes.
- Identificar productos críticos.
- Visualizar vencimientos próximos.
- Generar alertas visuales.

También permitirá:

- Consultar movimientos de stock.
- Visualizar historial de reposiciones.

Controles implementados:

- Alertas de stock crítico.
- Alertas de vencimiento.
- Registro automático de movimientos.
- Actualización en tiempo real del inventario.

3.3.6 Módulo de Reportes y Balances

Permitirá obtener información estadística y administrativa del kiosco.

Funcionamiento del módulo:

El usuario podrá generar reportes:

- Diarios.
- Semanales.
- Mensuales.

El sistema mostrará:

- Ventas realizadas.
- Ganancias.
- Productos más vendidos.
- Movimientos de caja.
- Compras registradas.

Controles implementados:

- Filtros por fecha.
- Validación de rangos de búsqueda.
- Generación automática de balances.
- Exportación de información.

3.3.7 Módulo de Autenticación y Seguridad

Este módulo garantizará la seguridad del acceso al sistema.

Funcionamiento del módulo:

El usuario deberá ingresar:

- Nombre de usuario.
- Contraseña.

El sistema:

1. Cifrará la contraseña ingresada.
2. Comparará el hash con la base de datos.
3. Validará permisos.
4. Permitirá o rechazará el acceso.

También incluirá:

- Recuperación de contraseña.
- Cambio obligatorio de contraseña temporal.
- Cierre seguro de sesión.

Controles implementados:

Contraseñas cifradas con SHA256.

- Validación de credenciales.
- Restricción según roles.
- Recuperación segura mediante correo electrónico.
- Control de sesión activa.

3.4 Restricciones

El sistema estará desarrollado únicamente para las funcionalidades y especificaciones acordadas con el cliente al inicio del proyecto, reemplazando los métodos manuales y herramientas actualmente utilizadas en la administración del kiosco.

La Empresa desarrolladora cumplirá con las acciones establecidas durante el desarrollo e implementación del sistema, respetando los requerimientos funcionales y técnicos definidos para el proyecto.

Tareas de la Empresa Desarrolladora

- Desarrollo del sistema de acuerdo con los requerimientos y especificaciones establecidas.
- Realización de reuniones periódicas con el cliente para seguimiento y validación del proyecto.
- Instalación y configuración del sistema en el entorno operativo del kiosco.
- Capacitación básica del personal para el uso correcto del software.
- Elaboración de documentación y material de ayuda para los usuarios.
- Soporte técnico frente a problemas o fallas imprevistas originadas durante el uso del sistema.

El sistema no contemplará la gestión completa de empleados, ya que únicamente registrará usuarios necesarios para operar dentro del software, almacenando solamente la información requerida para autenticación, roles y permisos.

Además, el sistema presentará las siguientes restricciones técnicas y operativas:

- Funcionamiento exclusivo en sistemas operativos Windows 10 o superiores.
- Sistema desarrollado únicamente para entorno de escritorio.
- Administración limitada a una única sucursal.
- No incluirá integración con facturación electrónica.
- No poseerá integración bancaria ni pasarelas de pago online.
- No contemplará gestión logística externa ni distribución de productos.
- Dependencia de SQL Server como sistema gestor de base de datos.

3.4.1 Otros Requisitos del Producto

La organización deberá garantizar un entorno de trabajo adecuado para asegurar el correcto funcionamiento del sistema y el cumplimiento de los requisitos del producto.

El sistema deberá ofrecer:

- Interfaz gráfica intuitiva y sencilla.
- Navegación rápida y organizada entre módulos.
- Seguridad básica mediante autenticación y control de permisos.
- Persistencia y almacenamiento seguro de la información.
- Integridad y consistencia de los datos registrados.
- Facilidad de uso para el personal operativo.
- Estabilidad durante jornadas prolongadas de trabajo.

3.4.2 Estándares Aplicables

Principales estándares considerados

Estándares de calidad

- ISO 9001.
- ISO/IEC 25000 .

Estándares de desarrollo

- ISO/IEC 12207 .
- IEEE 830 .

Estándares de seguridad

- ISO/IEC 27001 .
- Buenas prácticas OWASP para validación y seguridad de accesos.

Estándares de codificación

- Desarrollo realizado en lenguaje C#.
- Uso de arquitectura en capas:
    - Presentación.
    - Lógica.

    - Acceso a datos.
- Uso de convenciones de nombres propias de C#.
- Implementación de consultas SQL parametrizadas.
- Validación de datos en formularios.
- Manejo de excepciones para control de errores.
- Separación entre interfaz gráfica y lógica del sistema.
- Cifrado de contraseñas mediante SHA256.
- Compatibilidad con .NET 8, Windows Forms y SQL Server.

3.4.3 Requisitos de Sistema

Requisitos funcionales

El sistema deberá permitir:

- Autenticación de usuarios.
- Gestión de productos.
- Registro de ventas.
- Registro de compras.
- Control de inventario.
- Generación de reportes y balances.
- Administración de usuarios y permisos.

Requisitos no funcionales

El sistema deberá garantizar:

- Seguridad de acceso mediante roles y contraseñas cifradas.
- Integridad y consistencia de datos.
- Compatibilidad con Windows 10 o superior.
- Estabilidad durante jornadas prolongadas de uso.
- Rendimiento adecuado en operaciones diarias.

3.4.4 Requisitos de Desempeño

Velocidad de respuesta

- Inicio de sesión rápido, tiempo máximo 1.5 segundos.
- Registro de ventas en tiempo real, tiempo máximo 2 segundos.
- Consultas rápidas de productos y stock , máximo 3 segundos.
- Generación de reportes en menos de 5 segundos dependiendo del volumen de información.

Capacidad

- Soporte para múltiples operaciones diarias, un solo usuario.
- Manejo simultáneo de consultas y registros sin degradación notable.

Disponibilidad

- Funcionamiento continuo durante toda la jornada laboral del kiosco, 99.9% uptime.

Eficiencia de recursos

- Uso optimizado de memoria y procesamiento.
- Estabilidad en equipos de recursos moderados.

3.4.5 Requisitos de Entorno

Los requisitos de entorno especifican las condiciones necesarias para el correcto funcionamiento del sistema dentro del kiosco.

Hardware

- Computadora principal para operación del sistema.
- Procesador y memoria compatibles con .NET 8.
- Dispositivos básicos de entrada y salida.

Software

- Windows 10 o superior.
- SQL Server instalado y configurado.
- .NET Runtime compatible.

Red y conectividad

- Conexión local entre aplicación y base de datos.
- Acceso a internet para recuperación de contraseñas por correo electrónico.

Entorno operativo

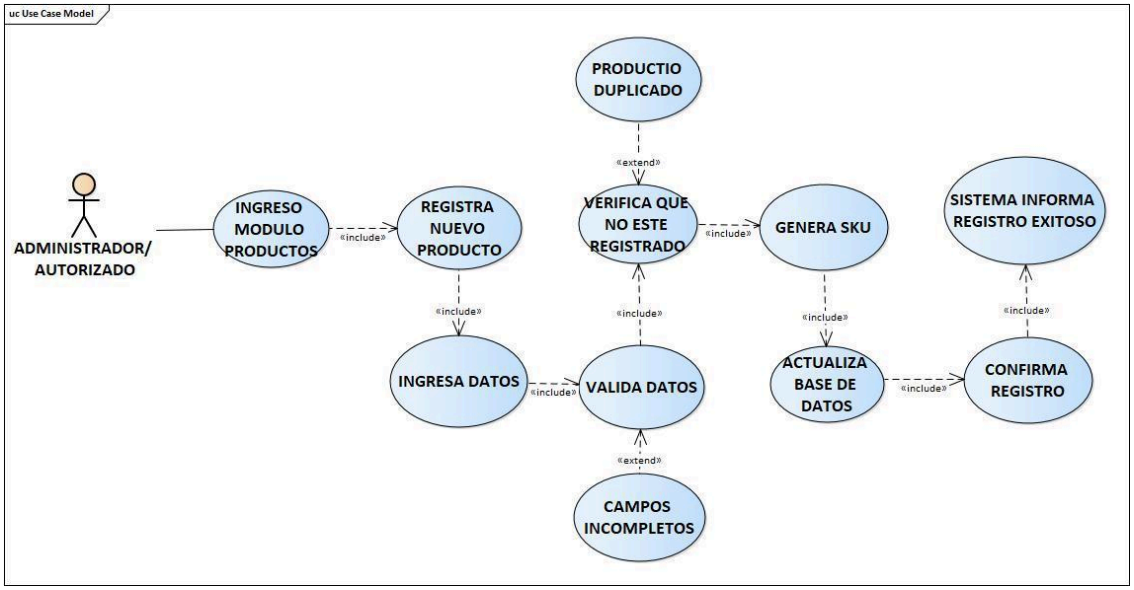
- Kiosco comercial de atención al público.
- Uso continuo diario.
- Atención simultánea a clientes.
- Operación local desde computadora principal de caja.

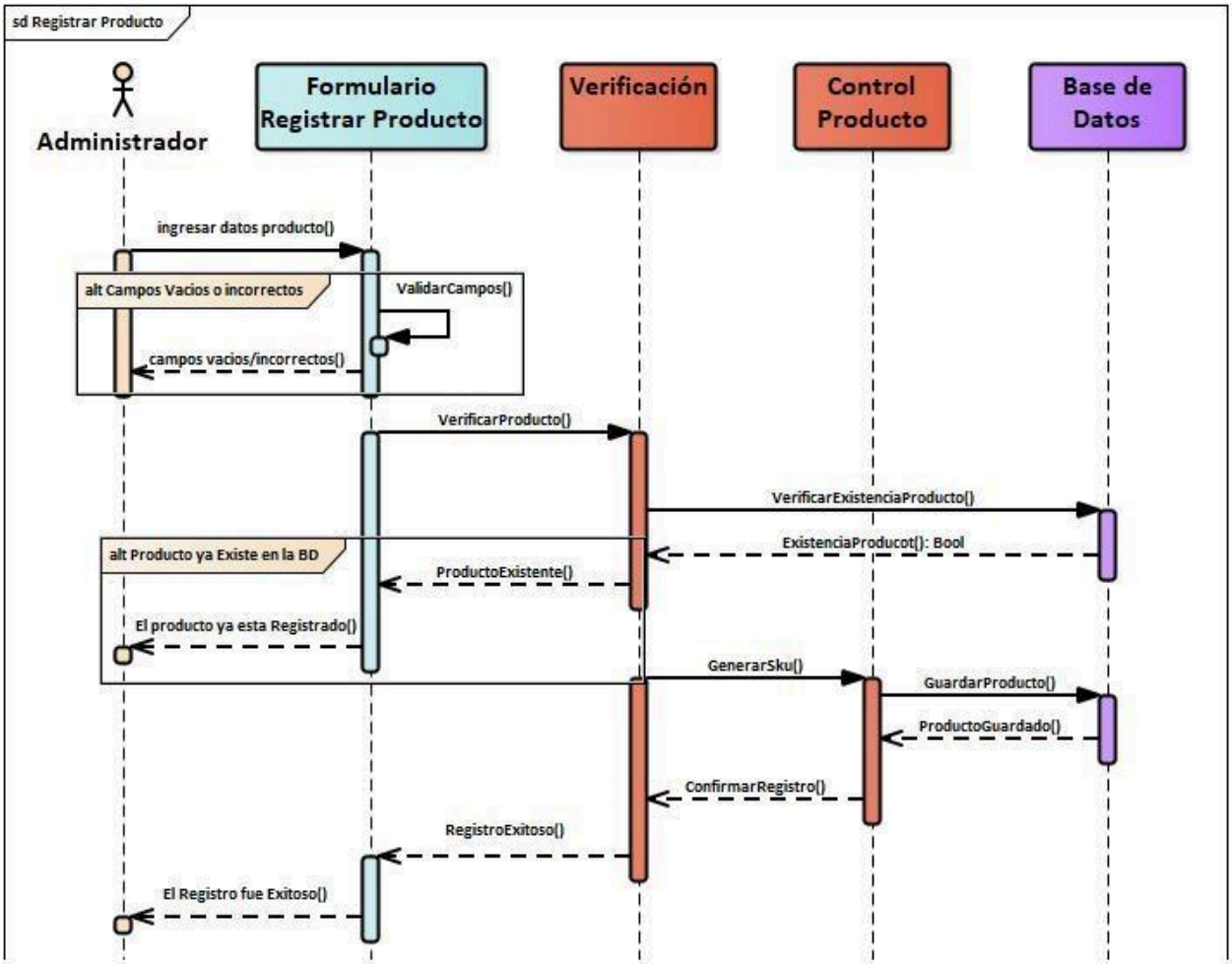
3.5 Casos de Uso

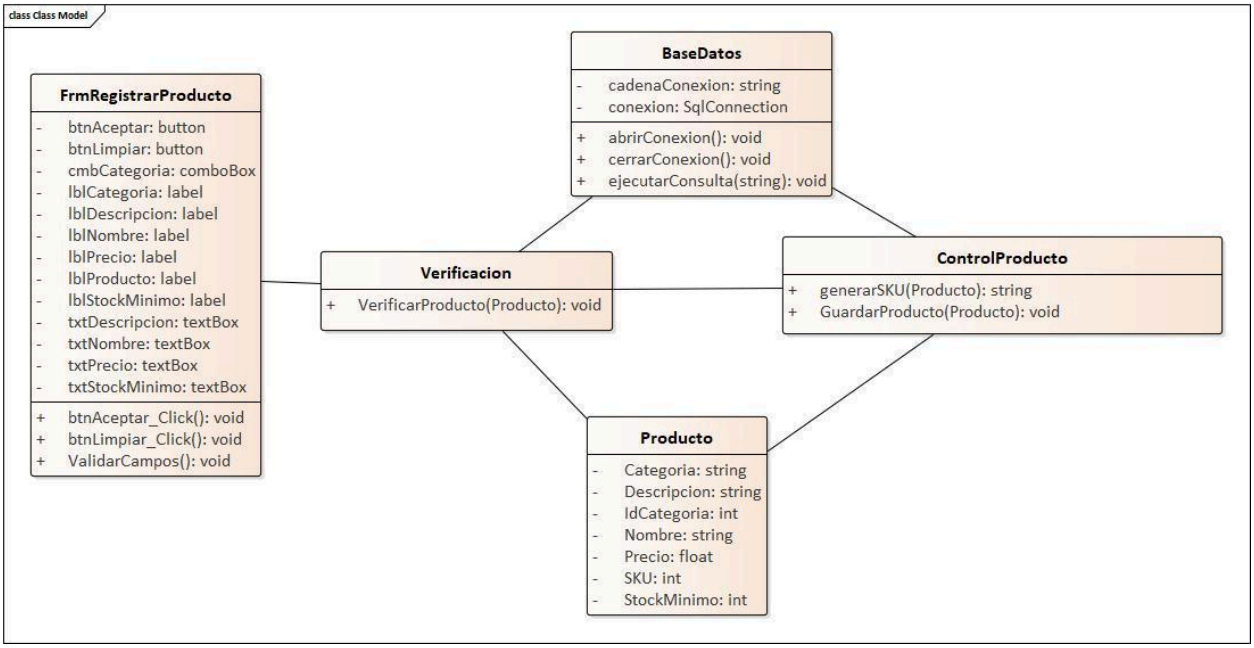
Casos de Uso de Negocio

CU01 - Registrar Producto

ID y Nombre	CU01 Registrar Producto	
Descripción	Permite al dueño o empleado autorizado cargar un nuevo producto al inventario del kiosco, asignando atributos como nombre, precio, categoría y stock inicial.	
Actores	Administrador - Empleado autorizado.	
Pre-Condición	El usuario debe haber iniciado sesión y tener permisos de registro.	
Post-Condición	El producto queda registrado en la base de datos y disponible en el inventario.	
FLUJO	Flujo Normal	Flujo Alternativo
	1. El usuario accede al módulo de productos.	
	2. Selecciona la opción “Registrar producto”.	
	3. Ingresa los datos obligatorios: nombre, categoría y stock minimo. No obligatorios : precio.	
	4. El sistema valida los datos ingresados.	4.1. Datos incompletos o inválidos: El sistema detecta campos vacíos o incorrectos. Muestra un mensaje de error y solicita corrección.
	5.El sistema verifica que el producto no se encuentre registrado previamente.	5.1 Producto duplicado: El sistema detecta que el SKU ya existe. Informa al usuario y solicita corrección.
	6. El sistema Genera un Sku para el producto	
	7. El sistema guarda el producto en la base de datos.	
	8. El sistema confirma el registro exitoso.	


uc Use Case Model
PRODUCTIO DUPLICADO
«extend»
ADMINISTRADOR/ AUTORIZADO
INGRESO MODULO PRODUCTOS
«include»
REGISTRA NUEVO PRODUCTO
VERIFICA QUE NO ESTE REGISTRADO
«include»
GENERA SKU
SISTEMA INFORMA REGISTRO EXITOSO
«include»
«include»
«include»
«include»
INGRESA DATOS
«include»
VALIDA DATOS
ACTUALIZA BASE DE DATOS
«include»
CONFIRMA REGISTRO
«extend»
CAMPOS INCOMPLETOS

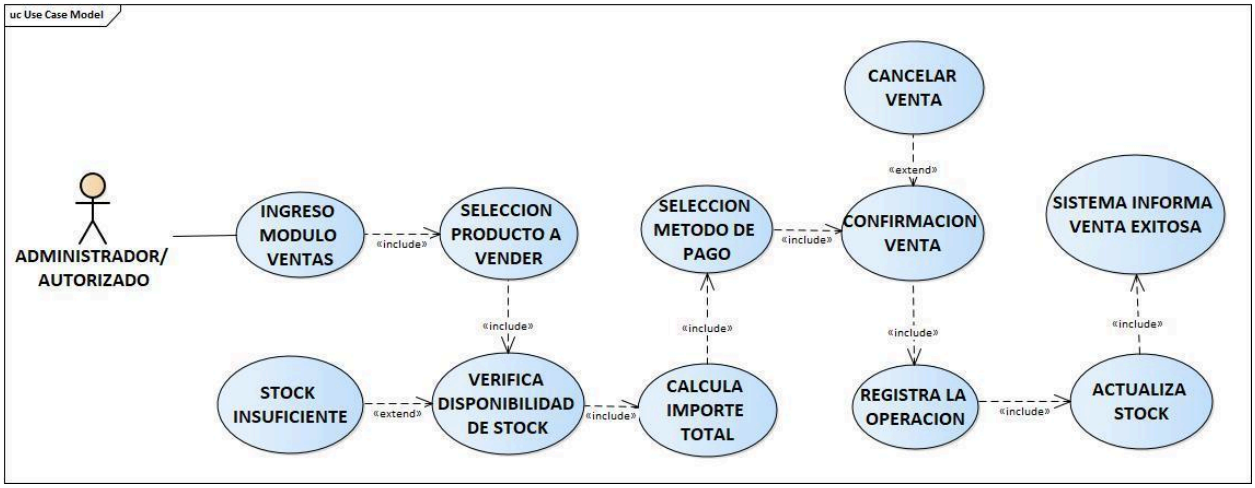

sd Registrar Producto
Administrador
Formulario Registrar Producto
Verificación
Control Producto
Base de Datos
ingresar datos producto()
alt Campos Vacios o incorrectos
ValidarCampos()
campos vacios/incorrectos()
VerificarProducto()
VerificarExistenciaProducto()
ExistenciaProducot(): Bool
alt Producto ya Existe en la BD
ProductoExistente()
El producto ya esta Registrado()
GenerarSku()
GuardarProducto()
ProductoGuardado()
ConfirmarRegistro()
RegistroExitoso()
El Registro fue Exitoso()

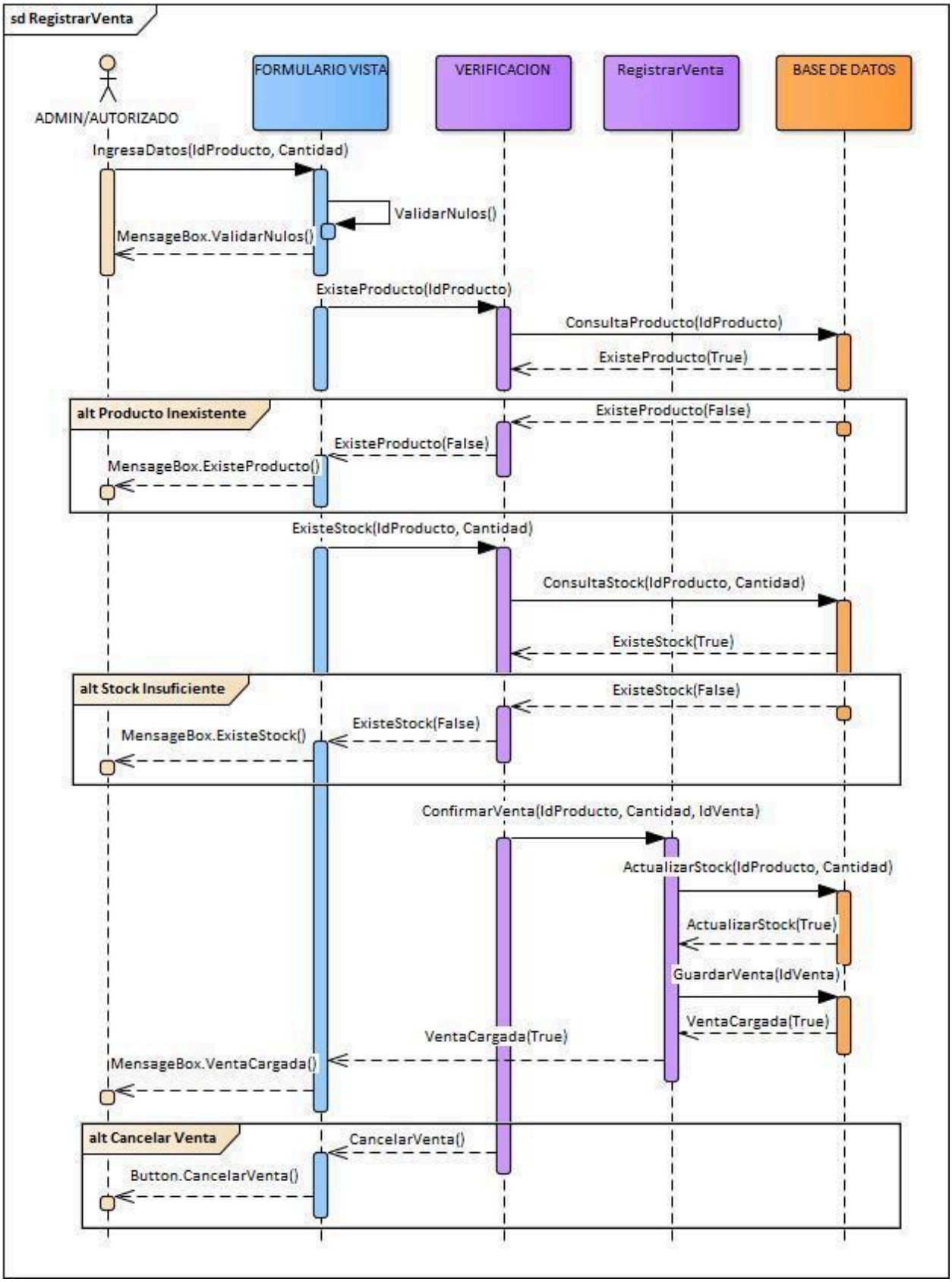


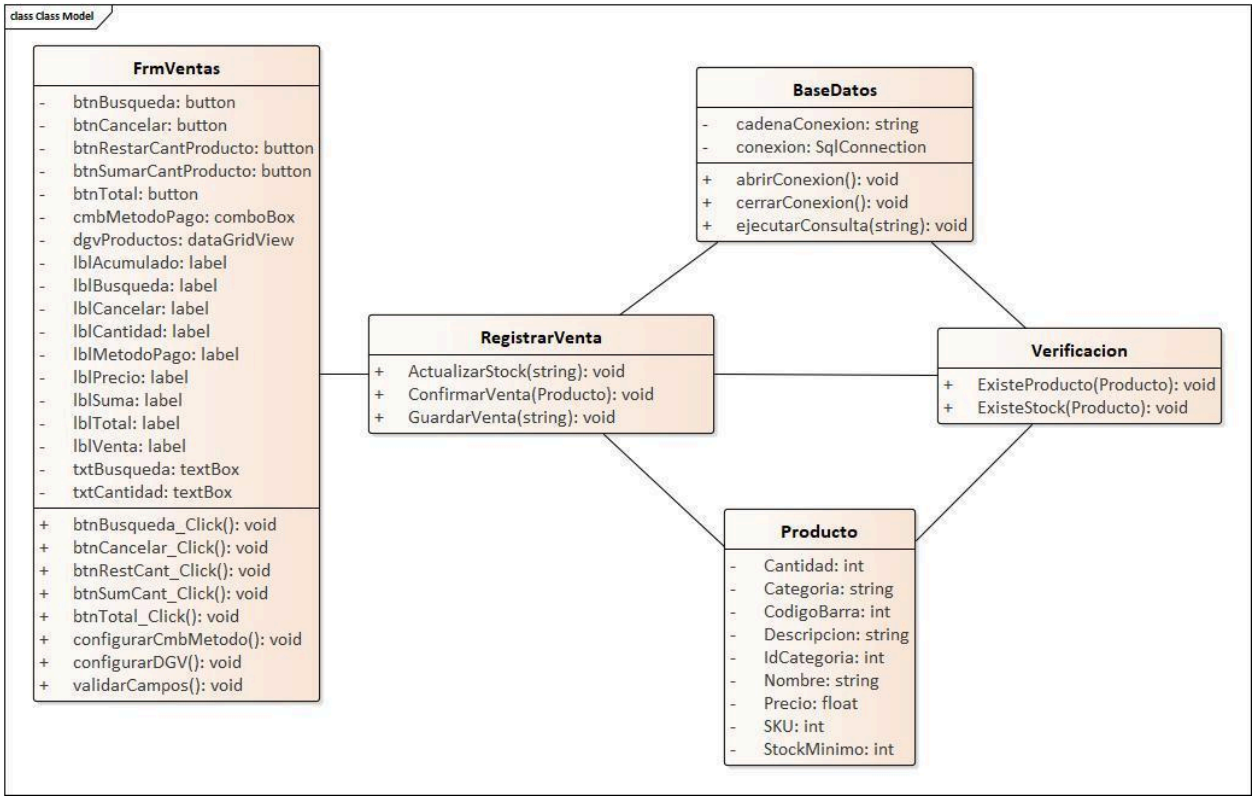
CU02 - Registrar Venta

ID y Nombre	CU02 Registrar Venta	
Descripción	Permite al usuario autorizado registrar una venta de productos dentro del sistema. El proceso contempla la selección de artículos, la verificación automática de stock disponible, el cálculo del importe total de la venta y la actualización del inventario. Una vez confirmada la operación, el sistema registra la transacción en la base de datos y genera el movimiento correspondiente para su posterior consulta en reportes y balances.	
Actores	Administrador - Empleado autorizado.	
Pre-Condición	El usuario debe haber iniciado sesión. El usuario debe poseer permisos para registrar ventas. Los productos deben estar previamente registrados en el sistema.	
Post-Condición	La venta queda registrada. El stock de los productos vendidos se actualiza automáticamente. Se registra el movimiento correspondiente para balances e informes.	
FLUJO	**Flujo Normal**	**Flujo Alternativo**
	1. El usuario accede al módulo Ventas.	

	2. Se seleccionan los productos a vender y sus cantidades.	
	3. El sistema verifica la disponibilidad de stock.	3.1 Stock insuficiente El sistema informa que no existe stock suficiente para completar la venta. El usuario puede modificar las cantidades o cancelar la operación.
	4. El sistema calcula el importe total.	
	5. El usuario confirma la venta.	5.1 Cancelación de la venta El usuario decide cancelar la operación. El sistema descarta la información ingresada y no registra cambios.
	6. El usuario elige el medio de pago.	
	7. El sistema registra la operación.	
	8. El sistema actualiza el stock.	
	9. El sistema informa que la venta fue realizada correctamente.	



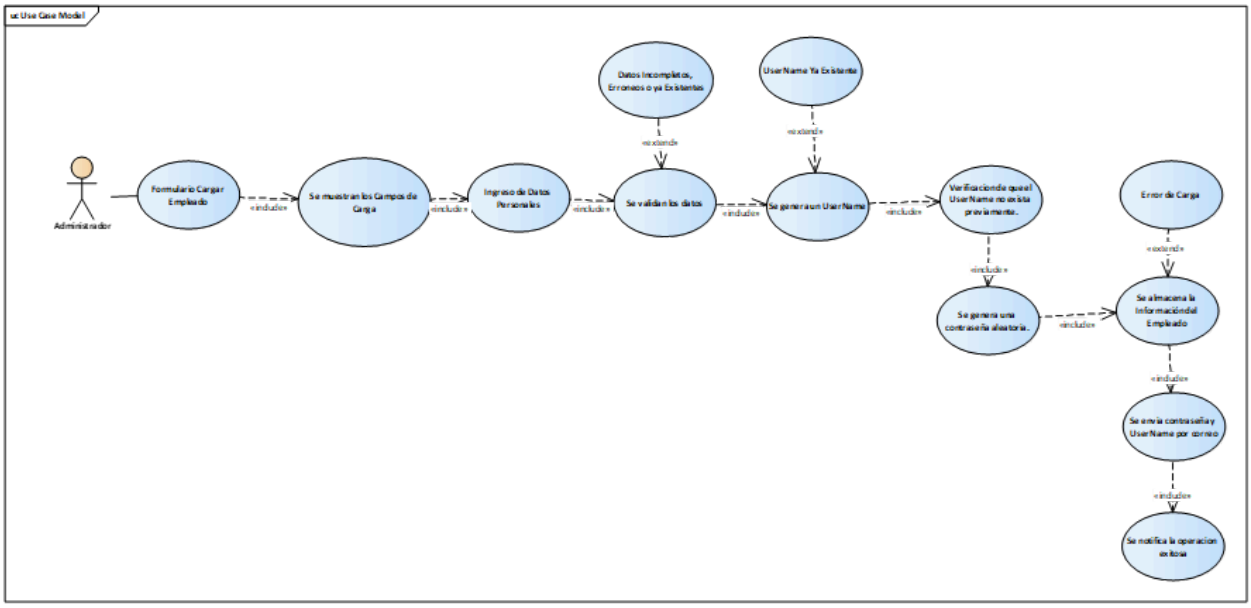

sd RegistrarVenta
ADMIN/AUTORIZADO
FORMULARIO VISTA
VERIFICACION
RegistrarVenta
BASE DE DATOS
IngresaDatos(IdProducto, Cantidad)
ValidarNulos()
MensageBox.ValidarNulos()
ExisteProducto(IdProducto)
ConsultaProducto(IdProducto)
ExisteProducto(True)
alt Producto Inexistente
ExisteProducto(False)
ExisteProducto(False)
MensageBox.ExisteProducto()
ExisteStock(IdProducto, Cantidad)
ConsultaStock(IdProducto, Cantidad)
ExisteStock(True)
alt Stock Insuficiente
ExisteStock(False)
ExisteStock(False)
MensageBox.ExisteStock()
ConfirmarVenta(IdProducto, Cantidad, IdVenta)
ActualizarStock(IdProducto, Cantidad)
ActualizarStock(True)
GuardarVenta(IdVenta)
VentaCargada(True)
VentaCargada(True)
MensageBox.VentaCargada()
alt Cancelar Venta
CancelarVenta()
Button.CancelarVenta()

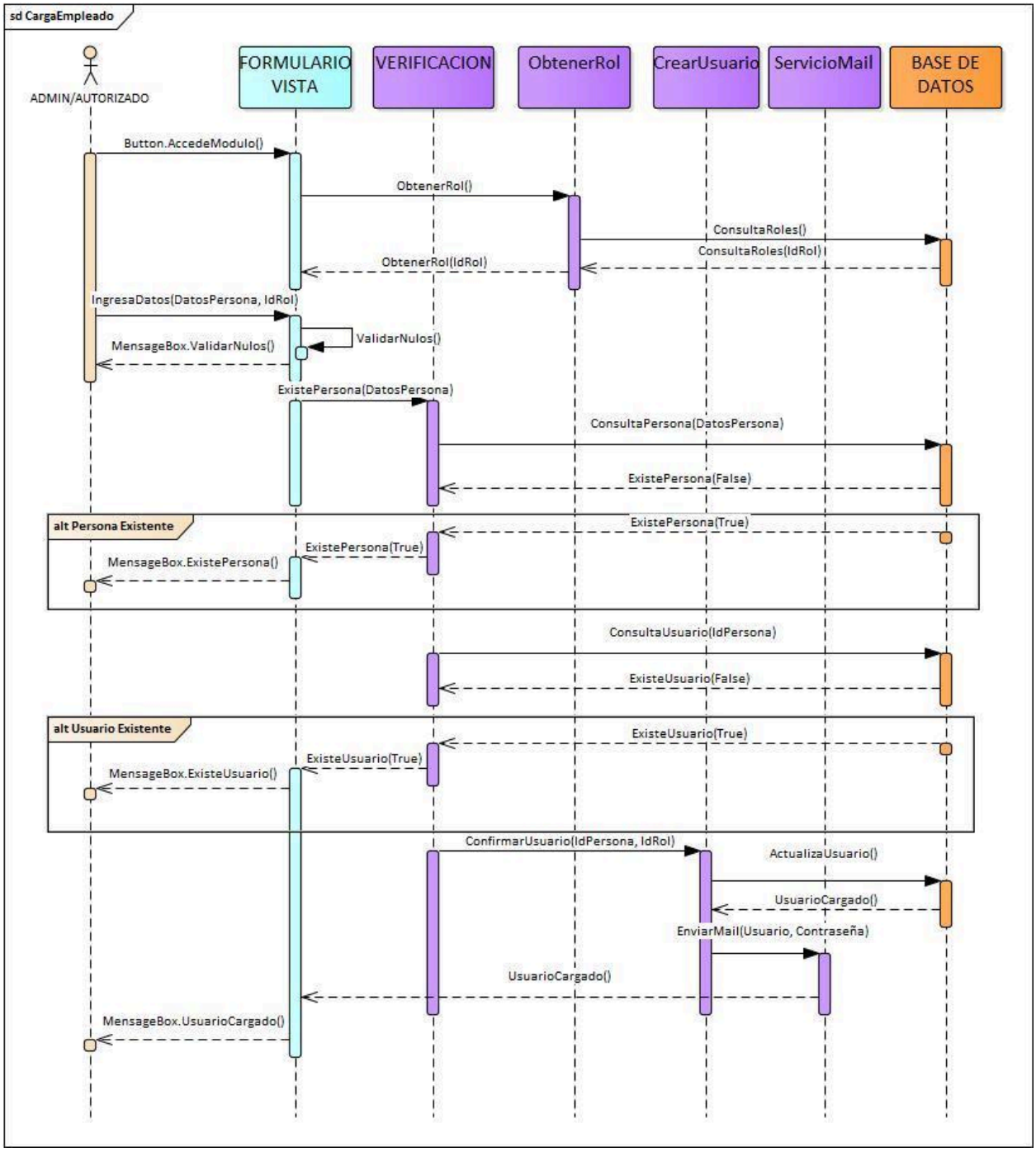


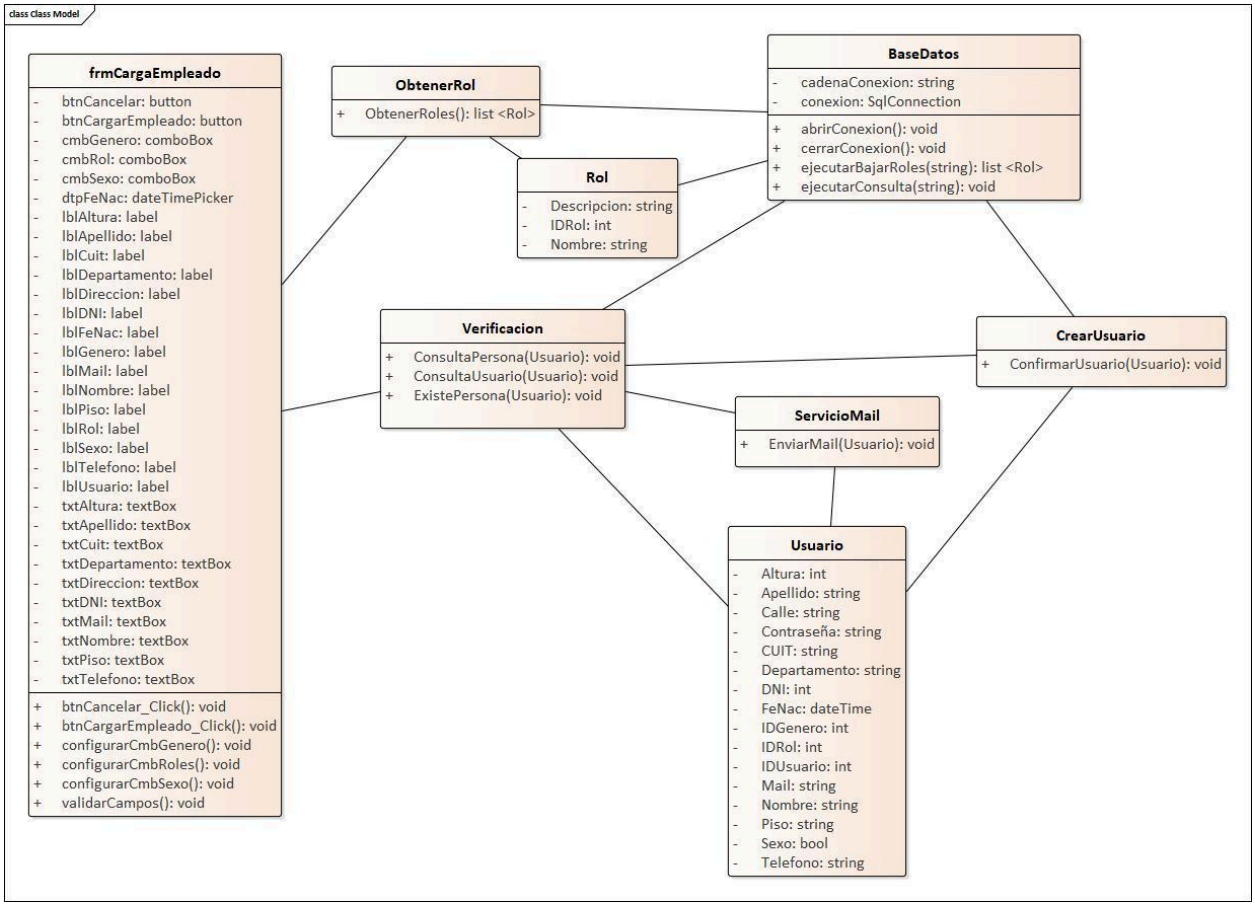
CU03 - Carga Empleado

ID y Nombre	CU03 Carga Empleado	
Descripción	Permite al Administrador registrar un nuevo empleado como usuario del sistema. Durante el proceso se ingresan los datos personales, el correo electrónico y el rol que desempeñará dentro de la organización. El sistema genera automáticamente un nombre de usuario y una contraseña temporal, enviándola por el respectivo email. Por último registra la información en la base de datos.	
Actores	Administrador	
Pre-Condición	El administrador debe haber iniciado sesión. Deben existir roles registrados en el sistema.	
Post-Condición	El usuario queda registrado. Se genera un nombre de usuario único. Se genera una contraseña temporal. El sistema almacena la información para futuros accesos.	
FLUJO	Flujo Normal	Flujo Alternativo

	1. El administrador accede al formulario Cargar Empleado.	
	2. El sistema muestra los campos de carga.	
	3. El administrador ingresa nombre, apellido, correo, rol y demás datos personales.	
	4. El sistema valida los datos ingresados.	4.1 Datos incompletos El sistema informa que existen campos obligatorios sin completar o haya algún error de tipeo. El administrador debe corregir la información.
	5. El sistema verifica si el empleado ya está registrado	5.1 El empleado ya esta registrado, el administrador debe ingresar un empleado sin registrar.
	6. El sistema genera automáticamente el nombre de usuario en base al nombre y apellido.	
	7. El sistema verifica que el nombre de usuario no exista.	6.1 Usuario existente El sistema detecta que el nombre de usuario ya existe. El sistema genera una variante del nombre de usuario. Continúa el proceso.
	8. El sistema genera una contraseña temporal aleatoria.	
	9. El sistema almacena la información del empleado en la base de datos.	8.1 Error al registrar El sistema informa que no fue posible completar el registro. No se almacenan cambios.
	10. El sistema envía la contraseña temporal y el nombre de usuario por el correo electrónico asignado.	
	11. El sistema informa que el registro fue exitoso.	


uc Use Case Model
Administrador
Formulario Cargar Empleado
«include»
Se muestran los Campos de Carga
«include»
Ingreso de Datos Personales
«include»
Se validan los datos
«include»
Se genera un UserName
«include»
Verificacion de que el UserName no exista previamente..
«include»
Se genera una contraseña aleatoria..
«include»
Datos Incompletos, Erroneos o ya Existentes
«extend»
UserName Ya Existente
«extend»
Error de Carga
«extend»
Se almacena la Informacion del Empleado
«include»
Se envia contraseña y UserName por correo
«include»
Se notifica la operacion exitosa


sd CargaEmpleado
ADMIN/AUTORIZADO
FORMULARIO VISTA
VERIFICACION
ObtenerRol
CrearUsuario
ServicioMail
BASE DE DATOS
Button.AccedeModulo()
ObtenerRol()
ConsultaRoles()
ConsultaRoles(IdRol)
ObtenerRol(IdRol)
IngresaDatos(DatosPersona, IdRol)
ValidarNulos()
MensageBox.ValidarNulos()
ExistePersona(DatosPersona)
ConsultaPersona(DatosPersona)
ExistePersona(False)
alt Persona Existente
ExistePersona(True)
ExistePersona(True)
MensageBox.ExistePersona()
ConsultaUsuario(IdPersona)
ExisteUsuario(False)
alt Usuario Existente
ExisteUsuario(True)
ExisteUsuario(True)
MensageBox.ExisteUsuario()
ConfirmarUsuario(IdPersona, IdRol)
ActualizaUsuario()
UsuarioCargado()
EnviarMail(Usuario, Contraseña)
UsuarioCargado()
MensageBox.UsuarioCargado()

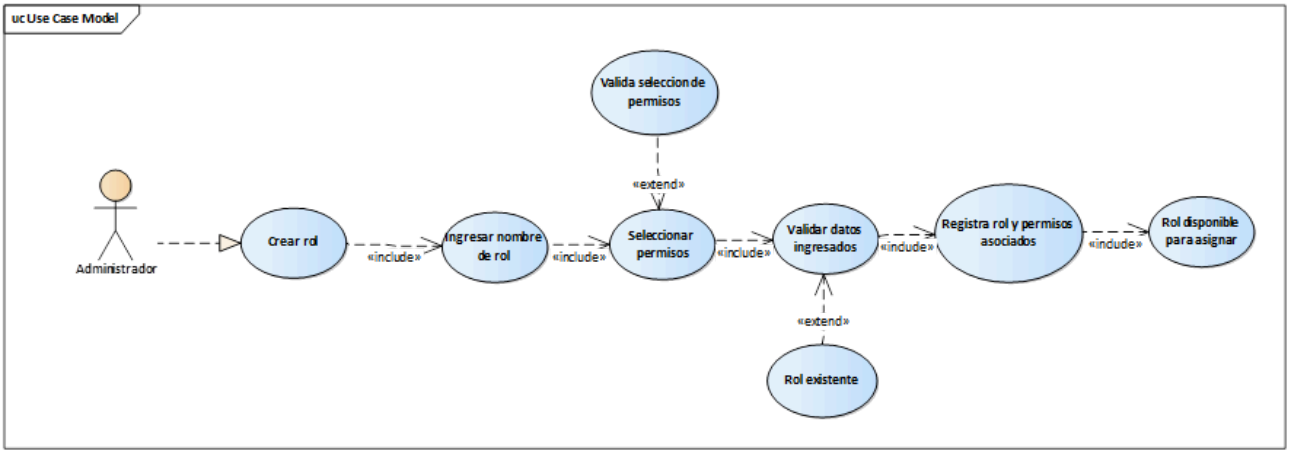


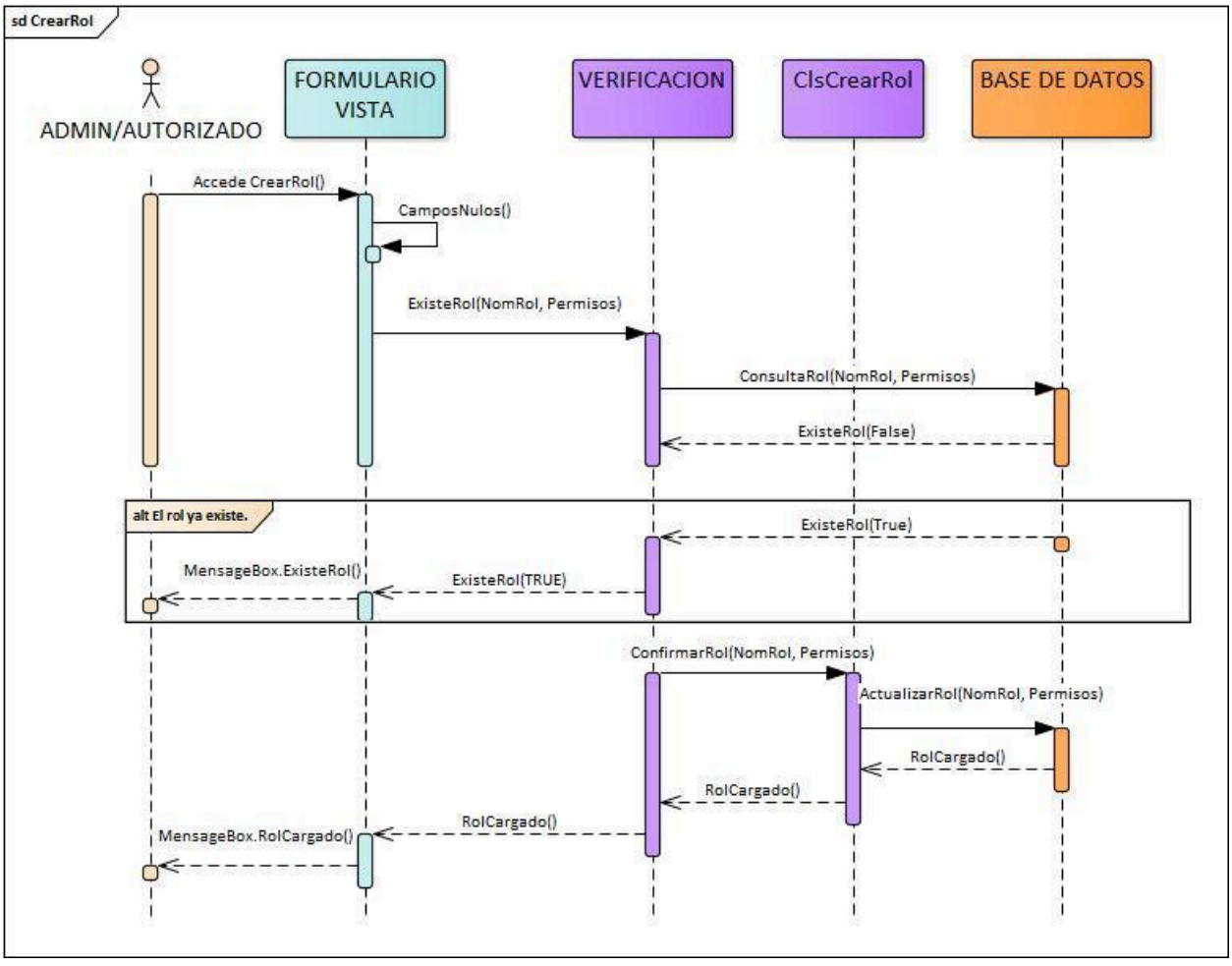
CU04 - Crear Rol

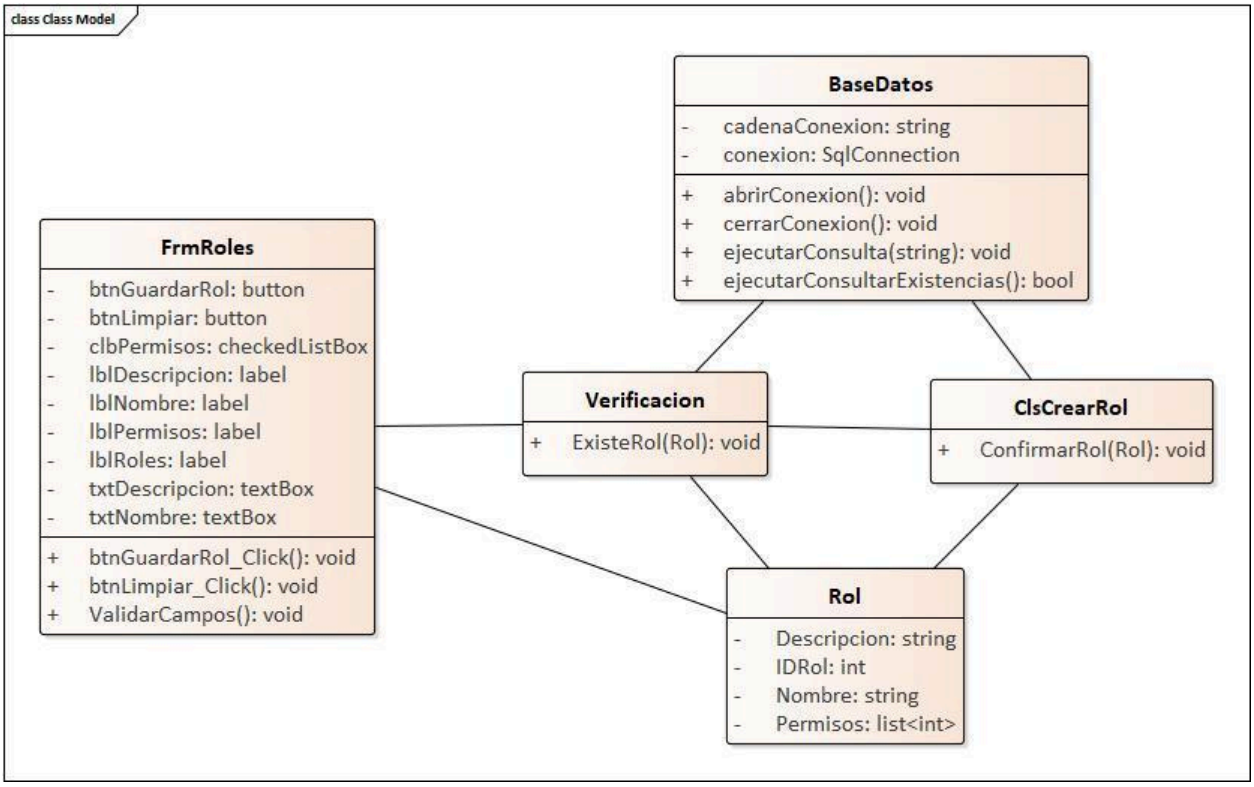
ID y Nombre	CU04 Crear Rol
Descripción	Permite al Administrador crear nuevos roles dentro del sistema y definir los permisos que tendrán asociados. Para ello, el administrador deberá ingresar un nombre identificatorio para el rol y seleccionar, mediante una lista de permisos disponibles, aquellas funcionalidades a las que los futuros usuarios asignados a dicho rol podrán acceder. Una vez completada la selección, el sistema validará que el nombre del rol no exista previamente y procederá a registrar el nuevo rol en la base de datos. Posteriormente, almacenará las asociaciones entre el rol creado y cada uno de los permisos seleccionados, generando automáticamente los registros correspondientes en la tabla de relación entre roles y permisos. Finalizado el proceso, el nuevo rol quedará disponible dentro del sistema y podrá ser asignado a los usuarios durante su creación o modificación. De esta manera, el administrador podrá personalizar los niveles de acceso según las necesidades

	operativas de la organización, sin necesidad de realizar cambios en el código fuente del sistema.	
Actores	Administrador	
Pre-Condición	El usuario debe haber iniciado sesión. El usuario debe poseer permisos de administración. Deben existir permisos registrados en el sistema.	
Post-Condición	El rol queda registrado en la base de datos. Los permisos seleccionados quedan asociados al rol. El rol se encuentra disponible para asignar a nuevos usuarios.	
FLUJO	**Flujo Normal**	**Flujo Alternativo**
	1. El administrador accede al formulario Crear Rol.	
	2. El administrador ingresa el nombre del rol.	
	3. El sistema muestra el listado de permisos disponibles.	
	4. El administrador selecciona uno o más permisos.	4.1 Ningún permiso seleccionado El sistema informa que debe seleccionarse al menos un permiso. El administrador continúa la carga.
	5. El sistema valida los datos ingresados.	
	6. El sistema verifica que el nombre del rol no exista.	6.1 Nombre de rol existente El sistema informa que ya existe un rol con ese nombre. El administrador debe ingresar otro nombre.
	7. El sistema registra el nuevo rol.	
	8. El sistema registra las asociaciones entre el rol y los permisos seleccionados.	
	9. El sistema confirma la creación del rol.	

	10. El nuevo rol queda disponible para ser asignado a los usuarios.	



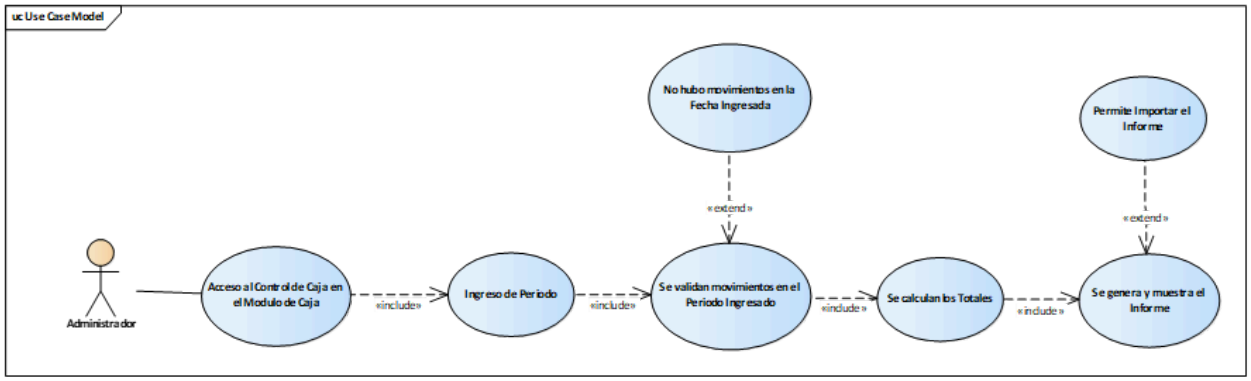


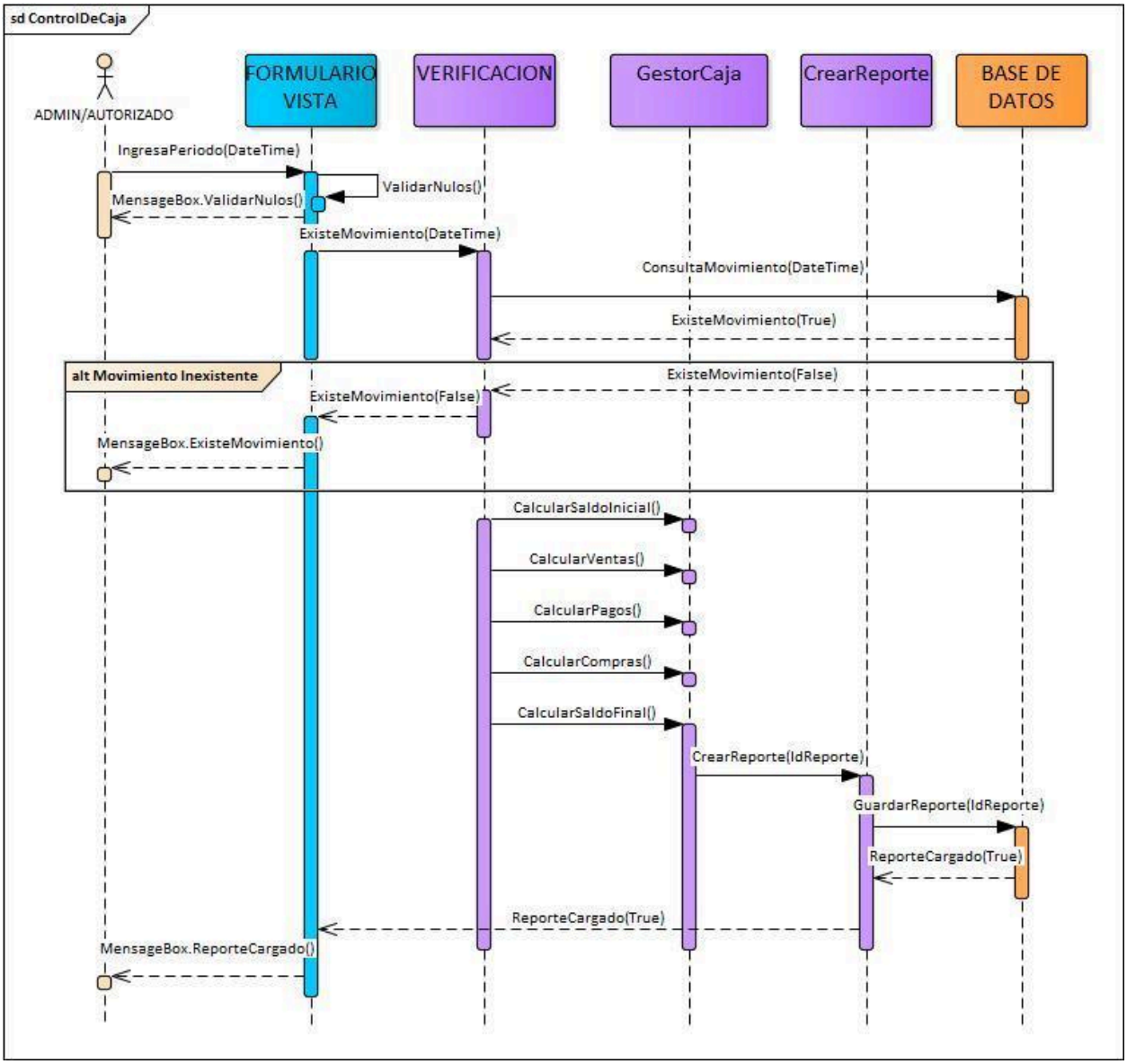


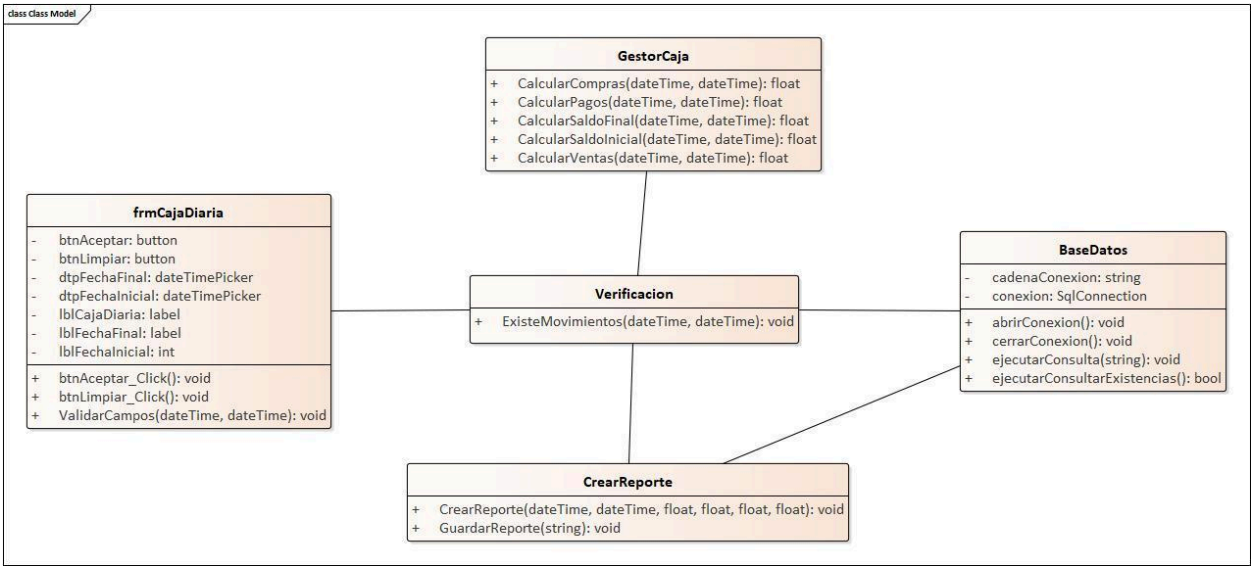
CU05 - Control de Caja Diaria

ID y Nombre	CU05 Control de caja diaria	
Descripción	Permite Dueño o empleado autorizado realizar el cierre de caja al finalizar la jornada, verificando las ventas registradas, los gastos, y calculando el saldo final	
Actores	Administrador	
Pre-Condición	Administrador debe estar logueado, deben existir movimientos de caja (compras, ventas) en la jornada	
Post-Condición	El sistema genera y guarda un reporte de caja diaria con saldo inicial, movimientos y saldo final. El reporte queda disponible para consultar, exportar o imprimir	
FLUJO	Flujo Normal	Flujo Alternativo
	1. El administrador accede al módulo de caja.	

	2. Selecciona la opción “Control de caja”.	
	3. El sistema solicita el periodo (por defecto: día actual)	
	4. El usuario confirma el periodo.	
	5. El sistema valida que existan movimientos registrados en el periodo.	5.1. No hay movimientos: el sistema informa que no se puede generar el control de caja.
	6. El sistema calcula: Saldo Inicial del día. Total de ventas registrada. Total de compras registradas. Total de gastos. Saldo final.	
	7. El sistema genera el reporte y muestra en pantalla	



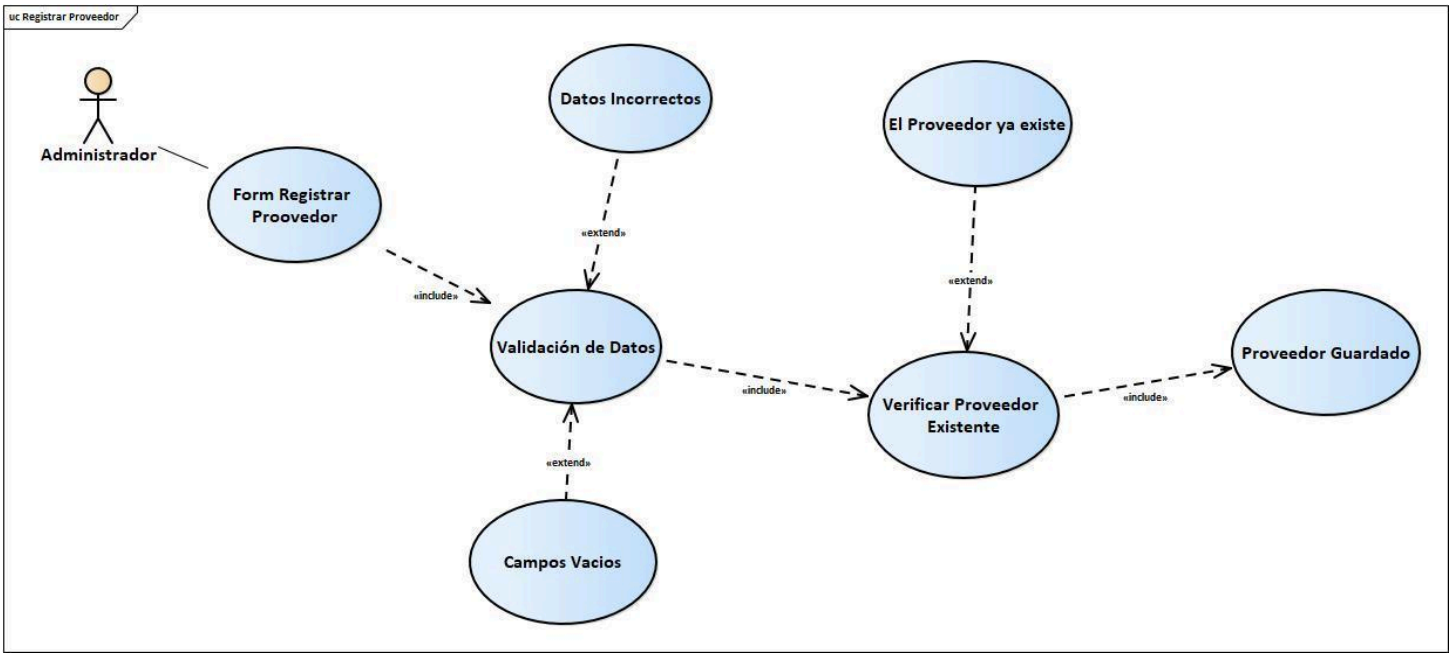

sd ControlDeCaja
ADMIN/AUTORIZADO
FORMULARIO VISTA
VERIFICACION
GestorCaja
CrearReporte
BASE DE DATOS
IngresaPeriodo(DateTime)
ValidarNulos()
MensageBox.ValidarNulos()
ExisteMovimiento(DateTime)
ConsultaMovimiento(DateTime)
ExisteMovimiento(True)
alt Movimiento Inexistente
ExisteMovimiento(False)
ExisteMovimiento(False)
MensageBox.ExisteMovimiento()
CalcularSaldoInicial()
CalcularVentas()
CalcularPagos()
CalcularCompras()
CalcularSaldoFinal()
CrearReporte(IdReporte)
GuardarReporte(IdReporte)
ReporteCargado(True)
ReporteCargado(True)
MensageBox.ReporteCargado()

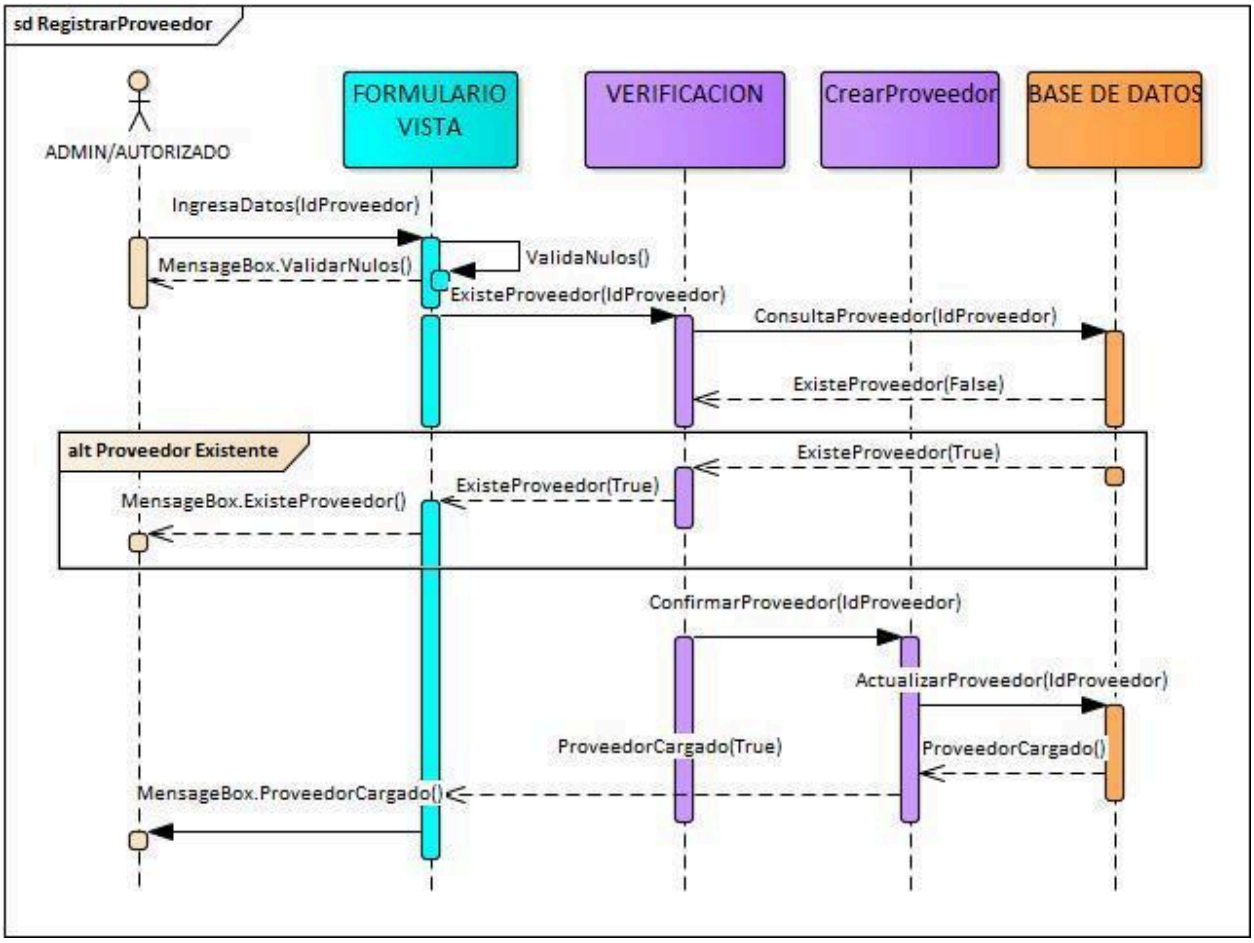


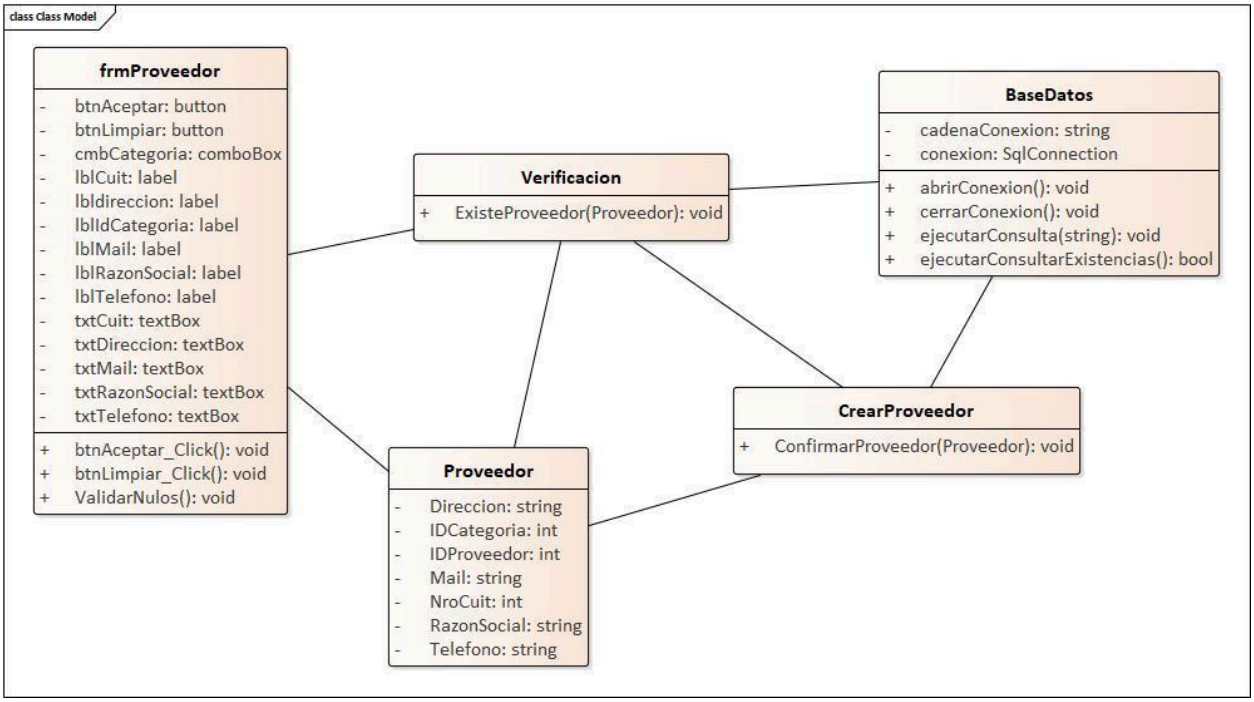
CU06 - Registrar Proveedor

ID y Nombre	CU06 Registrar Proveedor	
Descripción	Permite al dueño o empleado autorizado registrar a un proveedor nuevo al sistema.	
Actores	Administrador.	
Pre-Condición	El administrador debe estar logueado con los permisos correspondientes.	
Post-Condición	El proveedor queda ingresado al sistema.	
FLUJO	Flujo Normal	Flujo Alternativo
	1.El Administrador accede al módulo de proveedores.	
	2. El Administrador selecciona la opción registrar proveedor.	
	3. El Administrador ingresa los datos obligatorios del proveedor.	
	4. El sistema valida los datos ingresados.	4.1.Campos Vacíos. El sistema detecta campos incompletos e informa al usuario en pantalla solicitando completar los campos.

		4.2 Campos con datos incorrectos. El sistema detecta campos llenados con datos incorrectos e informa al usuario en pantalla que corrija los datos ingresados.
	5. El sistema Verifica que el proveedor no se encuentra registrado previamente.	5.1. Proveedor ya Registrado. El sistema revisa que el proveedor ya existe y solicita al administrador en pantalla que debe ingresar un proveedor sin registrar.
	6. El sistema guarda el proveedor en la base de datos	
	7. El sistema informa en pantalla que el Registro fue exitoso.	



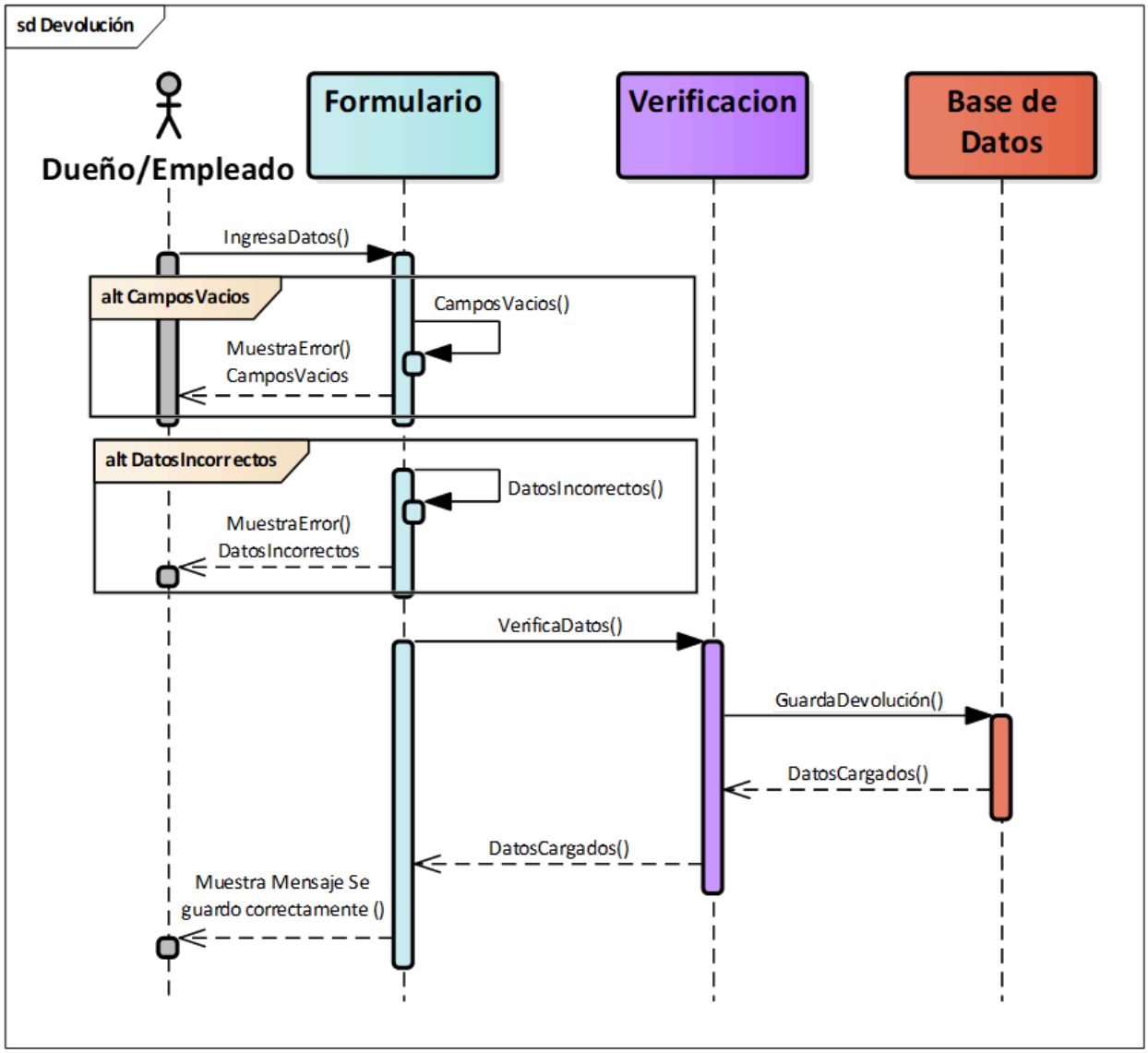

sd RegistrarProveedor
ADMIN/AUTORIZADO
FORMULARIO VISTA
VERIFICACION
CrearProveedor
BASE DE DATOS
IngresaDatos(IdProveedor)
MensageBox.ValidarNulos()
ValidaNulos()
ExisteProveedor(IdProveedor)
ConsultaProveedor(IdProveedor)
ExisteProveedor(False)
alt Proveedor Existente
ExisteProveedor(True)
ExisteProveedor(True)
MensageBox.ExisteProveedor()
ConfirmarProveedor(IdProveedor)
ActualizarProveedor(IdProveedor)
ProveedorCargado(True)
ProveedorCargado()
MensageBox.ProveedorCargado()

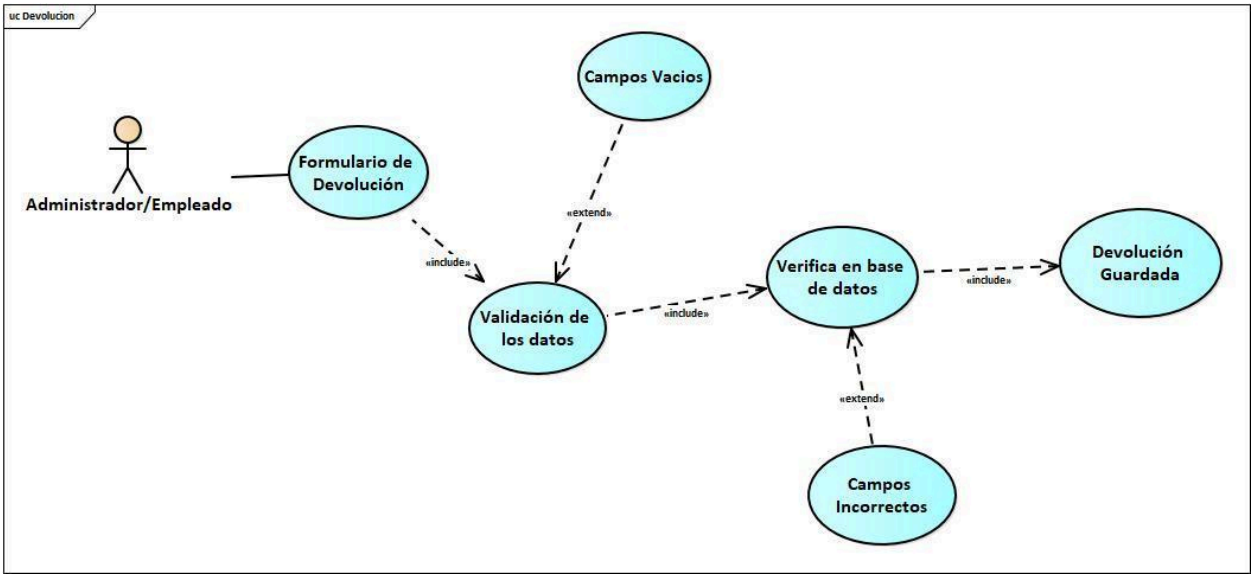


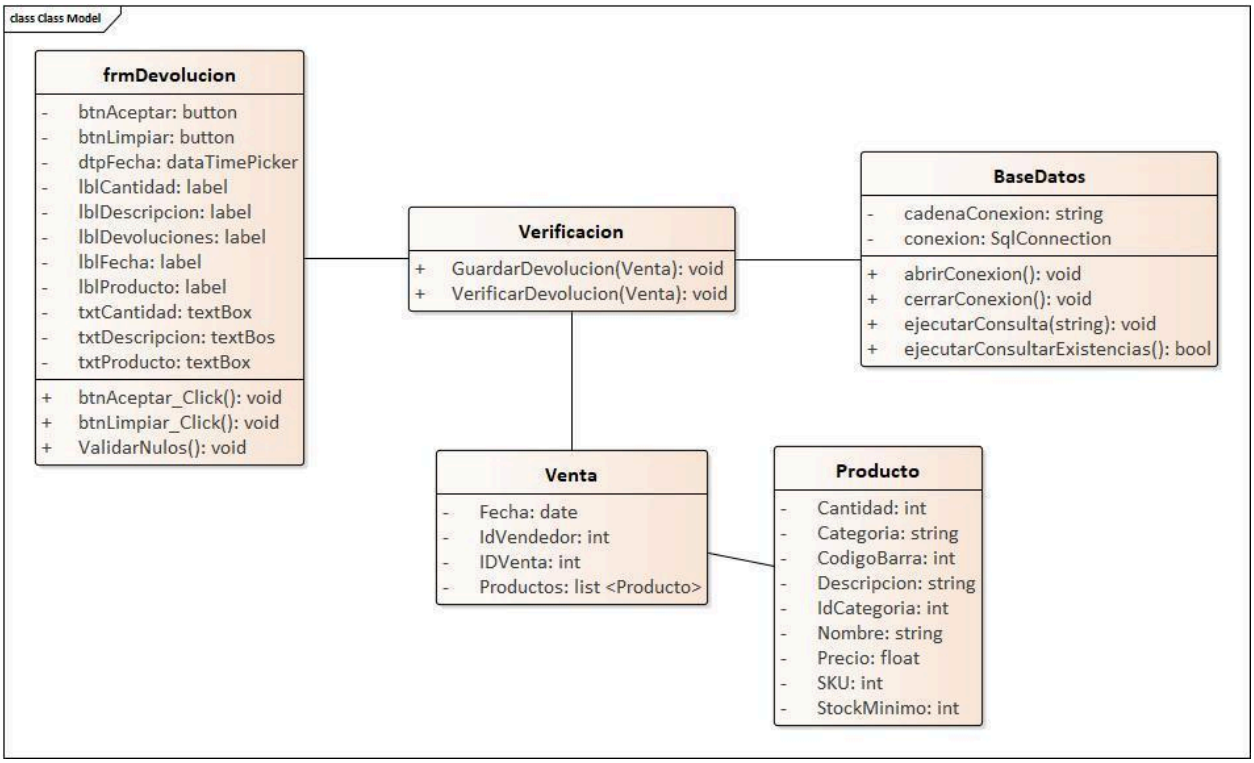
CU07 - Devolución

ID y Nombre	CU07 Devolución	
Descripción	Permite al dueño o empleado autorizado registrar las devoluciones	
Actores	Administrador.	
Pre-Condición	El administrador debe estar logueado con los permisos correspondientes.	
Post-Condición	El proveedor queda ingresado al sistema.	
FLUJO	Flujo Normal	Flujo Alternativo
	1.El Administrador accede al módulo de Devolución	
	2. El Administrador/Empleado Selecciona la opción de ‘’Nueva Devolución’’	
	3. El Administrador ingresa los datos obligatorios	
	4. El sistema valida los campos ingresados y los	4.1.Campos Vacíos.

	verifica en la base de datos.	El sistema detecta campos incompletos e informa al usuario en pantalla solicitando completar los campos. 4.2 Campos con datos incorrectos. El sistema detecta campos llenados con datos incorrectos e informa al usuario en pantalla que corrija los datos ingresados.
	5. El sistema guarda la devolución en la base de datos	
	6. El sistema informa en pantalla que la carga de la devolución fue exitosa.	

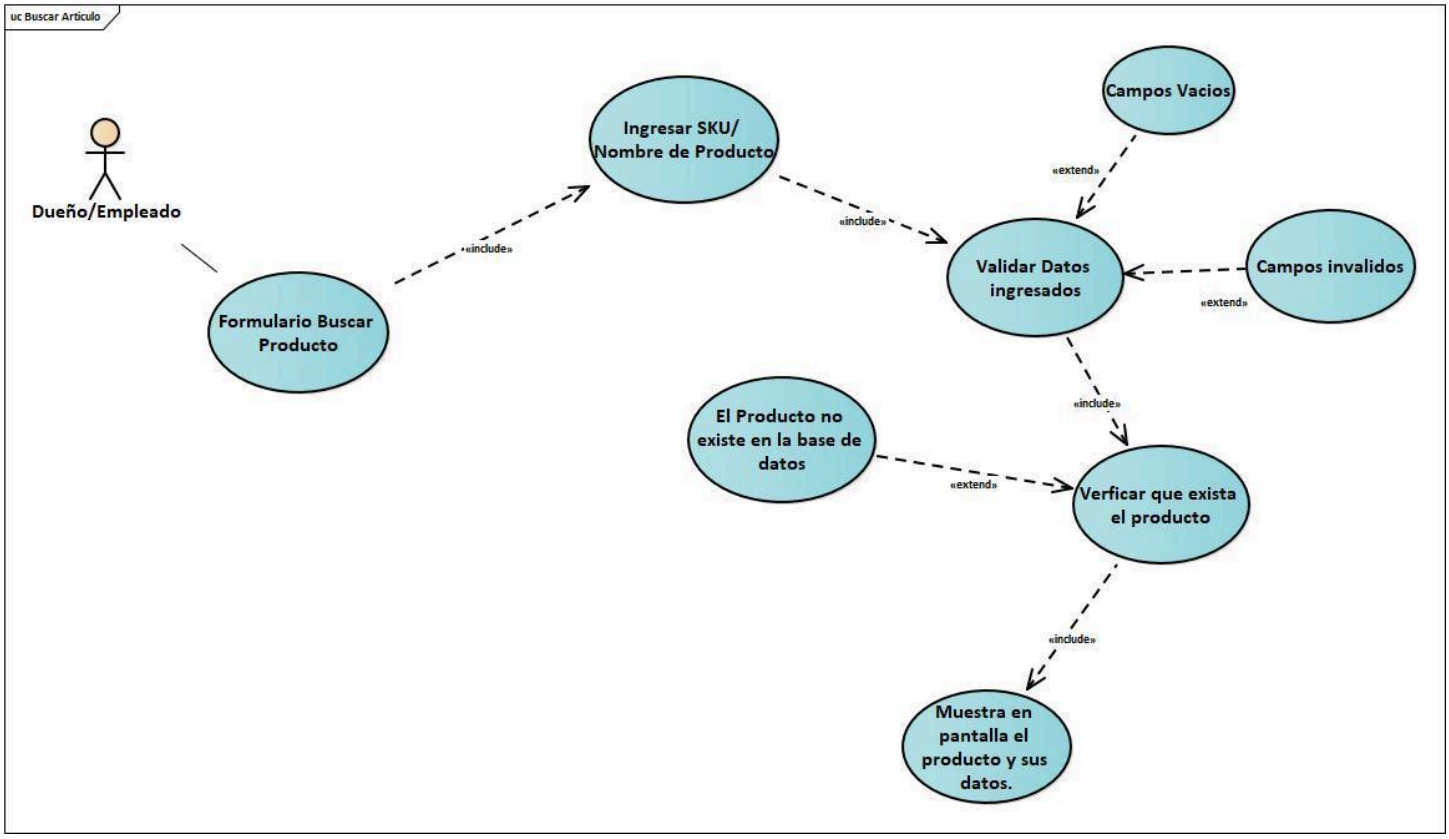

sd Devolución
Dueño/Empleado
Formulario
Verificacion
Base de Datos
IngresaDatos()
alt CamposVacios
CamposVacios()
MuestraError()
CamposVacios
alt DatosIncorrectos
DatosIncorrectos()
MuestraError()
DatosIncorrectos
VerificaDatos()
GuardaDevolución()
DatosCargados()
DatosCargados()
Muestra Mensaje Se guardo correctamente ()

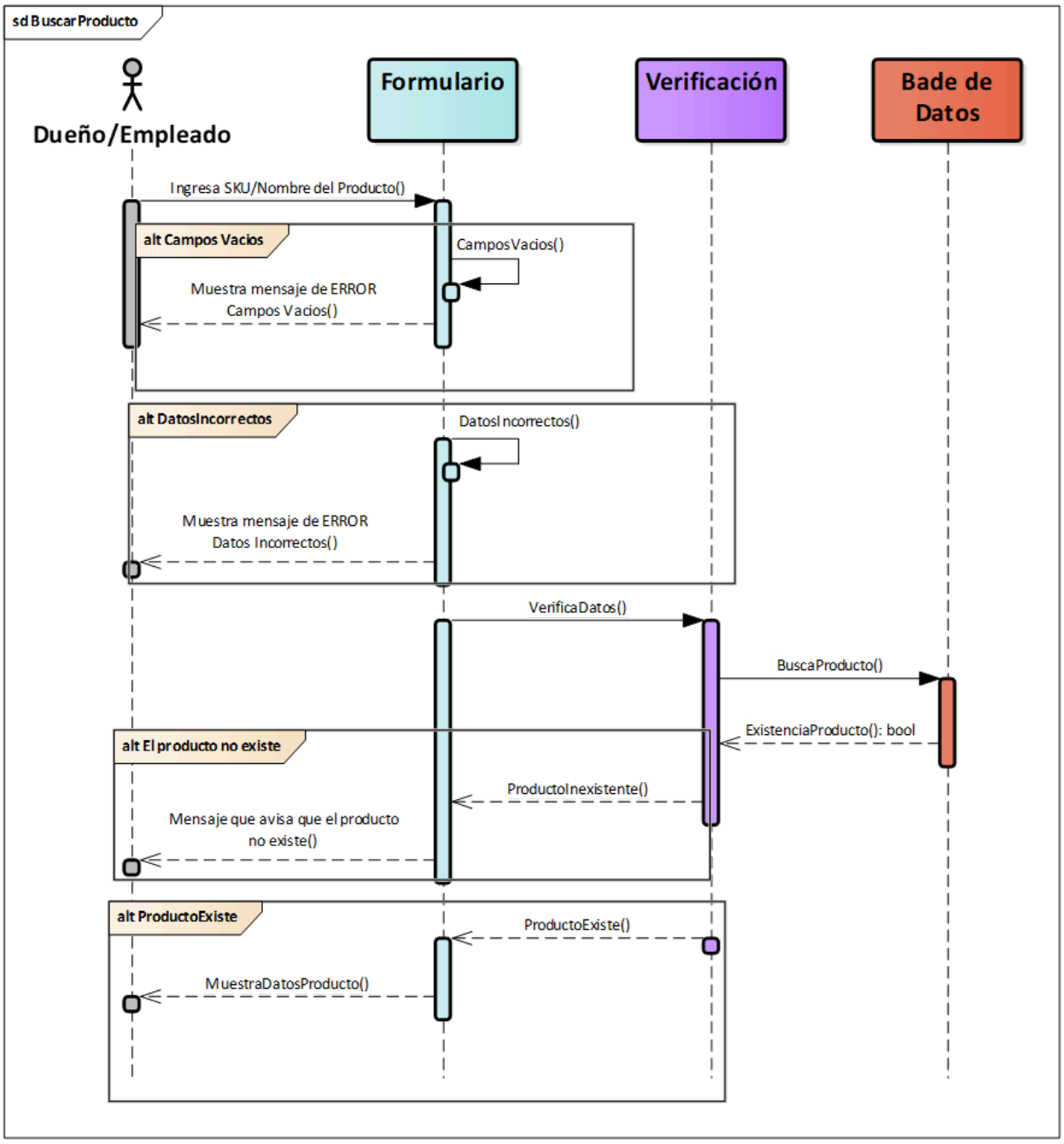


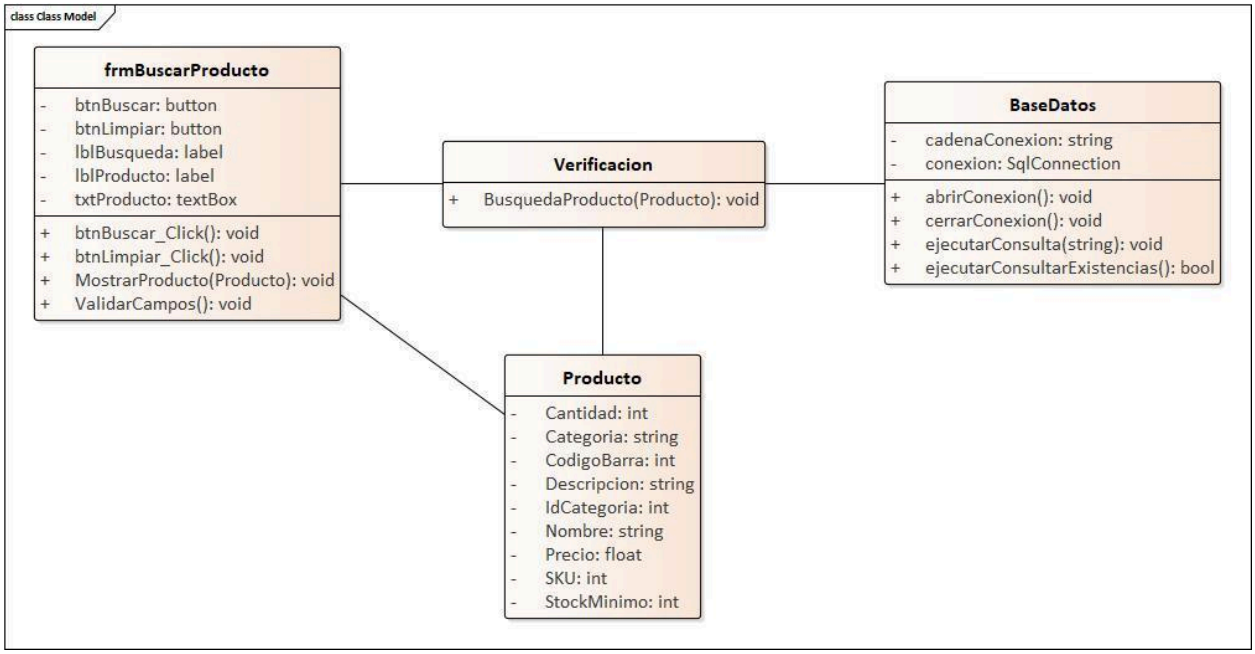


CU08 - Buscar Artículo

ID y Nombre	**CU08 Buscar Artículo**	
Descripción	Permite al dueño o empleado autorizado buscar los articulos por nombre o SKU.	
Actores	Dueño, empleado autorizado.	
Pre-Condición	El dueño o empleado debe estar logueado con los permisos correspondientes.	
Post-Condición	El sistema muestra en pantalla los datos del producto.	
FLUJO	**Flujo Normal**	**Flujo Alternativo**
	1.El Administrador accede al módulo de Búsqueda de Artículo.	
	2. El Administrador/Empleado ingresa el nombre o SKU.	
	3. El sistema valida los datos ingresados.	3.1.Campos Vacíos. El sistema detecta que el campo está vacío. 3.2 Campos con datos incorrectos. El sistema detecta el campo llenado con datos incorrectos e informa al usuario en pantalla que corrija los datos ingresados.
	4. El sistema verifica que el producto exista en la base de datos.	4.1. No está registrado. El sistema avisa que el artículo no se encuentra en la base de datos.
	5. El sistema muestra los datos del artículo seleccionado..	


uc Buscar Artículo
Dueño/Empleado
Formulario Buscar Producto
«include»
Ingresar SKU/ Nombre de Producto
«include»
Validar Datos ingresados
Campos Vacios
«extend»
Campos invalidos
«extend»
«include»
El Producto no existe en la base de datos
«extend»
Verficar que exista el producto
«include»
Muestra en pantalla el producto y sus datos.

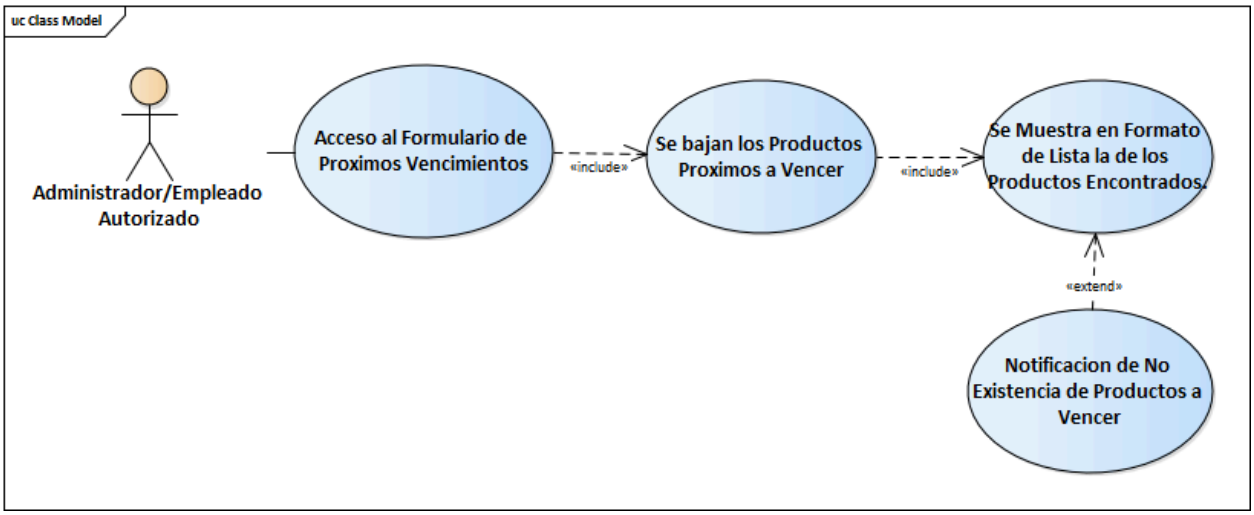

sd Buscar Producto
Dueño/Empleado
Formulario
Verificación
Bade de Datos
Ingresa SKU/Nombre del Producto()
alt Campos Vacios
CamposVacios()
Muestra mensaje de ERROR
Campos Vacios()
alt DatosIncorrectos
DatosIncorrectos()
Muestra mensaje de ERROR
Datos Incorrectos()
VerificaDatos()
BuscaProducto()
ExistenciaProducto(): bool
alt El producto no existe
ProductoInexistente()
Mensaje que avisa que el producto
no existe()
alt ProductoExiste
ProductoExiste()
MuestraDatosProducto()

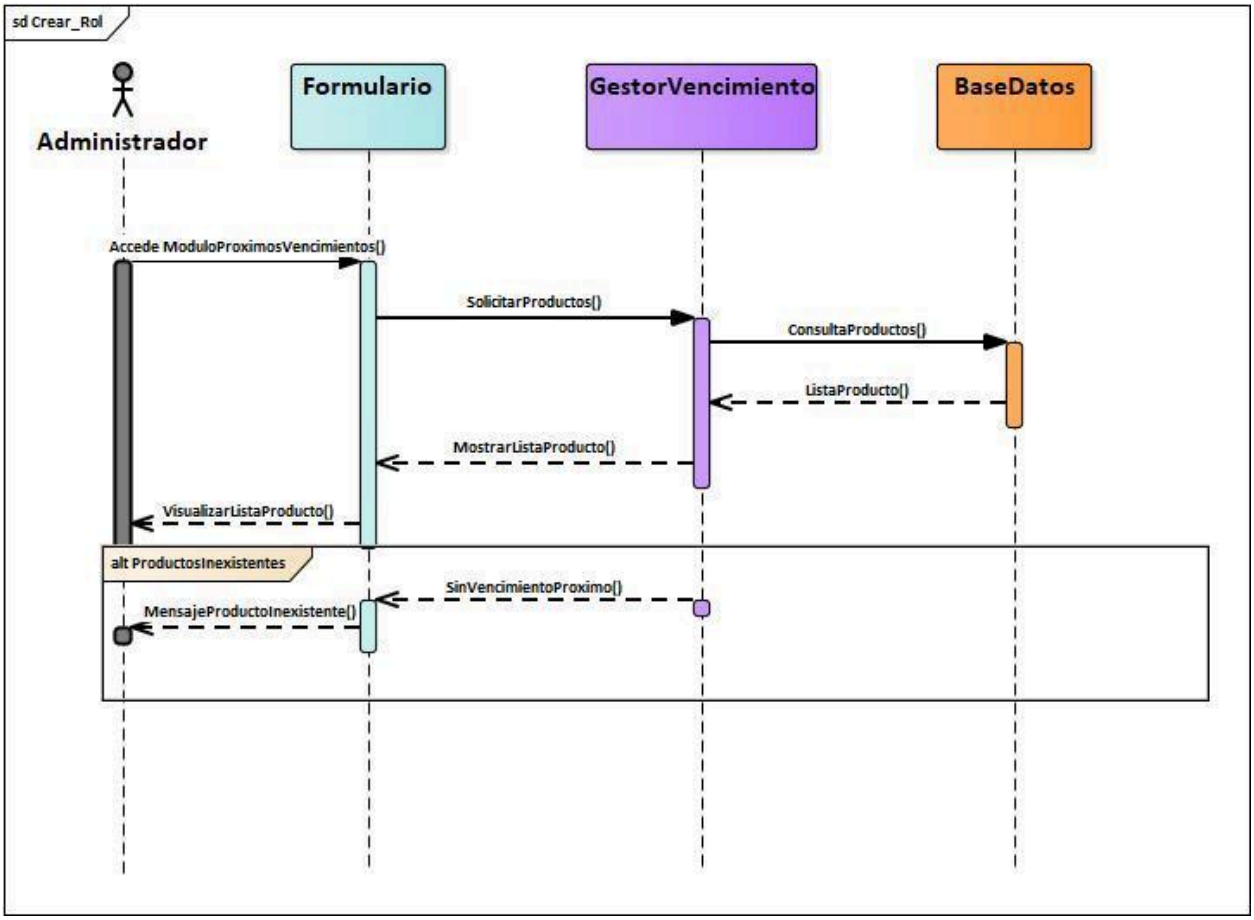


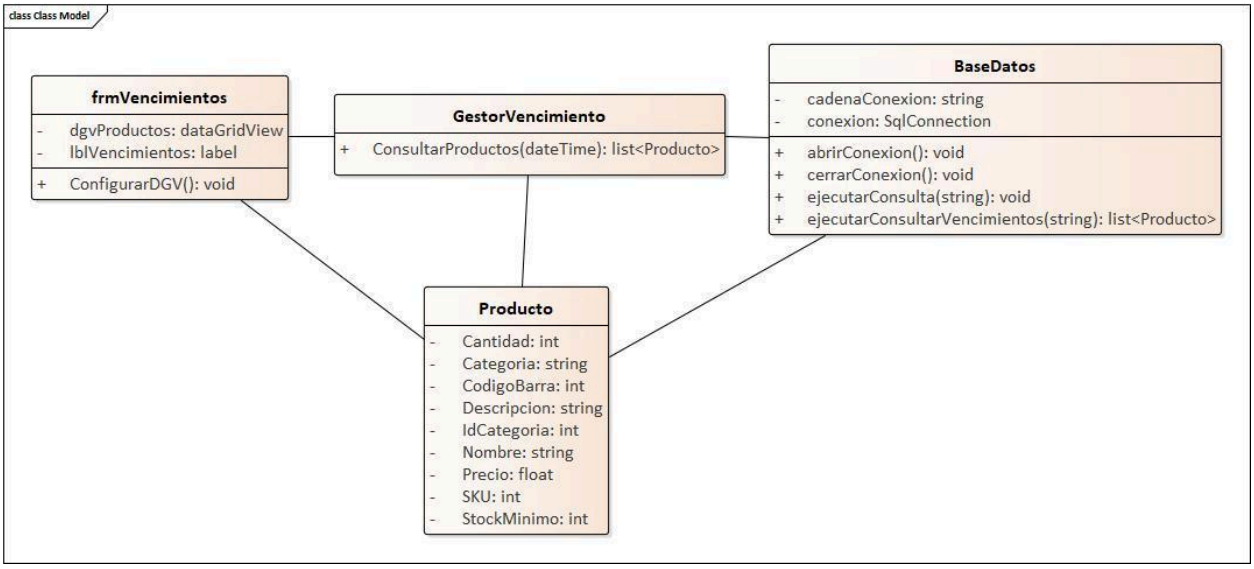
CU09 - Productos Próximos a Vencer

ID y Nombre	**CU09 Productos próximos a vencer**	
Descripción	Permite al dueño o empleado autorizado poder ver los productos que estén por vencer de forma próxima.	
Actores	Dueño, empleado autorizado	
Pre-Condición	El dueño o empleado debe estar logueado con los permisos correspondientes.	
Post-Condición	El sistema muestra en pantalla la lista de productos próximos a vencer.	
FLUJO	**Flujo Normal**	**Flujo Alternativo**
	1.El Administrador accede al módulo de Próximos Vencimientos.	
	2. El sistema baja de la base de datos los productos próximos a vencer.	
	3. El sistema muestra todos los productos detectados junto a sus datos.	3.1. No hay Productos:

		En caso de no existir productos próximos a vencer notificará al Usuario con un mensaje.



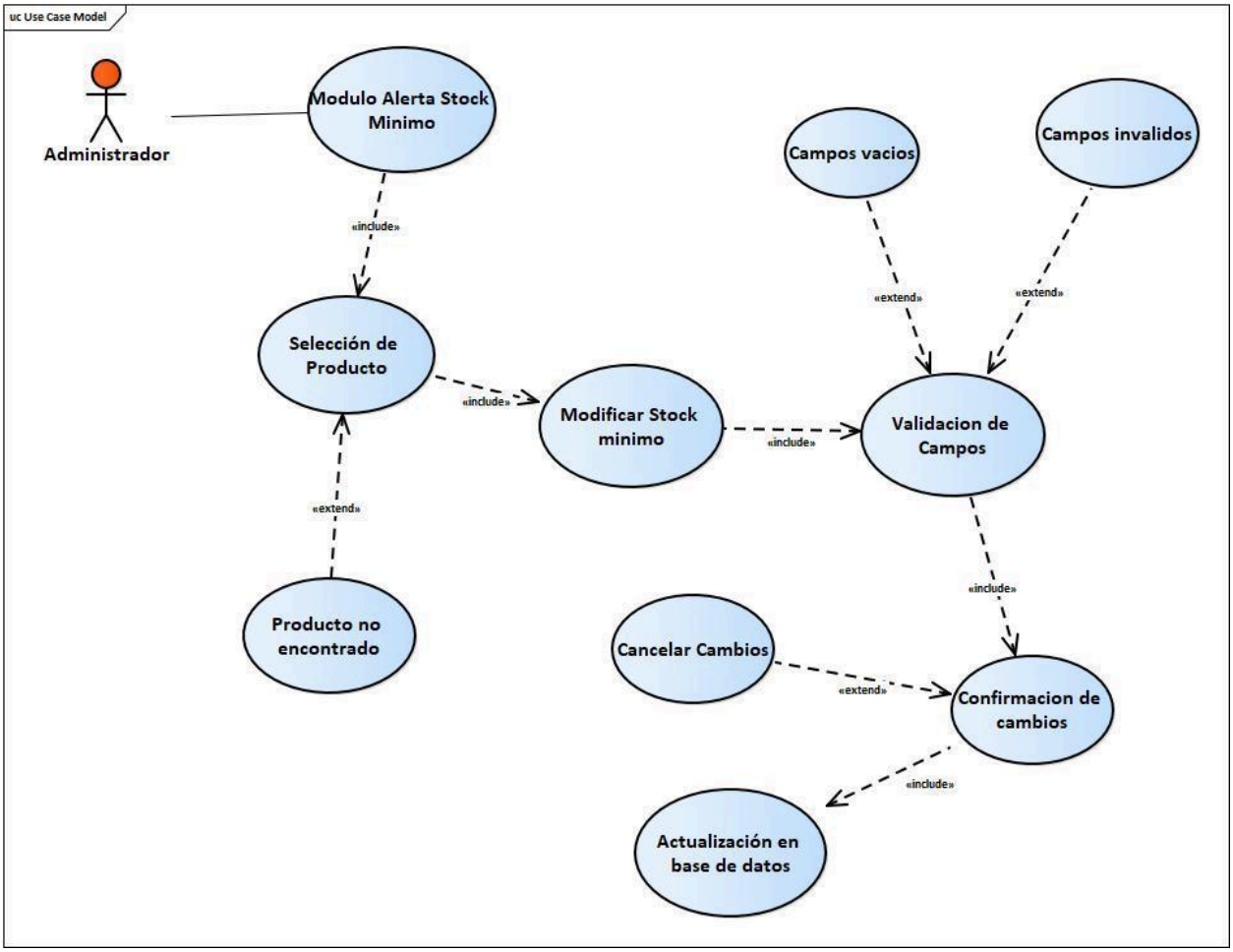


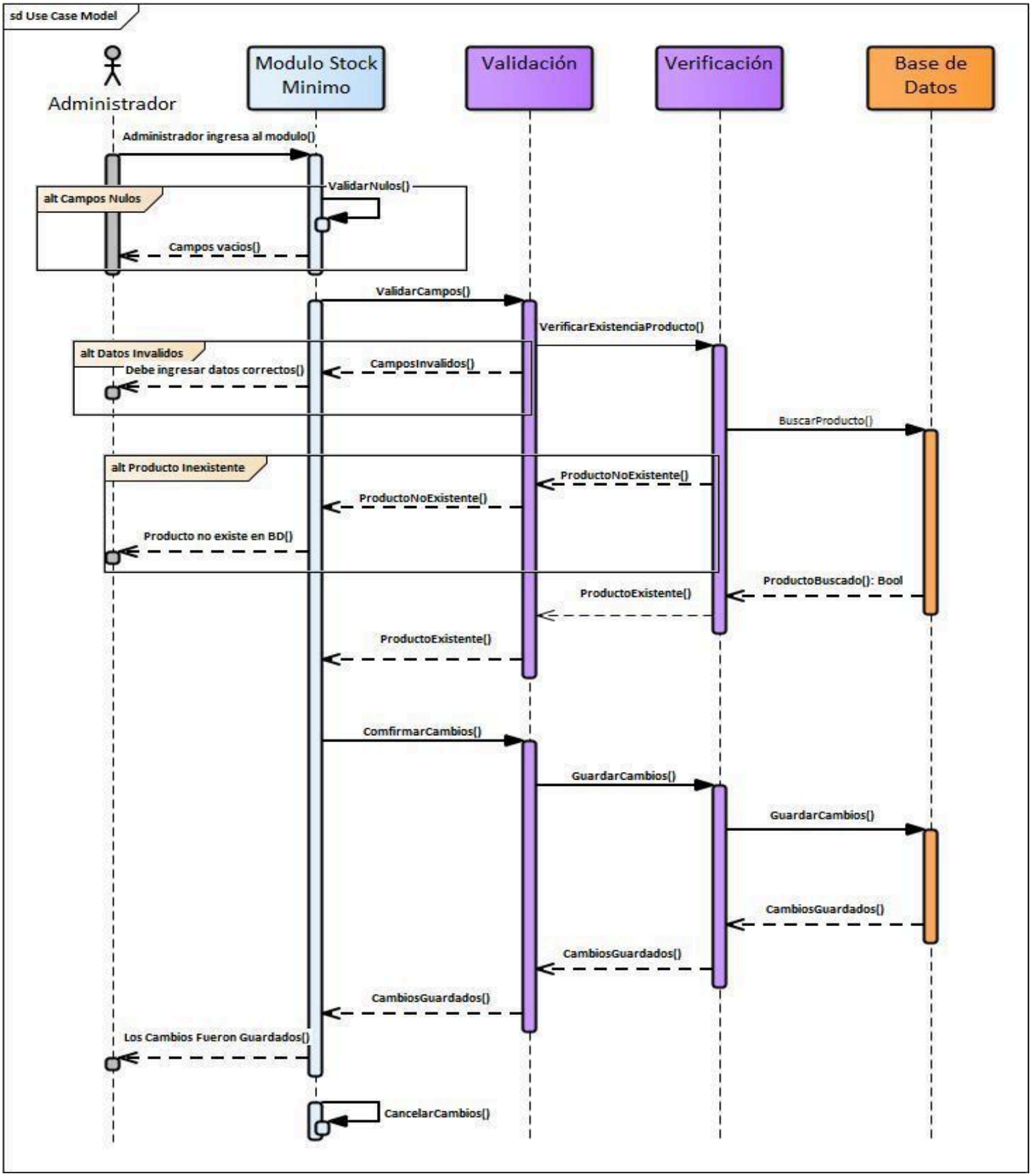


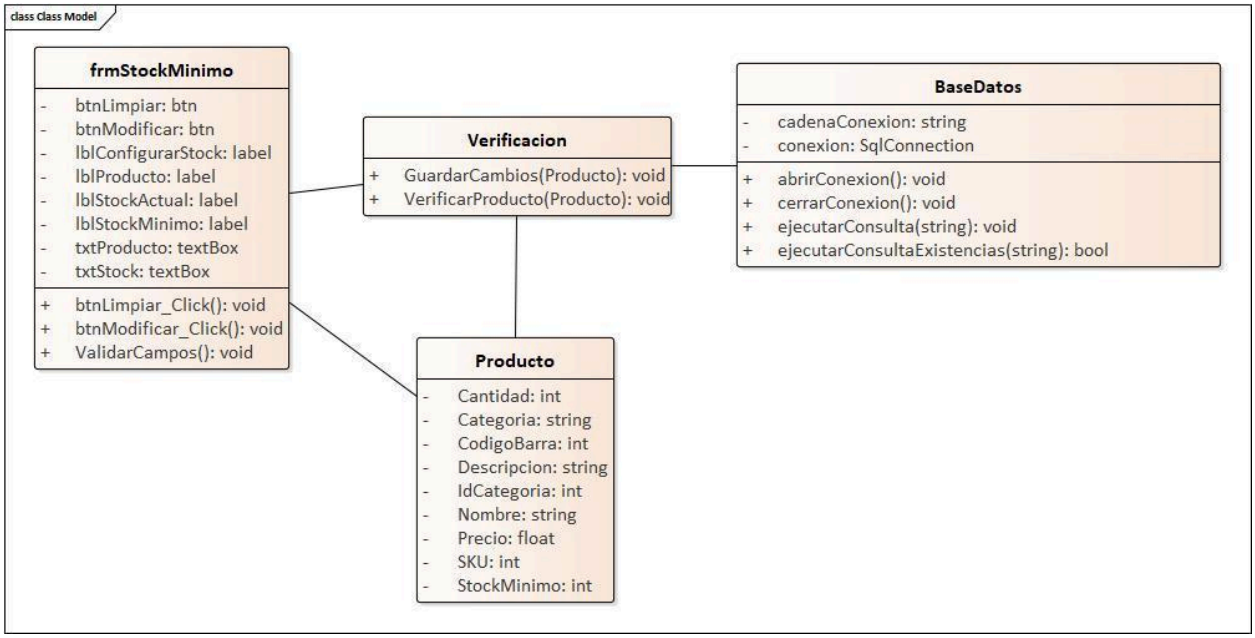
CU10 - Modificar Alerta Stock Mínimo

ID y Nombre	**CU10 Modificar alerta stock mínimo**	
Descripción	Permite al dueño o empleado autorizado a modificar el stock mínimo o máximo de cada producto ajustando la configuración de alerta.	
Actores	Administrador.	
Pre-Condición	El administrador debe estar logueado con los permisos correspondientes.	
Post-Condición	Queda actualizada la alerta del stock mínimo y se dispara según los parámetros configurados..	
FLUJO	**Flujo Normal**	**Flujo Alternativo**
	1.El Administrador accede al módulo de Generar alerta de stock.	
	2. El Administrador busca el producto a modificar.	2.1 El sistema no encuentra el producto, informa al administrador que el producto no se encuentra en la base de datos.

	3. El sistema muestra los datos del producto, incluyendo el stock mínimo.	
	4. El administrador/empleado modifica el stock mínimo.	
	5. El sistema valida los datos ingresados.	5.1.Campos Vacíos. El sistema detecta que el campo está vacío. 5.2 Campos con datos incorrectos. El sistema detecta el campo llenado con datos incorrectos e informa al usuario en pantalla que corrija los datos ingresados.
	6. El administrador confirma la modificación.	6.1 Cancelar los cambios del stock minimo.
	7. El sistema actualiza el valor en la base de datos.	
	8. El sistema muestra que la modificación fue exitosa.	


uc Use Case Model
Administrador
Modulo Alerta Stock Minimo
«include»
Selección de Producto
«include»
Modificar Stock minimo
«include»
Validacion de Campos
Campos vacios
Campos invalidos
«extend»
«extend»
«extend»
Producto no encontrado
«include»
Cancelar Cambios
«extend»
Confirmacion de cambios
«include»
Actualización en base de datos


sd Use Case Model
Administrador
Modulo Stock Minimo
Validación
Verificación
Base de Datos
Administrador ingresa al modulo()
alt Campos Nulos
ValidarNulos()
Campos vacios()
ValidarCampos()
VerificarExistenciaProducto()
alt Datos Invalidos
CamposInvalidos()
Debe ingresar datos correctos()
BuscarProducto()
alt Producto Inexistente
ProductoNoExistente()
ProductoNoExistente()
Producto no existe en BD()
ProductoBuscado(): Bool
ProductoExistente()
ProductoExistente()
ComfirmarCambios()
GuardarCambios()
GuardarCambios()
CambiosGuardados()
CambiosGuardados()
CambiosGuardados()
Los Cambios Fueron Guardados()
CancelarCambios()


class Class Model
frmStockMinimo
- btnLimpiar: btn
- btnModificar: btn
- lblConfigurarStock: label
- lblProducto: label
- lblStockActual: label
- lblStockMinimo: label
- txtProducto: textBox
- txtStock: textBox
+ btnLimpiar_Click(): void
+ btnModificar_Click(): void
+ ValidarCampos(): void
Verificacion
+ GuardarCambios(Producto): void
+ VerificarProducto(Producto): void
BaseDatos
- cadenaConexion: string
- conexion: SqlConnection
+ abrirConexion(): void
+ cerrarConexion(): void
+ ejecutarConsulta(string): void
+ ejecutarConsultaExistencias(string): bool
Producto
- Cantidad: int
- Categoria: string
- CodigoBarra: int
- Descripcion: string
- IdCategoria: int
- Nombre: string
- Precio: float
- SKU: int
- StockMinimo: int

CU11 - Modificar / dar de baja producto

ID y Nombre	**CU11 Modificar / dar de baja producto**	
Descripción	Permite al dueño o empleado autorizado modificar los datos de un producto ya registrado (nombre, descripción, código de barras, marca, rubro, IVA, unidad de medida y precio de venta) o darlo de baja lógica (discontinuarlo). Un producto discontinuado deja de mostrarse para la venta, pero no se elimina de la base de datos, por lo que se conserva su historial de ventas, compras y movimientos de stock.	
Actores	Administrador - Empleado autorizado (permiso 1.1.2 Baja/Modificación de Artículos).	
Pre-Condición	El usuario debe haber iniciado sesión y poseer el permiso 1.1.2 Baja/Modificación de Artículos. El producto debe estar registrado en el sistema.	
Post-Condición	Los datos del producto quedan actualizados, o el producto queda en estado Discontinuado. Se recalculan los dígitos verificadores de la tabla de artículos y la operación queda registrada en la bitácora.	
FLUJO	**Flujo Normal**	**Flujo Alternativo**
	1. El usuario accede al módulo Artículos > Gestión de Artículos.	
	2. El usuario busca el producto por código de barras, SKU o nombre (CU08 Buscar Artículo).	2.1 Producto no encontrado. El sistema informa que el producto no se encuentra registrado.
	3. El usuario selecciona el producto y el sistema muestra sus datos.	
	4. El usuario elige modificar los datos del producto.	4.1 Discontinuar producto. 4.1.1 El usuario presiona "Discontinuar". 4.1.2 El sistema solicita confirmación. 4.1.3 El usuario confirma y el sistema marca el producto como inactivo; continúa en el paso 8. 4.1.4 Si el usuario cancela, el sistema no realiza cambios y vuelve al paso 3.
	5. El usuario modifica los datos necesarios: nombre, descripción, código de barras, marca, rubro, IVA, unidad de medida o precio de venta.	

	6. El sistema valida los datos ingresados.	6.1 Campos vacíos. El sistema detecta campos obligatorios sin completar e informa al usuario. 6.2 Datos incorrectos. El sistema detecta datos inválidos (por ejemplo, un precio de venta menor o igual a cero) y solicita corregirlos. 6.3 Código de barras duplicado. El sistema detecta que el código de barras pertenece a otro producto e informa al usuario que ingrese uno distinto.
	7. El usuario confirma la modificación presionando "Guardar".	7.1 Cancelar los cambios. El sistema descarta los cambios y vuelve al paso 3.
	8. El sistema actualiza el producto en la base de datos y recalcula los dígitos verificadores.	
	9. El sistema registra la operación en la bitácora.	
	10. El sistema informa que la operación se realizó con éxito.	

Diagrama de caso de uso

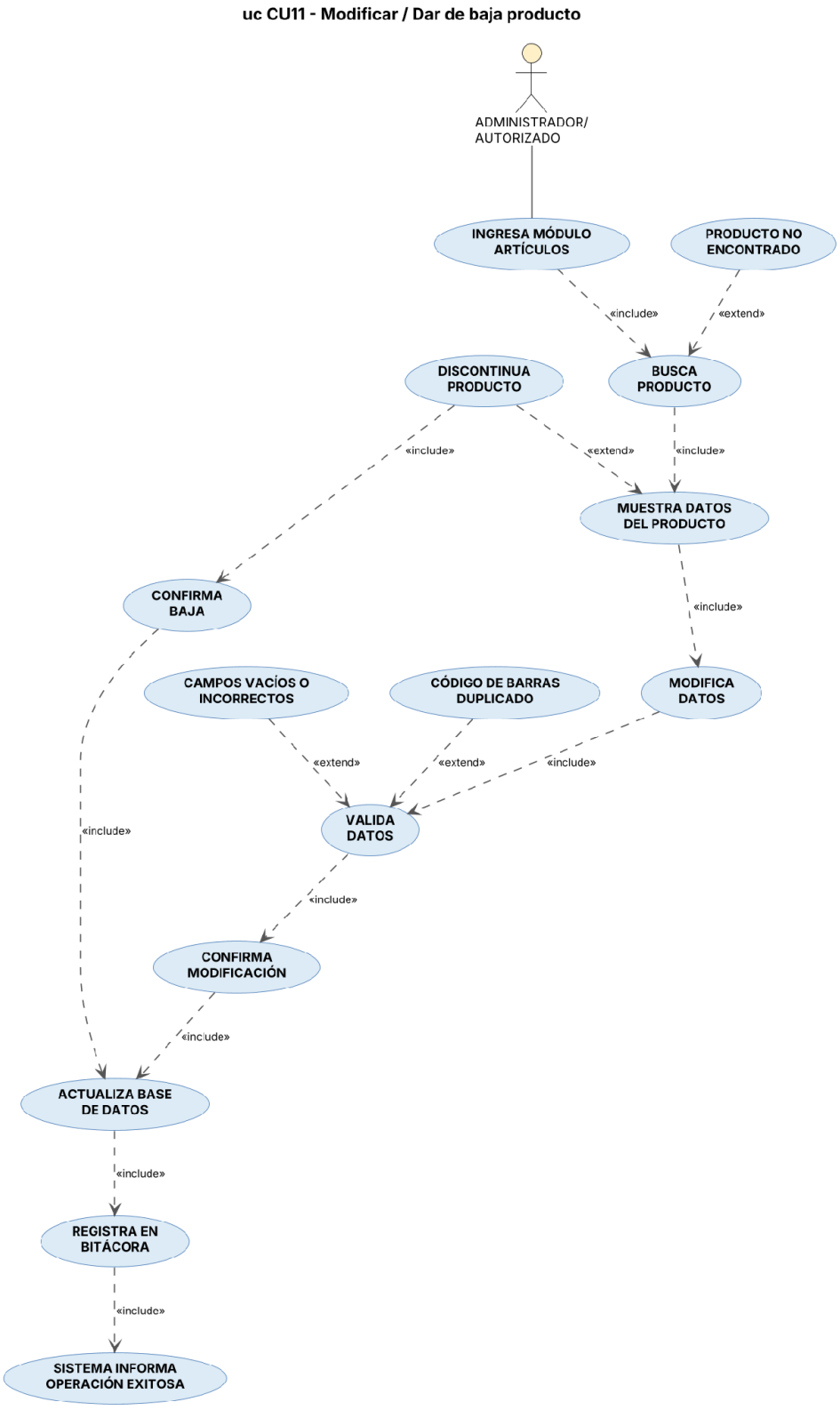


Diagrama de secuencia

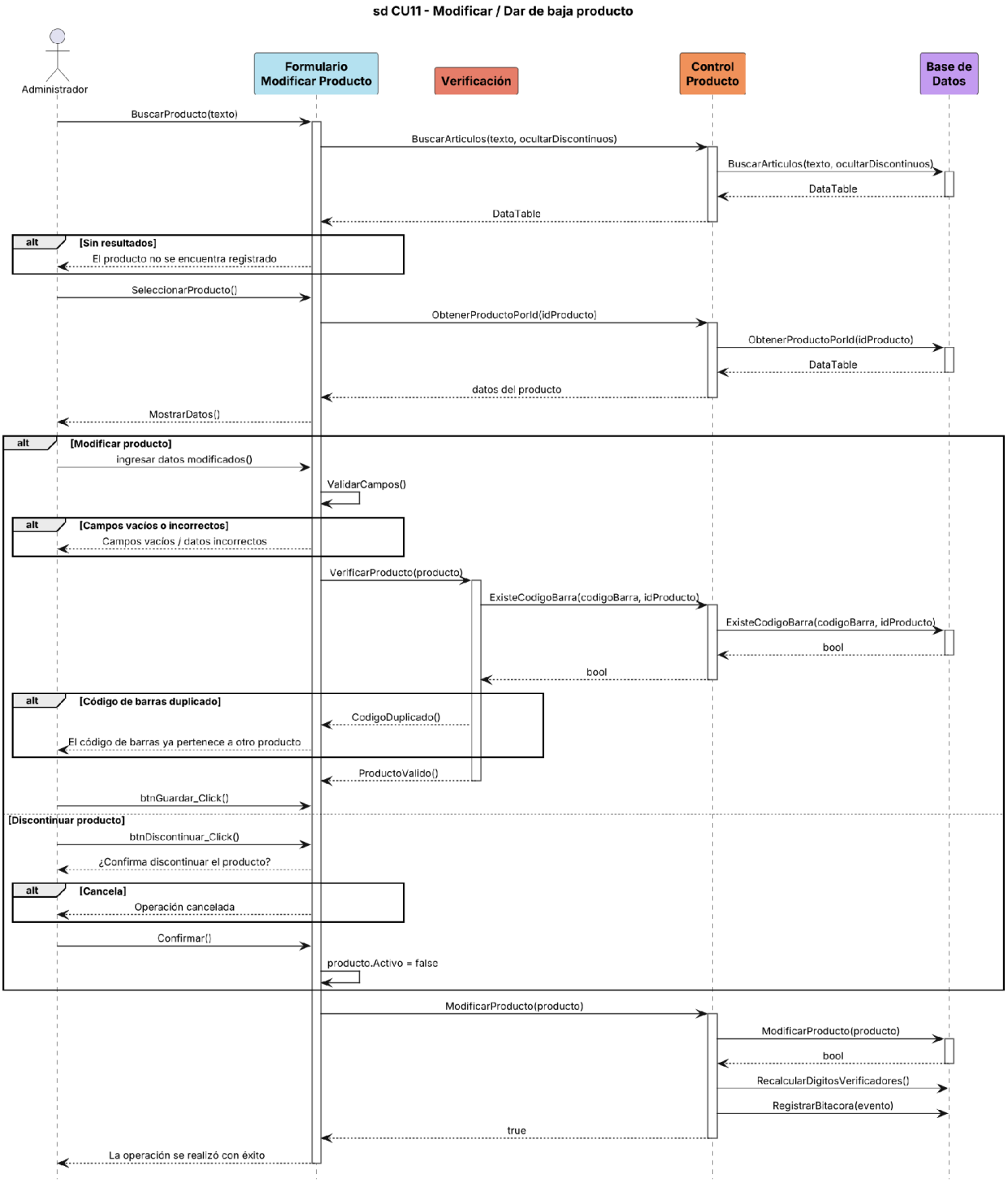
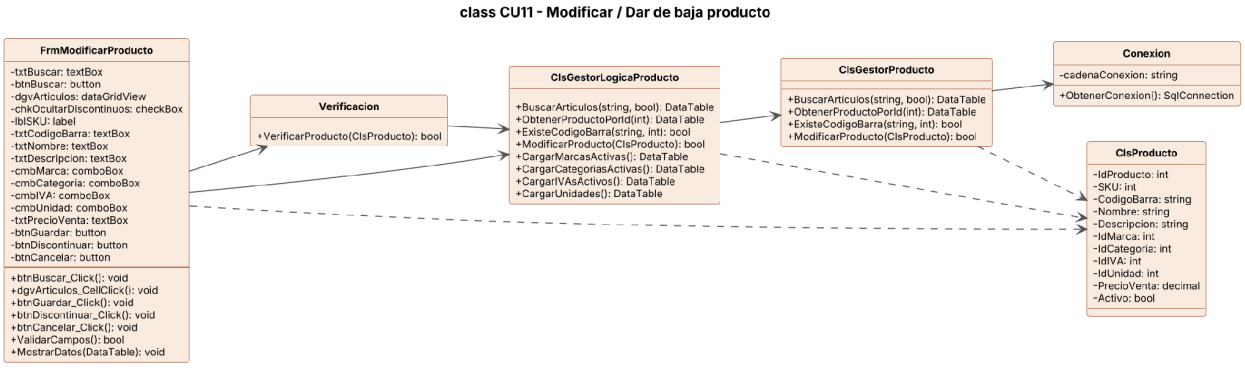


Diagrama de clases



CU12 - Cargar stock a artículo existente

ID y Nombre	CU12 Cargar stock a artículo existente	
Descripción	Permite al dueño o empleado autorizado sumar unidades al stock de un producto que ya está registrado, indicando la cantidad y los datos del lote (código de lote, fecha de fabricación y fecha de vencimiento). El sistema actualiza la cantidad disponible y registra el movimiento de stock.	
Actores	Administrador - Empleado autorizado (permiso 1.1.3 Carga de Artículos).	
Pre-Condición	El usuario debe haber iniciado sesión y tener el permiso 1.1.3 Carga de Artículos. El producto debe estar registrado y activo.	
Post-Condición	El stock del producto queda actualizado y se registra el nuevo lote. Se registra el movimiento de stock de tipo Ingreso con la fecha, la cantidad y el usuario.	
FLUJO	**Flujo Normal**	**Flujo Alternativo**
	1. El usuario accede al módulo Artículos > Carga de Artículos.	
	2. El usuario busca el producto por código de barras, SKU o nombre (CU08 Buscar Artículo).	2.1 Producto no encontrado. El sistema informa que el producto no se encuentra registrado.
	3. El sistema muestra los datos del producto y su stock actual.	3.1 Producto discontinuado. El sistema informa que no se puede cargar stock a un producto inactivo.
	4. El usuario ingresa la cantidad a cargar y los datos del lote: código de lote, fecha de fabricación y fecha de vencimiento.	
	5. El sistema valida los datos ingresados.	5.1 Campos vacíos. El sistema informa los campos obligatorios sin completar. 5.2 Cantidad inválida. El sistema informa que la cantidad debe ser mayor a cero. 5.3 Fecha de vencimiento inválida. El sistema informa que la fecha de vencimiento no puede ser anterior a la fecha actual.
	6. El usuario confirma la carga.	6.1 Cancelar. El sistema descarta los datos y vuelve al paso 3.
	7. El sistema registra el lote y actualiza el stock del producto.	
	8. El sistema registra el movimiento de stock de tipo Ingreso.	

	9. El sistema informa que el stock se cargó correctamente y muestra la nueva cantidad disponible.	

Diagrama de caso de uso

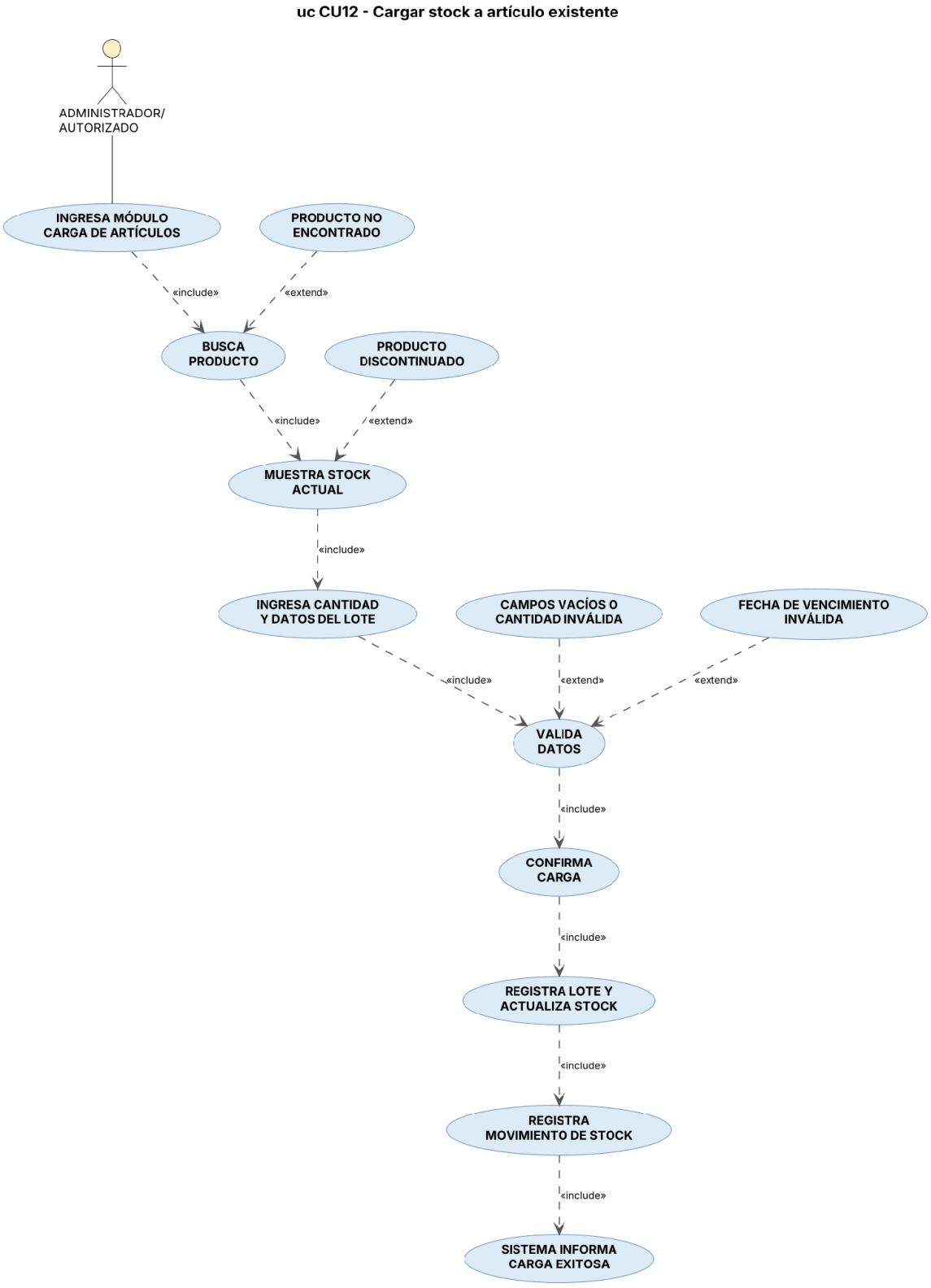


Diagrama de secuencia

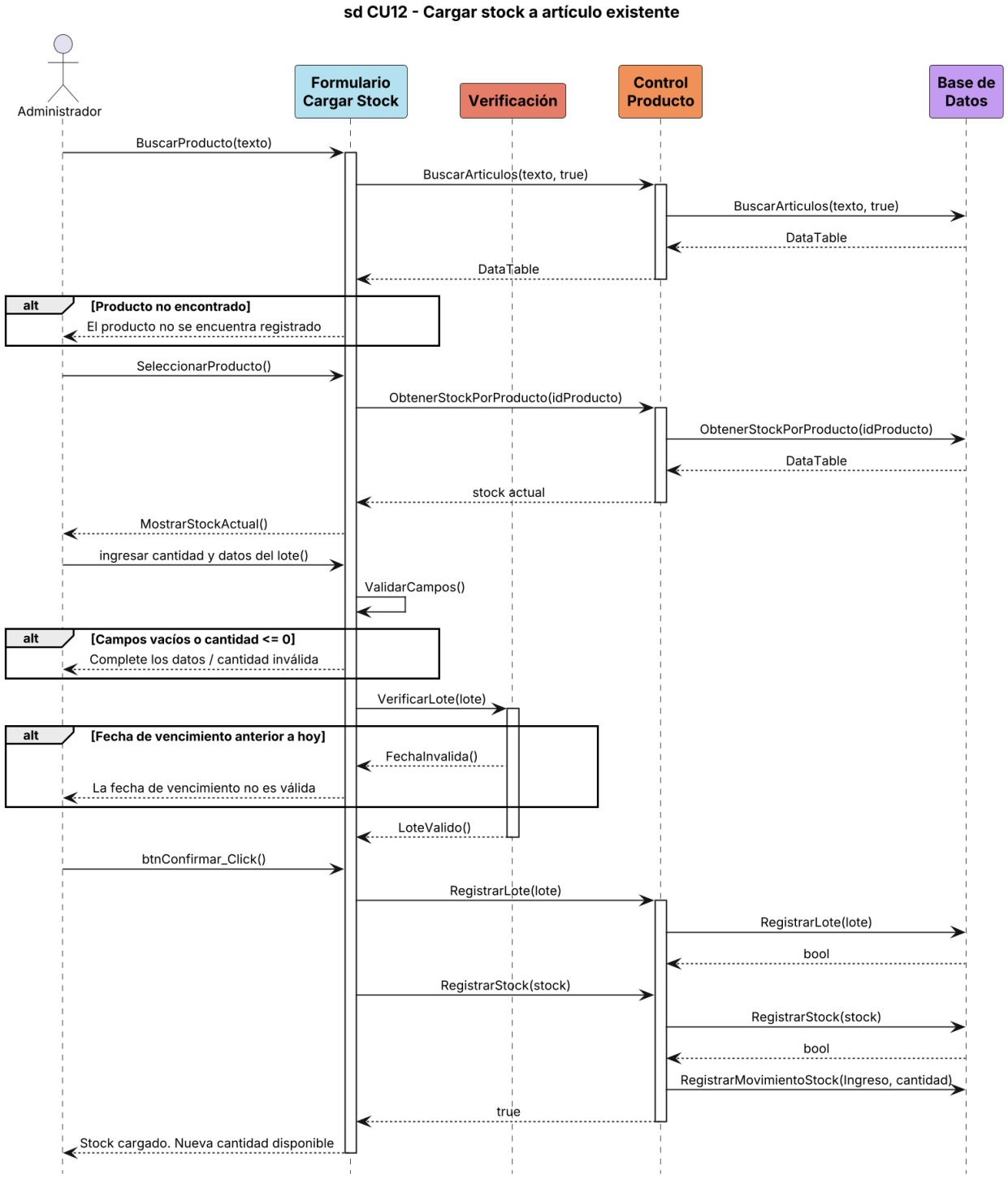
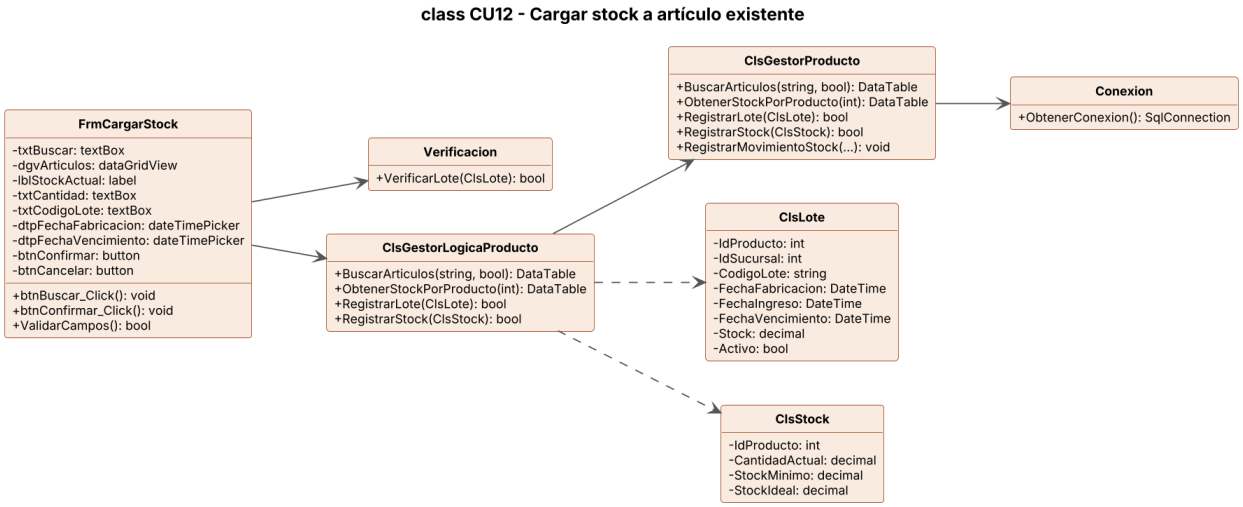


Diagrama de clases



CU13 - Gestionar rubros

ID y Nombre	**CU13 Gestionar rubros**	
Descripción	Permite al dueño o empleado autorizado administrar los rubros (categorías) con los que se clasifican los productos: dar de alta un rubro nuevo, modificar su nombre, discontinuarlo o volver a habilitarlo. Los rubros discontinuados no se eliminan, solo dejan de ofrecerse al cargar productos.	
Actores	Administrador - Empleado autorizado (permiso 1.2.0 Gestión de Rubros).	
Pre-Condición	El usuario debe haber iniciado sesión y tener el permiso 1.2.0 Gestión de Rubros (o 1.2.1 / 1.2.2 según la operación).	
Post-Condición	El rubro queda registrado, modificado, discontinuado o habilitado según la operación elegida. La operación queda registrada en la bitácora.	
FLUJO	**Flujo Normal**	**Flujo Alternativo**
	1. El usuario accede al módulo Artículos > Gestión de Rubros.	
	2. El sistema muestra el listado de rubros con su estado (Activo / Discontinuado).	
	3. El usuario presiona "Nuevo rubro".	3.1 Modificar rubro. El usuario selecciona un rubro, presiona "Modificar" y edita el nombre; continúa en el paso 5. 3.2 Discontinuar rubro. El usuario selecciona un rubro activo y presiona "Discontinuar"; el sistema pide confirmación y continúa en el paso 7. 3.3 Habilitar rubro. El usuario selecciona un rubro discontinuado y presiona "Habilitar"; continúa en el paso 7.
	4. El usuario ingresa el nombre del rubro.	
	5. El sistema valida los datos ingresados.	5.1 Campo vacío. El sistema informa que debe ingresar un nombre. 5.2 Rubro existente. El sistema informa que ya existe un rubro con ese nombre.
	6. El usuario confirma la operación.	6.1 Cancelar. El sistema descarta los cambios y vuelve al paso 2.
	7. El sistema guarda los cambios en la base de datos.	

	8. El sistema registra la operación en la bitácora.	
	9. El sistema informa que la operación se realizó con éxito y actualiza el listado.	

Diagrama de caso de uso

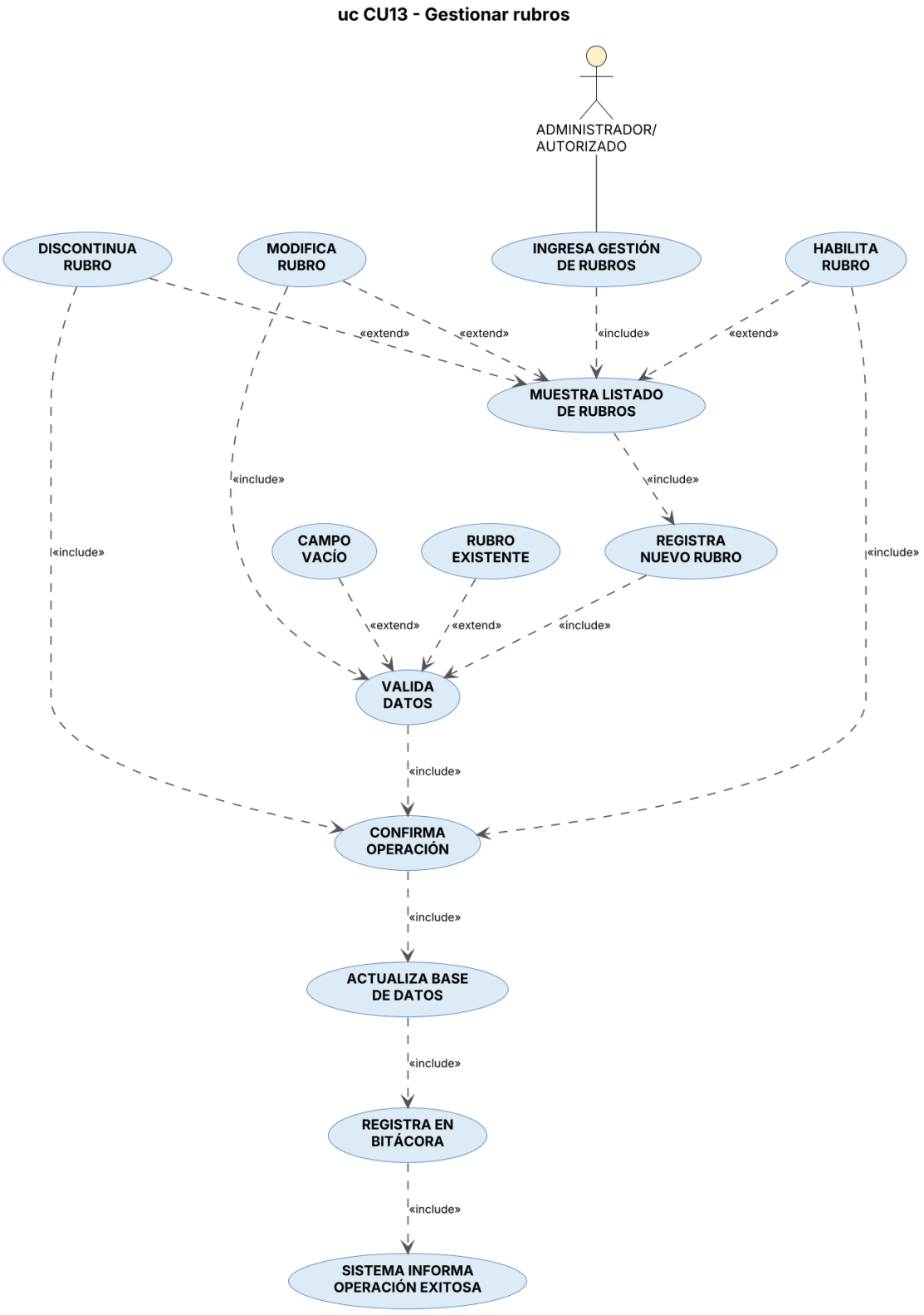


Diagrama de secuencia

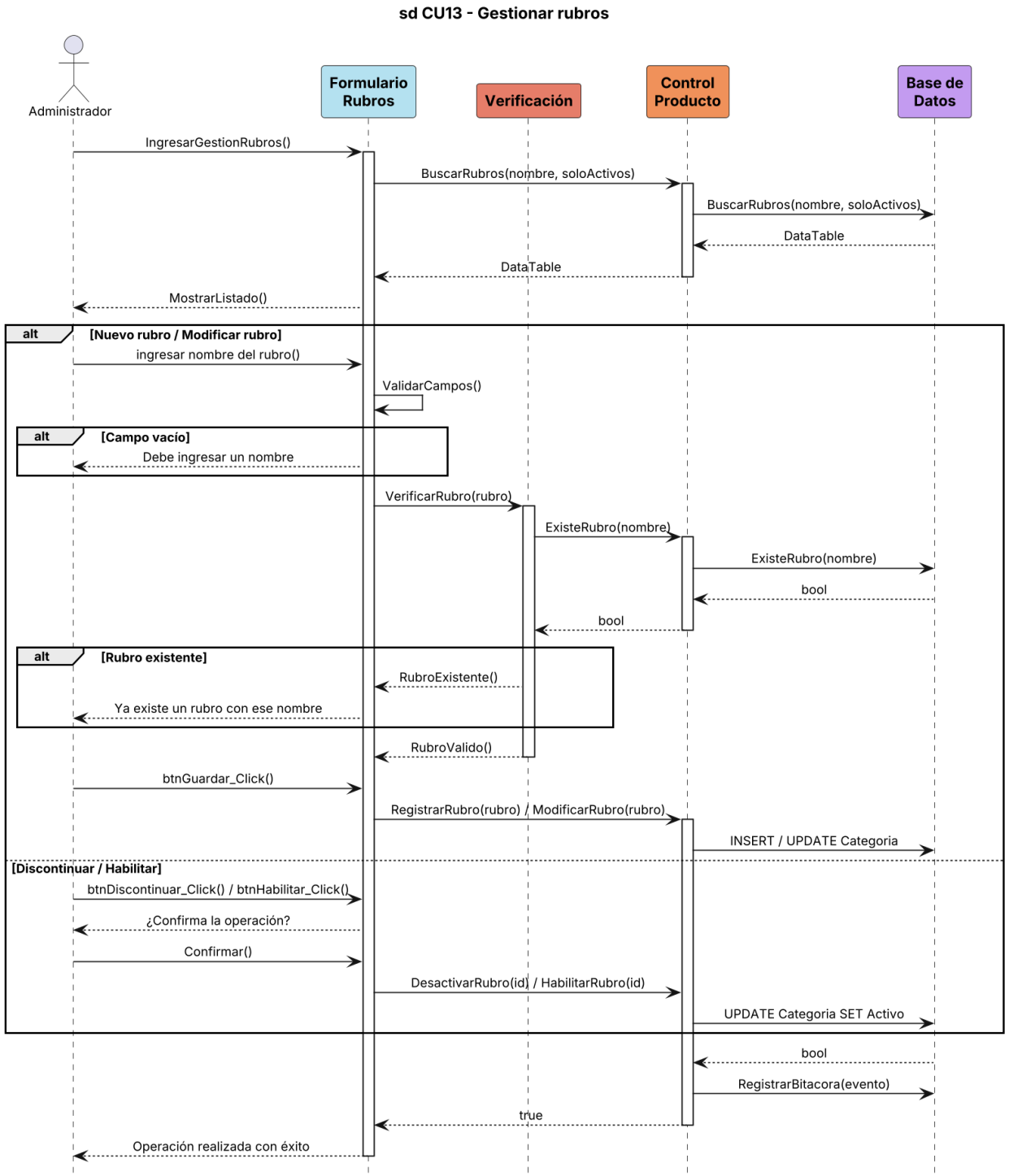
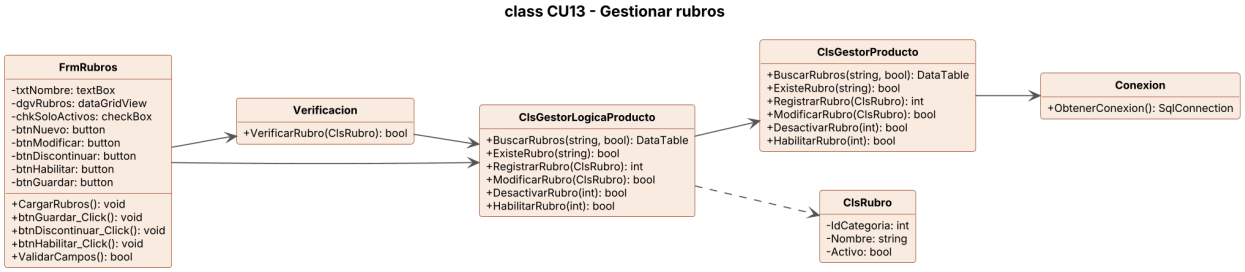


Diagrama de clases



CU14 - Gestionar marcas

ID y Nombre	**CU14 Gestionar marcas**	
Descripción	Permite al dueño o empleado autorizado administrar las marcas de los productos: registrar una marca nueva, modificarla, discontinuarla o volver a habilitarla. Las marcas discontinuadas no se eliminan; se pueden ver activando el filtro correspondiente.	
Actores	Administrador - Empleado autorizado (permiso 1.3.0 Gestión de Marcas).	
Pre-Condición	El usuario debe haber iniciado sesión y tener el permiso 1.3.0 Gestión de Marcas (o 1.3.1 / 1.3.2 según la operación).	
Post-Condición	La marca queda registrada, modificada, discontinuada o habilitada según la operación elegida. La operación queda registrada en la bitácora.	
FLUJO	**Flujo Normal**	**Flujo Alternativo**
	1. El usuario accede al módulo Artículos > Gestión de Marcas.	
	2. El sistema muestra el listado de marcas con su estado (Activo / Discontinuado).	
	3. El usuario busca la marca por nombre, pudiendo filtrar solo las activas.	3.1 Sin resultados. El sistema informa que no se encontraron marcas.
	4. El usuario presiona "Nueva marca".	4.1 Modificar marca. El usuario selecciona una marca y presiona "Modificar"; continúa en el paso 5. 4.2 Discontinuar marca. El usuario selecciona una marca activa y presiona "Discontinuar"; el sistema pregunta "¿Está seguro de discontinuar la marca?" y continúa en el paso 8. 4.3 Habilitar marca. El usuario selecciona una marca discontinuada y presiona "Habilitar"; continúa en el paso 8.
	5. El usuario ingresa o modifica el nombre de la marca.	
	6. El sistema valida los datos ingresados.	6.1 Campo vacío. El sistema informa que debe ingresar un nombre. 6.2 Marca existente. El sistema informa que ya existe una marca con ese nombre.

	7. El usuario confirma la operación.	7.1 Cancelar. El sistema descarta los cambios y vuelve al paso 2.
	8. El sistema guarda los cambios en la base de datos.	
	9. El sistema registra la operación en la bitácora.	
	10. El sistema informa que la operación se realizó con éxito y actualiza el listado.	

Diagrama de caso de uso

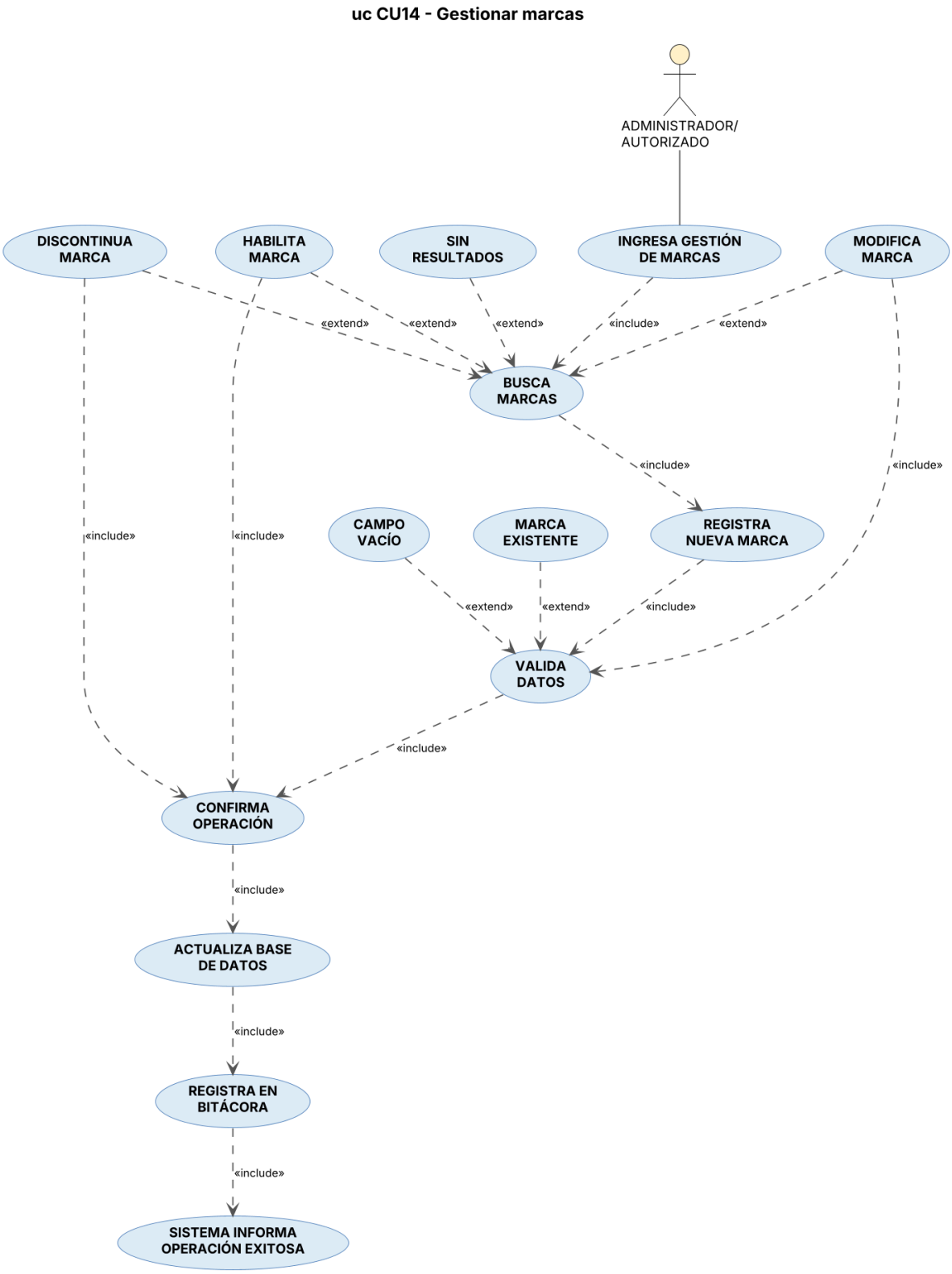


Diagrama de secuencia

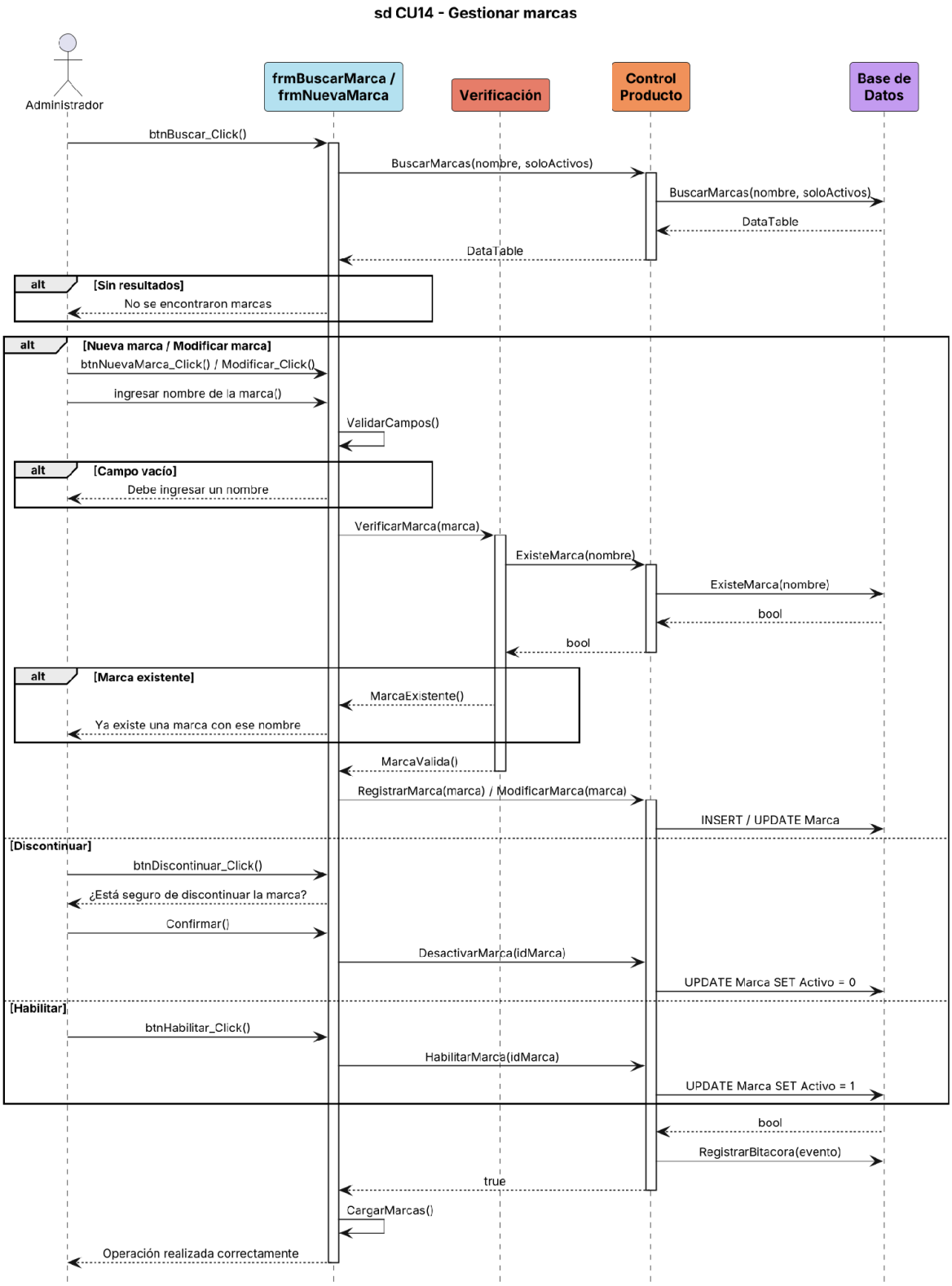
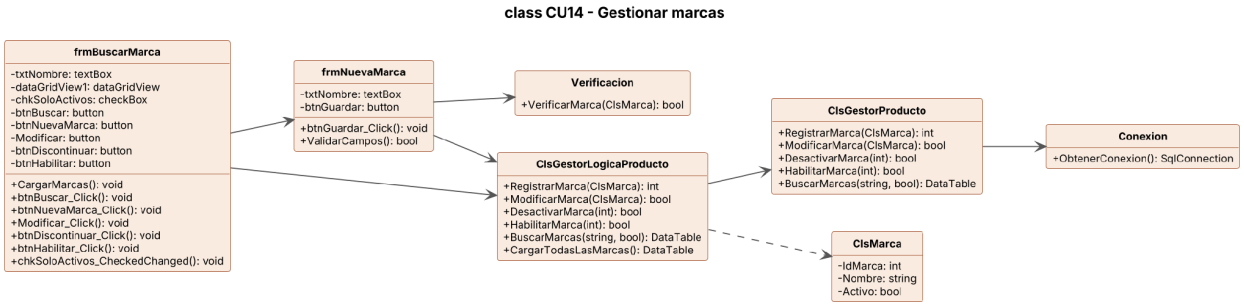


Diagrama de clases



CU15 - Importar artículos desde Excel

ID y Nombre	**CU15 Importar artículos desde Excel**	
Descripción	Permite al dueño o empleado autorizado cargar varios artículos de una sola vez desde un archivo Excel. El sistema ofrece una planilla modelo con una hoja de instrucciones, lee cada fila del archivo, valida los datos y registra los productos válidos junto con su stock, lote y proveedor. Al finalizar informa cuántas filas se importaron y cuáles tuvieron errores.	
Actores	Administrador - Empleado autorizado (permiso 1.4.0 Importar Artículos desde Excel).	
Pre-Condición	El usuario debe haber iniciado sesión y tener el permiso 1.4.0. El archivo debe estar en formato Excel (.xlsx) y respetar la estructura de la planilla modelo.	
Post-Condición	Los artículos válidos quedan registrados con su stock, lote y relación con el proveedor. Las filas con errores no se importan y se informan al usuario.	
FLUJO	**Flujo Normal**	**Flujo Alternativo**
	1. El usuario accede al módulo Artículos > Importar Artículos desde Excel.	
	2. El usuario presiona "Seleccionar" y elige el archivo Excel.	2.1 Descargar planilla modelo. El usuario presiona "Descargar planilla"; el sistema genera el archivo Excel con la hoja de instrucciones y lo guarda en la ruta elegida. 2.2 No se seleccionó archivo. El sistema informa "Debe seleccionar un archivo de Excel".
	3. El usuario presiona "Aceptar" para iniciar la importación.	
	4. El sistema abre el archivo y lee cada fila.	4.1 Archivo inválido. El sistema informa que no pudo leer el archivo o que no respeta el formato de la planilla.
	5. El sistema valida los datos de cada fila: campos obligatorios, precios mayores o iguales a cero, marca, rubro y proveedor existentes, y código de barras no repetido.	5.1 Fila con errores. El sistema omite la fila y guarda el motivo para el informe final.
	6. El sistema registra cada producto válido con su stock, lote y relación con el proveedor.	
	7. El sistema registra la importación en la bitácora.	
	8. El sistema informa la cantidad de artículos importados y el detalle de las filas con errores.	

Diagrama de caso de uso

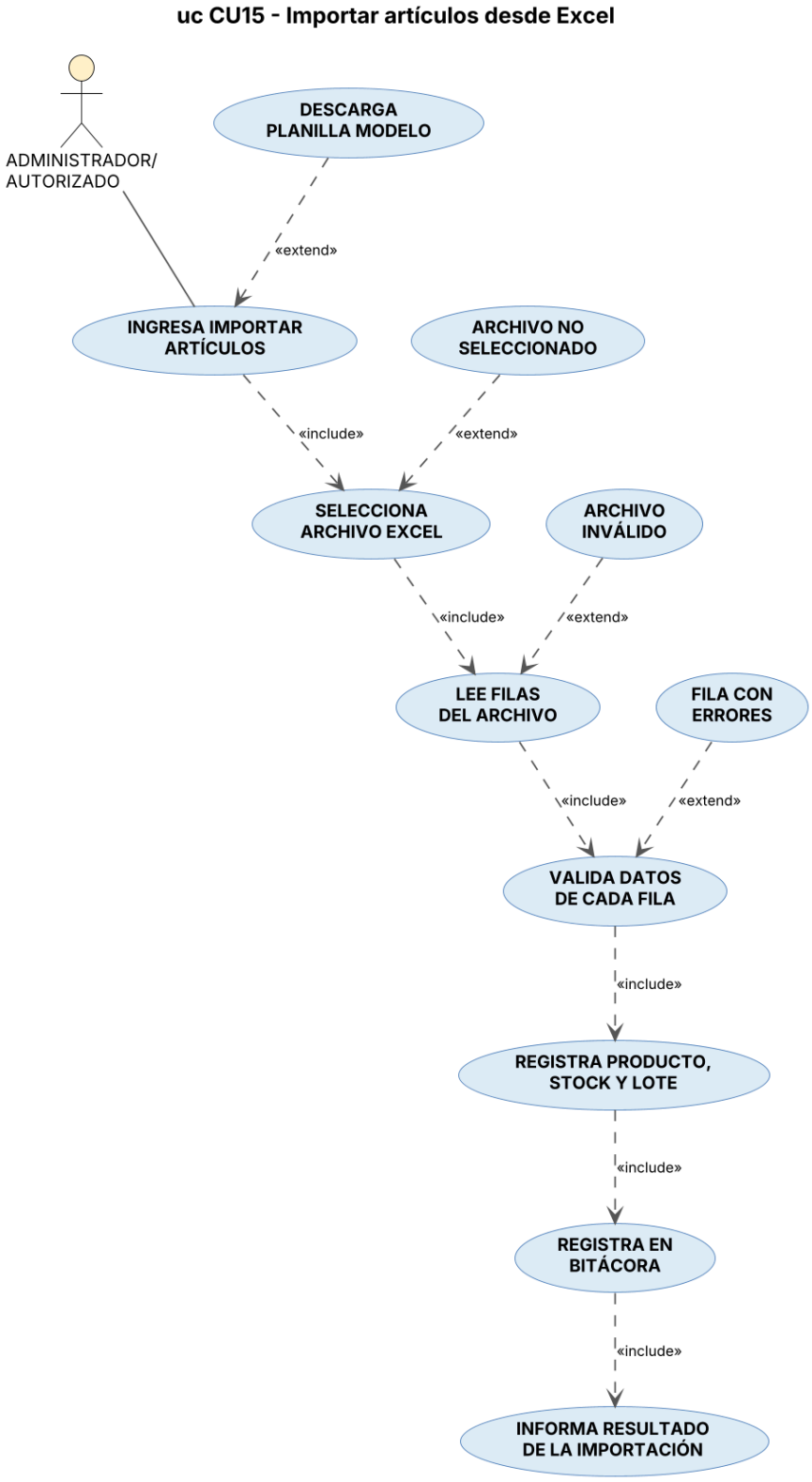


Diagrama de secuencia

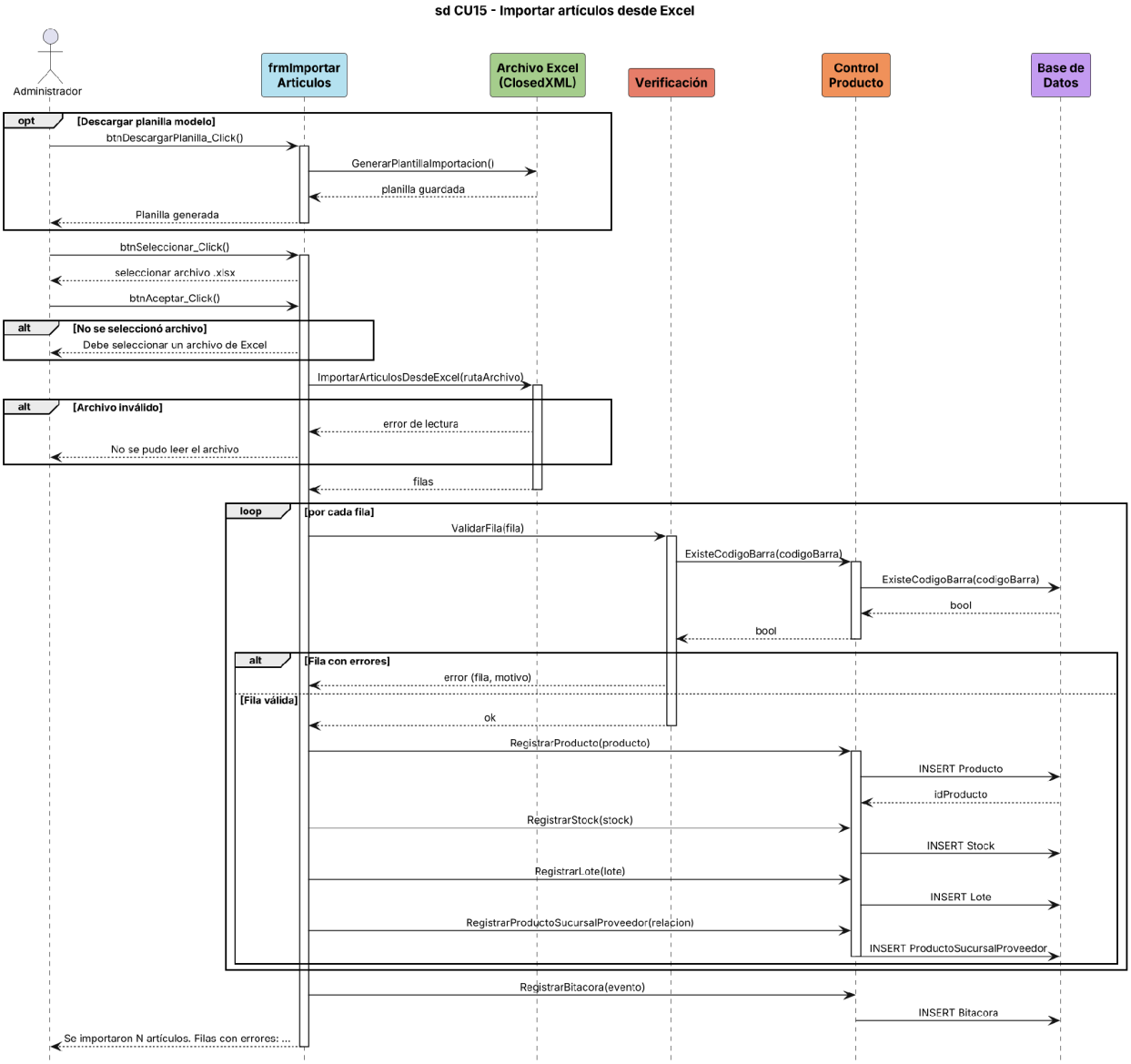
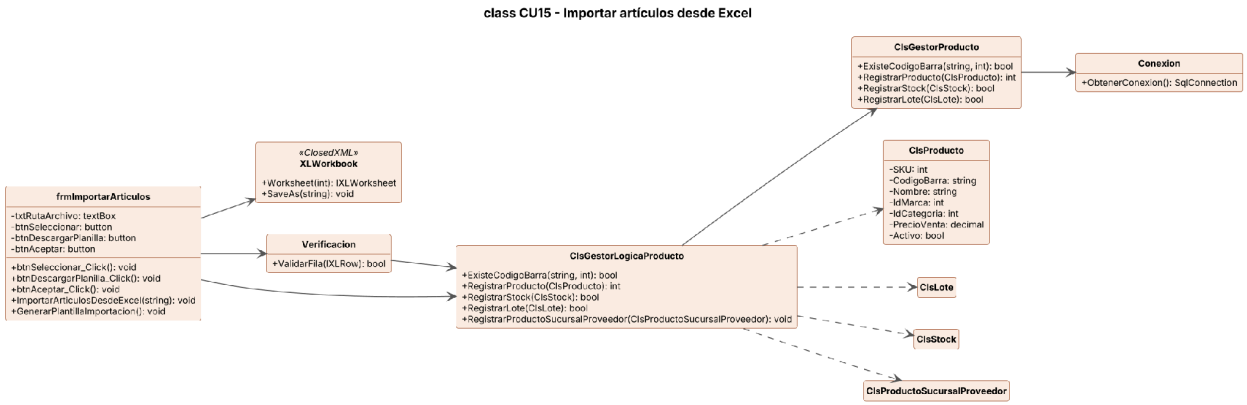


Diagrama de clases



CU16 - Exportar artículos a Excel

ID y Nombre	**CU16 Exportar artículos a Excel**	
Descripción	Permite al dueño o empleado autorizado generar un archivo Excel con el listado de artículos, filtrado por proveedor, sucursal del proveedor, rubro, marca y estado. Sirve para controlar precios y stock fuera del sistema o para enviar listados a proveedores.	
Actores	Administrador - Empleado autorizado (permiso 1.5.0 Exportar Artículos a Excel).	
Pre-Condición	El usuario debe haber iniciado sesión y tener el permiso 1.5.0. Debe haber artículos registrados.	
Post-Condición	Se genera un archivo Excel con los artículos que cumplen los filtros, en la ubicación elegida por el usuario.	
FLUJO	**Flujo Normal**	**Flujo Alternativo**
	1. El usuario accede al módulo Artículos > Exportar Artículos a Excel.	
	2. El sistema carga los filtros disponibles: proveedores, rubros, marcas y estados.	
	3. El usuario selecciona los filtros deseados (al elegir un proveedor, el sistema carga sus sucursales).	
	4. El usuario presiona "Exportar".	
	5. El sistema obtiene los artículos que cumplen los filtros.	5.1 Sin resultados. El sistema informa que no hay artículos para exportar con los filtros elegidos.
	6. El usuario elige la carpeta y el nombre del archivo.	6.1 Cancelar. El sistema no genera el archivo y vuelve al paso 3.
	7. El sistema genera el archivo Excel.	7.1 Error al guardar. El sistema informa que no pudo guardar el archivo (por ejemplo, porque está abierto).
	8. El sistema informa que el archivo se exportó correctamente.	

Diagrama de caso de uso

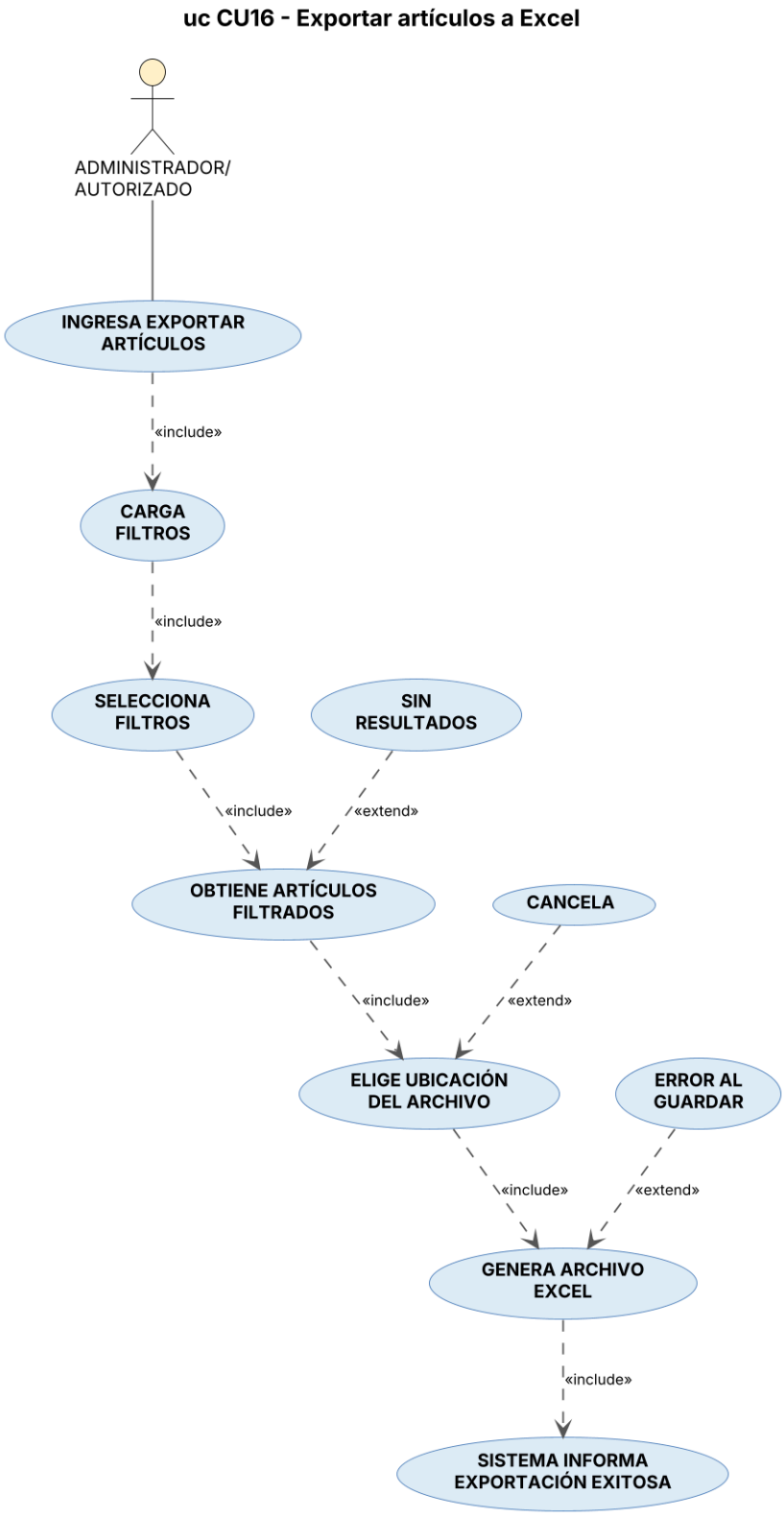


Diagrama de secuencia

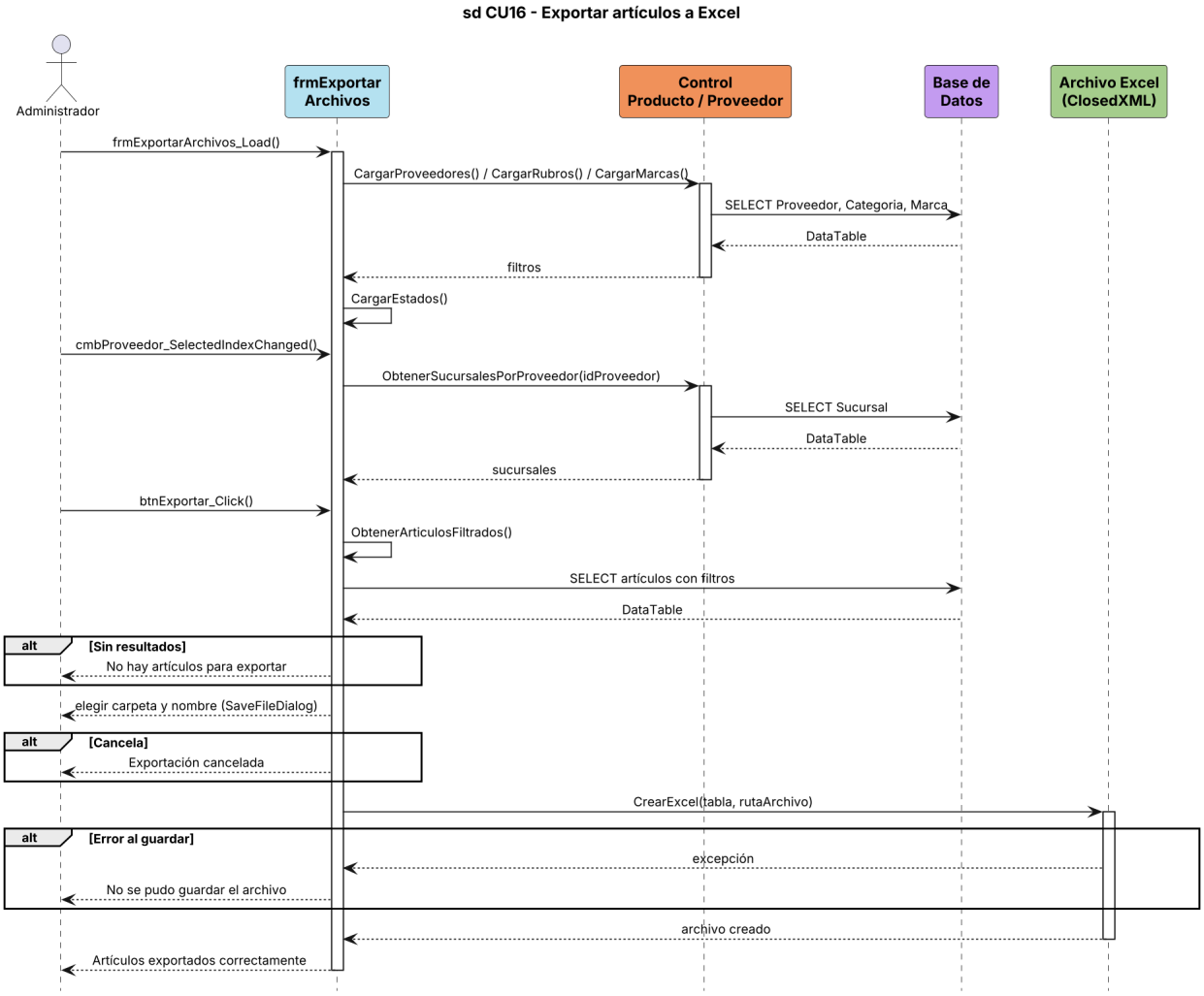
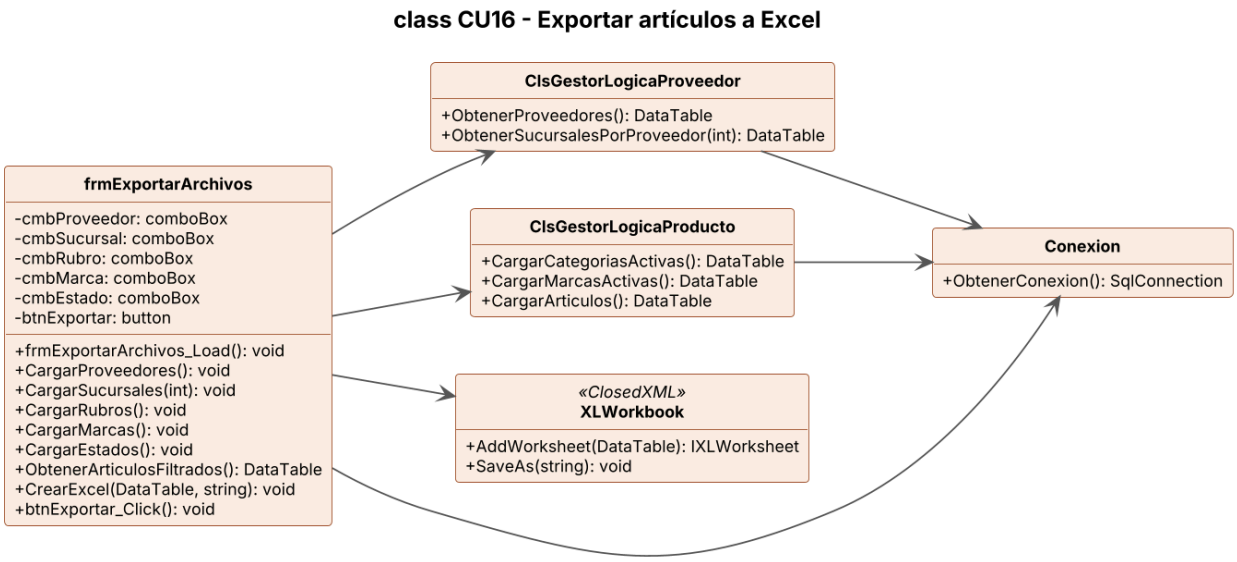


Diagrama de clases



CU17 - Consultar historial de ventas

ID y Nombre	**CU17 Consultar historial de ventas**	
Descripción	Permite al dueño o empleado autorizado consultar las ventas registradas en un rango de fechas, filtrarlas por producto o medio de pago y ver el detalle de cada venta: artículos, cantidades, precios, medios de pago utilizados y usuario que la realizó.	
Actores	Administrador - Empleado autorizado (permiso 2.0.0 Ventas).	
Pre-Condición	El usuario debe haber iniciado sesión y tener el permiso 2.0.0 Ventas.	
Post-Condición	Se muestran las ventas que cumplen los filtros y, si se elige una, su detalle. No se modifican datos.	
FLUJO	**Flujo Normal**	**Flujo Alternativo**
	1. El usuario accede al módulo Ventas > Administrador de Ventas.	
	2. El sistema muestra las ventas del día.	
	3. El usuario ingresa el rango de fechas y, opcionalmente, el producto o el medio de pago, y presiona "Buscar".	3.1 Rango de fechas inválido. El sistema informa que la fecha "desde" no puede ser posterior a la fecha "hasta".
	4. El sistema muestra las ventas que cumplen los filtros con fecha, total y usuario.	4.1 Sin resultados. El sistema informa que no hay ventas para los filtros elegidos.
	5. El usuario selecciona una venta y presiona "Ver detalle".	
	6. El sistema muestra el detalle de la venta: artículos, cantidades, precio unitario, subtotal y medios de pago.	

Diagrama de caso de uso

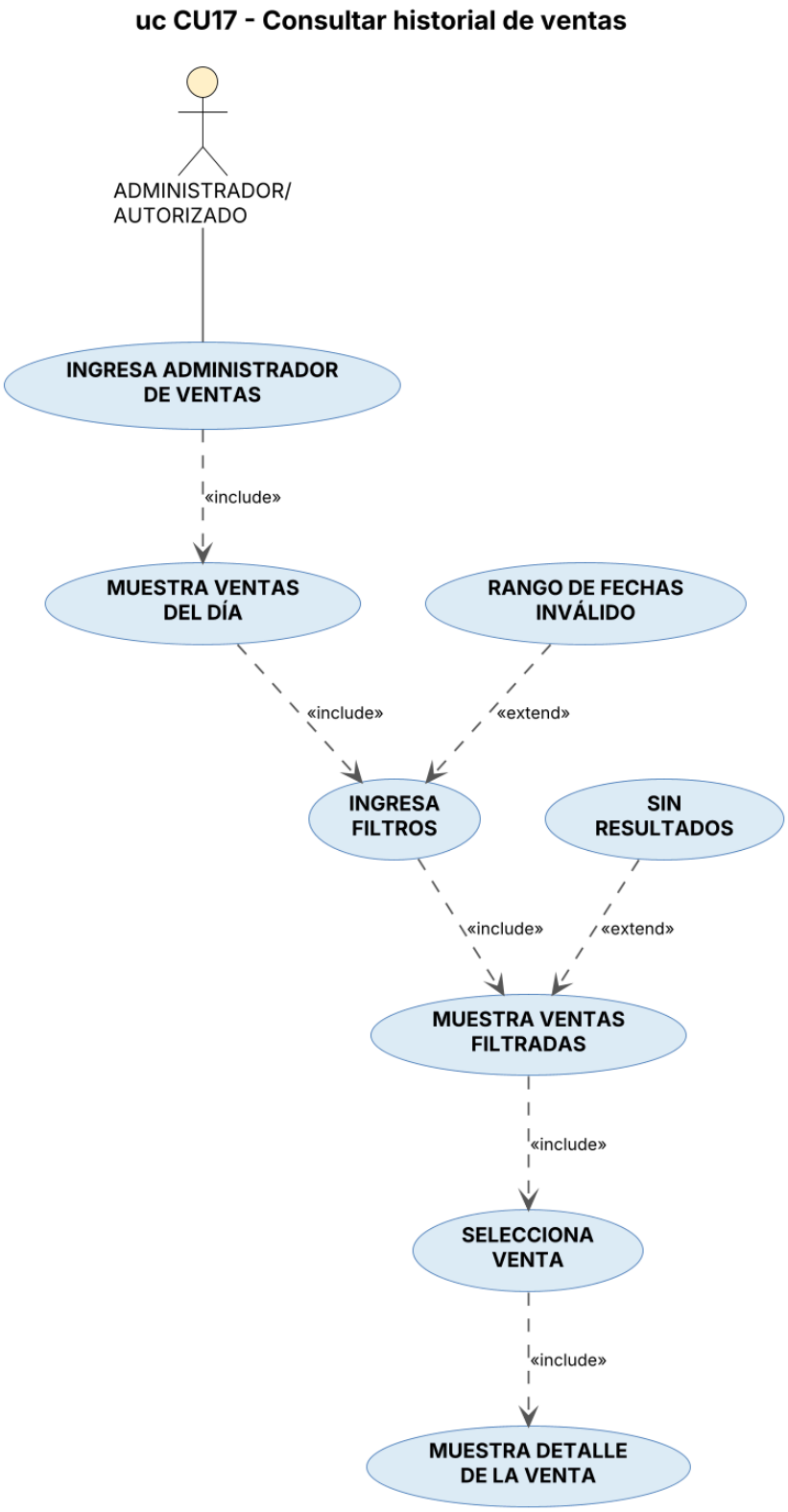


Diagrama de secuencia

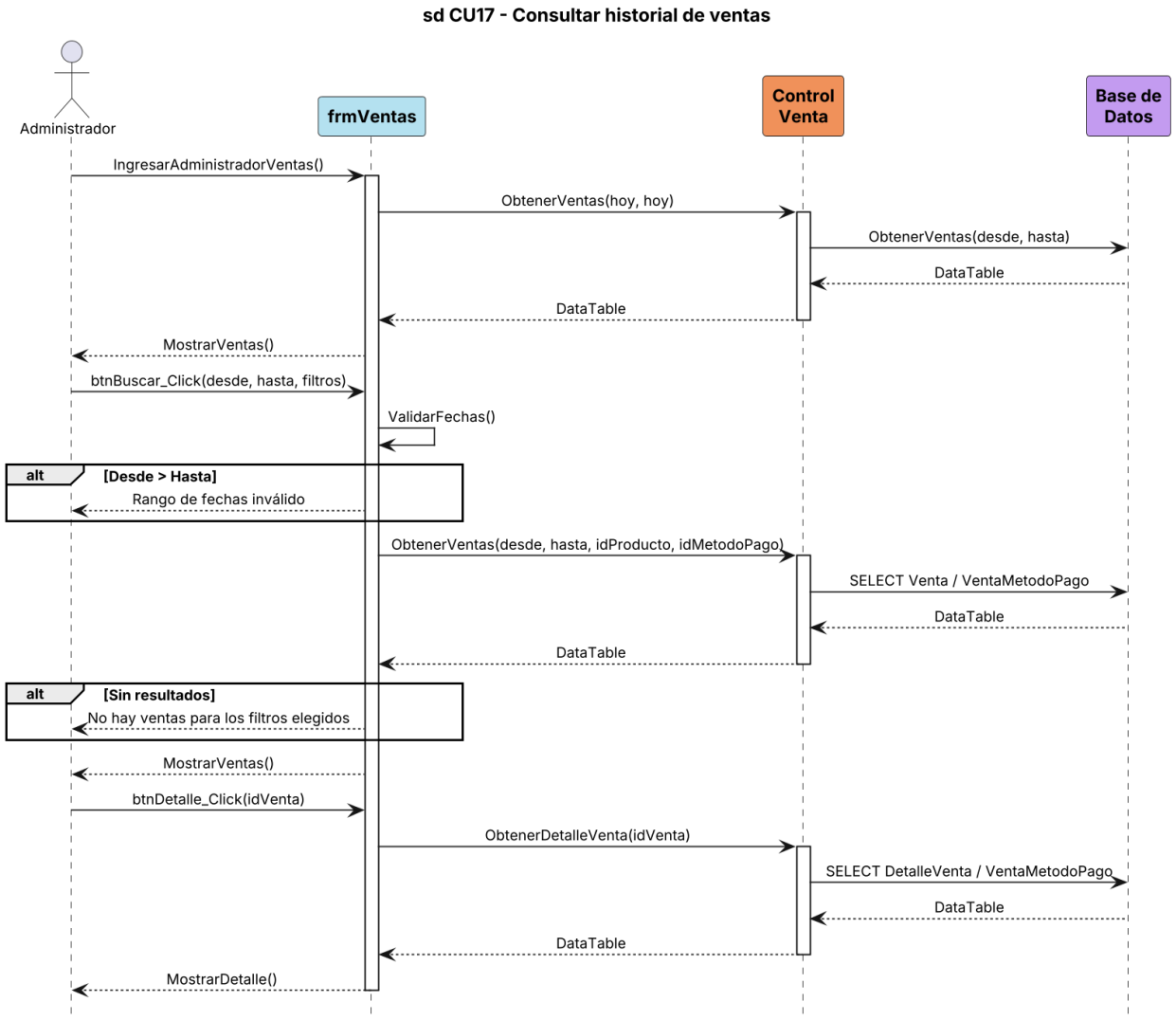
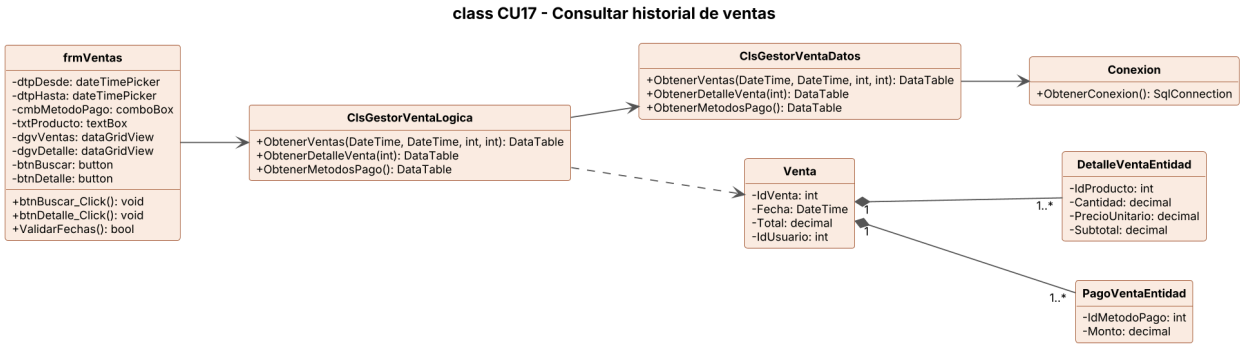


Diagrama de clases



CU18 - Consultar historial de devoluciones

ID y Nombre	**CU18 Consultar historial de devoluciones**	
Descripción	Permite al dueño o empleado autorizado consultar las devoluciones (notas de crédito) registradas, filtrarlas por fecha y ver el detalle de cada una: venta de origen, productos devueltos, cantidades, importe, motivo y usuario que la registró.	
Actores	Administrador - Empleado autorizado (permiso 2.2.2 Historial Devoluciones).	
Pre-Condición	El usuario debe haber iniciado sesión y tener el permiso 2.2.2 Historial Devoluciones.	
Post-Condición	Se muestran las devoluciones que cumplen los filtros y, si se elige una, su detalle. No se modifican datos.	
FLUJO	**Flujo Normal**	**Flujo Alternativo**
	1. El usuario accede al módulo Ventas > Devoluciones > Historial.	
	2. El usuario ingresa el rango de fechas y presiona "Buscar".	2.1 Rango de fechas inválido. El sistema informa que la fecha "desde" no puede ser posterior a la fecha "hasta".
	3. El sistema muestra las devoluciones del período con fecha, venta de origen, importe y usuario.	3.1 Sin resultados. El sistema informa que no hay devoluciones en el período.
	4. El usuario selecciona una devolución.	
	5. El sistema muestra el detalle: productos devueltos, cantidades, importe y motivo.	
	6. El usuario puede exportar el listado a Excel.	6.1 Error al exportar. El sistema informa que no pudo generar el archivo.

Diagrama de caso de uso

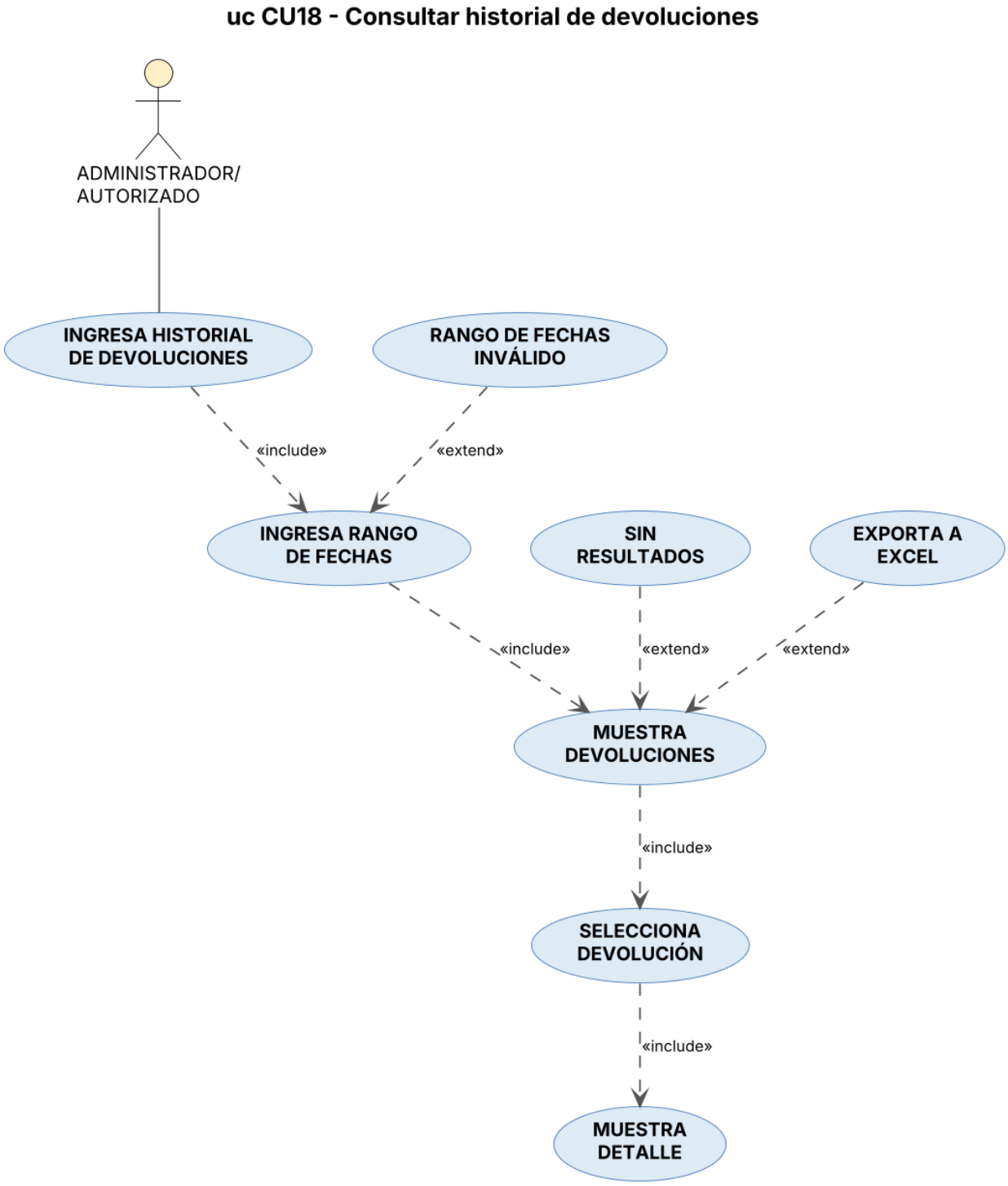


Diagrama de secuencia

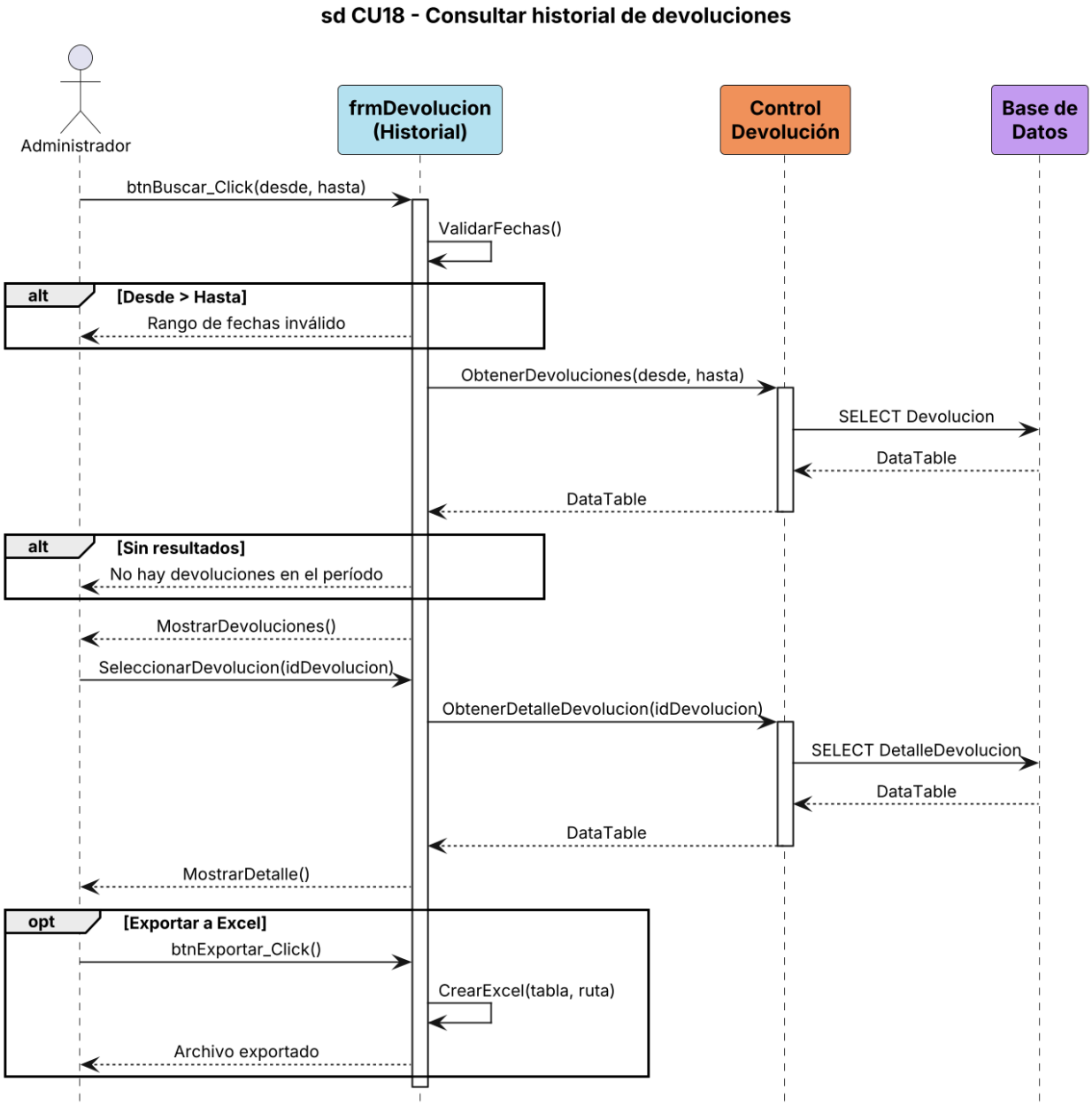
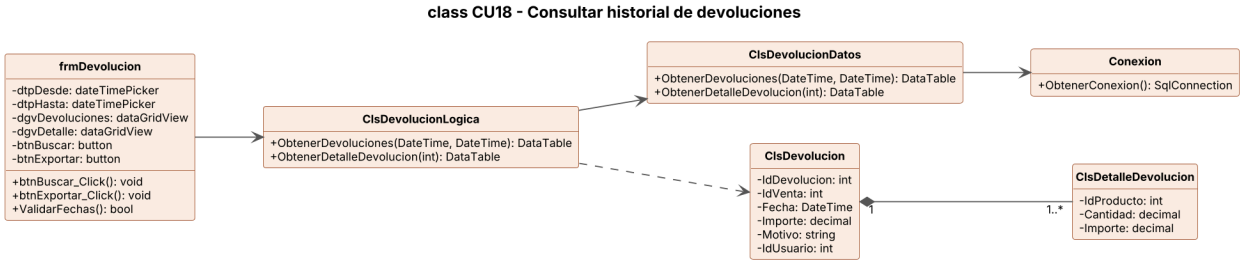


Diagrama de clases



CU19 - Crear promoción

ID y Nombre	CU19 Crear promoción	
Descripción	Permite al Administrador crear una promoción de venta indicando su nombre, los productos incluidos, el tipo de beneficio (precio especial o porcentaje de descuento), la cantidad mínima y el período de vigencia. Durante la vigencia, el sistema aplica la promoción automáticamente al registrar una venta que cumpla las condiciones.	
Actores	Administrador (permiso 2.3.0 Crear Promociones).	
Pre-Condición	El usuario debe haber iniciado sesión y tener el permiso 2.3.0 Crear Promociones. Los productos de la promoción deben estar registrados y activos.	
Post-Condición	La promoción queda registrada y activa para el período indicado. La operación queda registrada en la bitácora.	
FLUJO	**Flujo Normal**	**Flujo Alternativo**
	1. El Administrador presiona "Promociones" en la pantalla principal.	
	2. El sistema muestra las promociones vigentes.	
	3. El Administrador presiona "Nueva promoción".	
	4. El Administrador ingresa el nombre, el tipo de beneficio (precio especial o porcentaje), el valor, la cantidad mínima y las fechas de inicio y fin.	
	5. El Administrador agrega los productos incluidos en la promoción (CU08 Buscar Artículo).	5.1 Producto no encontrado. El sistema informa que el producto no se encuentra registrado.
	6. El sistema valida los datos ingresados.	6.1 Campos vacíos. El sistema informa los campos obligatorios sin completar. 6.2 Datos incorrectos. El sistema informa si el valor es menor o igual a cero, el porcentaje supera el 100 % o la fecha de fin es anterior a la de inicio. 6.3 Sin productos. El sistema informa que debe agregar al menos un producto. 6.4 Promoción superpuesta. El sistema informa que alguno de los productos ya tiene una promoción vigente en esas fechas.

	7. El Administrador confirma la promoción.	7.1 Cancelar. El sistema descarta los datos y vuelve al paso 2.
	8. El sistema registra la promoción y sus productos en la base de datos.	
	9. El sistema registra la operación en la bitácora.	
	10. El sistema informa que la promoción se creó con éxito.	

Diagrama de caso de uso

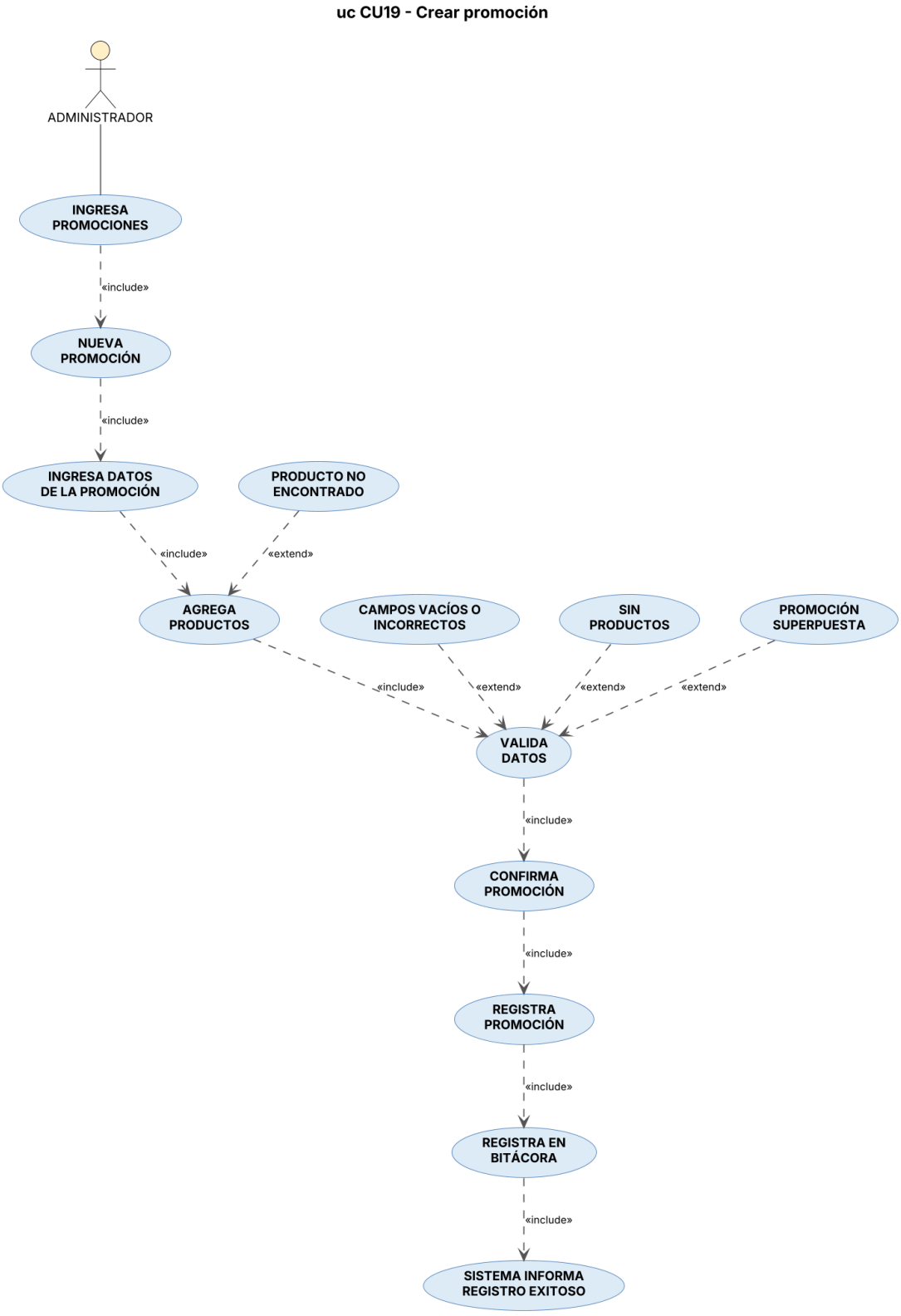


Diagrama de secuencia

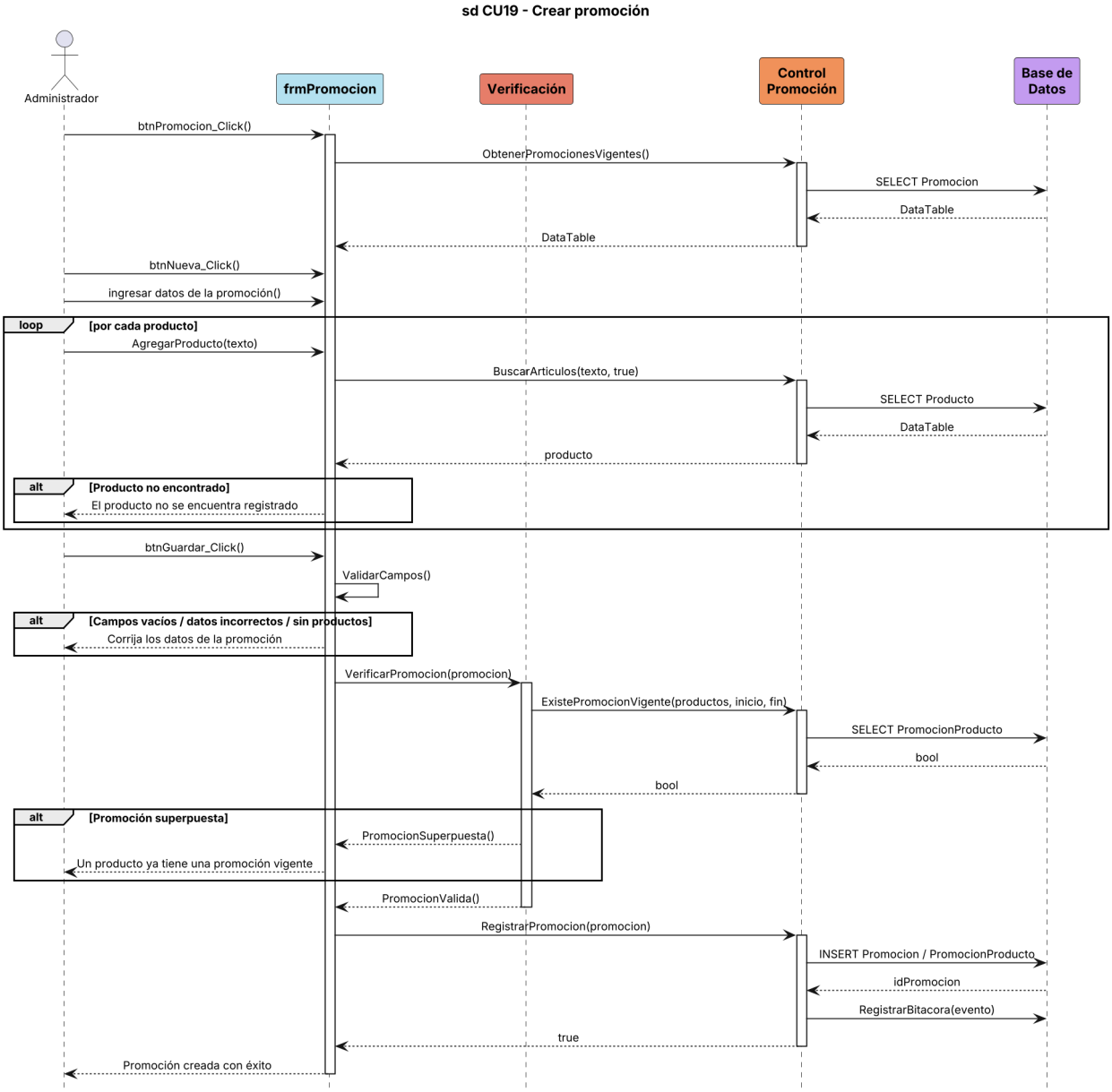
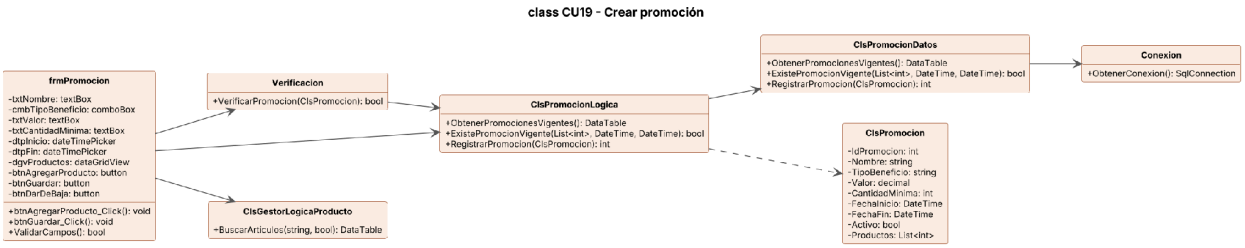


Diagrama de clases



CU20 - Dar de baja promoción

ID y Nombre	CU20 Dar de baja promoción	
Descripción	Permite al Administrador dar de baja una promoción vigente antes de su fecha de fin. La promoción deja de aplicarse a las ventas nuevas, pero se conserva en la base de datos junto con las ventas en las que ya se aplicó.	
Actores	Administrador (permiso 2.4.0 Baja Promociones).	
Pre-Condición	El usuario debe haber iniciado sesión y tener el permiso 2.4.0 Baja Promociones. Debe existir al menos una promoción activa.	
Post-Condición	La promoción queda inactiva y deja de aplicarse en las ventas. La operación queda registrada en la bitácora.	
FLUJO	**Flujo Normal**	**Flujo Alternativo**
	1. El Administrador presiona "Promociones" en la pantalla principal.	
	2. El sistema muestra las promociones vigentes.	2.1 Sin promociones vigentes. El sistema informa que no hay promociones activas.
	3. El Administrador selecciona la promoción y presiona "Dar de baja".	3.1 No se seleccionó promoción. El sistema informa que debe seleccionar una promoción.
	4. El sistema solicita confirmación.	
	5. El Administrador confirma la baja.	5.1 Cancelar. El sistema no realiza cambios y vuelve al paso 2.
	6. El sistema marca la promoción como inactiva.	
	7. El sistema registra la operación en la bitácora.	
	8. El sistema informa que la promoción se dio de baja y actualiza el listado.	

Diagrama de caso de uso

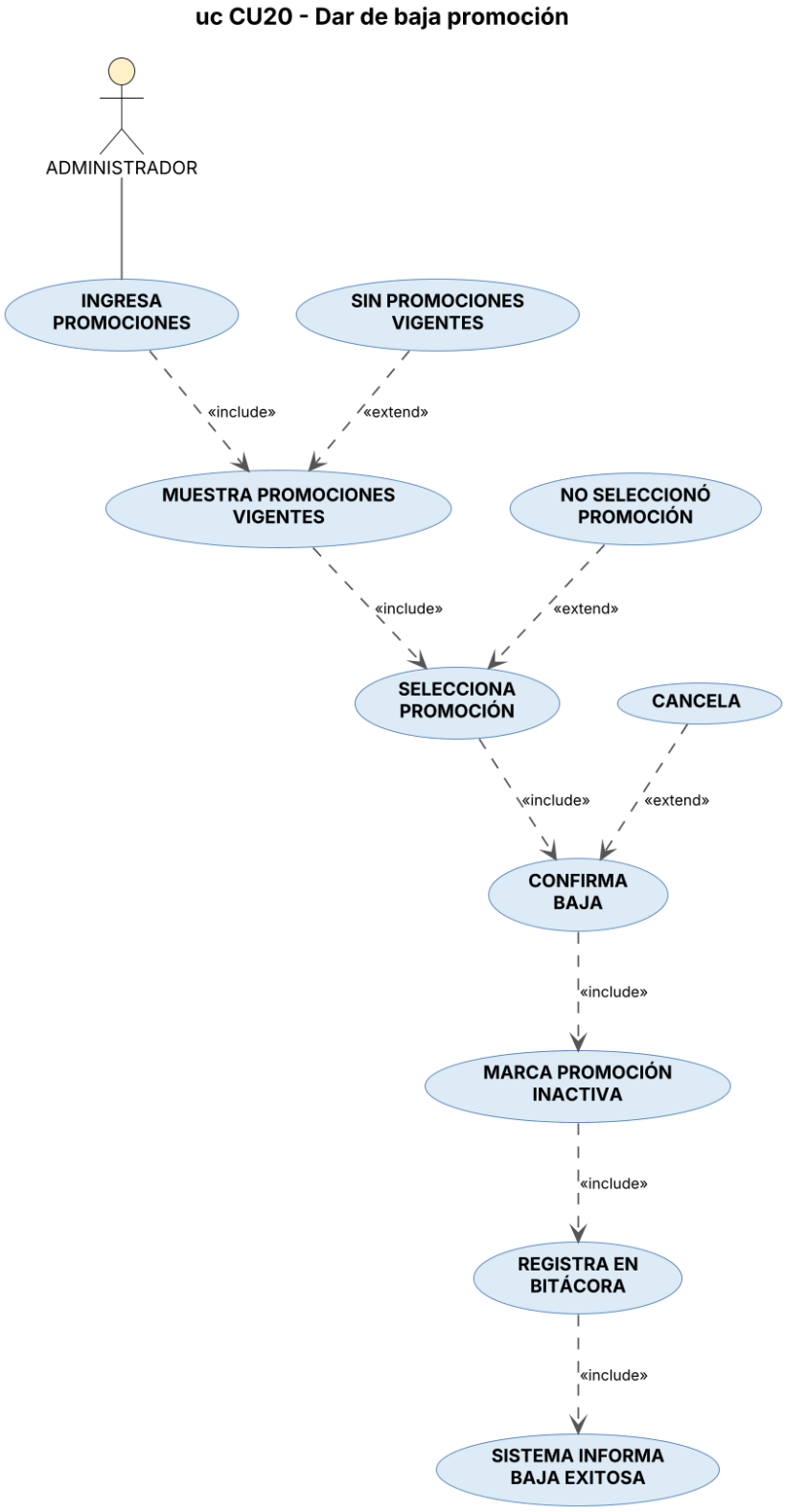


Diagrama de secuencia

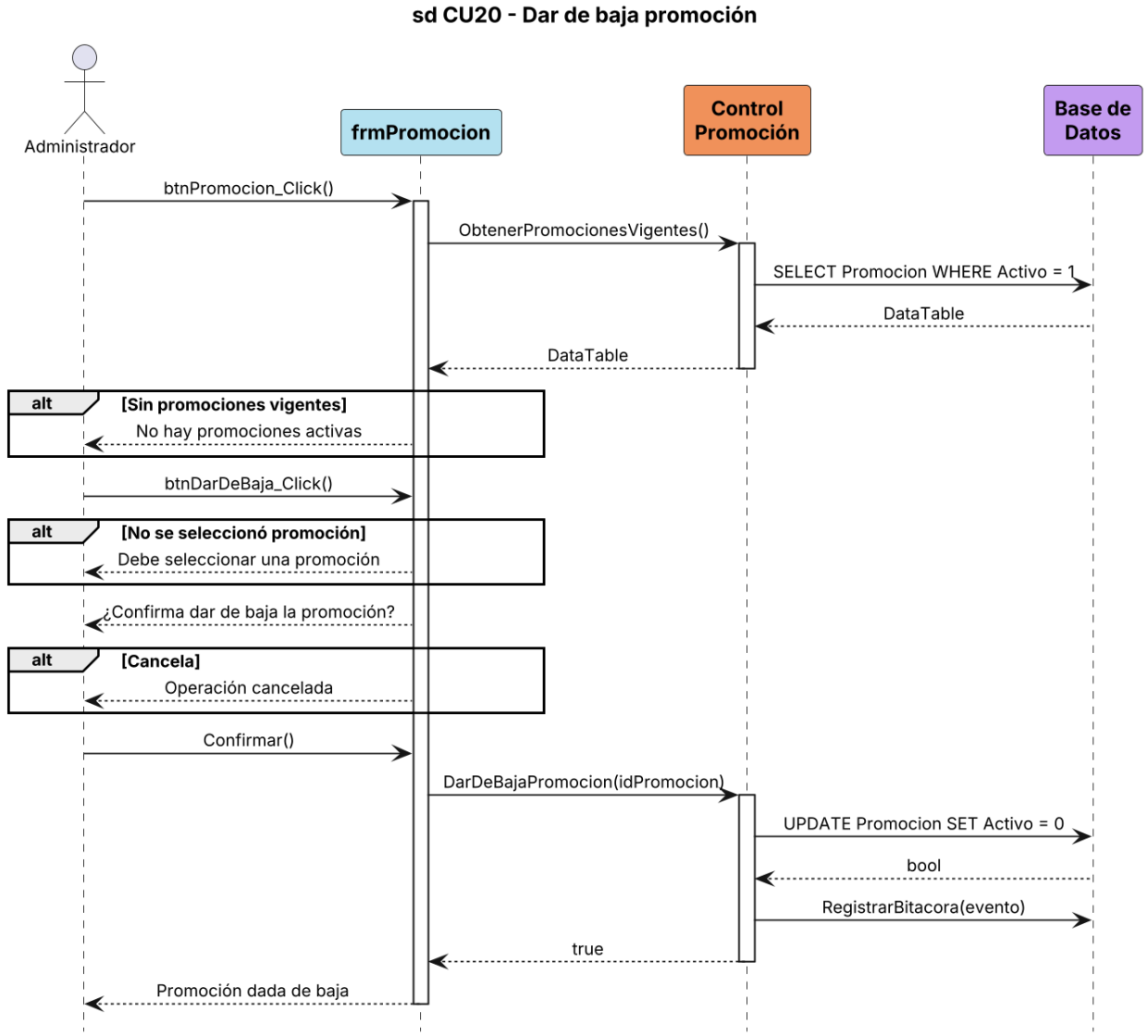
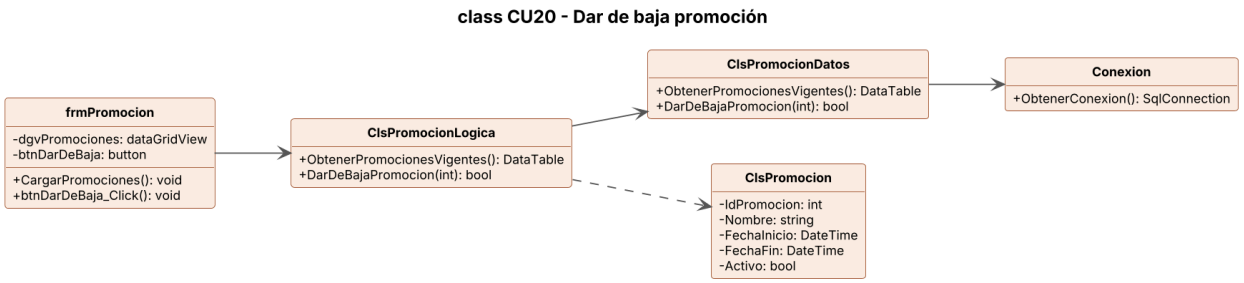


Diagrama de clases



CU21 - Registrar compra a proveedor

ID y Nombre	**CU21 Registrar compra a proveedor**	
Descripción	Permite al dueño o empleado de compras registrar la mercadería recibida de un proveedor, cargando el comprobante (tipo y número), los productos, las cantidades y los precios de compra sin IVA. El sistema calcula subtotal, IVA y total, actualiza el stock de cada producto y, si se indica, actualiza también el precio.	
Actores	Administrador - Empleado Compras (permiso 3.0.0 Compras).	
Pre-Condición	El usuario debe haber iniciado sesión y tener el permiso 3.0.0 Compras. El proveedor y los productos deben estar registrados.	
Post-Condición	La compra queda registrada con su detalle. El stock de cada producto se incrementa y se registran los movimientos de stock de tipo Ingreso. Si la compra es en cuenta corriente, queda pendiente de pago con el proveedor.	
FLUJO	**Flujo Normal**	**Flujo Alternativo**
	1. El usuario accede al módulo Compras y presiona "Nueva compra".	
	2. El usuario selecciona el proveedor (con autocompletado), el tipo de comprobante y carga el número de comprobante.	2.1 Proveedor no seleccionado. El sistema informa "Debe seleccionar un proveedor". 2.2 Comprobante incompleto. El sistema informa "Debe seleccionar un tipo de comprobante" o "Debe ingresar el número de comprobante".
	3. El usuario busca cada producto por código de barras o nombre y lo agrega a la grilla.	3.1 Producto no encontrado. El sistema informa que el producto no se encuentra registrado.
	4. El usuario ingresa la cantidad y el precio de compra sin IVA de cada producto.	4.1 Datos incorrectos. El sistema informa si la cantidad es menor o igual a cero o el precio es negativo.
	5. El sistema calcula el subtotal sin IVA, el IVA y el total de la compra.	
	6. El usuario indica si desea actualizar el precio de los productos y presiona "Guardar".	6.1 Sin productos. El sistema informa "Debe agregar al menos un producto a la compra". 6.2 Quitar producto. El usuario elimina una fila de la grilla y el sistema recalcula los totales.
	7. El sistema registra la compra y su detalle.	

	8. El sistema actualiza el stock de los productos, registra los movimientos de stock y, si corresponde, actualiza los precios.	
	9. El sistema registra la operación en la bitácora.	
	10. El sistema informa que la compra se registró correctamente.	

Diagrama de caso de uso

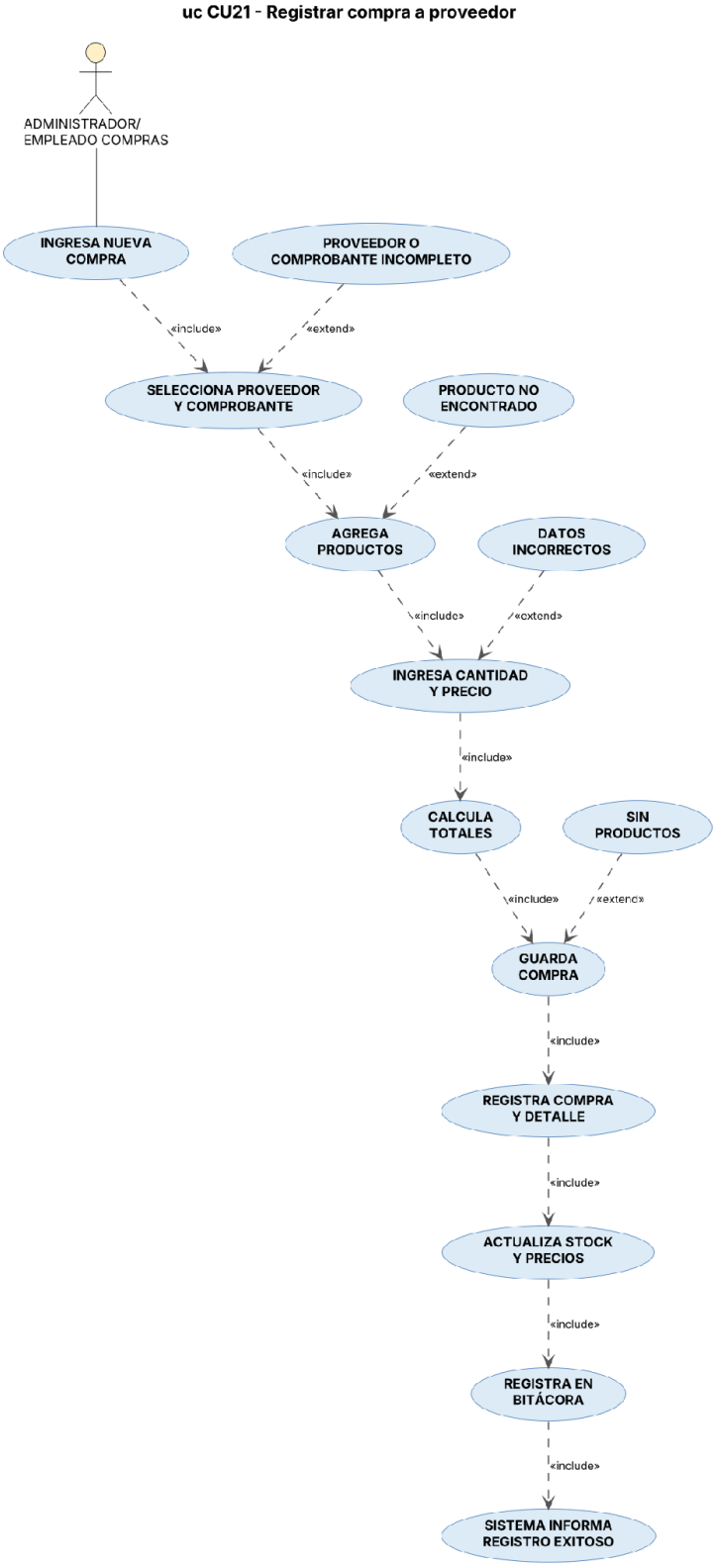


Diagrama de secuencia

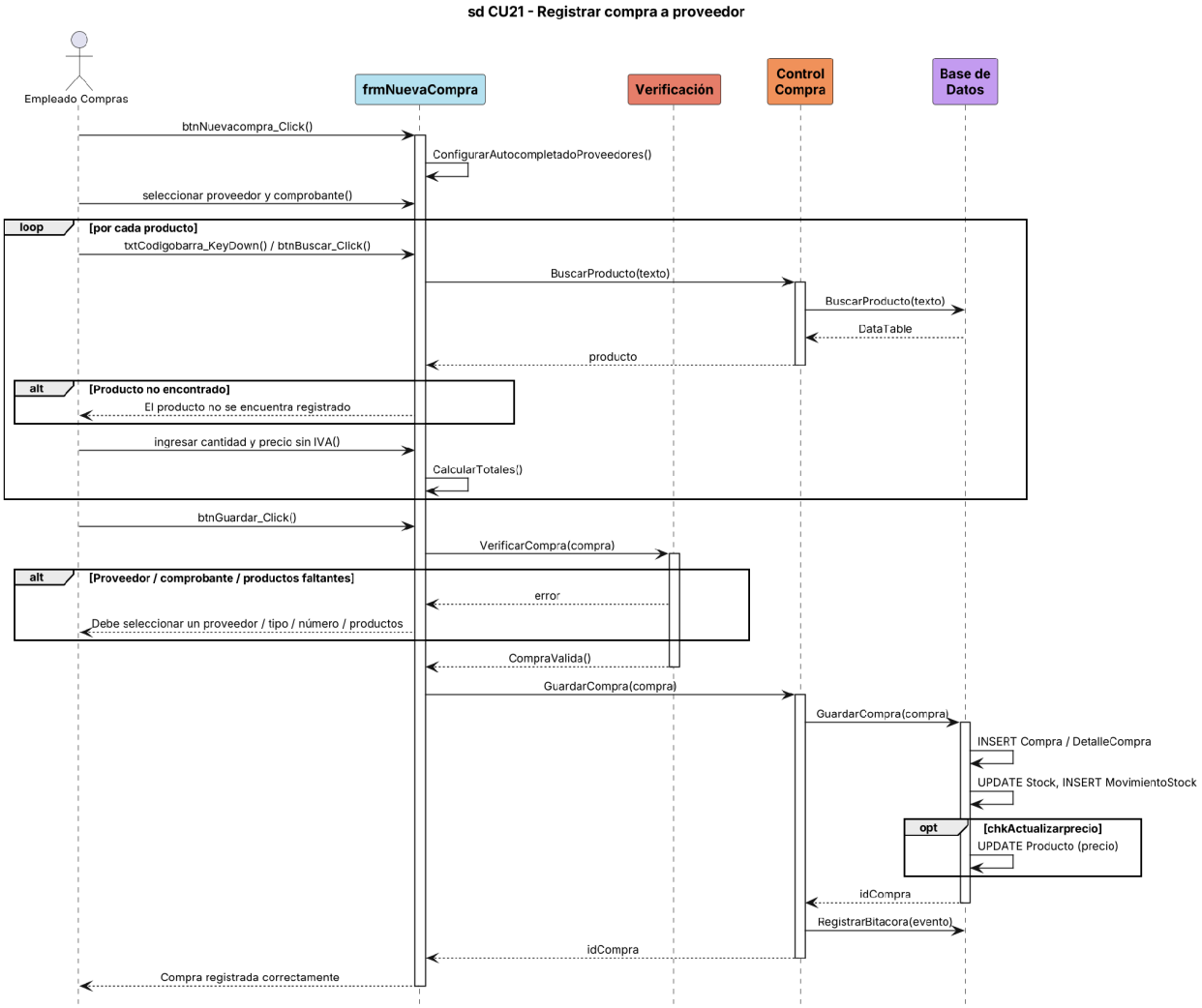
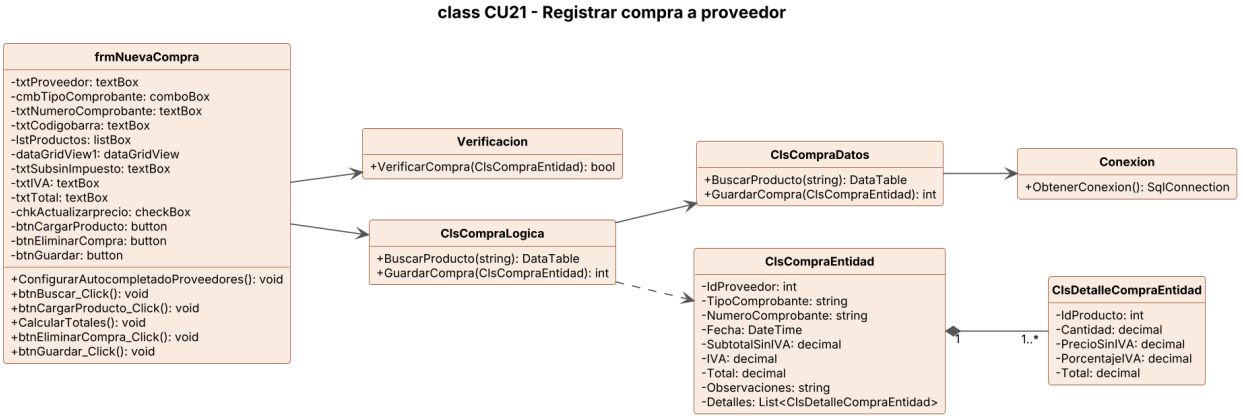


Diagrama de clases



CU22 - Consultar / anular compra

ID y Nombre	**CU22 Consultar / anular compra**	
Descripción	Permite al dueño o empleado de compras consultar las compras registradas en un rango de fechas, ver el detalle de cada una y anular una compra cargada por error indicando obligatoriamente el motivo. Al anular, el sistema descuenta del stock la mercadería que había ingresado con esa compra.	
Actores	Administrador - Empleado Compras (permiso 3.0.0 Compras).	
Pre-Condición	El usuario debe haber iniciado sesión y tener el permiso 3.0.0 Compras.	
Post-Condición	Consulta: se muestran las compras y su detalle, sin modificar datos. Anulación: la compra queda anulada con su motivo, se revierte el stock y la operación queda en la bitácora.	
FLUJO	**Flujo Normal**	**Flujo Alternativo**
	1. El usuario accede al módulo Compras.	
	2. El sistema muestra las compras del período actual.	
	3. El usuario ingresa el rango de fechas y presiona "Buscar".	3.1 Sin resultados. El sistema informa que no hay compras en el período.
	4. El usuario selecciona una compra y presiona "Detalle".	4.1 No se seleccionó compra. El sistema informa que debe seleccionar una compra.
	5. El sistema muestra el detalle: proveedor, comprobante, productos, cantidades, precios y totales.	
	6. El usuario cierra el detalle y finaliza la consulta.	6.1 Anular compra. 6.1.1 El usuario selecciona la compra y presiona "Anular". 6.1.2 El sistema solicita el motivo de la anulación. 6.1.3 Si el motivo está vacío, el sistema informa "Debe ingresar un motivo para anular la compra". 6.1.4 El sistema solicita confirmación; si el usuario cancela, vuelve al paso 2. 6.1.5 El sistema anula la compra, descuenta el stock ingresado y registra la operación en la bitácora. 6.1.6 El sistema informa que la compra fue anulada. 6.2 Compra ya anulada. El sistema informa que la compra ya se encuentra anulada.

Diagrama de caso de uso

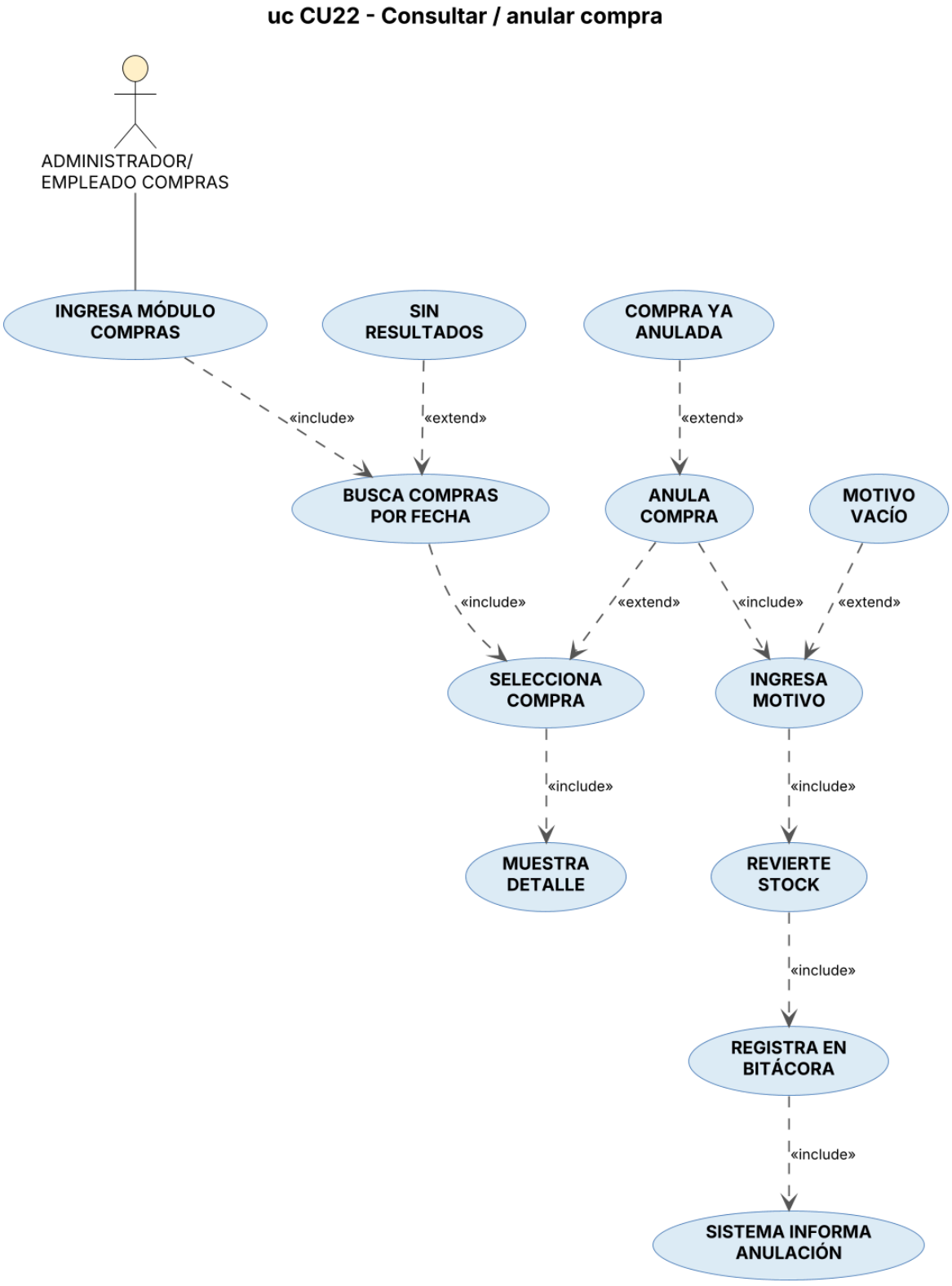


Diagrama de secuencia

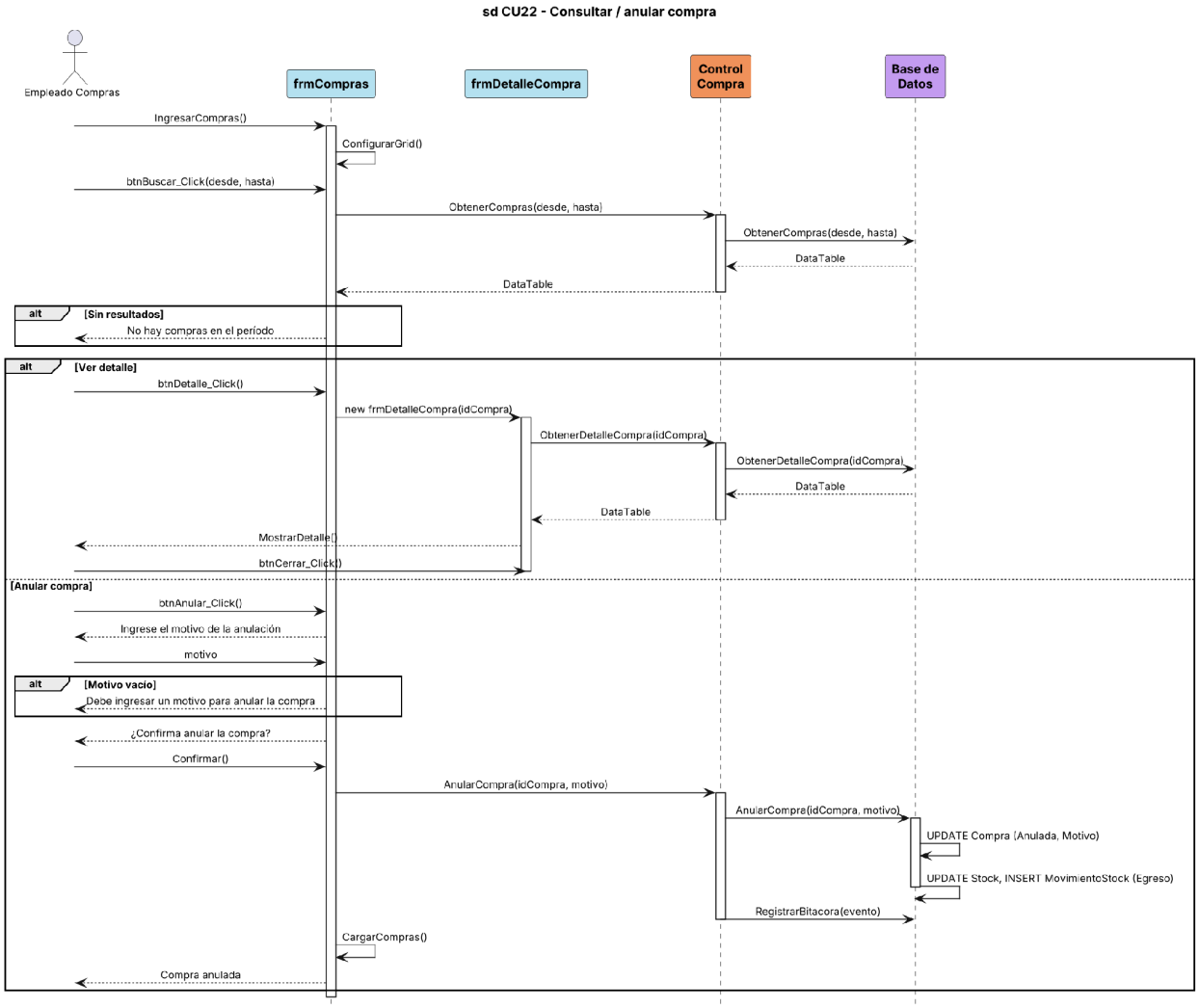
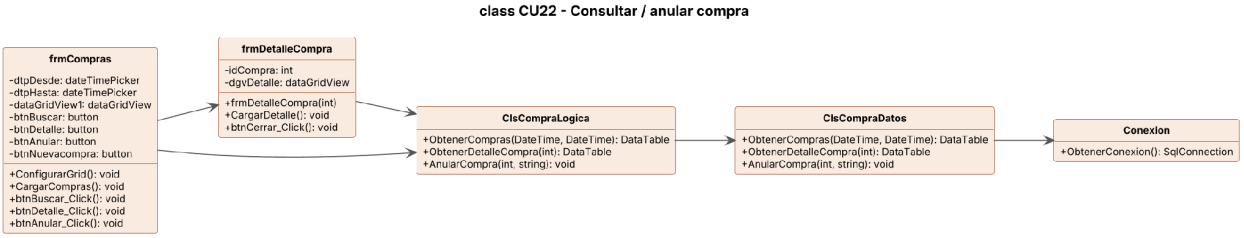


Diagrama de clases



CU23 - Generar pedido a proveedor

ID y Nombre	**CU23 Generar pedido a proveedor**	
Descripción	Permite al dueño o empleado de compras armar el pedido de mercadería para un proveedor. El sistema propone los productos de ese proveedor que están por debajo del punto de reposición, con la cantidad sugerida para llegar al stock ideal; el usuario ajusta las cantidades, agrega o quita productos y guarda el pedido, que puede exportar a Excel para enviarlo al proveedor.	
Actores	Administrador - Empleado Compras (permiso 3.0.0 Compras).	
Pre-Condición	El usuario debe haber iniciado sesión y tener el permiso 3.0.0 Compras. El proveedor debe estar registrado y tener productos asociados.	
Post-Condición	El pedido queda registrado en estado Pendiente con sus productos y cantidades. Opcionalmente se genera el archivo Excel del pedido.	
FLUJO	**Flujo Normal**	**Flujo Alternativo**
	1. El usuario accede al módulo Compras > Nuevo pedido.	
	2. El usuario selecciona el proveedor.	2.1 Proveedor sin productos. El sistema informa que el proveedor no tiene productos asociados.
	3. El sistema muestra los productos del proveedor que están por debajo del punto de reposición, con la cantidad sugerida (stock ideal menos stock actual).	3.1 Sin productos a reponer. El sistema informa que no hay productos por debajo del punto de reposición; el usuario puede agregarlos manualmente.
	4. El usuario ajusta las cantidades y agrega o quita productos.	
	5. El sistema valida el pedido.	5.1 Cantidades inválidas. El sistema informa que las cantidades deben ser mayores a cero. 5.2 Pedido vacío. El sistema informa que debe incluir al menos un producto.
	6. El usuario presiona "Guardar pedido".	6.1 Cancelar. El sistema descarta el pedido.
	7. El sistema registra el pedido en estado Pendiente.	
	8. El sistema informa que el pedido se generó correctamente.	8.1 Exportar pedido. El usuario presiona "Exportar a Excel" y el sistema genera el archivo para enviarlo al proveedor.

Diagrama de caso de uso

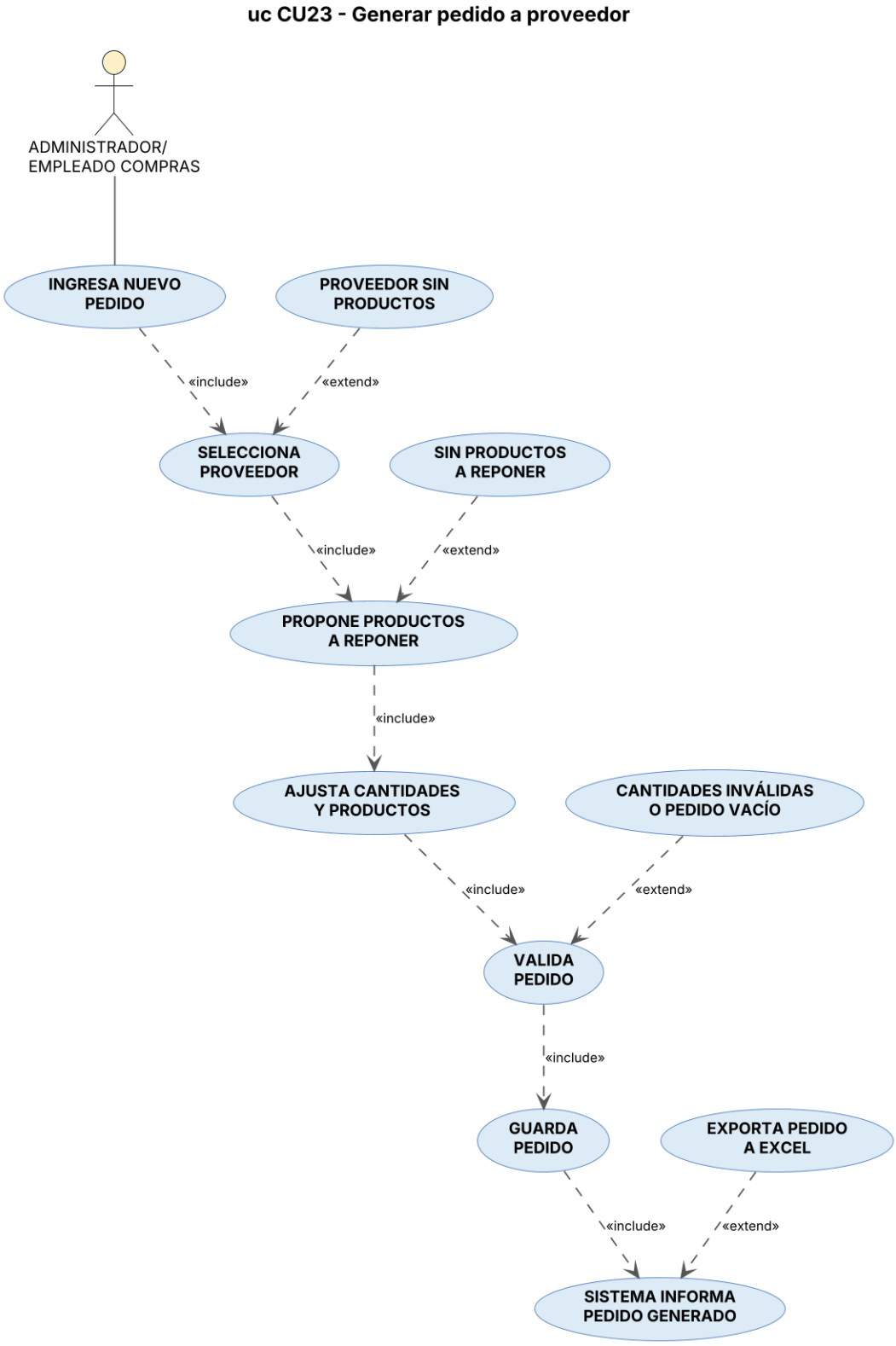


Diagrama de secuencia

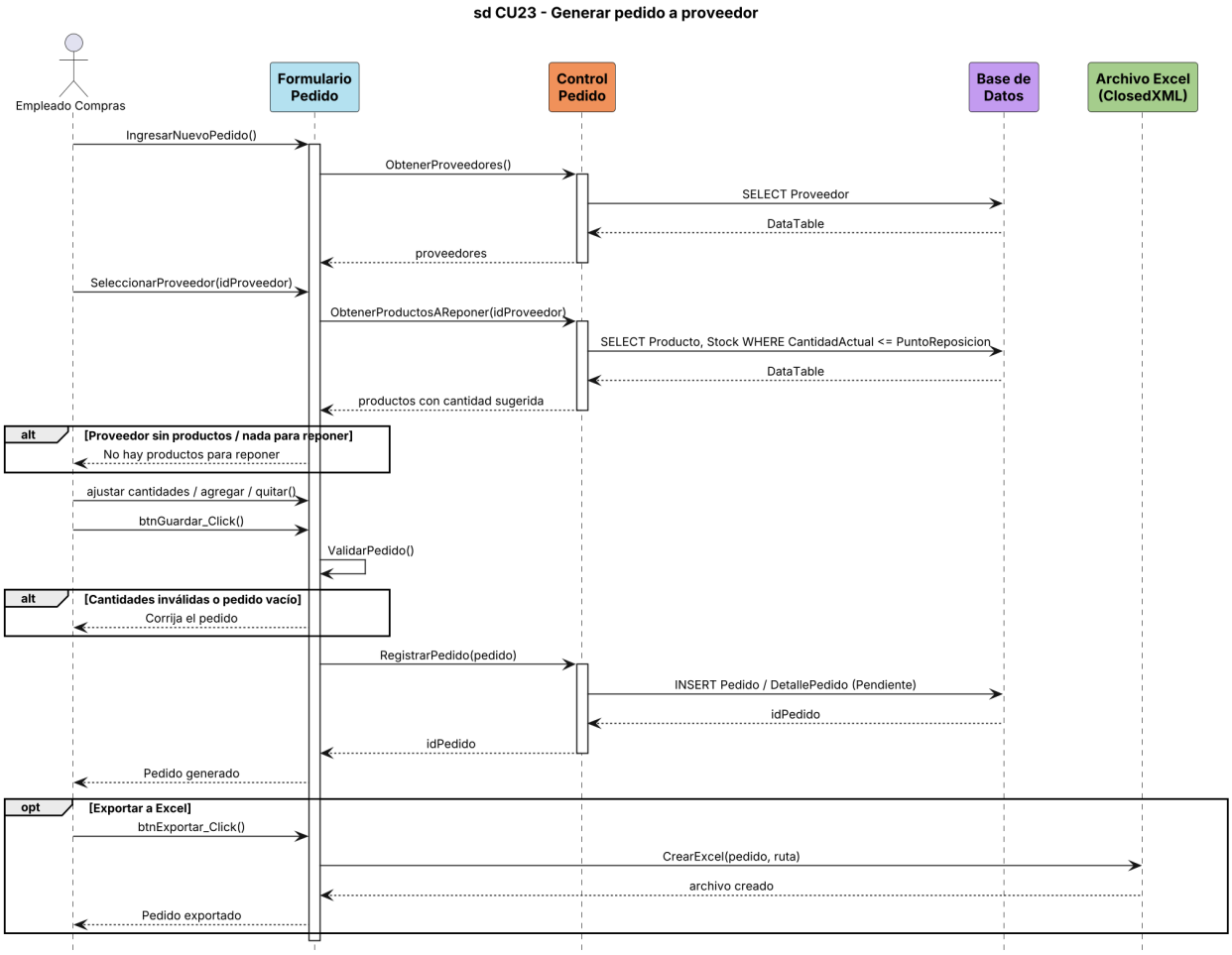
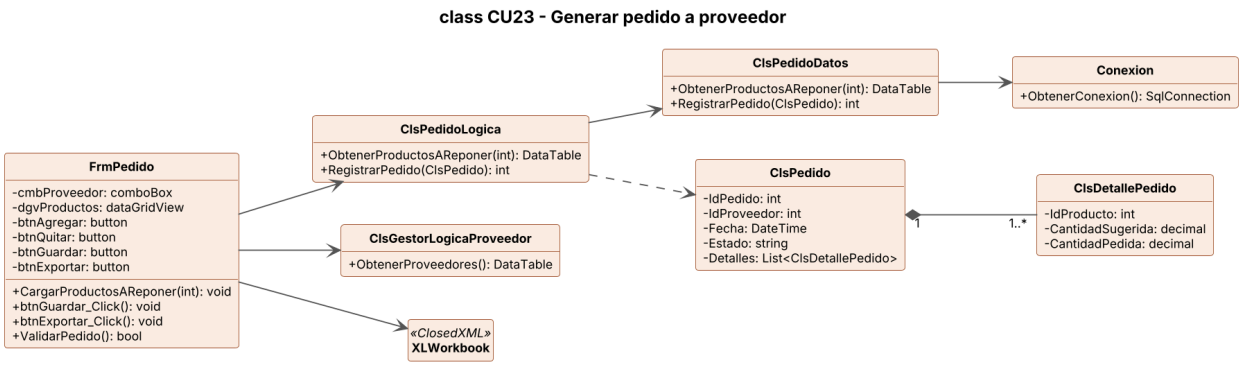


Diagrama de clases



CU24 - Modificar / dar de baja proveedor

ID y Nombre	CU24 Modificar / dar de baja proveedor	
Descripción	Permite al dueño o empleado autorizado modificar los datos de un proveedor ya registrado (razón social, CUIT, sucursales, teléfono, correo electrónico y dirección) o darlo de baja lógica. Un proveedor dado de baja no se puede elegir en compras nuevas, pero se conserva su historial de compras y su cuenta corriente.	
Actores	Administrador - Empleado autorizado (permiso 4.1.2 Baja/Modificación de Proveedores).	
Pre-Condición	El usuario debe haber iniciado sesión y tener el permiso 4.1.2. El proveedor debe estar registrado.	
Post-Condición	Los datos del proveedor quedan actualizados, o el proveedor queda inactivo. La operación queda registrada en la bitácora.	
FLUJO	**Flujo Normal**	**Flujo Alternativo**
	1. El usuario accede al módulo Proveedores > Administrador de Proveedores.	
	2. El usuario busca el proveedor por razón social o CUIT.	2.1 Proveedor no encontrado. El sistema informa que el proveedor no se encuentra registrado.
	3. El usuario selecciona el proveedor y el sistema muestra sus datos y sucursales.	
	4. El usuario elige modificar los datos.	4.1 Dar de baja proveedor. 4.1.1 El usuario presiona "Dar de baja". 4.1.2 Si el proveedor tiene saldo pendiente en cuenta corriente, el sistema lo informa y no permite la baja. 4.1.3 El sistema solicita confirmación; si el usuario cancela, vuelve al paso 3. 4.1.4 El sistema marca el proveedor como inactivo; continúa en el paso 7.
	5. El usuario modifica los datos necesarios.	
	6. El sistema valida los datos ingresados.	6.1 Campos vacíos. El sistema informa los campos obligatorios sin completar. 6.2 Datos incorrectos. El sistema informa si el CUIT o el correo no tienen un formato válido. 6.3 CUIT duplicado. El sistema informa que el CUIT pertenece a otro proveedor.
	7. El usuario confirma y el sistema actualiza el proveedor en la base de datos.	7.1 Cancelar. El sistema descarta los cambios y vuelve al paso 3.

	8. El sistema registra la operación en la bitácora.	
	9. El sistema informa que la operación se realizó con éxito.	

Diagrama de caso de uso

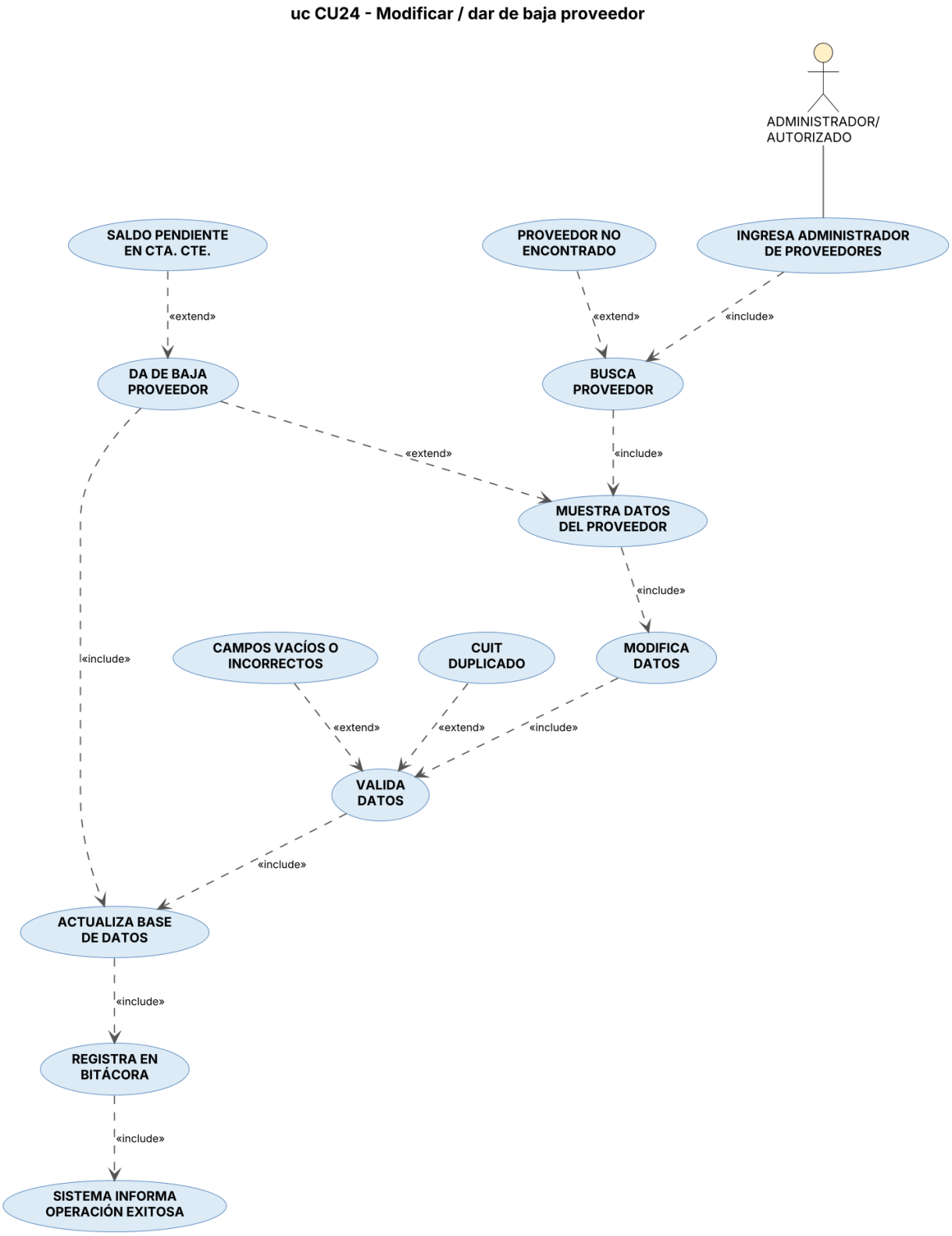


Diagrama de secuencia

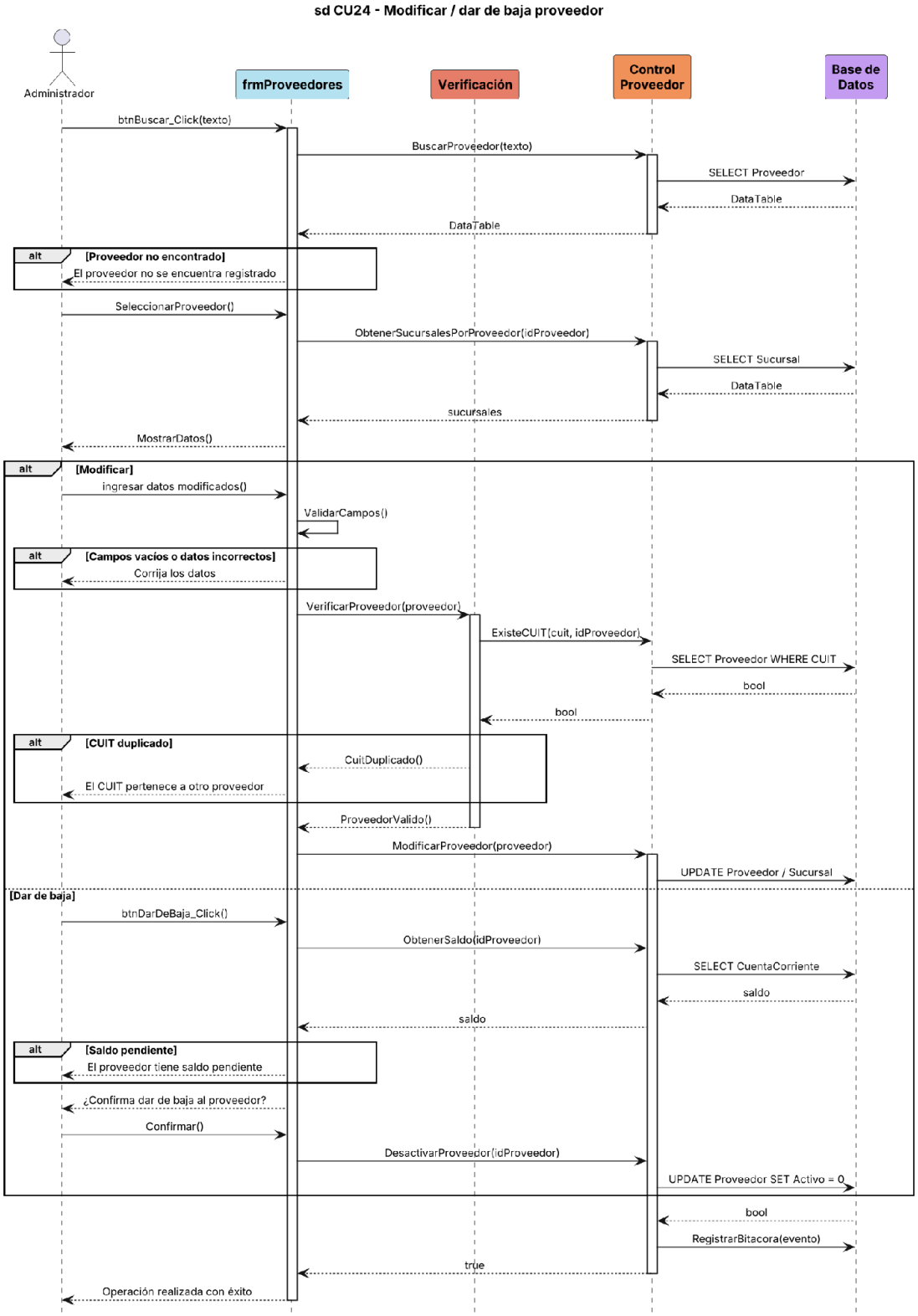
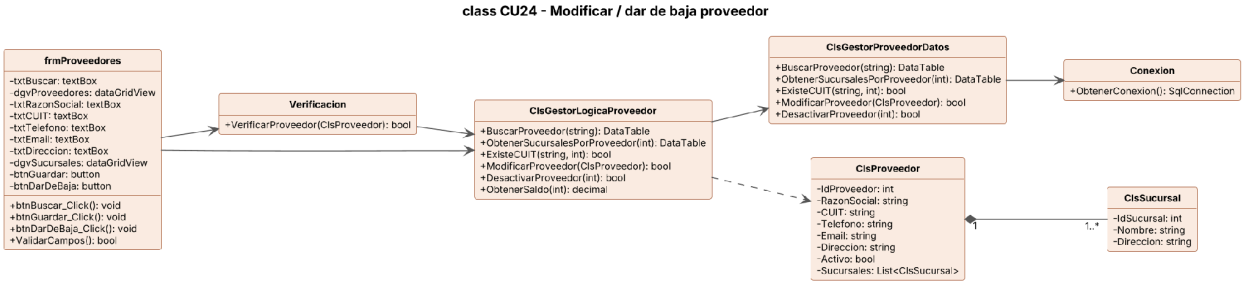


Diagrama de clases



CU25 - Registrar pago a proveedor (cuenta corriente)

ID y Nombre	**CU25 Registrar pago a proveedor (cuenta corriente)**	
Descripción	Permite al dueño o empleado de compras consultar la cuenta corriente de un proveedor (compras pendientes de pago, pagos realizados y saldo) y registrar un pago total o parcial, indicando el monto, el medio de pago y las compras que cancela. Si el pago es en efectivo, el sistema registra también el egreso de caja.	
Actores	Administrador - Empleado Compras (permiso 4.2.0 Cuenta Corriente de Proveedores).	
Pre-Condición	El usuario debe haber iniciado sesión y tener el permiso 4.2.0. El proveedor debe tener compras registradas en cuenta corriente. Para pagos en efectivo, la caja debe estar abierta.	
Post-Condición	El pago queda registrado y el saldo de la cuenta corriente del proveedor se actualiza. Si el pago es en efectivo, se registra el egreso de caja. La operación queda registrada en la bitácora.	
FLUJO	**Flujo Normal**	**Flujo Alternativo**
	1. El usuario accede al módulo Proveedores > Cuenta Corriente de Proveedores.	
	2. El usuario selecciona el proveedor.	
	3. El sistema muestra las compras pendientes, los pagos realizados y el saldo actual.	3.1 Sin saldo pendiente. El sistema informa que el proveedor no tiene deuda.
	4. El usuario presiona "Registrar pago" y selecciona las compras que cancela.	
	5. El usuario ingresa el monto, la fecha y el medio de pago.	
	6. El sistema valida los datos ingresados.	6.1 Monto inválido. El sistema informa que el monto debe ser mayor a cero. 6.2 Monto mayor al saldo. El sistema informa que el monto supera el saldo pendiente. 6.3 Caja cerrada. Si el pago es en efectivo y la caja no está abierta, el sistema lo informa.
	7. El usuario confirma el pago.	7.1 Cancelar. El sistema descarta el pago y vuelve al paso 3.
	8. El sistema registra el pago, actualiza el saldo y marca como pagadas las compras canceladas por completo.	

	9. Si el pago es en efectivo, el sistema registra el egreso en la caja.	
	10. El sistema registra la operación en la bitácora e informa que el pago se registró correctamente.	

Diagrama de caso de uso

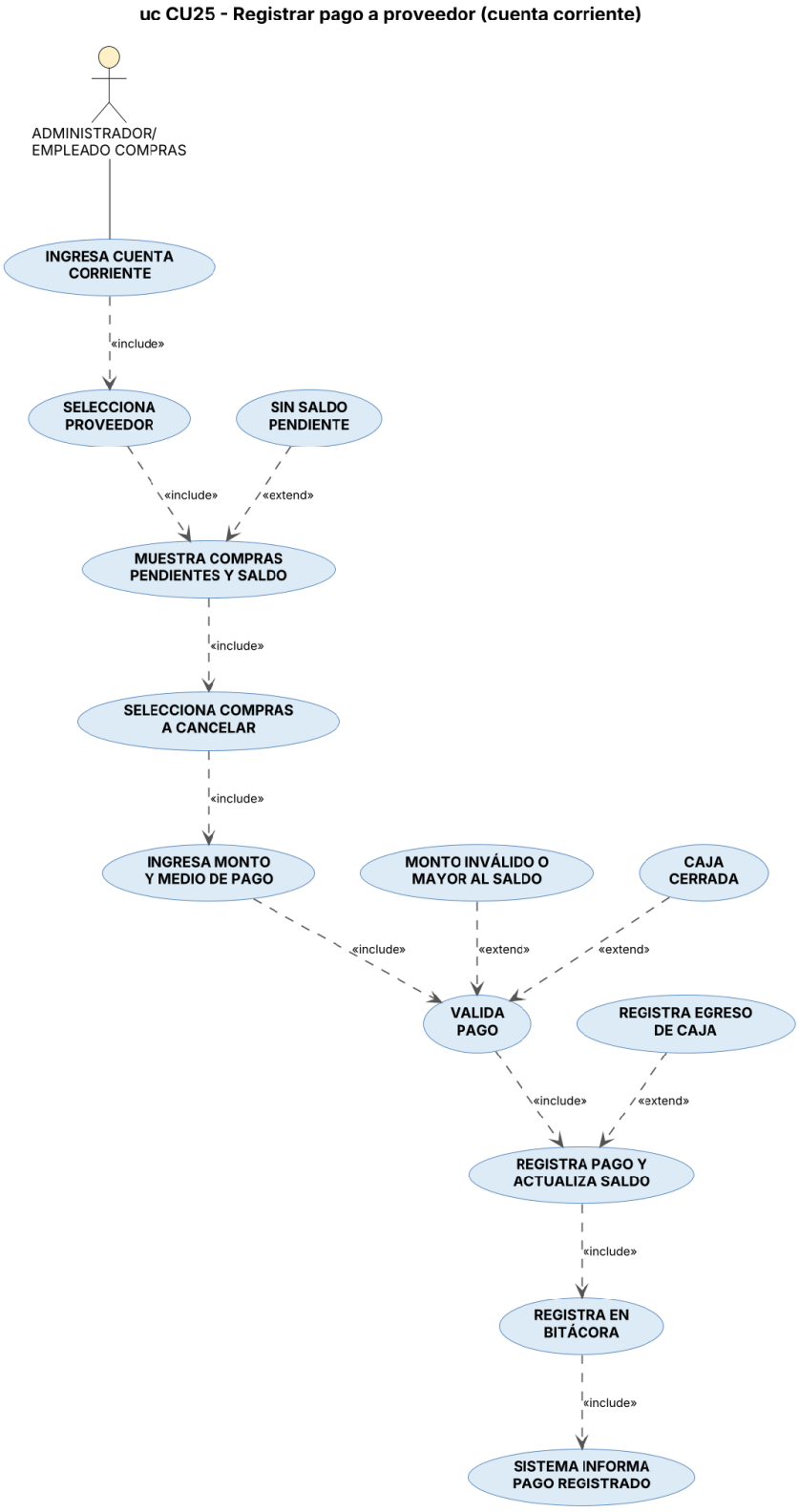


Diagrama de secuencia

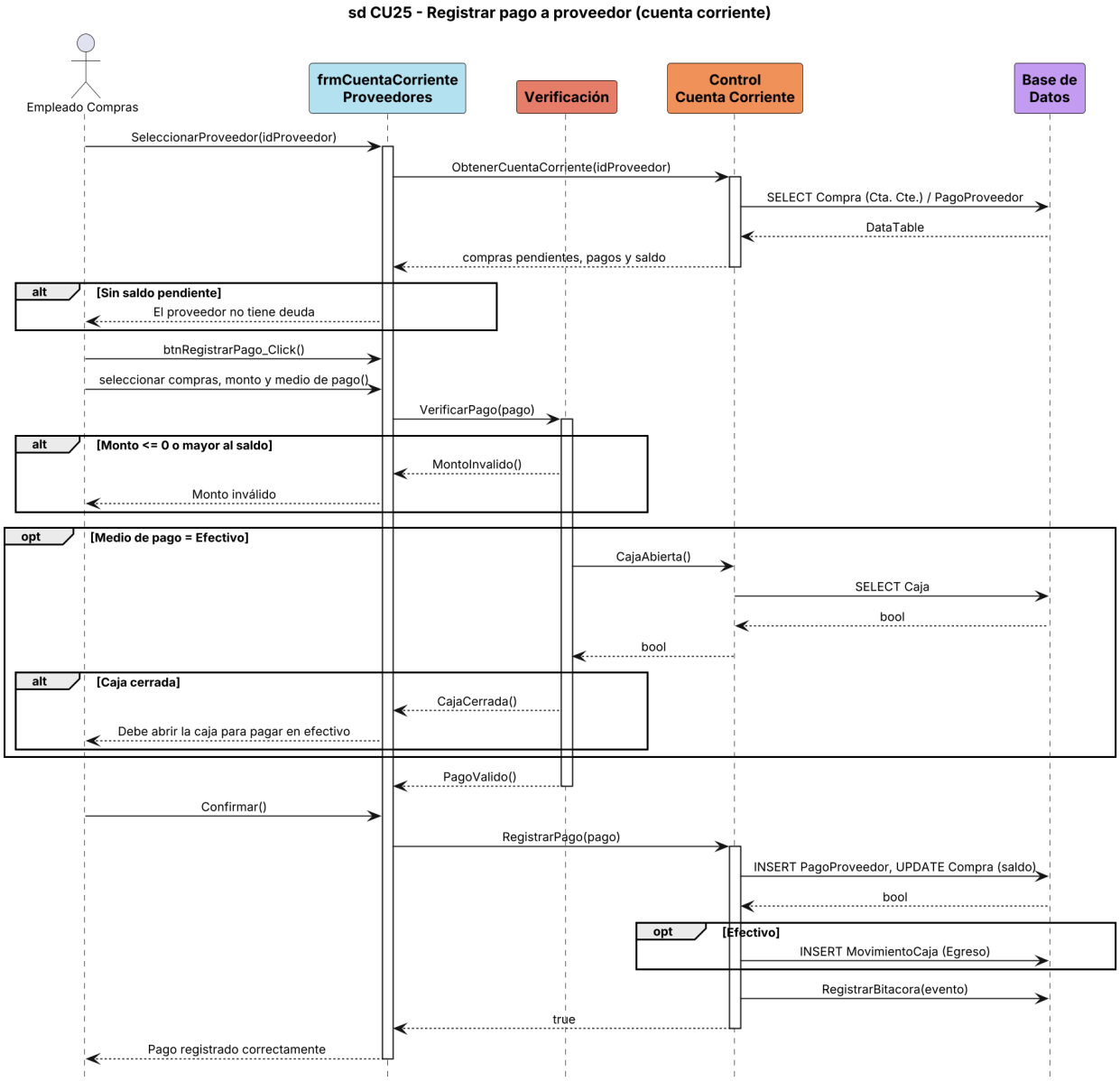
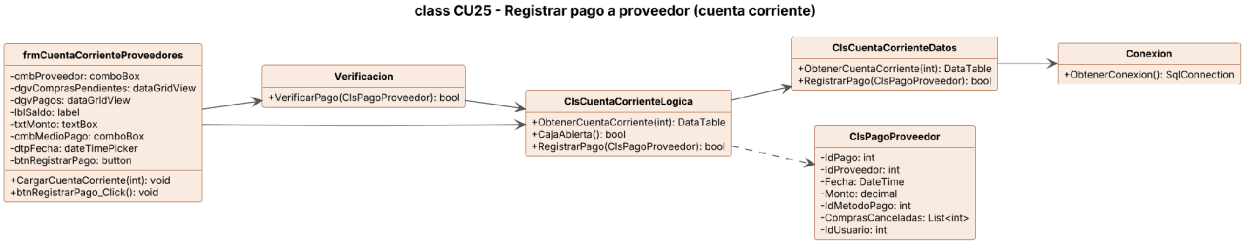


Diagrama de clases



CU26 - Consultar listado de precios

ID y Nombre	**CU26 Consultar listado de precios**	
Descripción	Permite a cualquier empleado con permiso consultar el listado de precios de venta de los productos activos, filtrarlo por rubro, marca o texto y exportarlo a Excel o imprimirlo para tenerlo en el mostrador.	
Actores	Administrador - Empleado General - Empleado Ventas (permiso 5.1.0 Listado de Precios).	
Pre-Condición	El usuario debe haber iniciado sesión y tener el permiso 5.1.0 Listado de Precios.	
Post-Condición	Se muestra el listado de precios filtrado. Opcionalmente se genera el archivo Excel o la impresión. No se modifican datos.	
FLUJO	**Flujo Normal**	**Flujo Alternativo**
	1. El usuario accede al módulo Listados > Listado de Precios.	
	2. El sistema muestra los productos activos con SKU, nombre, marca, rubro y precio de venta.	
	3. El usuario filtra por rubro, marca o texto.	3.1 Sin resultados. El sistema informa que no hay productos para los filtros elegidos.
	4. El sistema actualiza el listado según los filtros.	
	5. El usuario presiona "Exportar a Excel".	5.1 Imprimir. El usuario presiona "Imprimir" y el sistema envía el listado a la impresora. 5.2 Error al exportar. El sistema informa que no pudo generar el archivo.
	6. El sistema genera el archivo y informa que se exportó correctamente.	

Diagrama de caso de uso

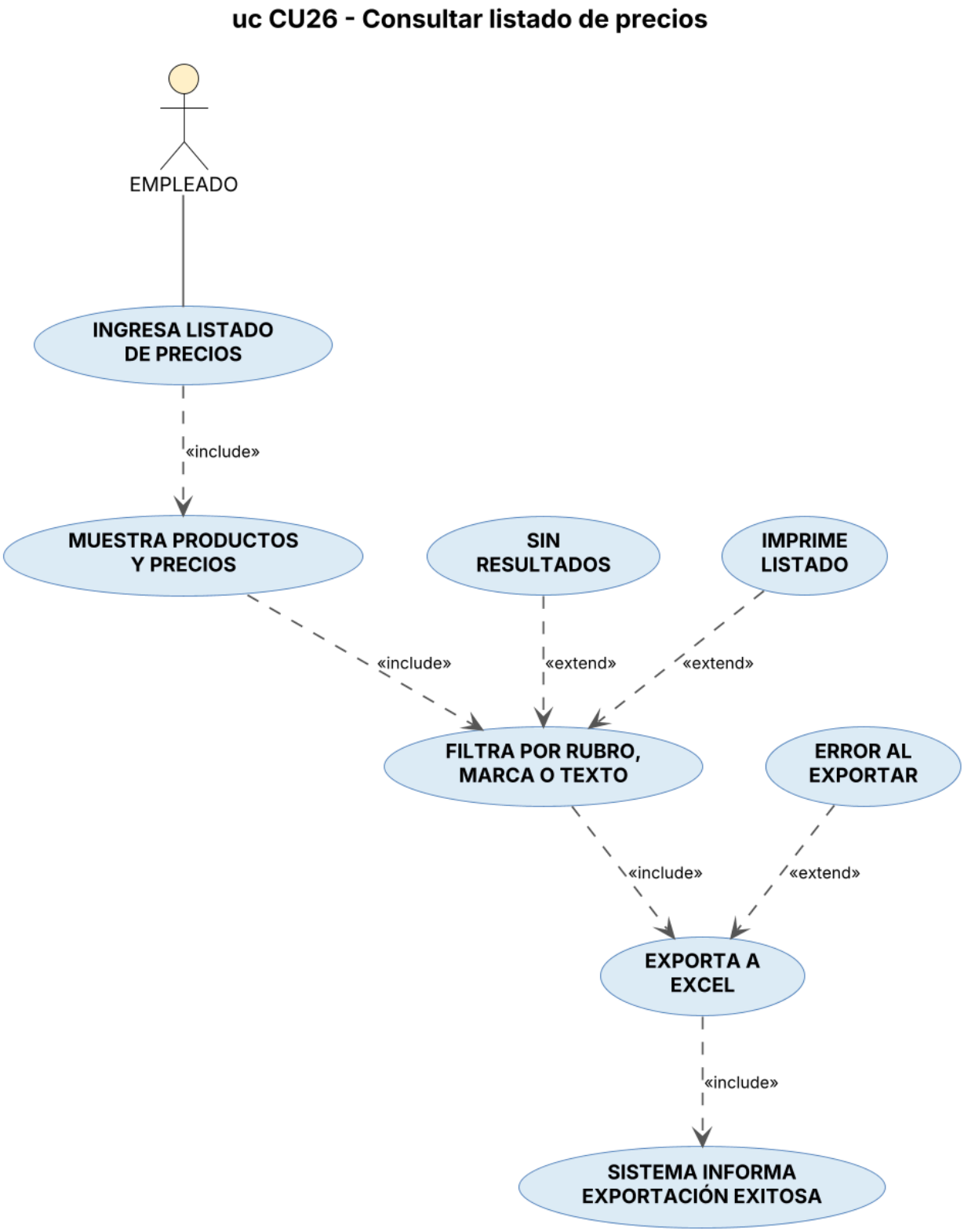


Diagrama de secuencia

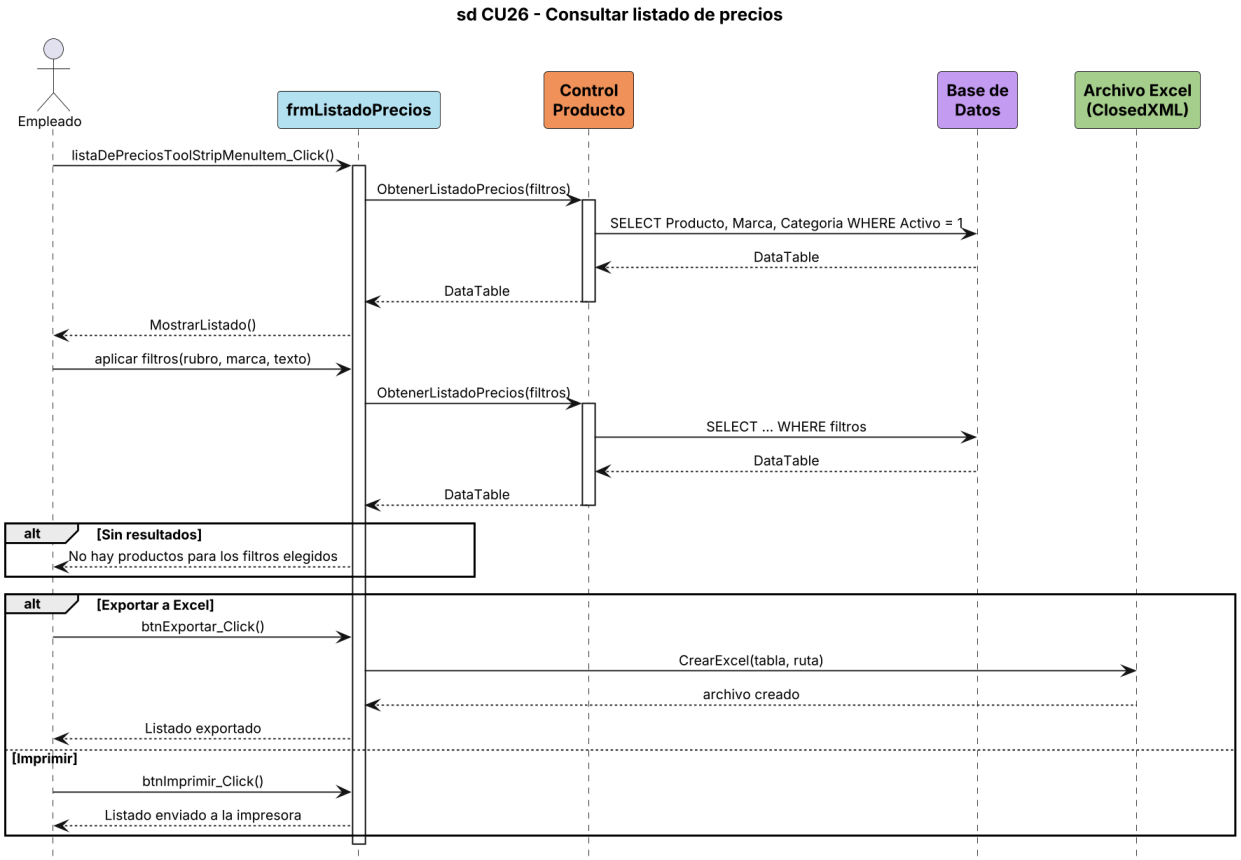
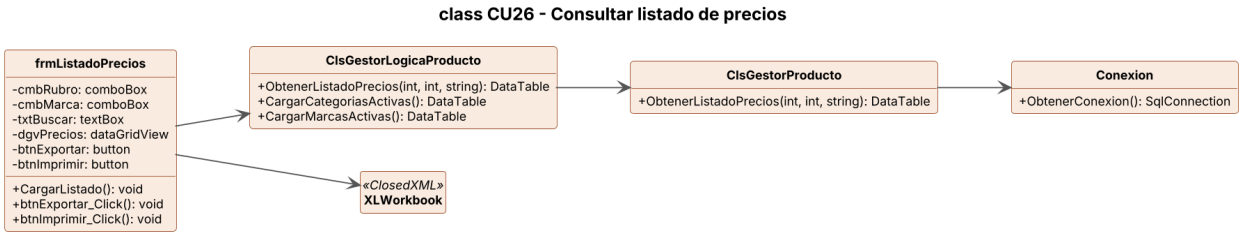


Diagrama de clases



CU27 - Consultar listado de reposición

ID y Nombre	CU27 Consultar listado de reposición	
Descripción	Permite al dueño o empleado de compras ver los productos cuyo stock actual está en o por debajo del punto de reposición, con la cantidad sugerida para llegar al stock ideal, agrupados por proveedor. Desde el listado puede exportarlo a Excel o generar un pedido al proveedor (CU23).	
Actores	Administrador - Empleado Compras (permiso 5.2.0 Listado de Reposición).	
Pre-Condición	El usuario debe haber iniciado sesión y tener el permiso 5.2.0 Listado de Reposición. Los productos deben tener configurado el stock mínimo y el stock ideal.	
Post-Condición	Se muestra el listado de productos a reponer. Opcionalmente se exporta o se genera un pedido. No se modifican datos del stock.	
FLUJO	**Flujo Normal**	**Flujo Alternativo**
	1. El usuario accede al módulo Listados > Listado de Reposición.	
	2. El sistema calcula los productos con stock actual menor o igual al punto de reposición y la cantidad sugerida (stock ideal menos stock actual).	2.1 Sin productos a reponer. El sistema informa que todos los productos tienen stock suficiente.
	3. El sistema muestra el listado agrupado por proveedor, resaltando los productos en stock crítico.	
	4. El usuario filtra por proveedor o rubro.	
	5. El usuario presiona "Exportar a Excel".	5.1 Generar pedido. El usuario selecciona un proveedor y presiona "Generar pedido"; el sistema abre el CU23 Generar pedido a proveedor con los productos cargados. 5.2 Error al exportar. El sistema informa que no pudo generar el archivo.
	6. El sistema genera el archivo y informa que se exportó correctamente.	

Diagrama de caso de uso

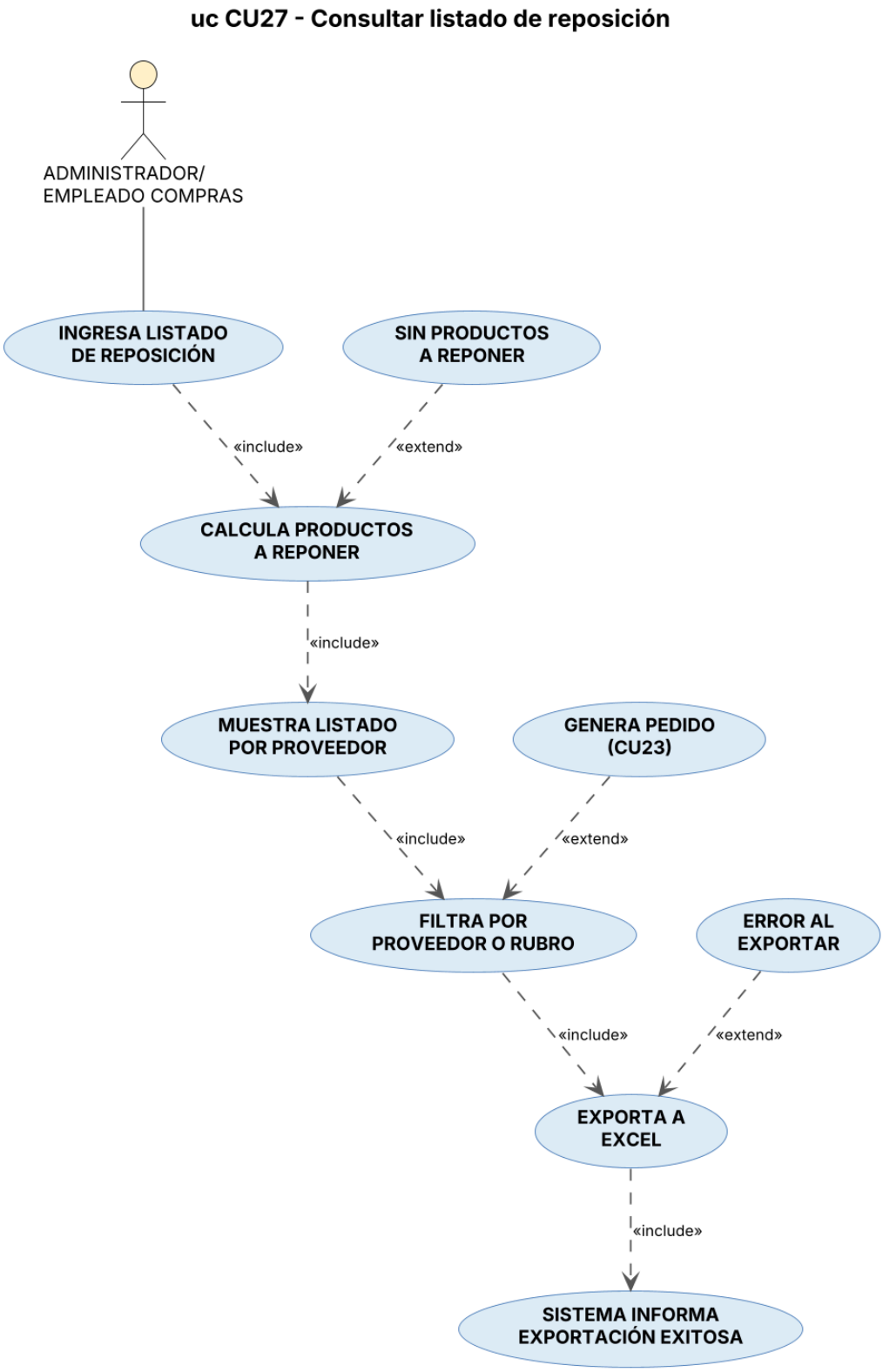


Diagrama de secuencia

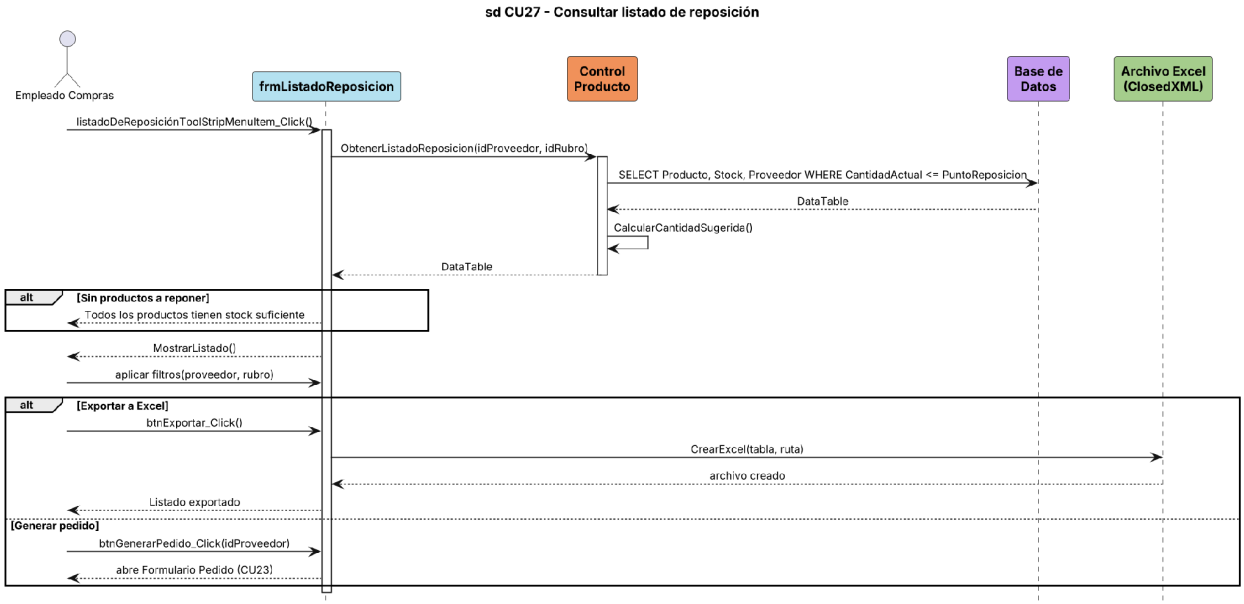
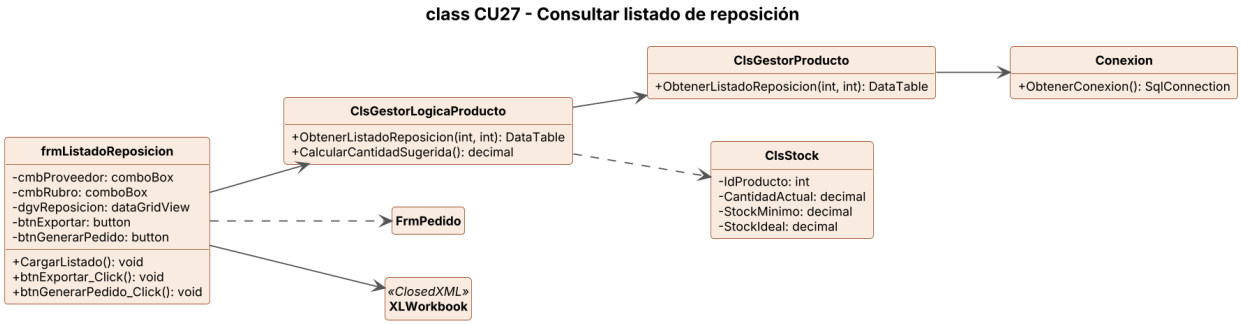


Diagrama de clases



CU28 - Generar informe de ventas

ID y Nombre	**CU28 Generar informe de ventas**	
Descripción	Permite al Administrador generar informes de ventas para un rango de fechas en cinco variantes: ventas por fecha, ventas con detalle por fecha, ventas por rubro, ventas por artículo y ventas por medio de pago. El informe se muestra en pantalla con sus totales y puede exportarse a Excel.	
Actores	Administrador (permisos 6.1.0 a 6.5.0 Informes).	
Pre-Condición	El usuario debe haber iniciado sesión y tener el permiso del informe elegido.	
Post-Condición	Se muestra el informe con sus totales. Opcionalmente se genera el archivo Excel. No se modifican datos.	
FLUJO	**Flujo Normal**	**Flujo Alternativo**
	1. El Administrador accede al módulo Informes y elige el tipo de informe: por fecha, con detalle por fecha, por rubro, por artículo o por medio de pago.	
	2. El sistema abre el formulario de informe con el tipo seleccionado.	
	3. El Administrador ingresa el rango de fechas y presiona "Generar".	3.1 Rango de fechas inválido. El sistema informa que la fecha "desde" no puede ser posterior a la fecha "hasta".
	4. El sistema obtiene las ventas del período agrupadas según el tipo de informe.	4.1 Sin resultados. El sistema informa que no hay ventas en el período.
	5. El sistema muestra el informe con cantidades, importes y el total general.	
	6. El Administrador presiona "Exportar a Excel".	6.1 No exportar. El Administrador cierra el informe. 6.2 Error al exportar. El sistema informa que no pudo generar el archivo.
	7. El sistema genera el archivo y informa que se exportó correctamente.	

Diagrama de caso de uso

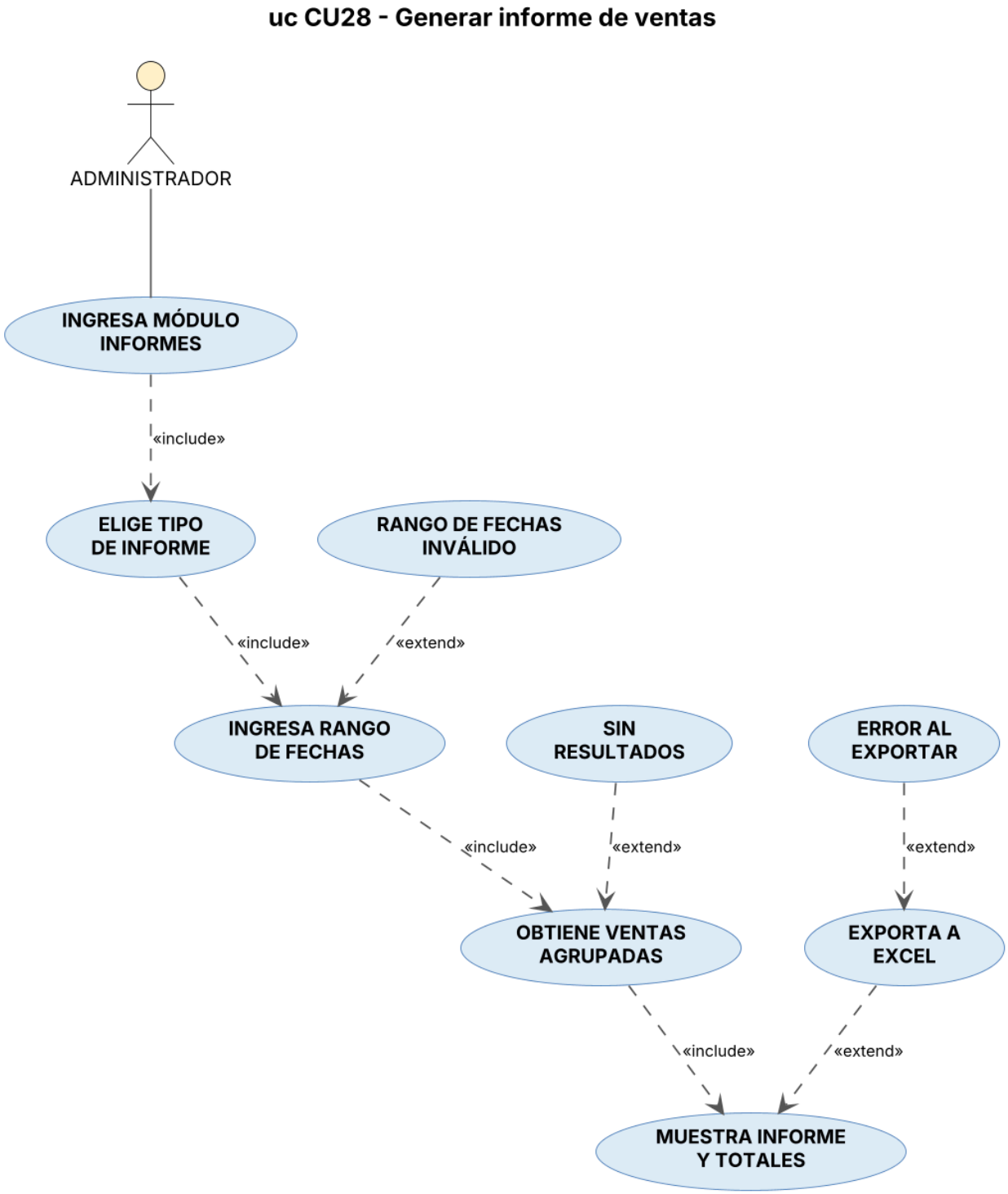


Diagrama de secuencia

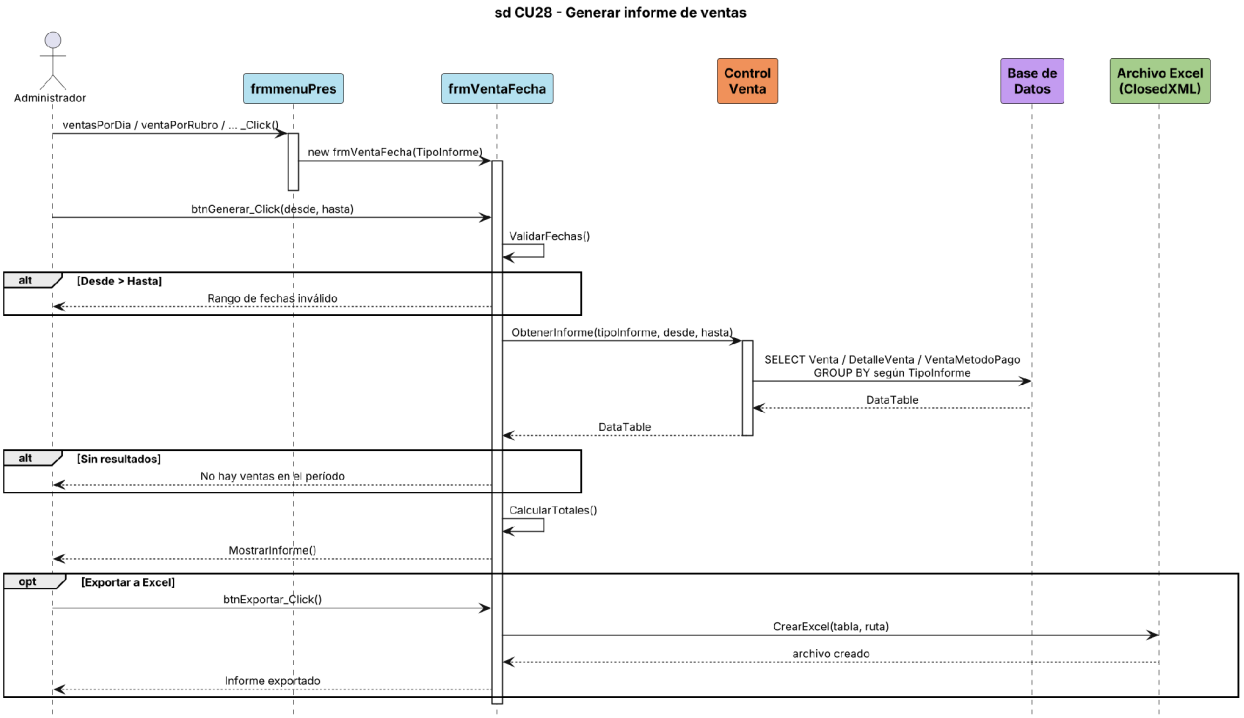
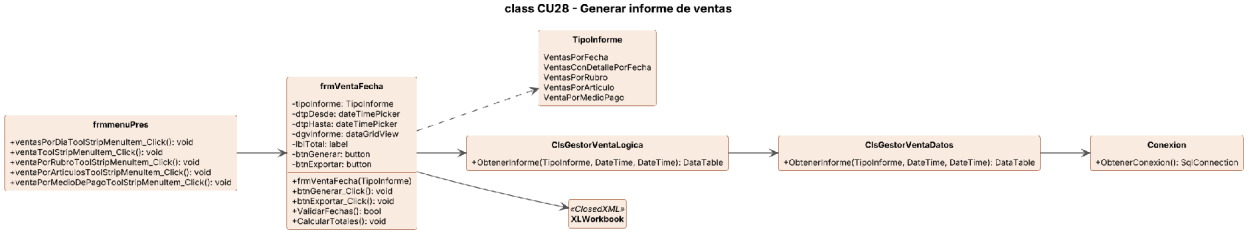


Diagrama de clases



CU29 - Generar balance de ingresos y egresos

ID y Nombre	CU29 Generar balance de ingresos y egresos	
Descripción	Permite al Administrador obtener el balance del kiosco para un período (diario, semanal o mensual): ingresos por ventas y por ingresos manuales de caja, egresos por compras pagadas, pagos a proveedores, devoluciones y egresos manuales, ganancia bruta (precio de venta menos costo de compra) y resultado final. El balance puede exportarse a Excel.	
Actores	Administrador (permiso 6.0.0 Informes).	
Pre-Condición	El usuario debe haber iniciado sesión y tener el permiso 6.0.0 Informes.	
Post-Condición	Se muestra el balance del período con el detalle de ingresos, egresos, ganancia y resultado. Opcionalmente se genera el archivo Excel. No se modifican datos.	
FLUJO	**Flujo Normal**	**Flujo Alternativo**
	1. El Administrador accede al módulo Informes > Balance.	
	2. El Administrador elige el período: diario, semanal, mensual o un rango de fechas.	2.1 Rango de fechas inválido. El sistema informa que la fecha "desde" no puede ser posterior a la fecha "hasta".
	3. El Administrador presiona "Generar".	
	4. El sistema obtiene los ingresos del período: ventas (por medio de pago) e ingresos manuales de caja.	4.1 Sin movimientos. El sistema informa que no hay movimientos en el período.
	5. El sistema obtiene los egresos del período: pagos a proveedores, devoluciones y egresos manuales de caja.	
	6. El sistema calcula la ganancia bruta de lo vendido (precio de venta menos costo de compra) y el resultado del período (ingresos menos egresos).	
	7. El sistema muestra el balance con el detalle de cada concepto y los totales.	
	8. El Administrador presiona "Exportar a Excel".	8.1 No exportar. El Administrador cierra el balance. 8.2 Error al exportar. El sistema informa que no pudo generar el archivo.
	9. El sistema genera el archivo y informa que se exportó correctamente.	

Diagrama de caso de uso

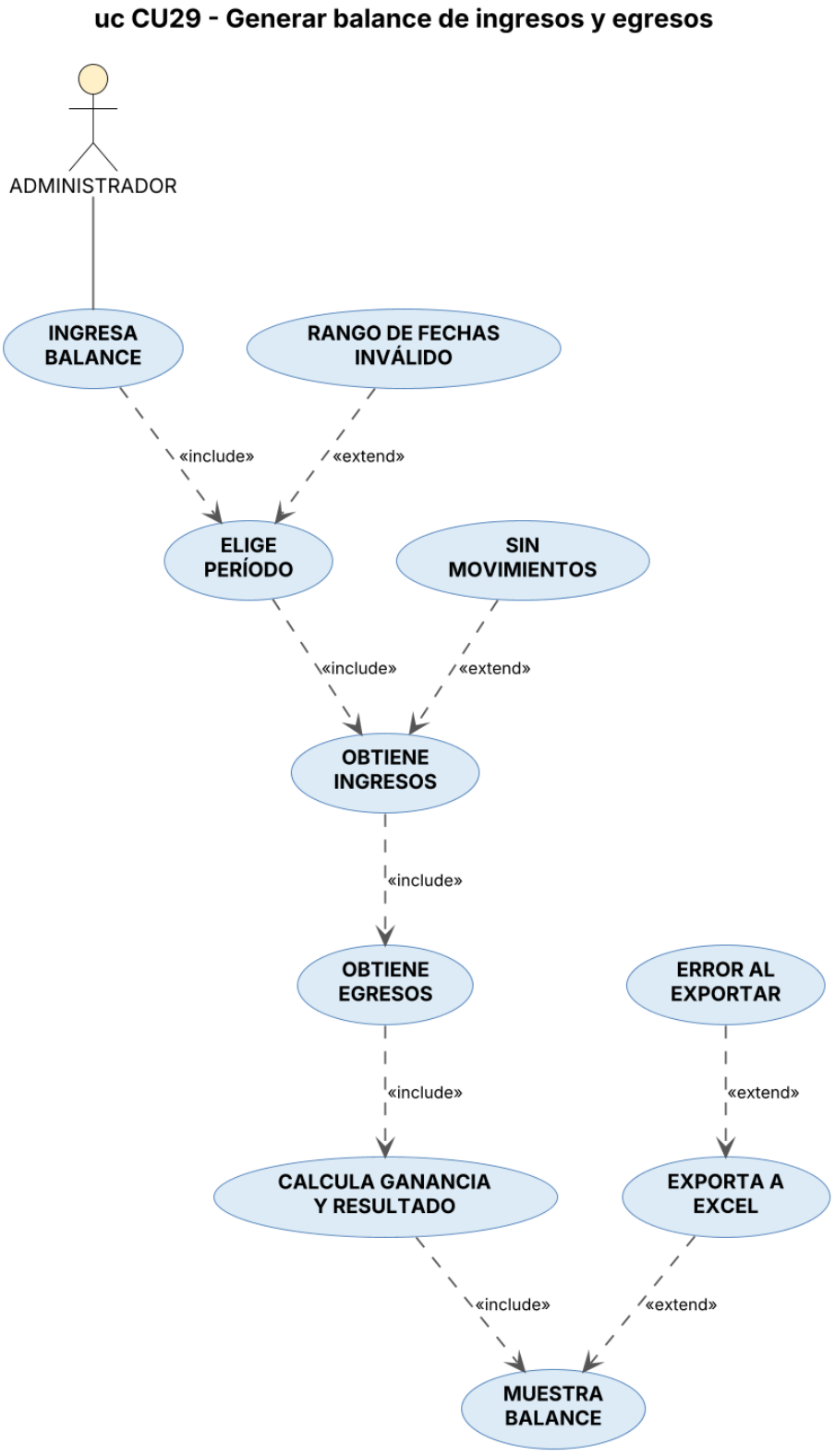


Diagrama de secuencia

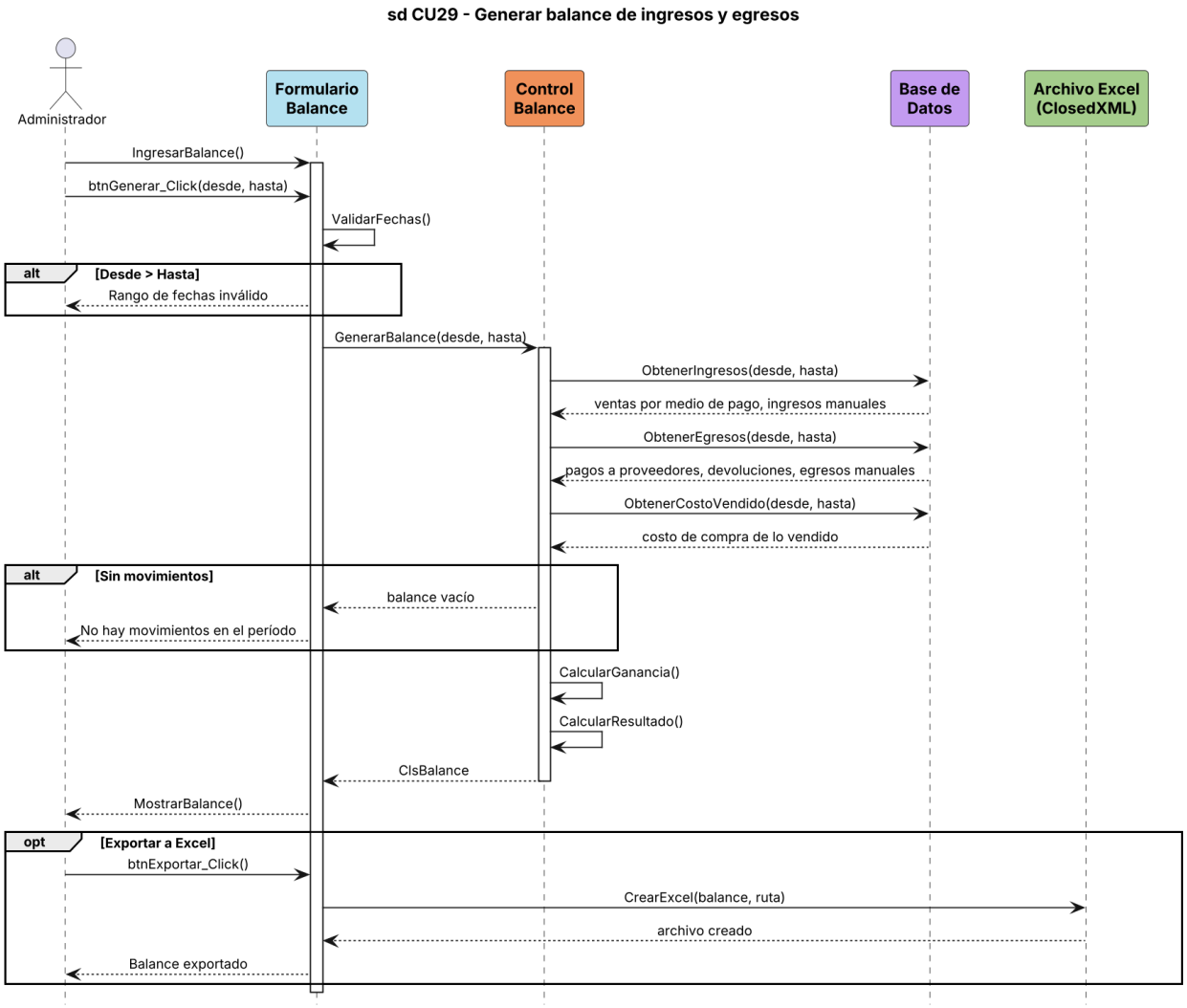
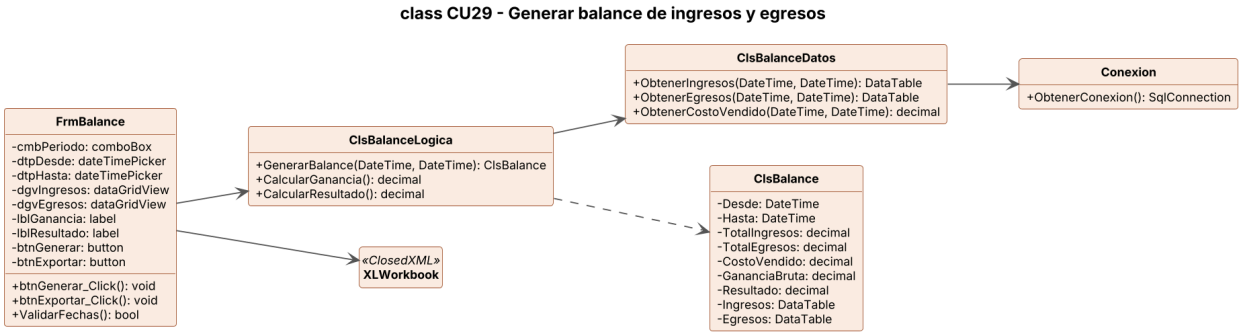


Diagrama de clases



CU30 - Consultar movimientos de stock

ID y Nombre	**CU30 Consultar movimientos de stock**	
Descripción	Permite al Administrador consultar el historial de movimientos de stock (ingresos por compras y cargas, egresos por ventas y anulaciones, devoluciones y ajustes) filtrado por fecha, producto y tipo de movimiento, con el usuario que realizó cada uno. El listado puede exportarse a Excel.	
Actores	Administrador (permiso 6.0.0 Informes).	
Pre-Condición	El usuario debe haber iniciado sesión y tener el permiso 6.0.0 Informes.	
Post-Condición	Se muestran los movimientos que cumplen los filtros. Opcionalmente se genera el archivo Excel. No se modifican datos.	
FLUJO	**Flujo Normal**	**Flujo Alternativo**
	1. El Administrador accede al módulo Informes > Movimientos de Stock.	
	2. El Administrador ingresa el rango de fechas y, opcionalmente, el producto y el tipo de movimiento (Ingreso / Egreso).	2.1 Rango de fechas inválido. El sistema informa que la fecha "desde" no puede ser posterior a la fecha "hasta".
	3. El Administrador presiona "Buscar".	
	4. El sistema muestra los movimientos con fecha y hora, producto, tipo, cantidad, observación y usuario.	4.1 Sin resultados. El sistema informa que no hay movimientos para los filtros elegidos.
	5. El Administrador presiona "Exportar a Excel".	5.1 No exportar. El Administrador cierra la consulta. 5.2 Error al exportar. El sistema informa que no pudo generar el archivo.
	6. El sistema genera el archivo y informa que se exportó correctamente.	

Diagrama de caso de uso

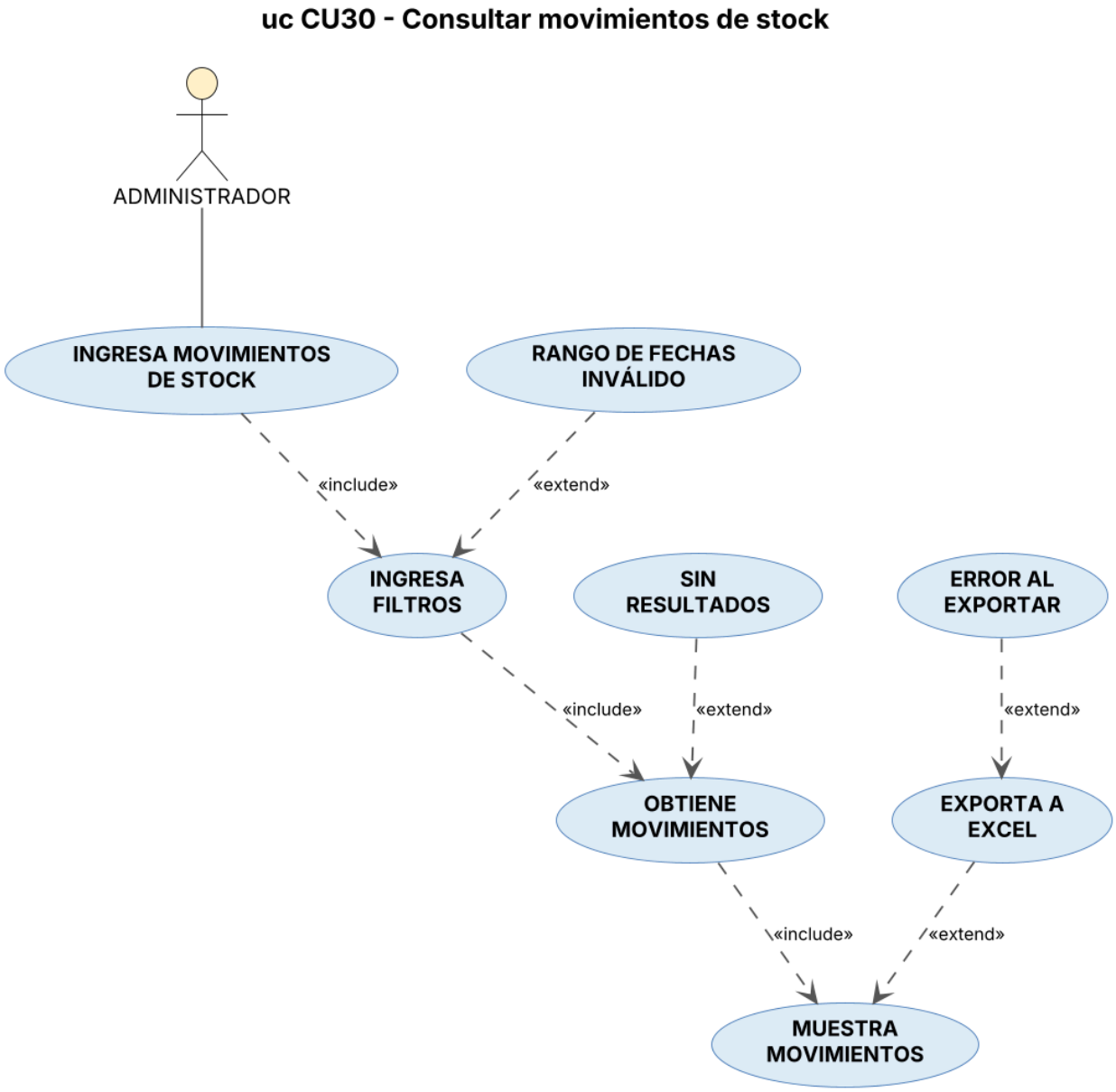


Diagrama de secuencia

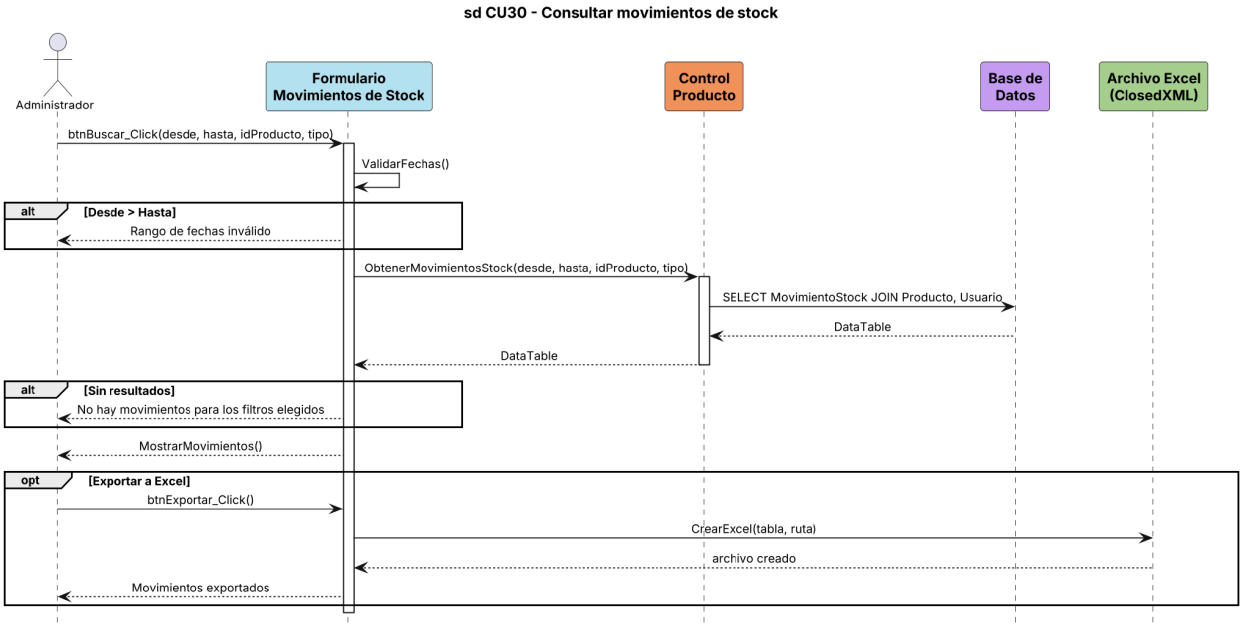
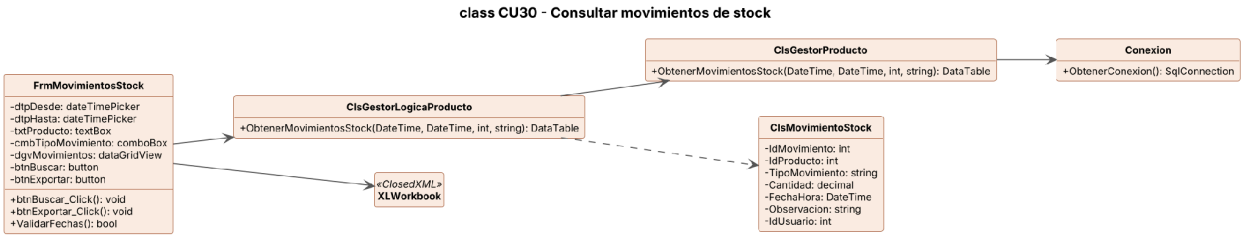


Diagrama de clases



CU31 - Abrir caja

ID y Nombre	**CU31 Abrir caja**	
Descripción	Permite al dueño o empleado autorizado abrir la caja al comenzar la jornada o el turno, ingresando el monto inicial en efectivo (fondo) con el que se empieza a trabajar. A partir de la apertura se habilitan las ventas, los cobros en efectivo y los movimientos manuales de caja, que luego se controlan en el cierre (CU05).	
Actores	Administrador - Empleado autorizado (permiso 7.3.0 Abrir Caja).	
Pre-Condición	El usuario debe haber iniciado sesión y tener el permiso 7.3.0 Abrir Caja. No debe haber otra caja abierta.	
Post-Condición	La caja queda abierta con el monto inicial, la fecha, la hora, el turno y el usuario. Se habilitan las ventas y los movimientos de caja. La operación queda registrada en la bitácora.	
FLUJO	**Flujo Normal**	**Flujo Alternativo**
	1. Al ingresar a la pantalla principal, el sistema verifica si hay una caja abierta.	1.1 Caja ya abierta. El sistema muestra la pantalla principal con la caja actual y no solicita la apertura.
	2. El sistema solicita el monto con el que se abre la caja.	
	3. El usuario ingresa el monto inicial (fondo).	
	4. El usuario presiona "Abrir caja".	4.1 Cancelar. El sistema no abre la caja y bloquea las ventas y los movimientos de caja hasta que se abra.
	5. El sistema valida el monto.	5.1 Campo vacío. El sistema informa que debe ingresar el monto inicial. 5.2 Monto inválido. El sistema informa que el monto no puede ser negativo.
	6. El sistema registra la apertura de caja con el monto, la fecha, la hora, el turno y el usuario.	
	7. El sistema registra la operación en la bitácora.	
	8. El sistema habilita las ventas y muestra la pantalla principal con la caja abierta.	

Diagrama de caso de uso

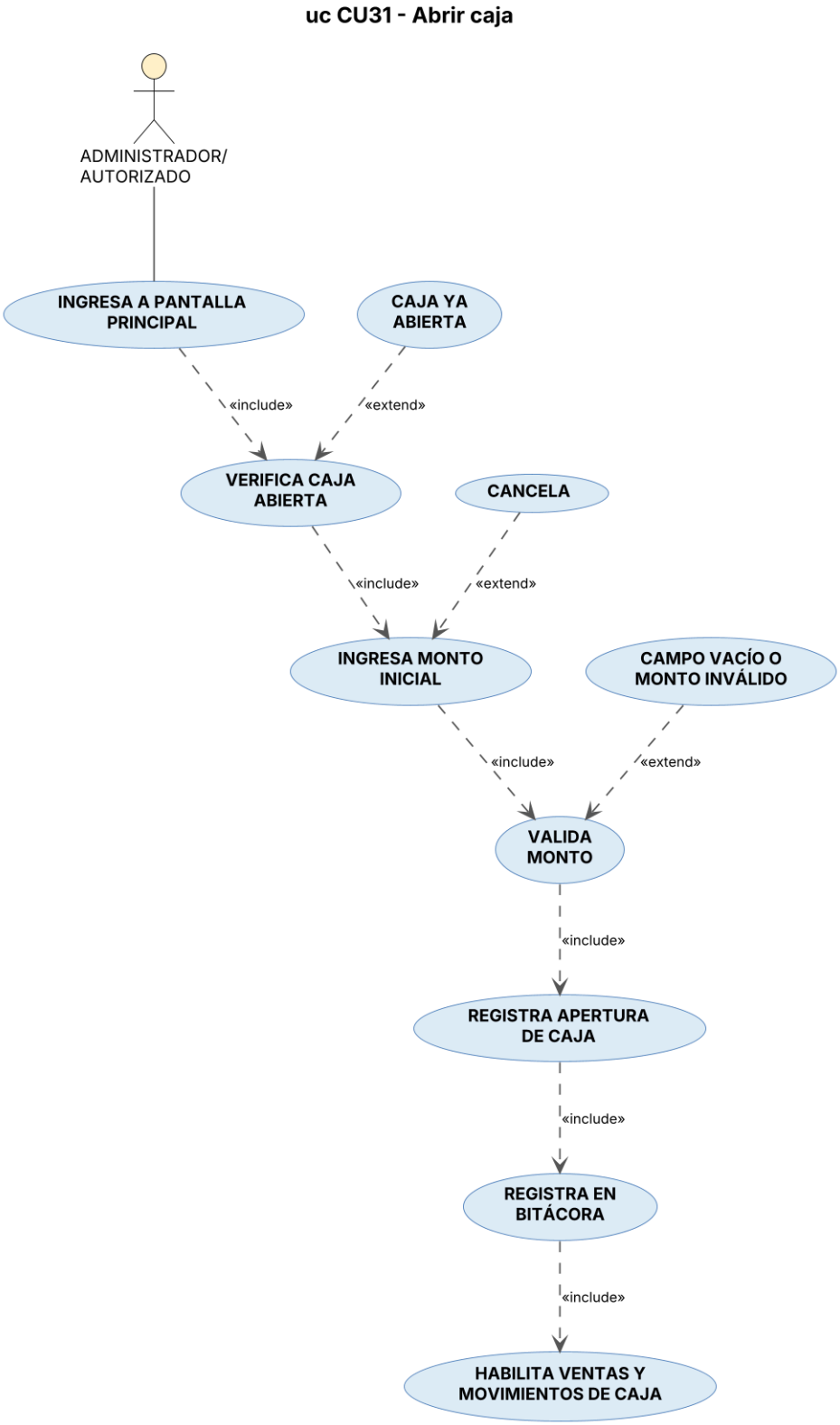


Diagrama de secuencia

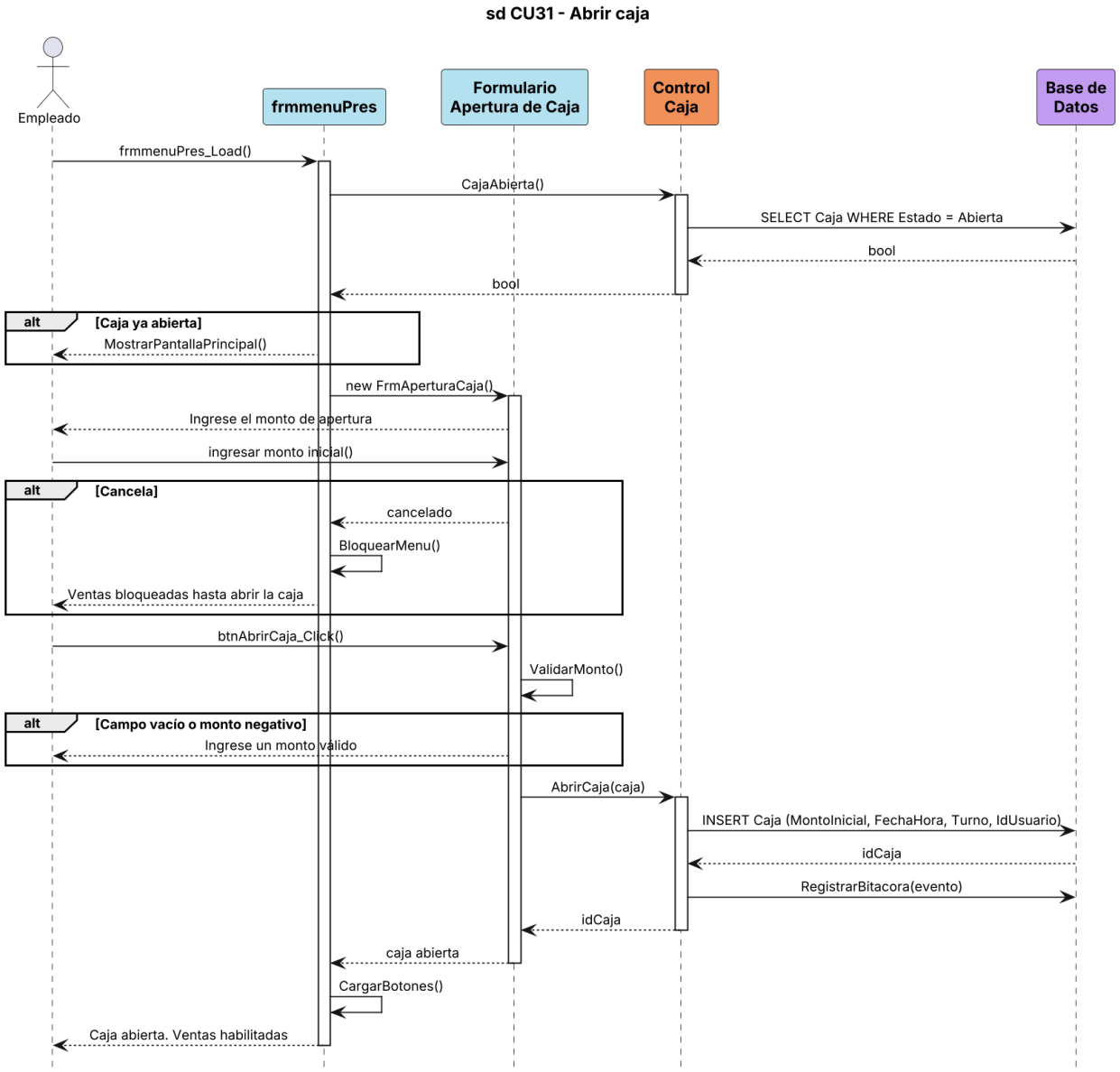
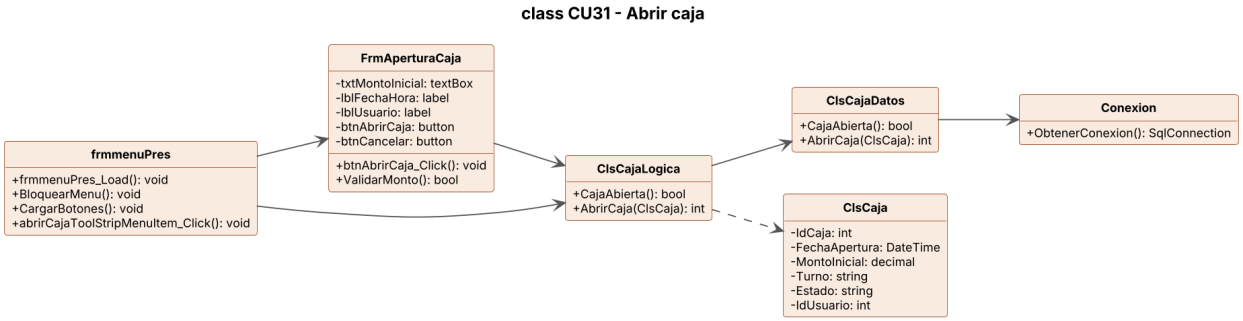


Diagrama de clases



CU32 - Registrar ingreso manual de caja

ID y Nombre	**CU32 Registrar ingreso manual de caja**	
Descripción	Permite al dueño o empleado autorizado registrar un ingreso de dinero en la caja que no surge de una venta (por ejemplo, un aporte de cambio o un cobro fuera de una venta), indicando el importe y el motivo. Por ser un movimiento manual, el sistema exige la justificación y envía una notificación por correo al Administrador.	
Actores	Administrador - Empleado autorizado (permiso 7.1.0 Nuevo Ingreso).	
Pre-Condición	El usuario debe haber iniciado sesión y tener el permiso 7.1.0 Nuevo Ingreso. La caja debe estar abierta (CU31).	
Post-Condición	El ingreso queda registrado en la caja con importe, motivo, fecha, hora y usuario, y el saldo de caja se actualiza. Se envía la notificación por correo al Administrador y la operación queda registrada en la bitácora.	
FLUJO	**Flujo Normal**	**Flujo Alternativo**
	1. El usuario accede al menú Caja > Nuevo Ingreso.	1.1 Caja cerrada. El sistema informa que debe abrir la caja antes de registrar movimientos.
	2. El sistema abre el formulario de ingreso de caja.	
	3. El usuario ingresa el importe.	
	4. El usuario ingresa el motivo (concepto) del ingreso.	
	5. El usuario presiona "Guardar".	5.1 Cancelar. El sistema cierra el formulario sin registrar el movimiento.
	6. El sistema valida los datos.	6.1 Campos vacíos. El sistema informa que debe ingresar el importe y el motivo. 6.2 Importe inválido. El sistema informa que el importe debe ser mayor a cero.
	7. El sistema registra el ingreso en la caja y actualiza el saldo.	
	8. El sistema envía la notificación por correo al Administrador con el detalle del movimiento.	8.1 Error de envío. El movimiento queda registrado igual; el sistema informa que no se pudo enviar el correo y lo deja registrado en la bitácora.
	9. El sistema registra la operación en la bitácora.	

	10. El sistema informa que el ingreso se registró correctamente y vuelve a la pantalla principal.	

Diagrama de caso de uso

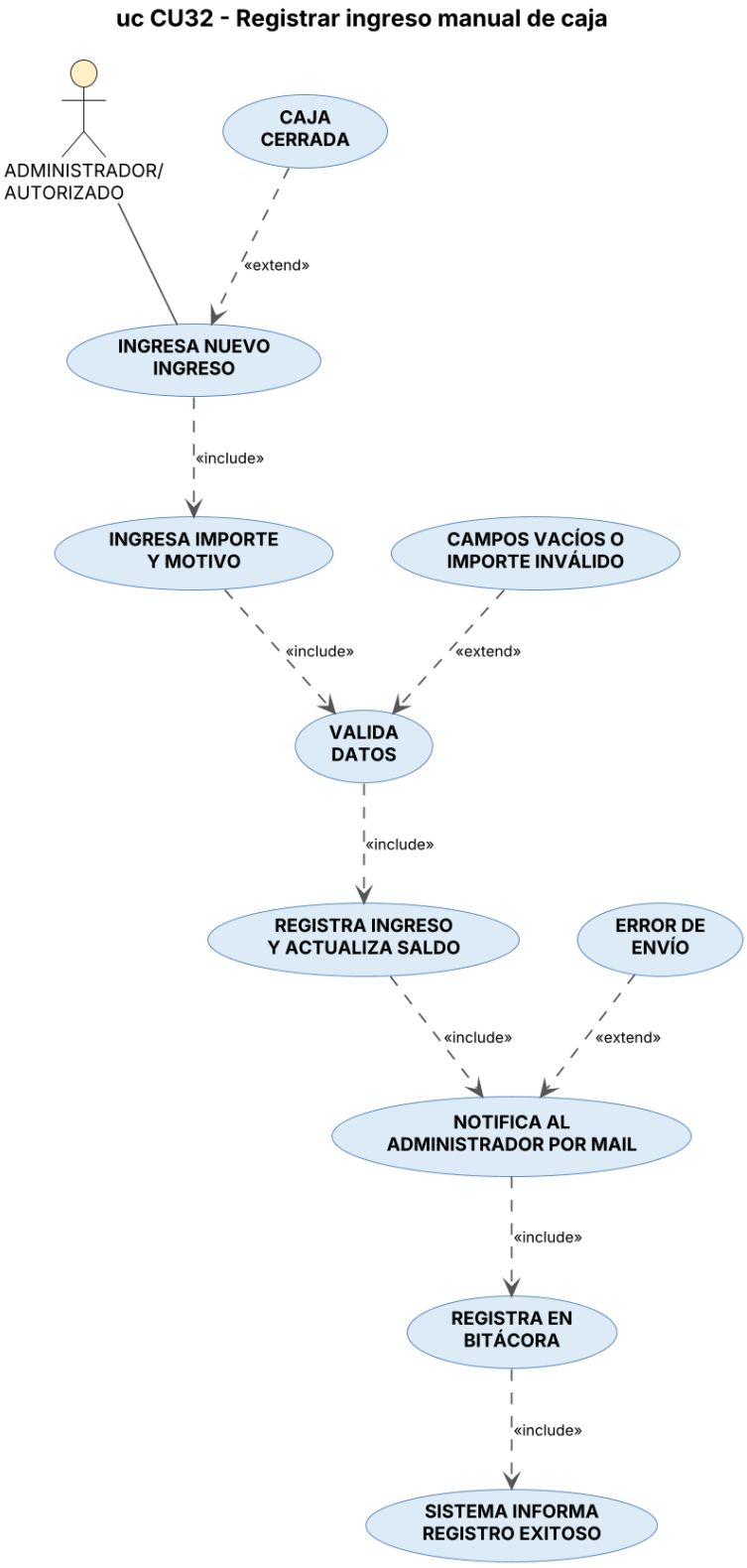


Diagrama de secuencia

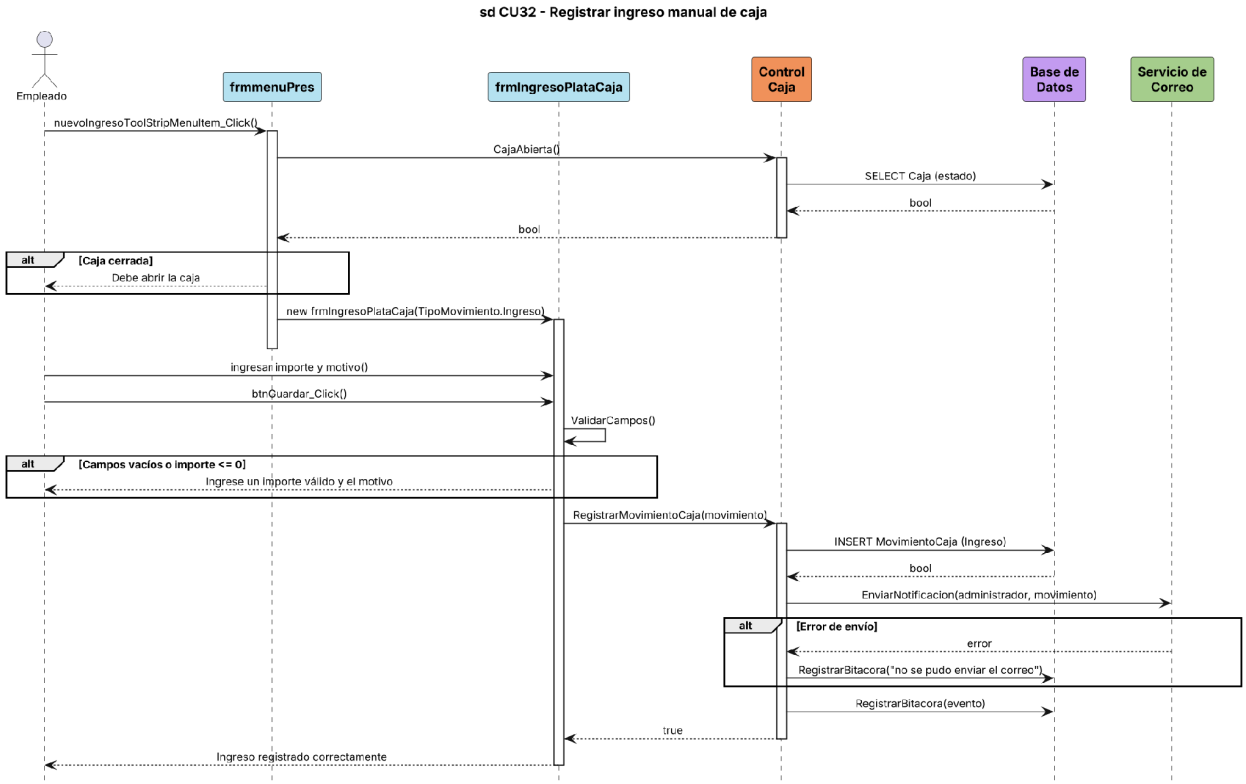
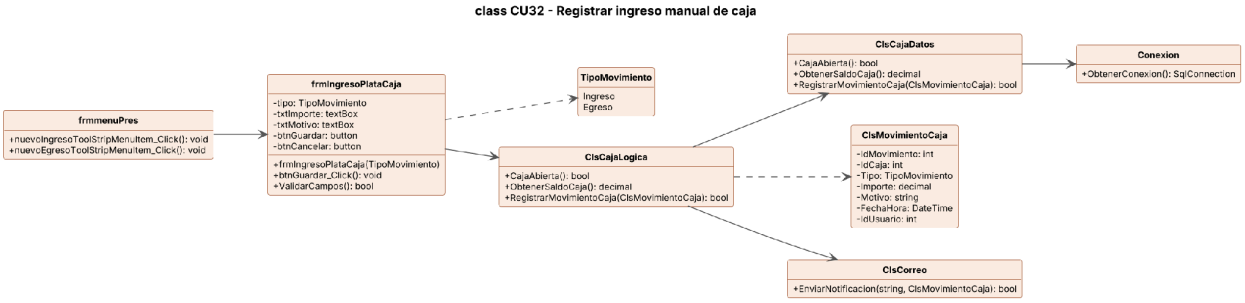


Diagrama de clases



CU33 - Registrar egreso manual de caja

ID y Nombre	**CU33 Registrar egreso manual de caja**	
Descripción	Permite al dueño o empleado autorizado registrar un egreso de dinero en la caja que no surge de una venta (por ejemplo, un pago a un proveedor en efectivo, un retiro o un vale de un empleado), indicando el importe y el motivo. Por ser un movimiento manual, el sistema exige la justificación y envía una notificación por correo al Administrador.	
Actores	Administrador - Empleado autorizado (permiso 7.2.0 Nuevo Egreso).	
Pre-Condición	El usuario debe haber iniciado sesión y tener el permiso 7.2.0 Nuevo Egreso. La caja debe estar abierta (CU31).	
Post-Condición	El egreso queda registrado en la caja con importe, motivo, fecha, hora y usuario, y el saldo de caja se actualiza. Se envía la notificación por correo al Administrador y la operación queda registrada en la bitácora.	
FLUJO	**Flujo Normal**	**Flujo Alternativo**
	1. El usuario accede al menú Caja > Nuevo Egreso.	1.1 Caja cerrada. El sistema informa que debe abrir la caja antes de registrar movimientos.
	2. El sistema abre el formulario de egreso de caja.	
	3. El usuario ingresa el importe.	
	4. El usuario ingresa el motivo (concepto) del egreso.	
	5. El usuario presiona "Guardar".	5.1 Cancelar. El sistema cierra el formulario sin registrar el movimiento.
	6. El sistema valida los datos.	6.1 Campos vacíos. El sistema informa que debe ingresar el importe y el motivo. 6.2 Importe inválido. El sistema informa que el importe debe ser mayor a cero. 6.3 Saldo insuficiente. El sistema informa que el importe supera el efectivo disponible en la caja.
	7. El sistema registra el egreso en la caja y actualiza el saldo.	
	8. El sistema envía la notificación por correo al Administrador con el detalle del movimiento.	8.1 Error de envío. El movimiento queda registrado igual; el sistema informa que no se pudo enviar el correo y lo deja registrado en la bitácora.

	9. El sistema registra la operación en la bitácora.	
	10. El sistema informa que el egreso se registró correctamente y vuelve a la pantalla principal.	

Diagrama de caso de uso

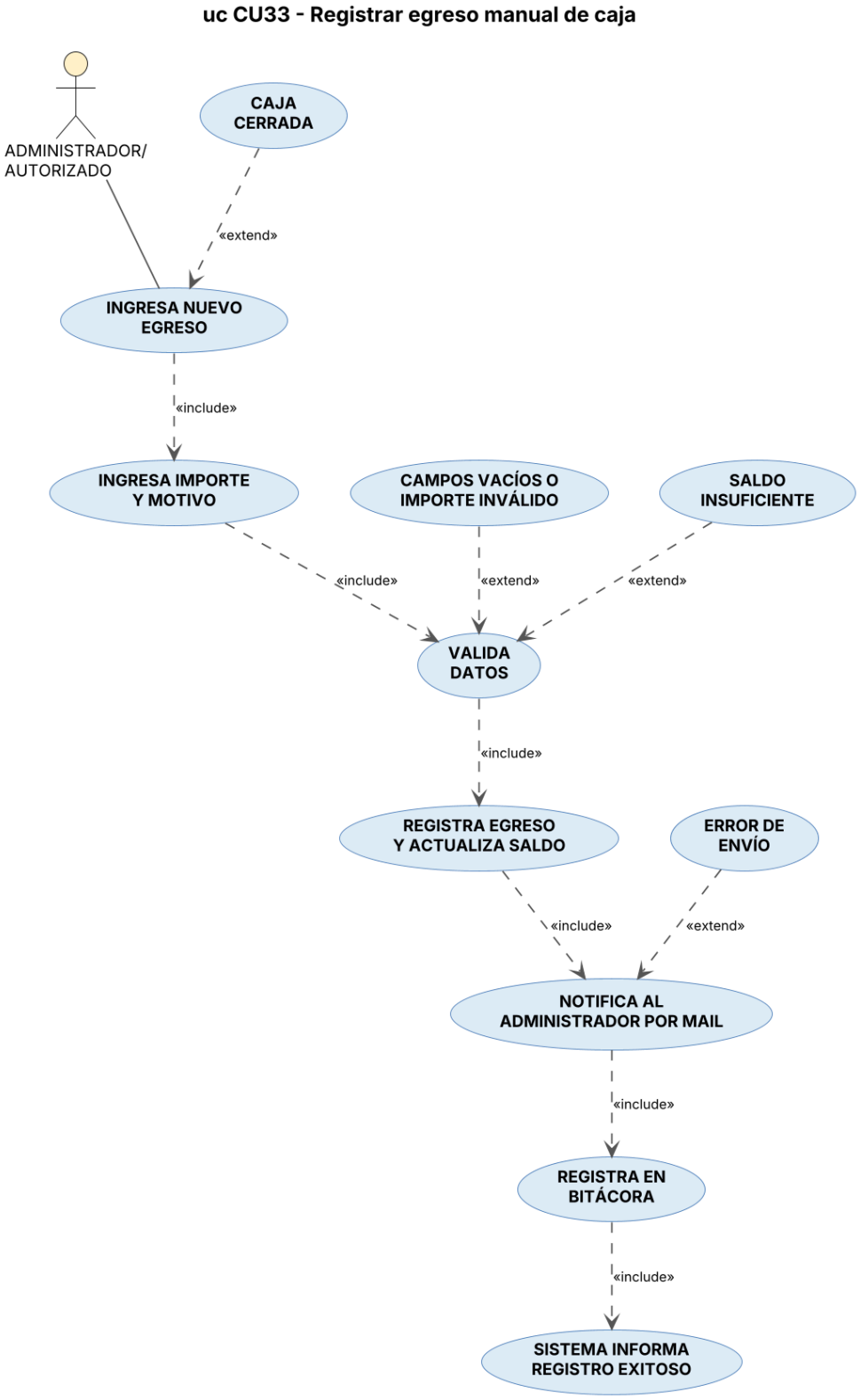


Diagrama de secuencia

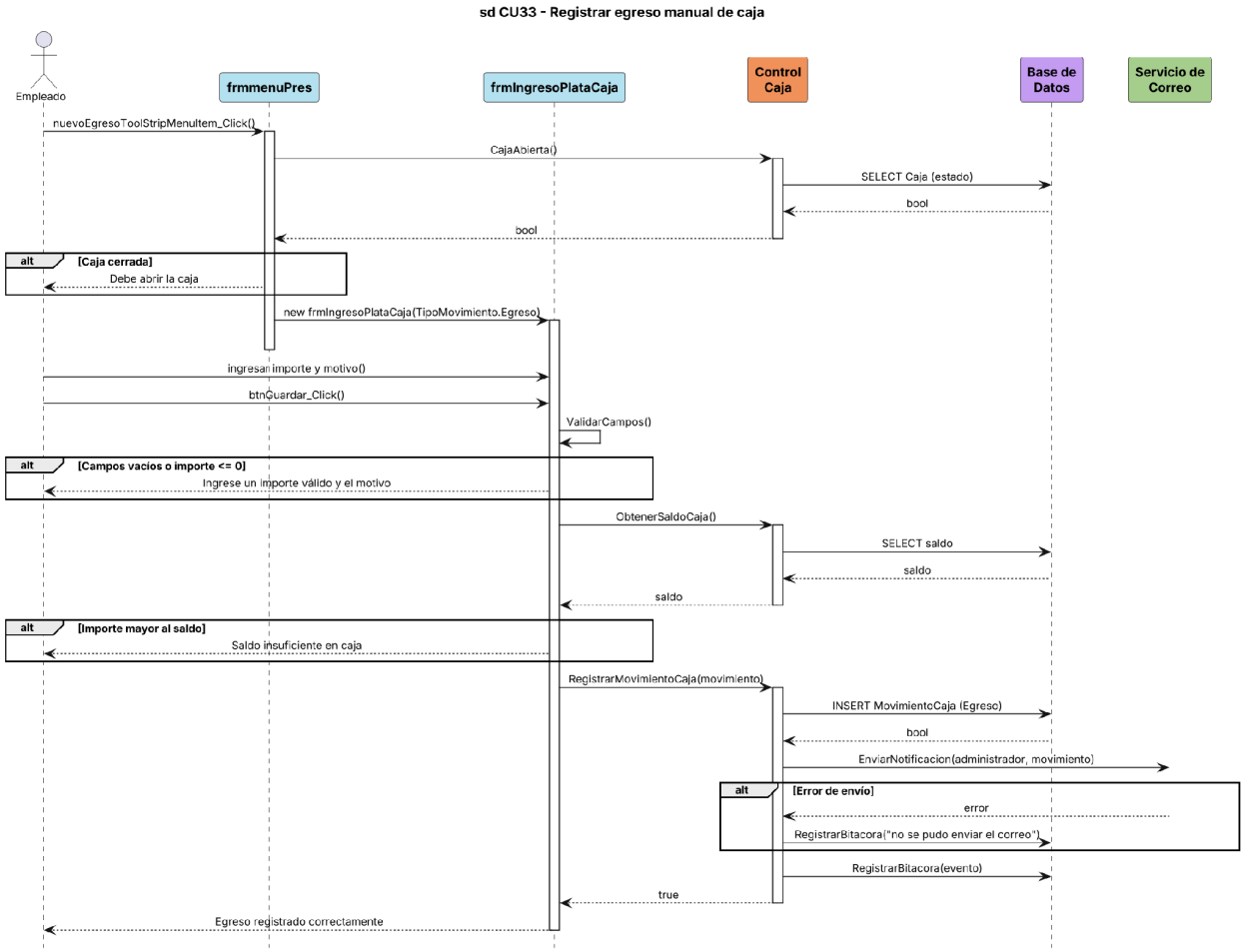
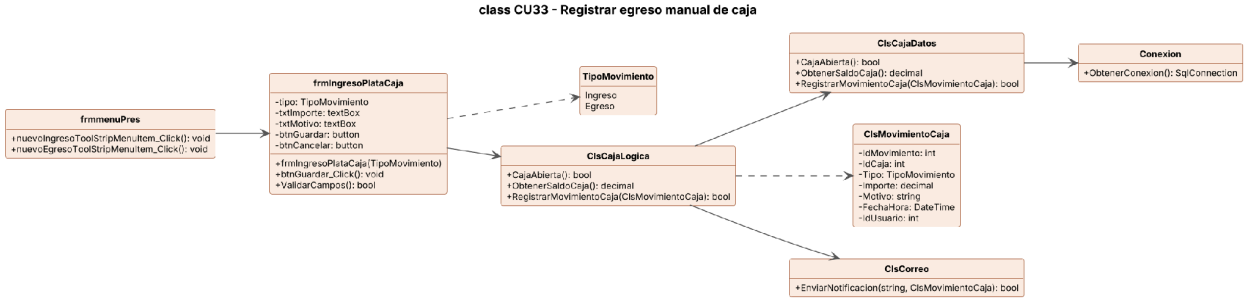


Diagrama de clases



CU34 - Consultar alertas de stock bajo / crítico

ID y Nombre	CU34 Consultar alertas de stock bajo / crítico	
Descripción	Permite a los empleados con permiso consultar las alarmas de stock: productos cuyo stock actual llegó al punto de reposición (stock bajo) o al stock crítico. El sistema muestra cada producto con su stock actual, el mínimo configurado y el nivel de la alarma, y avisa en la pantalla principal cuando hay alarmas nuevas.	
Actores	Administrador - Empleado General - Empleado Compras - Empleado Ventas (permiso 11.1.0 Visualizar Alarma).	
Pre-Condición	El usuario debe haber iniciado sesión y tener el permiso 11.1.0 Visualizar Alarma. Los productos deben tener configurado el stock mínimo (CU10).	
Post-Condición	Se muestran las alarmas activas de stock. No se modifican datos.	
FLUJO	**Flujo Normal**	**Flujo Alternativo**
	1. El usuario accede al menú Alarmas (o presiona el aviso de alarmas de la pantalla principal).	
	2. El sistema obtiene los productos activos cuyo stock actual es menor o igual al stock mínimo o al stock crítico.	2.1 Sin alarmas. El sistema informa que no hay productos con stock bajo o crítico.
	3. El sistema muestra las alarmas con producto, stock actual, stock mínimo y nivel (Bajo / Crítico), resaltando en rojo las críticas.	
	4. El usuario filtra por nivel o por rubro.	
	5. El usuario selecciona una alarma para ver el detalle del producto y su proveedor.	5.1 Ir al listado de reposición. El usuario presiona "Reponer" y el sistema abre el CU27 Listado de Reposición. 5.2 Dar de baja alarma. El usuario presiona "Dar de baja" y continúa en el CU35.

Diagrama de caso de uso

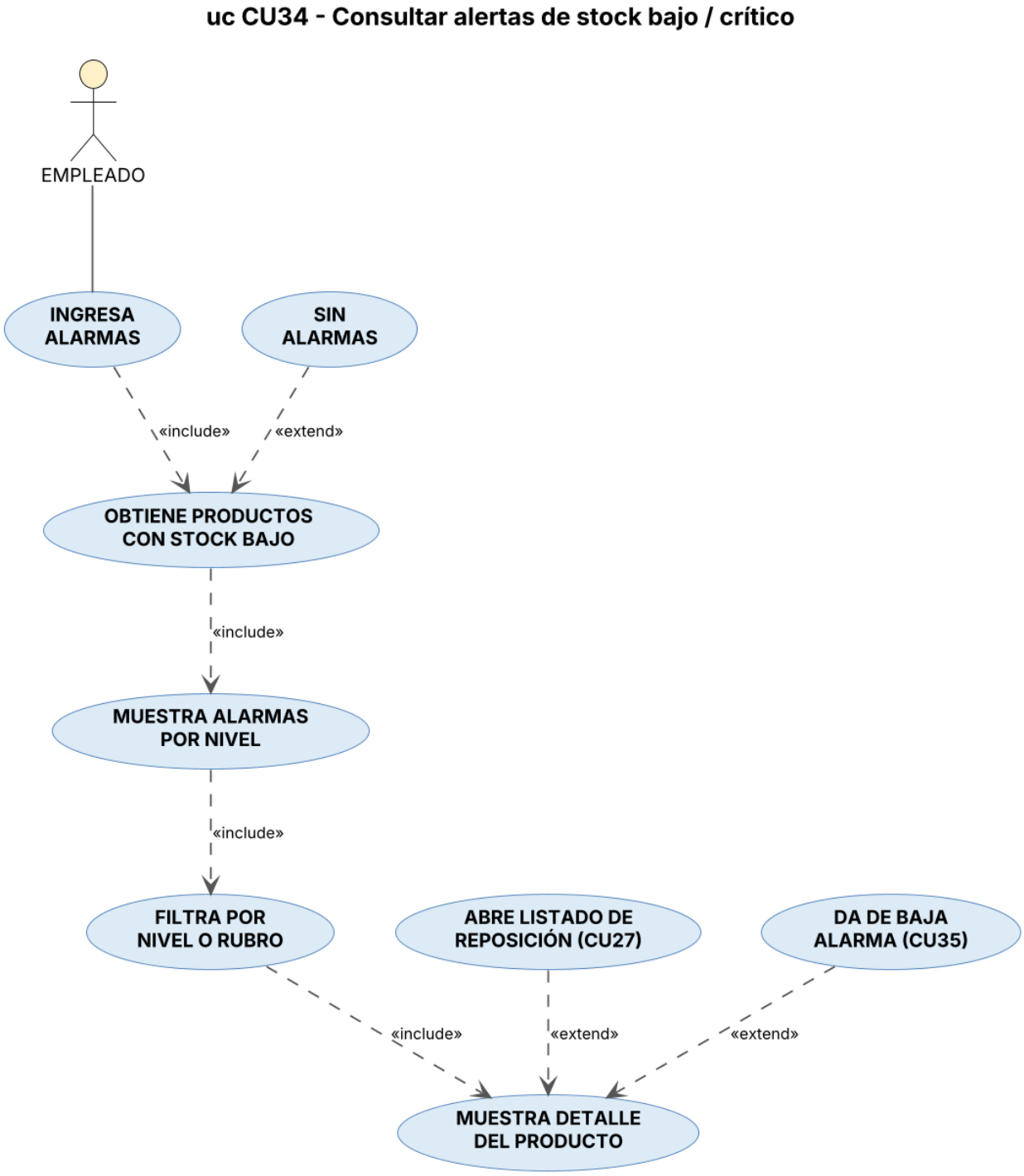


Diagrama de secuencia

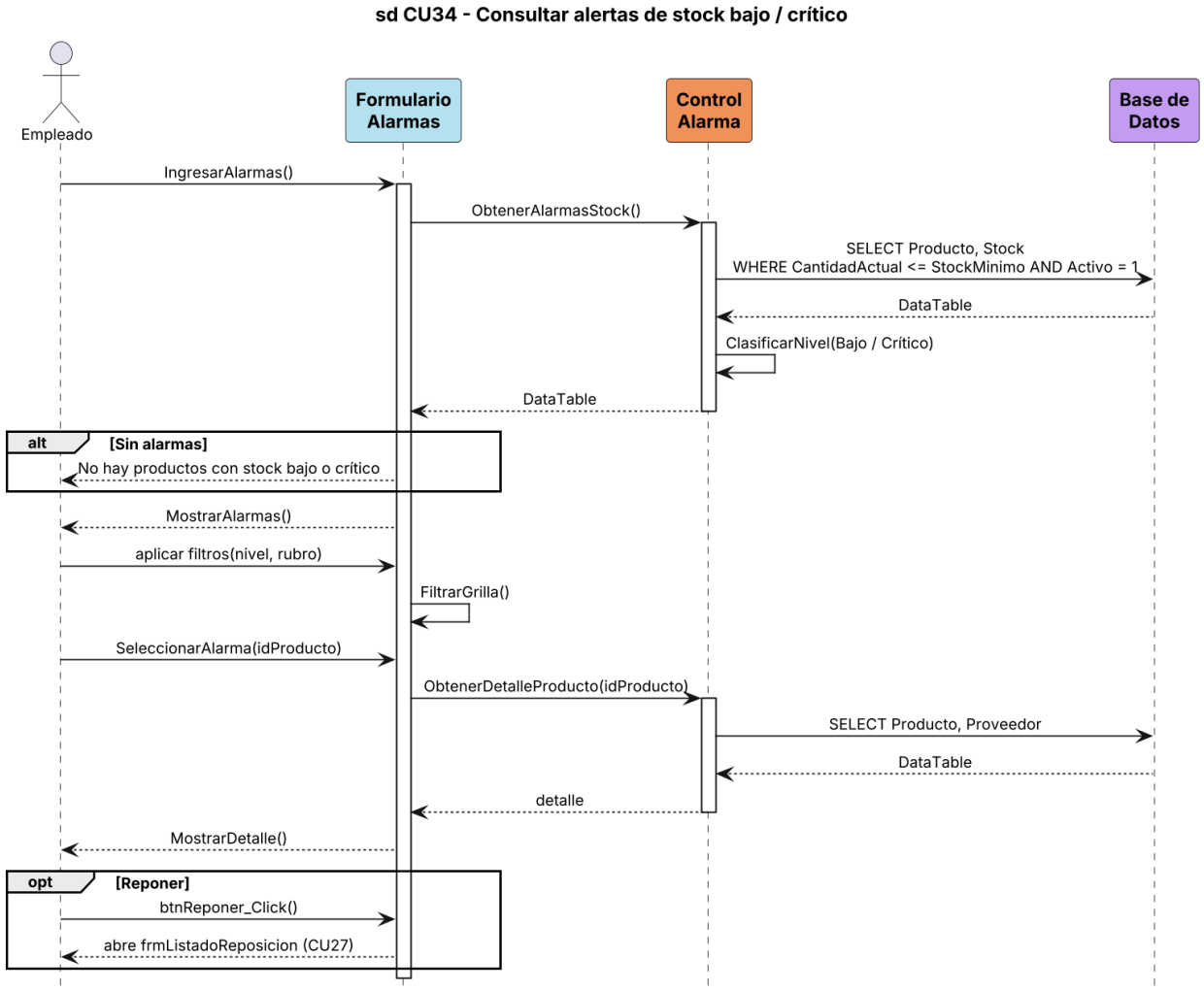
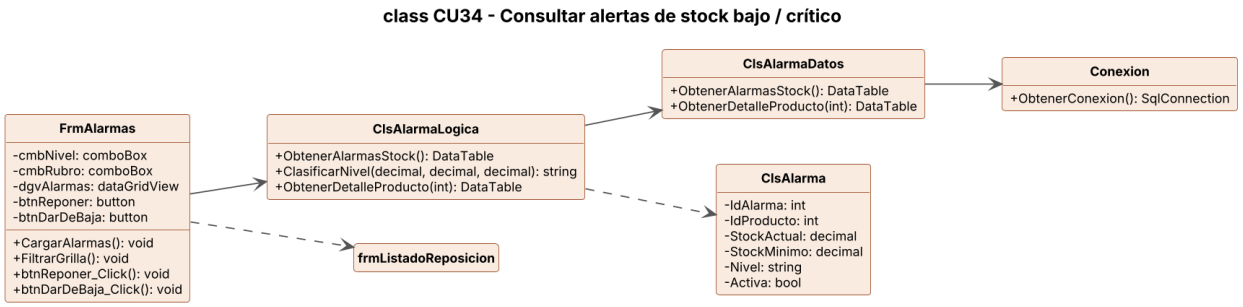


Diagrama de clases



CU35 - Dar de baja alarma

ID y Nombre	**CU35 Dar de baja alarma**	
Descripción	Permite al dueño o empleado autorizado dar de baja una alarma de stock que ya fue atendida (por ejemplo, porque el pedido ya se hizo), indicando un motivo, para que deje de mostrarse. Si el stock del producto se repone, la alarma no vuelve a aparecer; si el stock vuelve a bajar del mínimo, el sistema genera una alarma nueva.	
Actores	Administrador - Empleado autorizado (permiso 11.2.0 Dar de Baja Alarma).	
Pre-Condición	El usuario debe haber iniciado sesión y tener el permiso 11.2.0 Dar de Baja Alarma. Debe existir al menos una alarma activa.	
Post-Condición	La alarma queda inactiva con el motivo, la fecha y el usuario que la dio de baja. La operación queda registrada en la bitácora.	
FLUJO	**Flujo Normal**	**Flujo Alternativo**
	1. El usuario accede al menú Alarmas (CU34).	
	2. El sistema muestra las alarmas activas.	2.1 Sin alarmas. El sistema informa que no hay alarmas activas.
	3. El usuario selecciona la alarma y presiona "Dar de baja".	3.1 No se seleccionó alarma. El sistema informa que debe seleccionar una alarma.
	4. El sistema solicita el motivo de la baja.	
	5. El usuario ingresa el motivo y confirma.	5.1 Motivo vacío. El sistema informa que debe ingresar un motivo. 5.2 Cancelar. El sistema no realiza cambios y vuelve al paso 2.
	6. El sistema marca la alarma como inactiva con el motivo, la fecha y el usuario.	
	7. El sistema registra la operación en la bitácora.	
	8. El sistema informa que la alarma se dio de baja y actualiza el listado.	

Diagrama de caso de uso

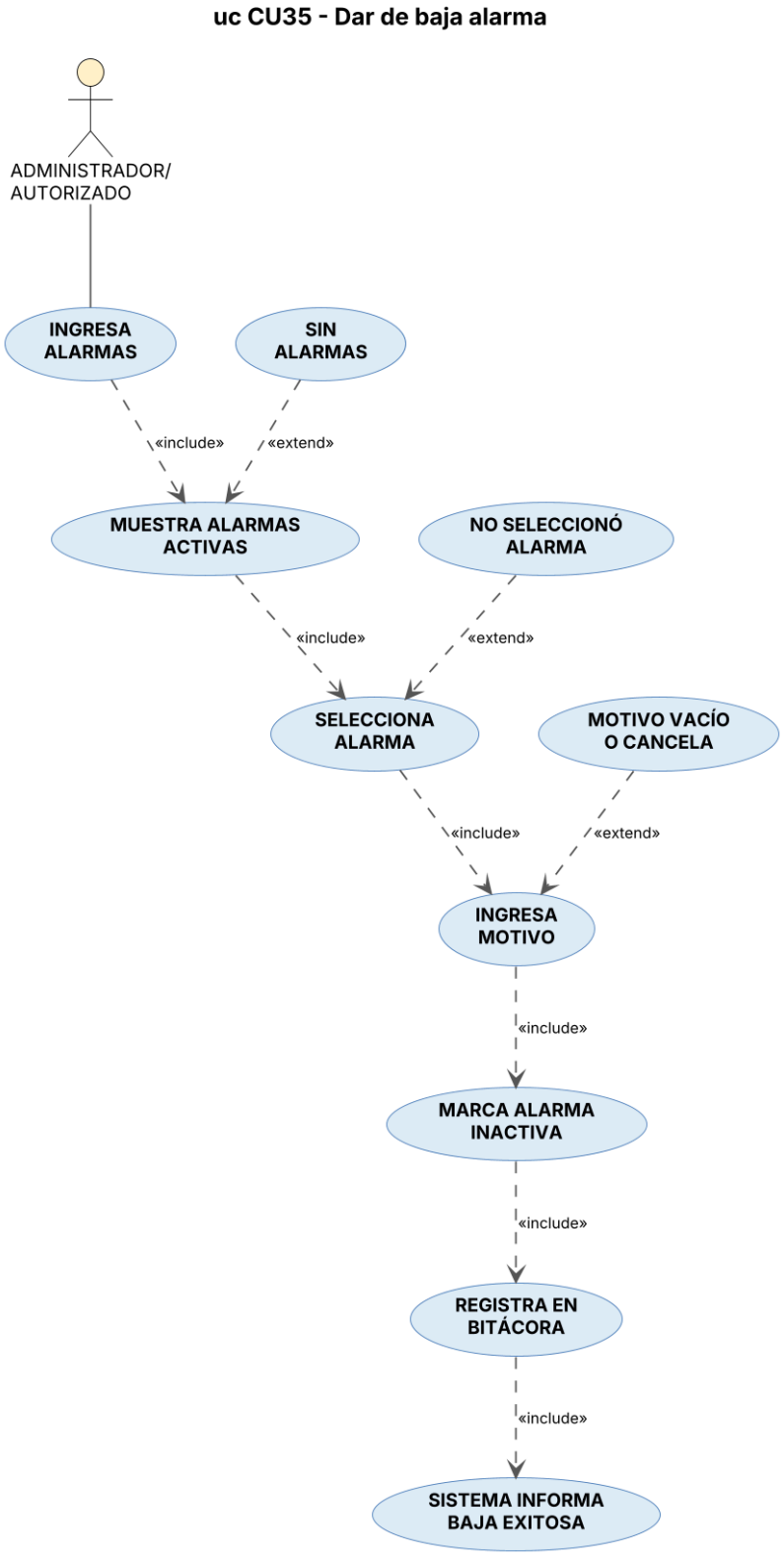


Diagrama de secuencia

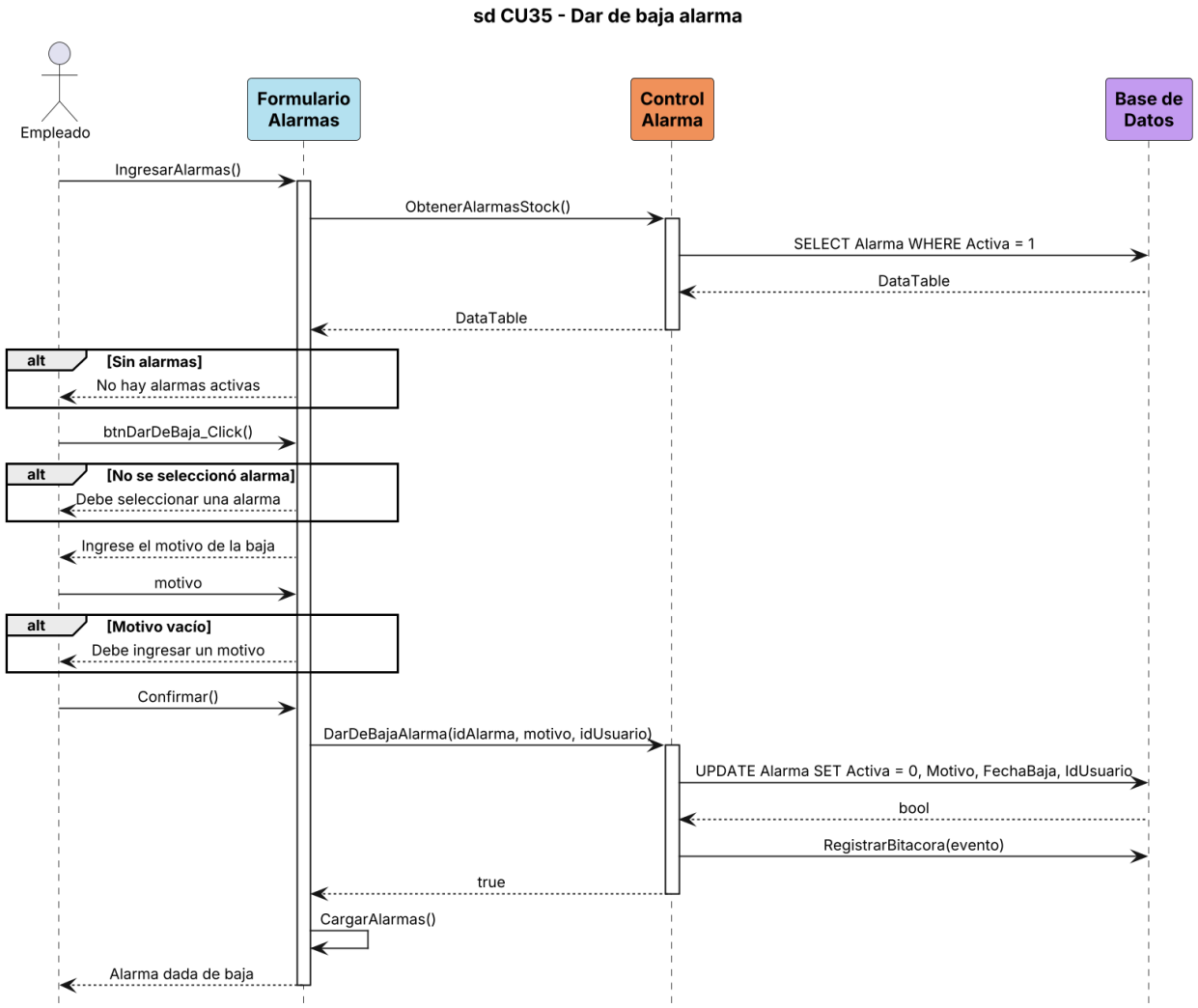
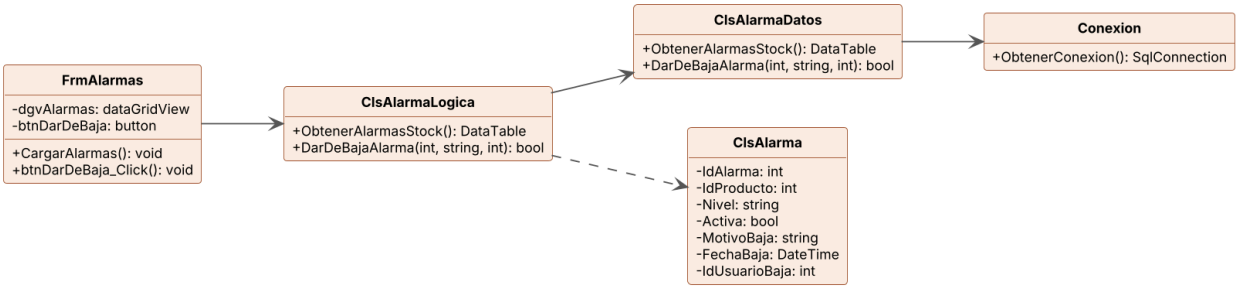


Diagrama de clases

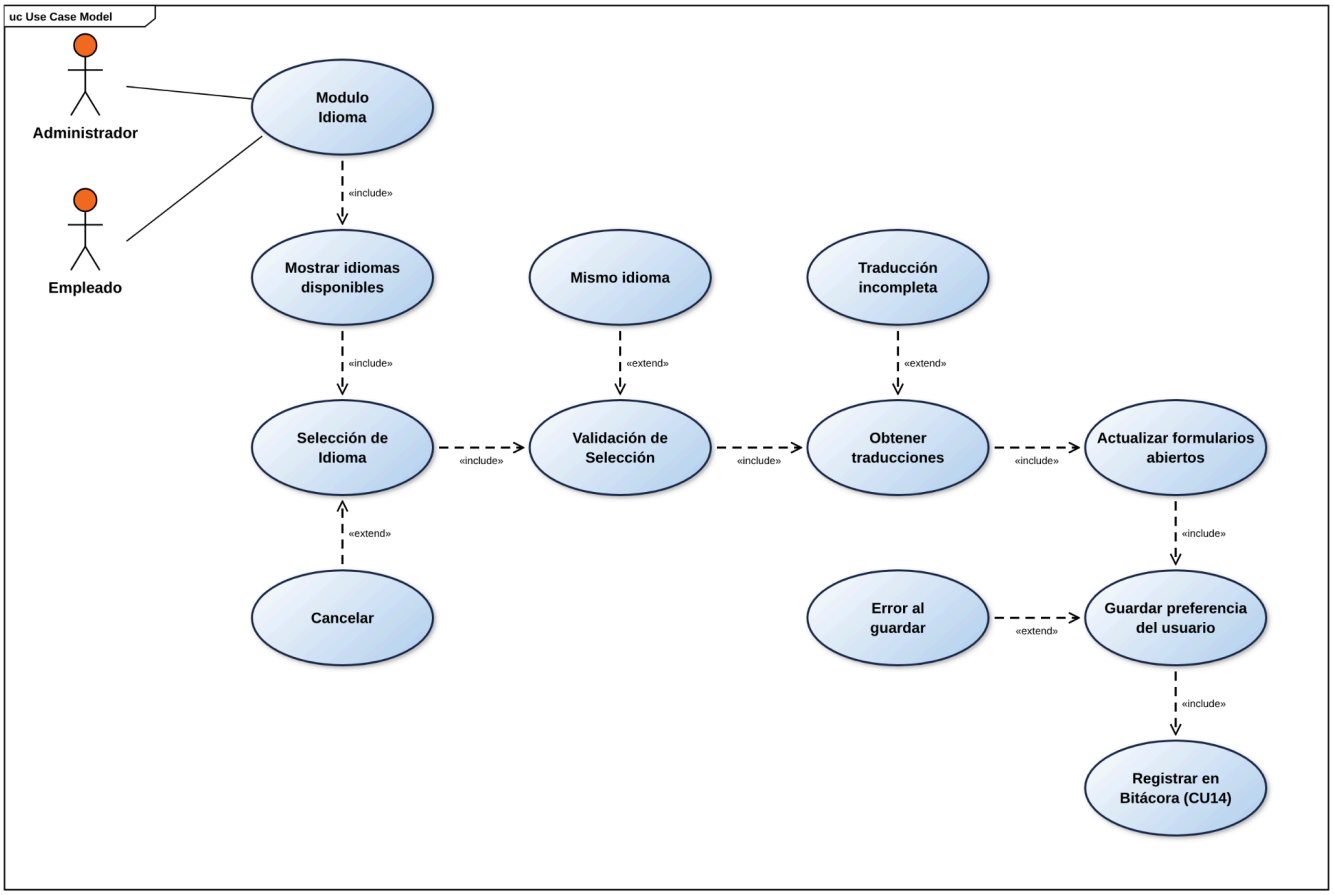


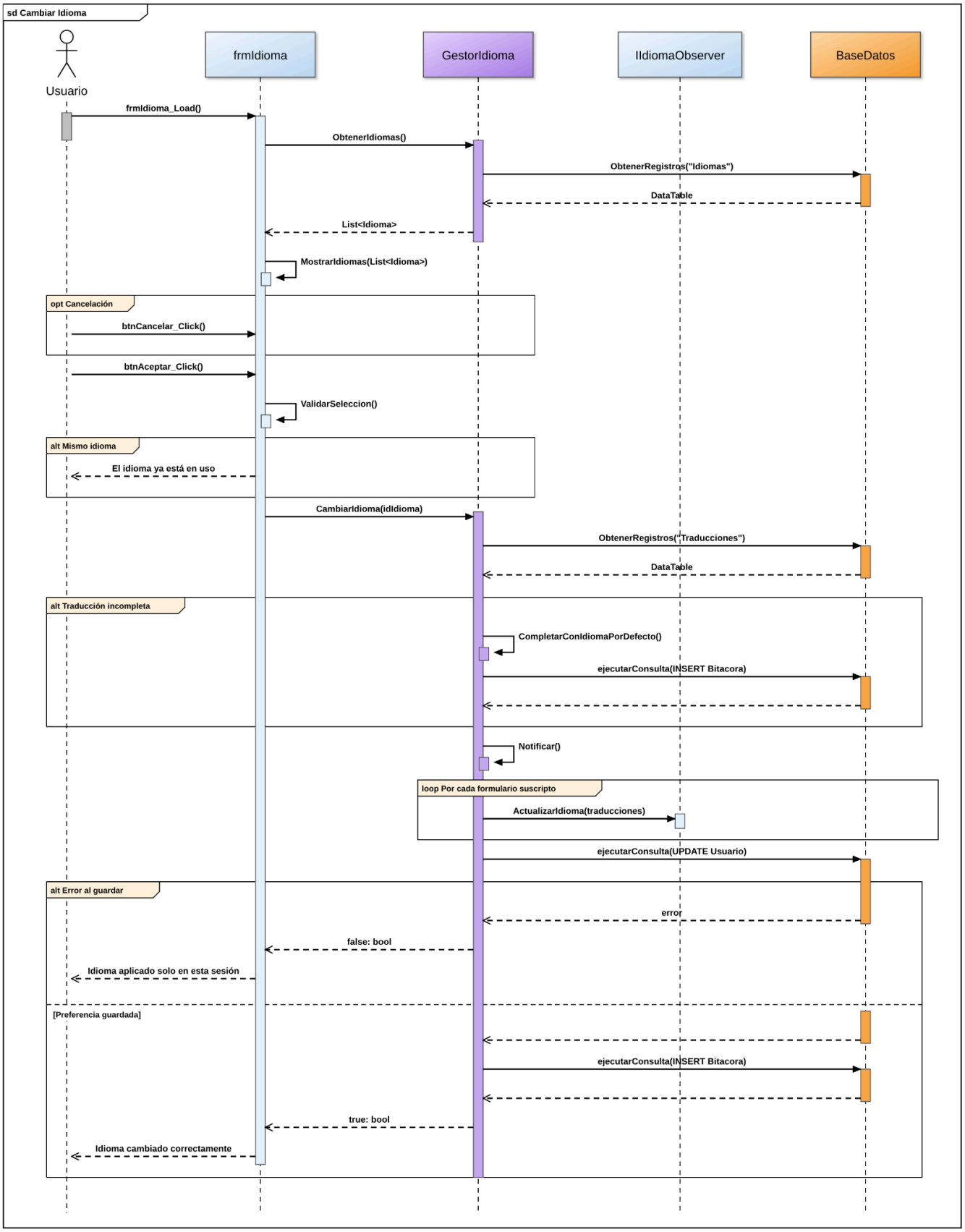
Casos de Uso de Seguridad

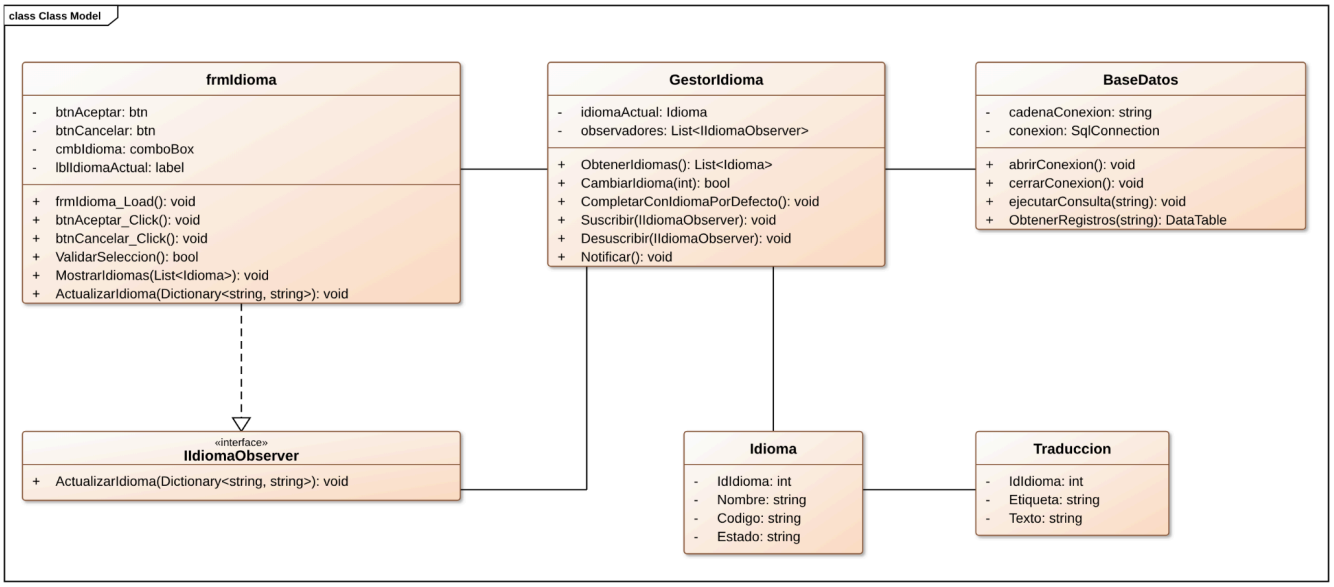
CUS01 - Cambiar Idioma

ID y Nombre	**CUS01 Cambiar Idioma**	
Descripción	Permite a cualquier usuario del sistema seleccionar el idioma en el que se muestra la interfaz. El sistema obtiene de la base de datos las traducciones del idioma elegido y notifica a todos los formularios abiertos (patrón Observer) para que actualicen sus textos sin reiniciar la aplicación. Además, guarda la elección como preferencia del usuario para aplicarla en su próximo inicio de sesión.	
Actores	Administrador - Empleado.	
Pre-Condición	El usuario debe haber iniciado sesión. Deben existir al menos dos idiomas activos con sus traducciones cargadas.	
Post-Condición	La interfaz se muestra en el idioma seleccionado. El idioma queda guardado como preferencia del usuario. El cambio queda registrado en la bitácora.	
FLUJO	**Flujo Normal**	**Flujo Alternativo**
	1. El usuario accede al módulo Ajustes > Idioma.	
	2. El sistema obtiene de la base de datos los idiomas disponibles y los muestra, indicando el idioma actual.	
	3. El usuario selecciona el idioma deseado y presiona "Aceptar".	3.1 Cancelación El usuario presiona "Cancelar". El sistema mantiene el idioma actual y cierra la ventana.
	4. El sistema valida la selección.	4.1 Mismo idioma El usuario seleccionó el idioma que ya está en uso. El sistema informa que el idioma ya está en uso y no realiza cambios.
	5. El sistema obtiene de la base de datos las traducciones del idioma seleccionado.	5.1 Traducción incompleta Existen etiquetas sin traducir. El sistema las completa con el idioma por defecto (Español) y registra el faltante en la bitácora.

	6. El sistema notifica a todos los formularios abiertos, que actualizan sus textos al nuevo idioma.	
	7. El sistema guarda el idioma como preferencia del usuario.	7.1 Error al guardar El sistema informa que el idioma se aplicó solamente para la sesión actual.
	8. El sistema registra el cambio en la bitácora e informa que el idioma se cambió correctamente.	



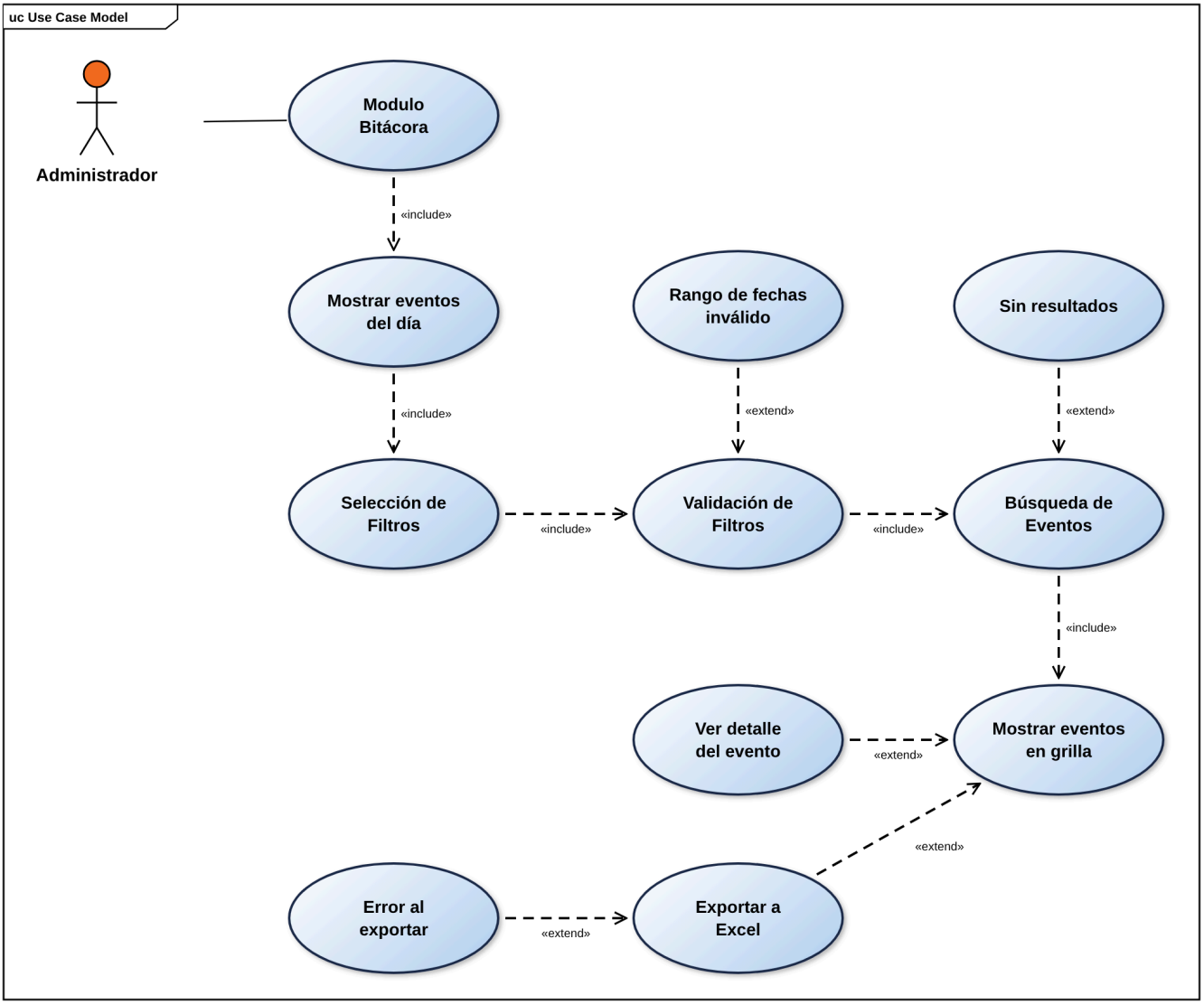

sd Cambiar Idioma
Usuario
frmIdioma
GestorIdioma
IIdiomaObserver
BaseDatos
frmIdioma_Load()
ObtenerIdiomas()
ObtenerRegistros("Idiomas")
DataTable
List<Idioma>
MostrarIdiomas(List<Idioma>)
opt Cancelación
btnCancelar_Click()
btnAceptar_Click()
ValidarSeleccion()
alt Mismo idioma
El idioma ya está en uso
CambiarIdioma(idIdioma)
ObtenerRegistros("Traducciones")
DataTable
alt Traducción incompleta
CompletarConIdiomaPorDefecto()
ejecutarConsulta(INSERT Bitacora)
Notificar()
loop Por cada formulario suscripto
ActualizarIdioma(traducciones)
ejecutarConsulta(UPDATE Usuario)
alt Error al guardar
error
false: bool
Idioma aplicado solo en esta sesión
[Preferencia guardada]
ejecutarConsulta(INSERT Bitacora)
true: bool
Idioma cambiado correctamente

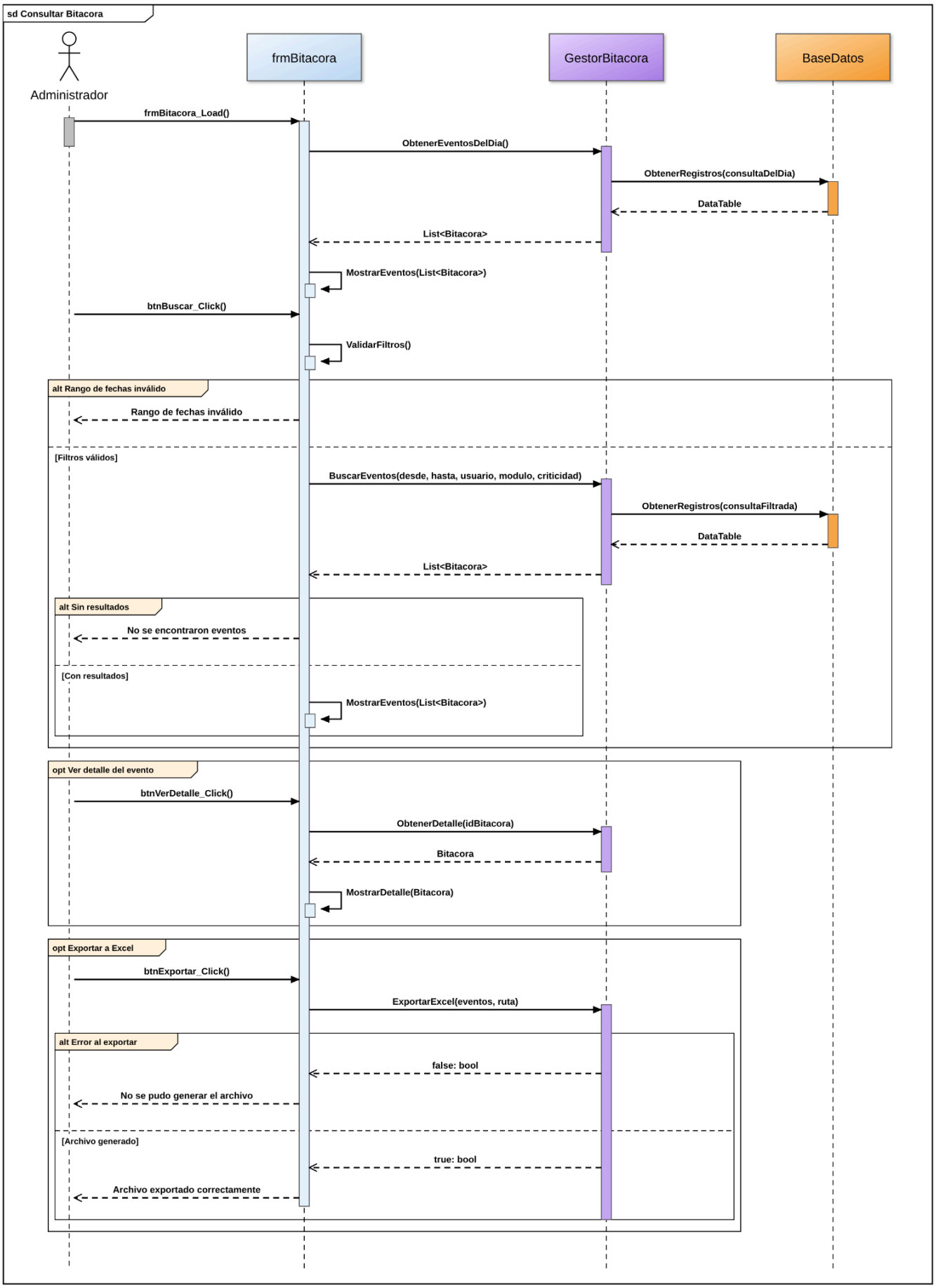


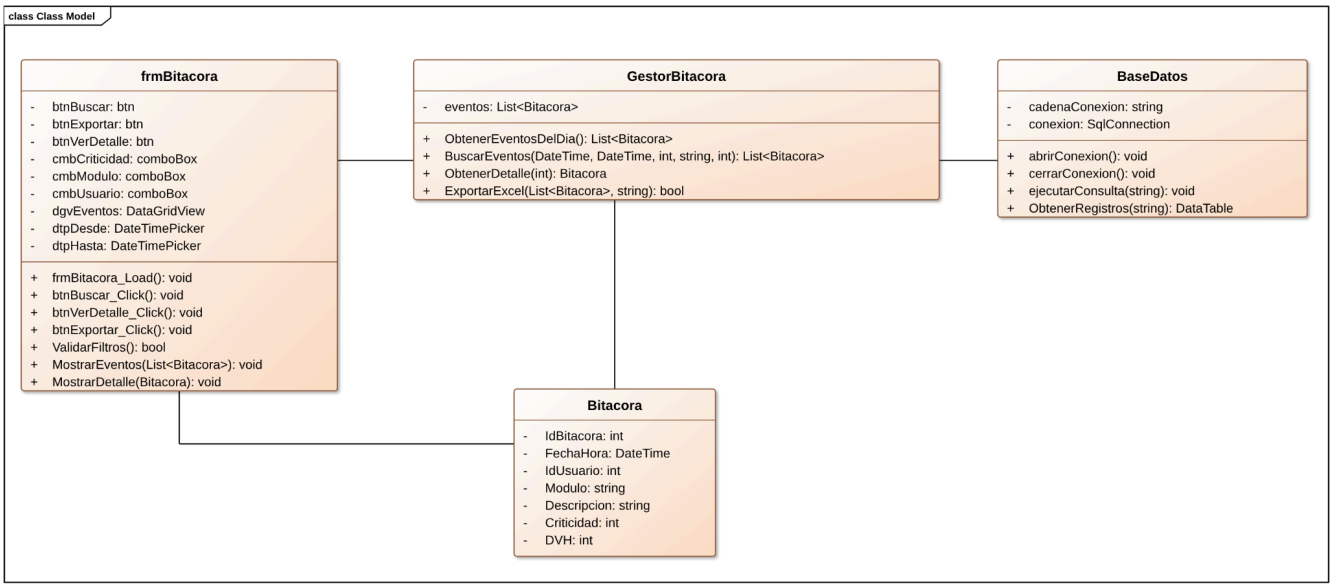
CUS02 - Consultar Bitácora

ID y Nombre	**CUS02 Consultar Bitácora**	
Descripción	Permite al Administrador consultar los eventos registrados en la bitácora del sistema. Al ingresar al módulo se muestran los eventos del día; luego el administrador puede filtrarlos por rango de fechas, usuario, módulo y criticidad, ver el detalle completo de un evento y exportar el resultado a un archivo Excel. Los registros de la bitácora solo pueden consultarse: no pueden modificarse ni eliminarse.	
Actores	Administrador.	
Pre-Condición	El administrador debe haber iniciado sesión. Debe poseer el permiso de Bitácora (8.2.0).	
Post-Condición	El sistema muestra en pantalla los eventos que cumplen con los filtros seleccionados. Opcionalmente, se muestra el detalle de un evento y/o el resultado queda exportado en un archivo Excel.	
FLUJO	**Flujo Normal**	**Flujo Alternativo**
	1. El administrador accede al módulo Ajustes > Bitácora.	

	2. El sistema obtiene de la base de datos los eventos del día actual y los muestra en la grilla, ordenados del más reciente al más antiguo.	
	3. El administrador selecciona los filtros (fecha desde, fecha hasta, usuario, módulo y/o criticidad) y presiona "Buscar".	
	4. El sistema valida los filtros ingresados.	4.1 Rango de fechas inválido La fecha desde es mayor que la fecha hasta. El sistema informa el error y solicita corrección.
	5. El sistema busca en la base de datos los eventos que cumplen con los filtros.	5.1 Sin resultados El sistema informa que no se encontraron eventos para los filtros seleccionados.
	6. El sistema muestra los eventos encontrados en la grilla.	
	7. El administrador selecciona un evento y presiona "Ver detalle" (opcional).	
	8. El sistema muestra el detalle completo del evento seleccionado.	
	9. El administrador presiona "Exportar a Excel" (opcional).	
	10. El sistema genera el archivo Excel con los eventos de la grilla e informa que se exportó correctamente.	10.1 Error al exportar El sistema informa que no se pudo generar el archivo (ruta sin permisos o archivo en uso).


uc Use Case Model
Administrador
Modulo Bitácora
«include»
Mostrar eventos del día
«include»
Selección de Filtros
«include»
Validación de Filtros
«include»
Búsqueda de Eventos
«include»
Mostrar eventos en grilla
Rango de fechas inválido
«extend»
Sin resultados
«extend»
Ver detalle del evento
«extend»
Error al exportar
«extend»
Exportar a Excel
«extend»

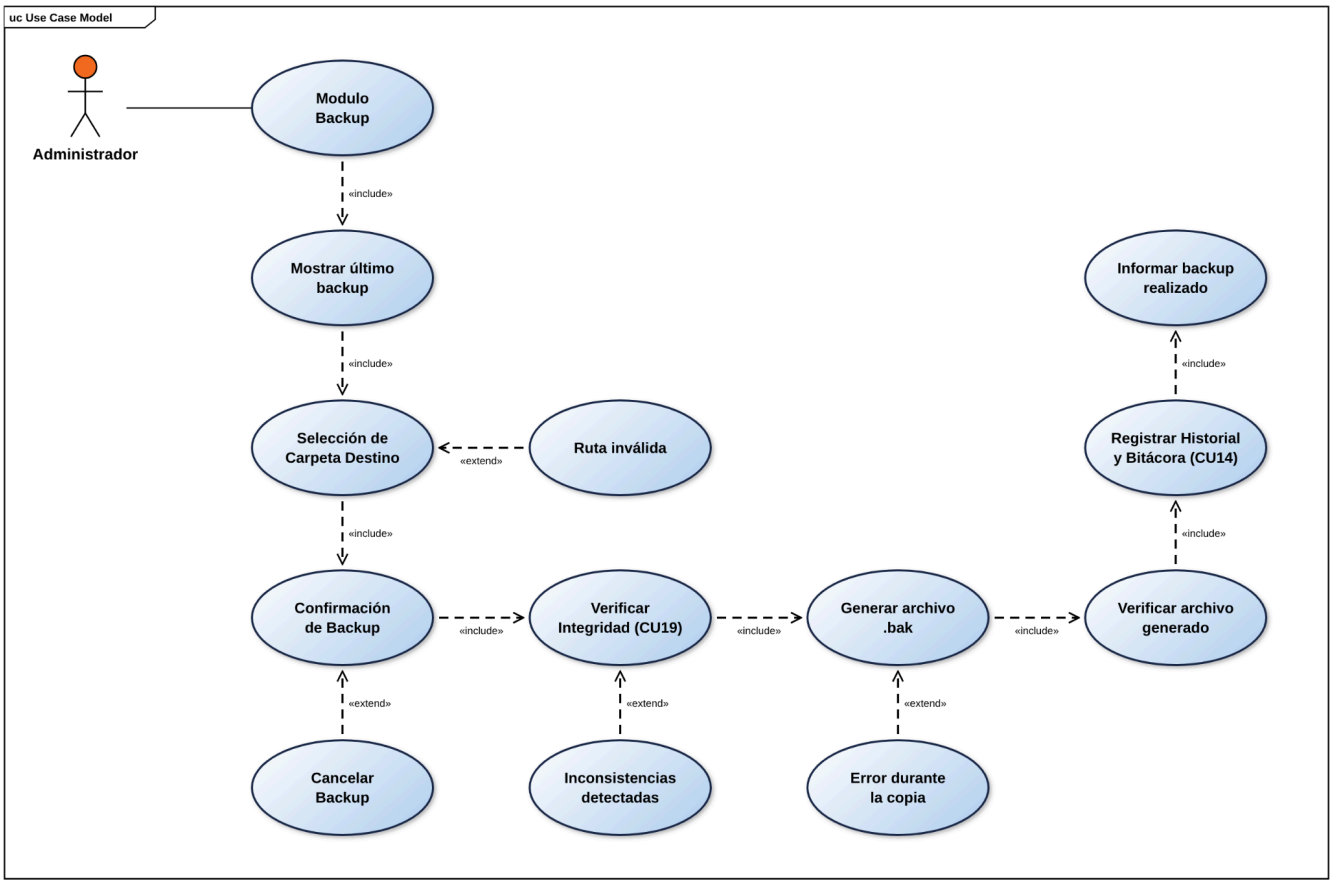

sd Consultar Bitacora
Administrador
frmBitacora
GestorBitacora
BaseDatos
frmBitacora_Load()
ObtenerEventosDelDia()
ObtenerRegistros(consultaDelDia)
DataTable
List<Bitacora>
MostrarEventos(List<Bitacora>)
btnBuscar_Click()
ValidarFiltros()
alt Rango de fechas inválido
Rango de fechas inválido
[Filtros válidos]
BuscarEventos(desde, hasta, usuario, modulo, criticidad)
ObtenerRegistros(consultaFiltrada)
DataTable
List<Bitacora>
alt Sin resultados
No se encontraron eventos
[Con resultados]
MostrarEventos(List<Bitacora>)
opt Ver detalle del evento
btnVerDetalle_Click()
ObtenerDetalle(idBitacora)
Bitacora
MostrarDetalle(Bitacora)
opt Exportar a Excel
btnExportar_Click()
ExportarExcel(eventos, ruta)
alt Error al exportar
false: bool
No se pudo generar el archivo
[Archivo generado]
true: bool
Archivo exportado correctamente

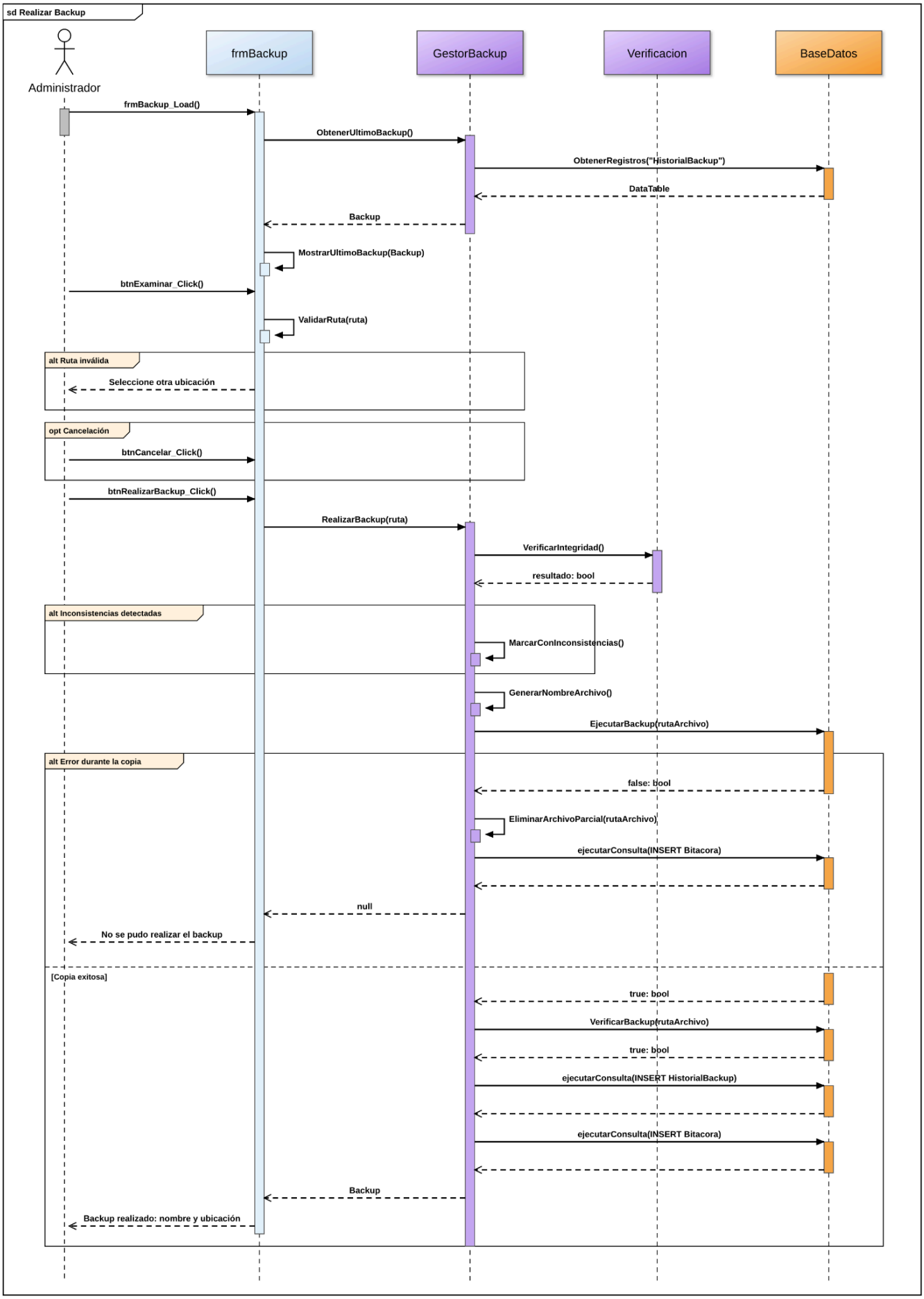


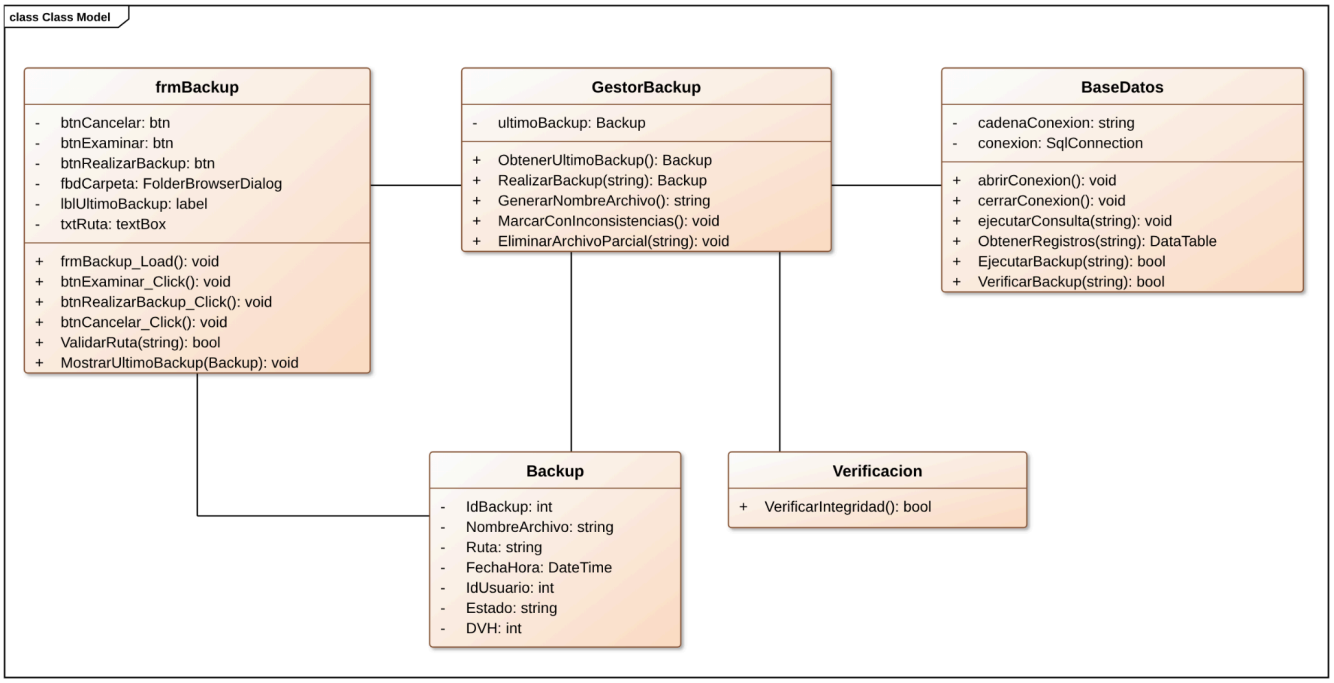
CUS03 - Realizar Backup

ID y Nombre	CUS03 Realizar Backup	
Descripción	Permite al Administrador generar una copia de seguridad completa de la base de datos de Gestorix en la carpeta que elija (disco local, pendrive o disco externo). Antes de copiar, el sistema verifica la integridad de los datos mediante los dígitos verificadores. El nombre del archivo se genera automáticamente con la fecha y hora (por ejemplo: Gestorix_20261020_2200.bak) y, una vez creado, el sistema comprueba que el archivo sea válido y lo registra en el historial de backups y en la bitácora.	
Actores	Administrador.	
Pre-Condición	El administrador debe haber iniciado sesión y poseer el permiso de Realizar Backup (8.3.0). La ubicación de destino debe estar disponible y contar con espacio suficiente.	
Post-Condición	Se genera el archivo de backup en la ubicación seleccionada. El backup queda registrado en el historial de backups. La operación queda registrada en la bitácora.	
FLUJO	**Flujo Normal**	**Flujo Alternativo**
	1. El administrador accede al módulo Ajustes > Backup.	

	2. El sistema obtiene de la base de datos el último backup realizado y lo muestra junto con la carpeta de destino por defecto.	
	3. El administrador presiona "Examinar" y selecciona la carpeta de destino.	3.1 Ruta inválida La carpeta no existe o no tiene permisos de escritura. El sistema solicita seleccionar otra ubicación.
	4. El administrador presiona "Realizar Backup".	4.1 Cancelación El administrador presiona "Cancelar". No se genera el backup y se cierra la ventana.
	5. El sistema verifica la integridad de la base de datos mediante los dígitos verificadores.	5.1 Inconsistencias detectadas El sistema continúa con el backup y lo marca como "Con inconsistencias" en el historial.
	6. El sistema genera el nombre del archivo y ejecuta el backup de la base de datos.	6.1 Error durante la copia Espacio insuficiente o dispositivo desconectado. El sistema elimina el archivo incompleto, registra el error en la bitácora e informa que no se pudo realizar el backup.
	7. El sistema verifica que el archivo generado sea válido.	
	8. El sistema registra el backup en el historial de backups y en la bitácora.	
	9. El sistema informa que el backup se realizó correctamente, indicando nombre y ubicación del archivo.	


uc Use Case Model
Administrador
Modulo Backup
«include»
Mostrar último backup
«include»
Selección de Carpeta Destino
«extend»
Ruta inválida
«include»
Confirmación de Backup
«include»
Verificar Integridad (CU19)
«include»
Generar archivo .bak
«include»
Verificar archivo generado
«include»
Registrar Historial y Bitácora (CU14)
«include»
Informar backup realizado
«extend»
Cancelar Backup
«extend»
Inconsistencias detectadas
«extend»
Error durante la copia


sd Realizar Backup
Administrador
frmBackup
GestorBackup
Verificacion
BaseDatos
frmBackup_Load()
ObtenerUltimoBackup()
ObtenerRegistros("HistorialBackup")
DataTable
Backup
MostrarUltimoBackup(Backup)
btnExaminar_Click()
ValidarRuta(ruta)
alt Ruta inválida
Seleccione otra ubicación
opt Cancelación
btnCancelar_Click()
btnRealizarBackup_Click()
RealizarBackup(ruta)
VerificarIntegridad()
resultado: bool
alt Inconsistencias detectadas
MarcarConInconsistencias()
GenerarNombreArchivo()
EjecutarBackup(rutaArchivo)
alt Error durante la copia
false: bool
EliminarArchivoParcial(rutaArchivo)
ejecutarConsulta(INSERT Bitacora)
null
No se pudo realizar el backup
[Copia exitosa]
true: bool
VerificarBackup(rutaArchivo)
true: bool
ejecutarConsulta(INSERT HistorialBackup)
ejecutarConsulta(INSERT Bitacora)
Backup
Backup realizado: nombre y ubicación

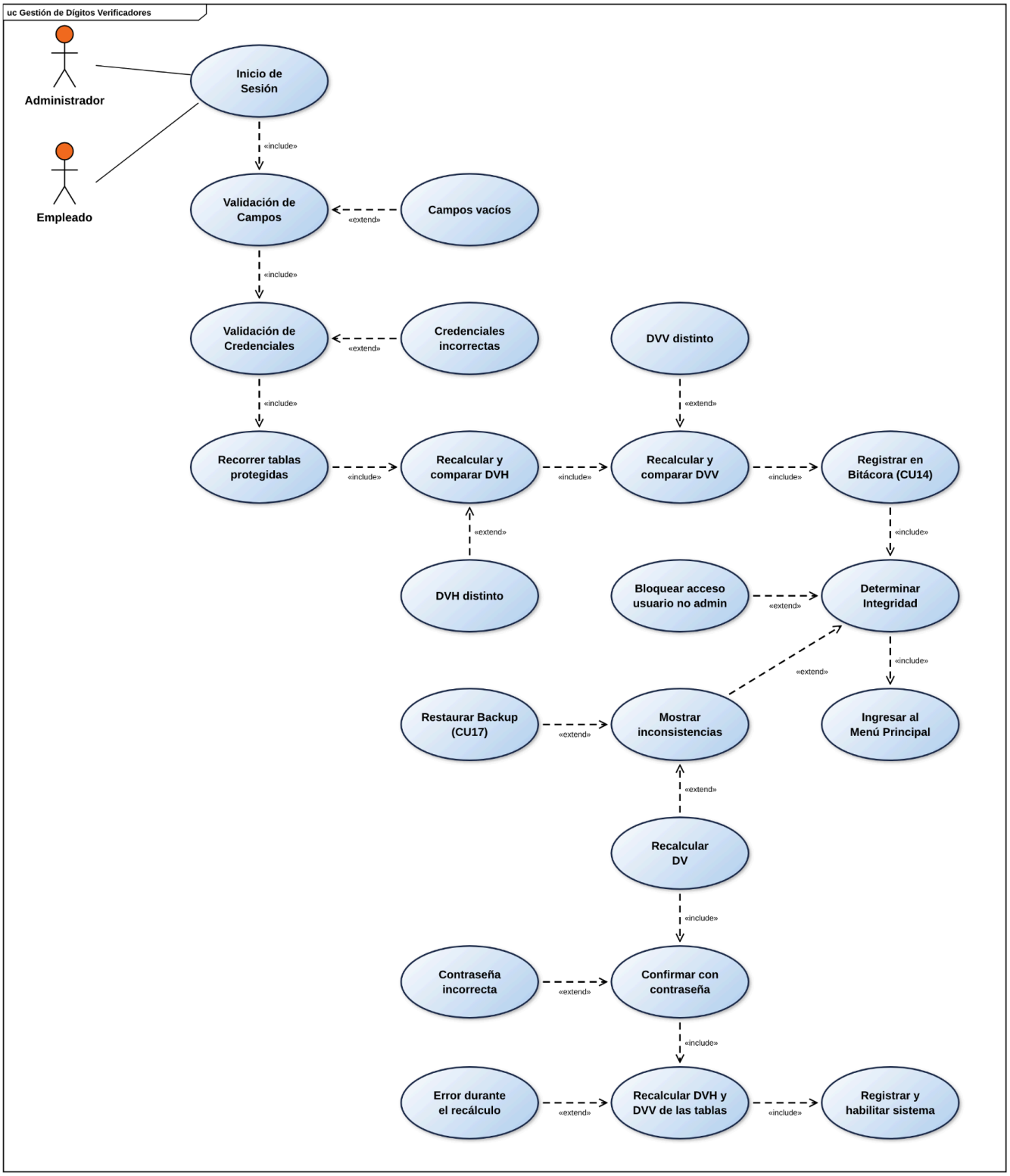


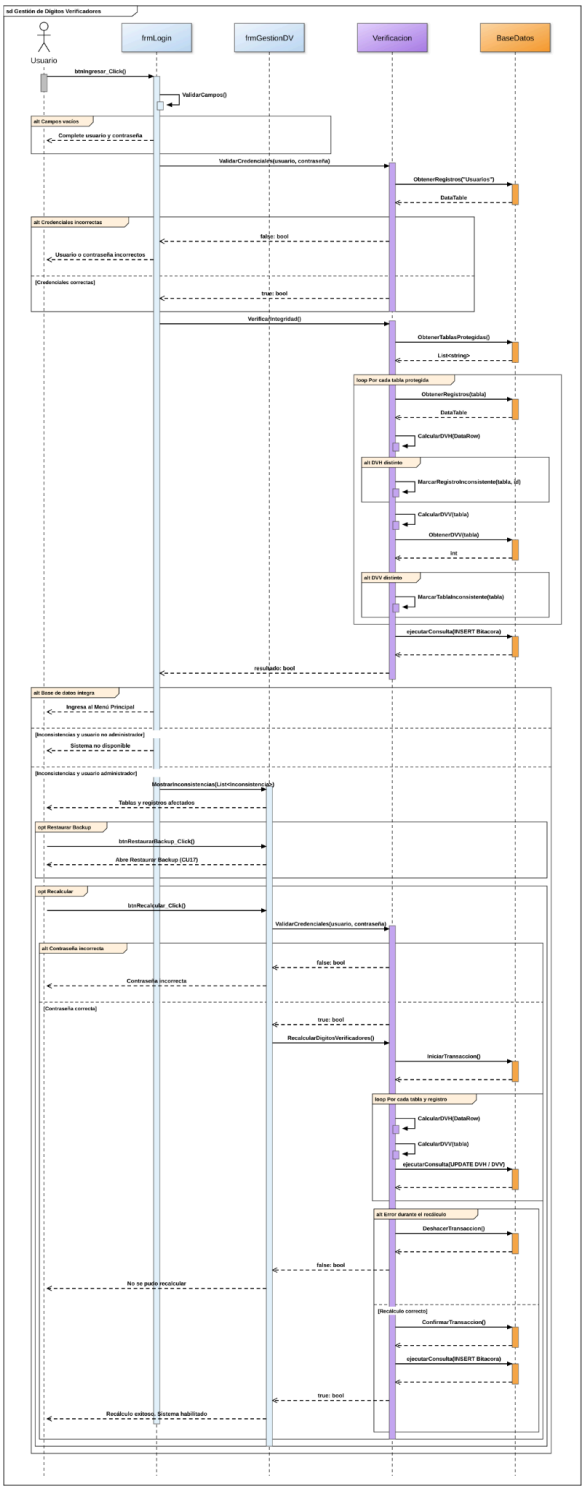
CUS04 - Gestión de Dígitos Verificadores

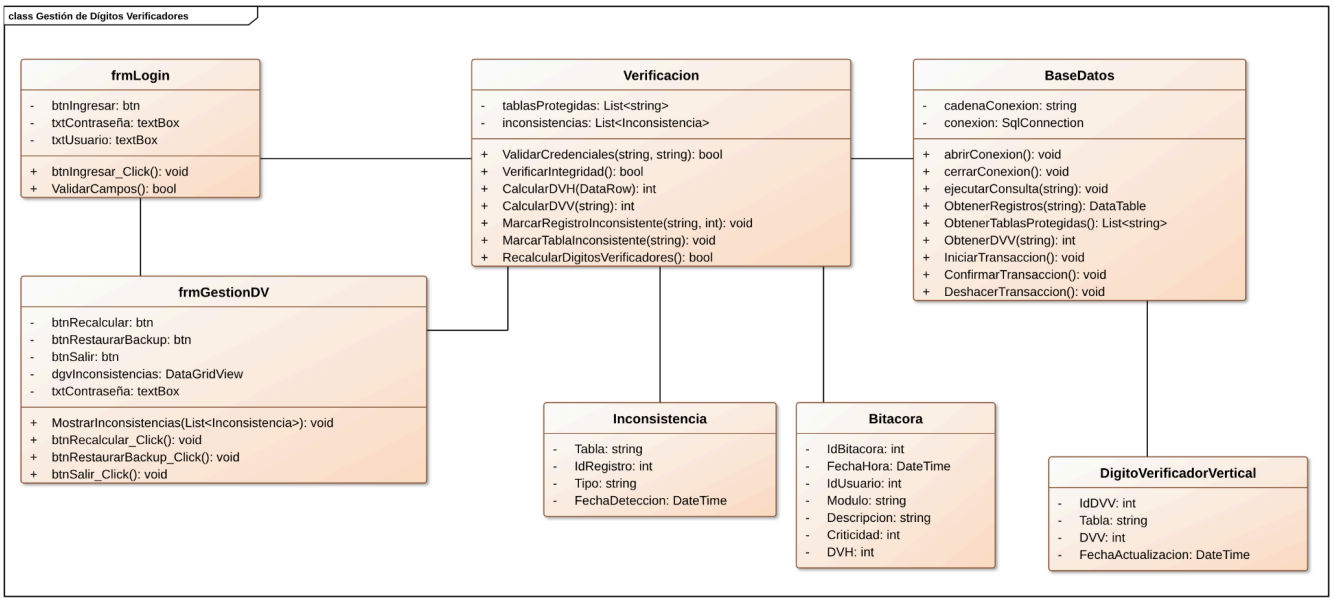
ID y Nombre	**CUS04 Gestión de Dígitos Verificadores**
Descripción	Permite controlar que la información de la base de datos no haya sido alterada por fuera del sistema. En cada inicio de sesión, luego de validar las credenciales, el sistema recorre las tablas protegidas, recalcula el Dígito Verificador Horizontal (DVH) de cada registro y el Dígito Verificador Vertical (DVV) de cada tabla, y los compara con los valores almacenados. Si la base de datos es íntegra, el usuario ingresa normalmente. Si hay inconsistencias, se bloquea el acceso a los usuarios no administradores y se muestra al Administrador la pantalla de Gestión de Dígitos Verificadores, desde donde puede recalcular los dígitos verificadores (aceptando los datos actuales como válidos) o restaurar un backup. Los DVH y DVV se calculan además automáticamente cada vez que se da de alta o se modifica un registro de una tabla protegida.
Actores	Administrador - Empleado.
Pre-Condición	El usuario debe estar registrado en el sistema. La base de datos debe estar accesible. Las tablas protegidas deben tener sus DVH y DVV calculados.

Post-Condición	Si los datos son íntegros, el usuario ingresa al menú principal. Si existen inconsistencias, quedan registradas en la bitácora y el acceso queda restringido al administrador. Si el administrador recalcula, todos los DVH y DVV quedan actualizados y el sistema vuelve a estar disponible.	
FLUJO	**Flujo Normal**	**Flujo Alternativo**
	1. El usuario ingresa su nombre de usuario y contraseña y presiona "Ingresar".	
	2. El sistema valida que los campos estén completos.	2.1 Campos vacíos El sistema informa que debe completar usuario y contraseña.
	3. El sistema valida las credenciales del usuario.	3.1 Credenciales incorrectas El sistema informa "Usuario o contraseña incorrectos" y no continúa.
	4. El sistema obtiene el listado de tablas protegidas.	
	5. Por cada tabla protegida, el sistema recalcula el DVH de cada registro y lo compara con el almacenado.	5.1 DVH distinto El sistema marca el registro como inconsistente (tabla e identificador).
	6. El sistema recalcula el DVV de cada tabla y lo compara con el almacenado.	6.1 DVV distinto El sistema marca la tabla como inconsistente (posibles registros agregados o eliminados por fuera del sistema).
	7. El sistema registra el resultado de la verificación en la bitácora.	
	8. El sistema determina que la base de datos es íntegra y el usuario ingresa al menú principal.	8.1 Inconsistencias y usuario no administrador El sistema informa "Sistema no disponible. Contacte al administrador" y no permite el ingreso. 8.2 Inconsistencias y usuario administrador El sistema abre la pantalla de Gestión de Dígitos Verificadores con las tablas y registros afectados, y

		las opciones Recalcular, Restaurar Backup o Salir. Continúa en el paso 9.
	9. El administrador presiona "Recalcular" e ingresa su contraseña.	9.1 Restaurar Backup El administrador presiona "Restaurar Backup" y se ejecuta el Caso de uso de Backup. 9.2 Contraseña incorrecta El sistema informa que la contraseña es incorrecta y no continúa.
	10. El sistema recalcula, dentro de una transacción, el DVH de todos los registros y el DVV de todas las tablas protegidas.	10.1 Error durante el recálculo El sistema deshace la transacción, mantiene los valores anteriores e informa el error.
	11. El sistema confirma la transacción y registra la operación en la bitácora.	
	12. El sistema informa que el recálculo fue exitoso y habilita el uso normal del sistema.	


uc Gestión de Dígitos Verificadores
Administrador
Empleado
Inicio de Sesión
Validación de Campos
Campos vacíos
Validación de Credenciales
Credenciales incorrectas
DVV distinto
Recorrer tablas protegidas
Recalcular y comparar DVH
Recalcular y comparar DVV
Registrar en Bitácora (CU14)
DVH distinto
Bloquear acceso usuario no admin
Determinar Integridad
Restaurar Backup (CU17)
Mostrar inconsistencias
Ingresar al Menú Principal
Recalcular DV
Contraseña incorrecta
Confirmar con contraseña
Error durante el recálculo
Recalcular DVH y DVV de las tablas
Registrar y habilitar sistema
«include»
«extend»


sd Gestión de Dígitos Verificadores
Usuario
frmLogin
frmGestionDV
Verificacion
BaseDatos
btningresar_Click()
ValidarCampos()
alt Campos vacíos
Complete usuario y contraseña
ValidarCredenciales(usuario, contraseña)
ObtenerRegistros("Usuarios")
DataTable
alt Credenciales incorrectas
false: bool
Usuario o contraseña incorrectos
[Credenciales correctas]
true: bool
VerificarIntegridad()
ObtenerTablasProtegidas()
List<string>
loop Por cada tabla protegida
ObtenerRegistros(tabla)
DataTable
CalcularDVH(DataRow)
alt DVH distinto
MarcarRegistroInconsistente(tabla, id)
CalcularDVV(tabla)
ObtenerDVV(tabla)
int
alt DVV distinto
MarcarTablaInconsistente(tabla)
ejecutarConsulta(INSERT Bitacora)
resultado: bool
alt Base de datos integra
Ingresa al Menú Principal
[Inconsistencias y usuario no administrador]
Sistema no disponible
[Inconsistencias y usuario administrador]
MostrarInconsistencias(List<InconsistenciaDto>)
Tablas y registros afectados
opt Restaurar Backup
btnRestaurarBackup_Click()
Abre Restaurar Backup (CU17)
opt Recalcular
btnRecalcular_Click()
ValidarCredenciales(usuario, contraseña)
alt Contraseña incorrecta
false: bool
Contraseña incorrecta
[Contraseña correcta]
true: bool
RecalcularDigitosVerificadores()
IniciarTransaccion()
loop Por cada tabla y registro
CalcularDVH(DataRow)
CalcularDVV(tabla)
ejecutarConsulta(UPDATE DVH / DVV)
alt Error durante el recálculo
DeshacerTransaccion()
false: bool
No se pudo recalcular
[Recálculo correcto]
ConfirmarTransaccion()
ejecutarConsulta(INSERT Bitacora)
true: bool
Recálculo exitoso. Sistema habilitado

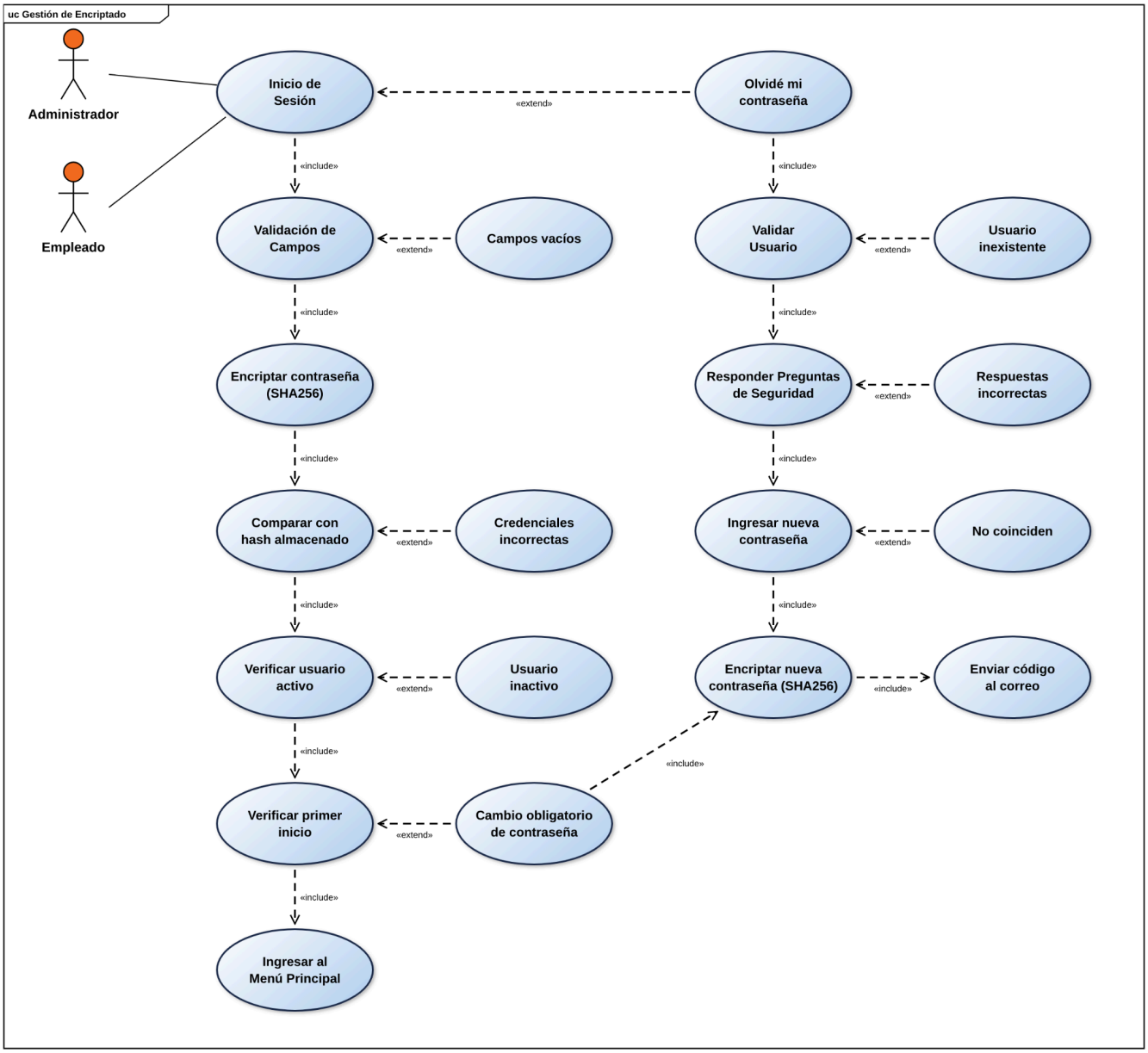


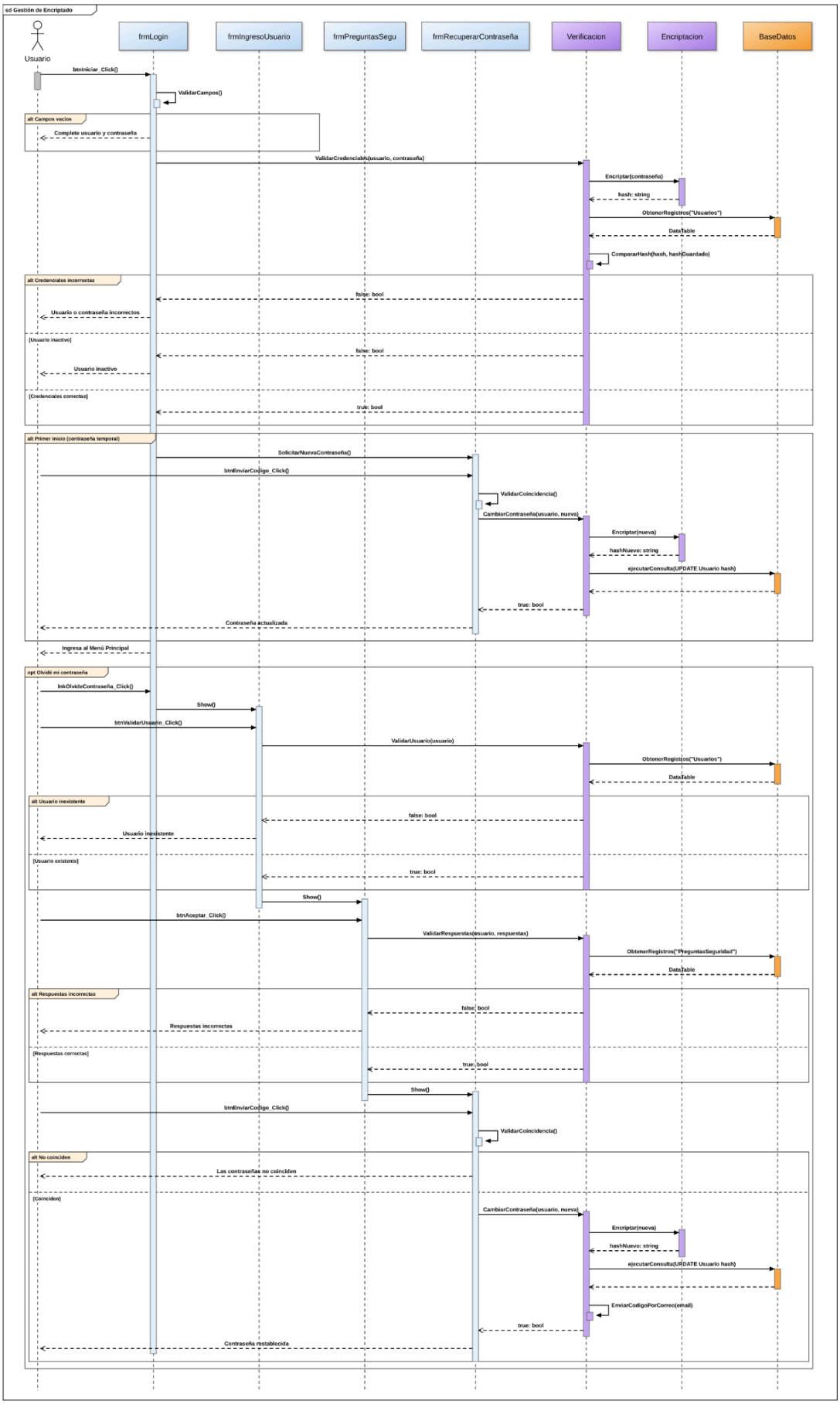
CUS05 - Gestión de Encriptado

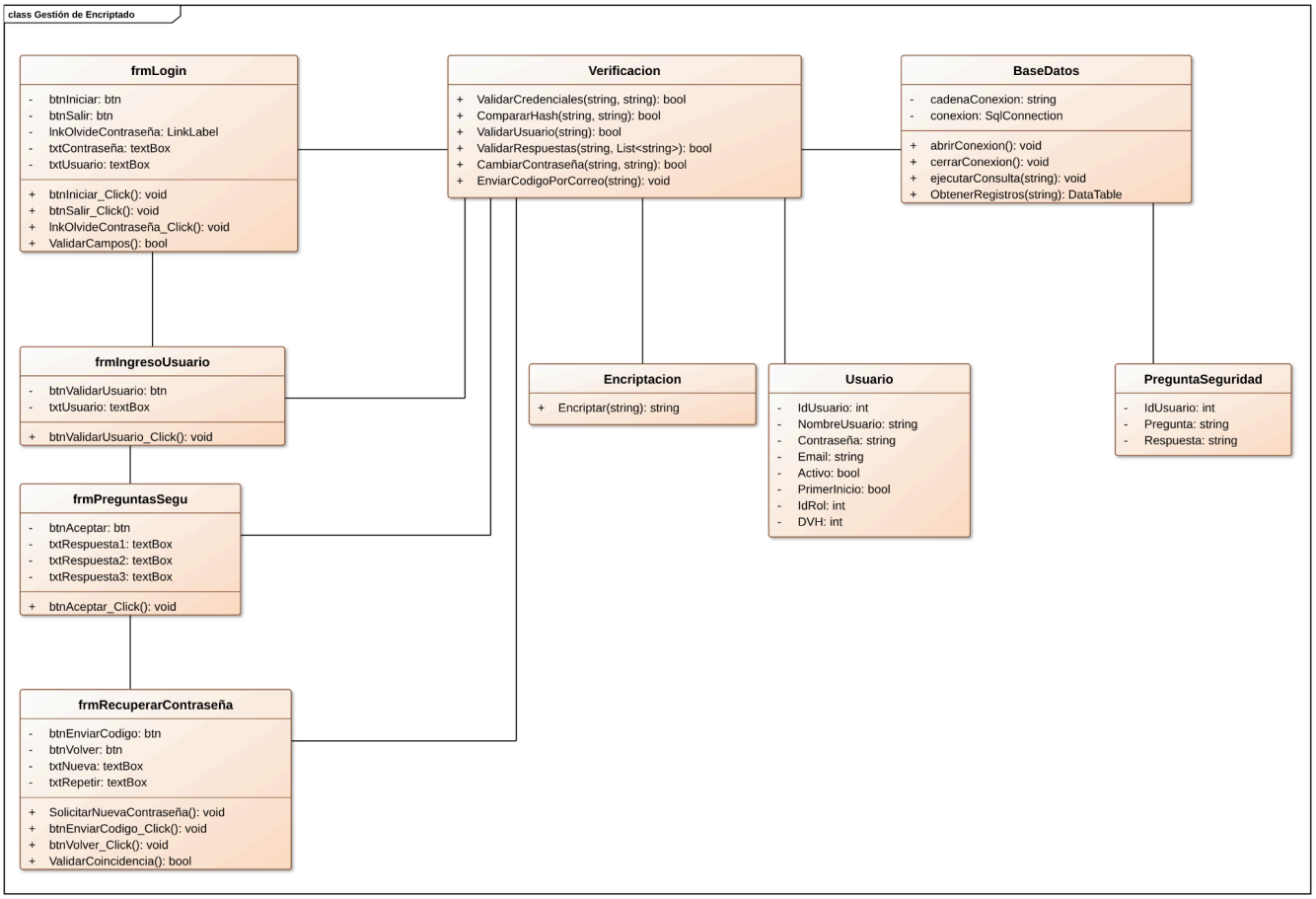
ID y Nombre	CUS05 Gestión de Encriptado	
Descripción	Permite al sistema validar el acceso de los usuarios y actualizar sus contraseñas utilizando encriptado SHA256. Al iniciar sesión, el sistema encripta la contraseña ingresada, la compara con el hash almacenado, verifica que el usuario esté activo y, si es su primer inicio, le exige reemplazar la contraseña temporal. Si el usuario olvidó su contraseña, puede recuperarla validando su usuario y respondiendo sus preguntas de seguridad; la nueva contraseña también se encripta con SHA256 antes de guardarse.	
Actores	Administrador - Empleado.	
Pre-Condición	El usuario debe estar registrado en el sistema. El usuario debe tener cargadas sus preguntas de seguridad. Para la recuperación, debe existir conexión a internet y un servicio de correo configurado.	
Post-Condición	El usuario ingresa al menú principal con los módulos habilitados según su rol. Cuando se cambia o recupera la contraseña, en la base de datos queda guardado únicamente su hash SHA256.	
FLUJO	**Flujo Normal**	**Flujo Alternativo**

	1. El usuario ingresa su nombre de usuario y contraseña en la pantalla Login y presiona "Iniciar".	1.1 Olvidé mi contraseña El usuario presiona "Olvidé mi contraseña". Continúa en el paso 8.
	2. El sistema valida que los campos estén completos.	2.1 Campos vacíos El sistema solicita completar usuario y contraseña.
	3. El sistema encripta la contraseña ingresada con SHA256.	
	4. El sistema obtiene el hash almacenado del usuario y lo compara con el hash obtenido.	4.1 Credenciales incorrectas El sistema informa "Usuario o contraseña incorrectos".
	5. El sistema verifica que el usuario se encuentre activo.	5.1 Usuario inactivo El sistema informa que el usuario está inactivo y no permite el ingreso.
	6. El sistema verifica si es el primer inicio de sesión del usuario.	6.1 Primer inicio (contraseña temporal) El sistema muestra la pantalla Recuperar Contraseña para que el usuario ingrese una nueva contraseña y la repita. El sistema valida que coincidan, la encripta con SHA256 y guarda el hash en reemplazo de la contraseña temporal.
	7. El sistema ingresa al menú principal, habilitando solo los módulos permitidos según el rol del usuario.	
	8. (Recuperación) En la pantalla Ingreso usuario, el usuario ingresa su nombre de usuario y presiona "Validar Usuario".	8.1 Usuario inexistente El sistema informa que el usuario no existe.
	9. El sistema muestra la pantalla Preguntas de Seguridad; el usuario responde y presiona "Aceptar".	9.1 Respuestas incorrectas El sistema informa que las respuestas son incorrectas y no continúa.
	10. El sistema muestra la pantalla Recuperar Contraseña; el usuario ingresa la nueva contraseña, la repite y presiona "Enviar Código".	10.1 No coinciden El sistema informa que las contraseñas no coinciden.

	11. El sistema encripta la nueva contraseña con SHA256 y guarda el hash.	
	12. El sistema envía un código de confirmación al correo electrónico del usuario e informa que la contraseña fue restablecida.	




sd Gestión de Encriptado
Usuario
frmLogin
frmIngresoUsuario
frmPreguntasSegu
frmRecuperarContraseña
Verificacion
Encriptacion
BaseDatos
btnIniciar_Click()
ValidarCampos()
alt Campos vacios
Complete usuario y contraseña
ValidarCredenciales(usuario, contraseña)
Encriptar(contraseña)
hash: string
ObtenerRegistros("Usuarios")
DataTable
CompararHash(hash, hashGuardado)
alt Credenciales incorrectas
false: bool
Usuario o contraseña incorrectos
[Usuario inactivo]
false: bool
Usuario inactivo
[Credenciales correctas]
true: bool
alt Primer inicio (contraseña temporal)
SolicitarNuevaContraseña()
btnEnviarCodigo_Click()
ValidarCoincidencia()
CambiarContraseña(usuario, nueva)
Encriptar(nueva)
hashNuevo: string
ejecutarConsulta(UPDATE Usuario hash)
true: bool
Contraseña actualizada
Ingresa al Menú Principal
opt Olvidé mi contraseña
lnkOlvideContraseña_Click()
Show()
btnValidarUsuario_Click()
ValidarUsuario(usuario)
ObtenerRegistros("Usuarios")
DataTable
alt Usuario inexistente
false: bool
Usuario inexistente
[Usuario existente]
true: bool
Show()
btnAceptar_Click()
ValidarRespuestas(usuario, respuestas)
ObtenerRegistros("PreguntasSeguridad")
DataTable
alt Respuestas incorrectas
false: bool
Respuestas incorrectas
[Respuestas correctas]
true: bool
Show()
btnEnviarCodigo_Click()
ValidarCoincidencia()
alt No coinciden
Las contraseñas no coinciden
[Coinciden]
CambiarContraseña(usuario, nueva)
Encriptar(nueva)
hashNuevo: string
ejecutarConsulta(URDATE Usuario hash)
EnviarCodigoPorCorreo(email)
true: bool
Contraseña restablecida



3.5.9 Especificaciones Adicionales

En este punto se detallan los requisitos y reglas que complementan a los casos de uso CUS01 a CUS05 y que no forman parte de su flujo paso a paso. Todos se basan en las definiciones del proyecto ya establecidas en los puntos 3.1 a 3.4 y 5.0 de este documento.

Consideraciones generales

● Las funcionalidades de los casos de uso CUS01 a CUS05 se desarrollan respetando los estándares de codificación definidos en 3.4.2: lenguaje C#, .NET 8, Windows Forms y SQL Server, arquitectura en capas (Presentación, Lógica y Acceso a datos), consultas SQL parametrizadas, validación de datos en formularios y manejo de excepciones.

● Funcionan sobre Windows 10 o superior, en la computadora de caja del kiosco, con la base de datos SQL Server instalada en forma local y para una única sucursal (3.4 y 3.4.5).

● Las pantallas mantienen la interfaz simple, clara y fácil de aprender definida en 3.1.6.2, ya que los usuarios poseen conocimientos informáticos básicos.

● Cualquier modificación importante de estas funciones deberá ser aprobada por el administrador (dueño del kiosco), según su perfil de stakeholder (3.1.6.3.1).

Desempeño

● La verificación de integridad (CUS04) y el encriptado de la contraseña (CUS05) se ejecutan durante el inicio de sesión, por lo que deben respetar el tiempo máximo de inicio de sesión de 1,5 segundos (3.4.4).

● La consulta de la bitácora (CUS02) debe responder en un máximo de 3 segundos y su exportación a Excel en menos de 5 segundos, igual que las consultas y reportes del sistema (3.4.4).

● Estas funciones deben mantener la estabilidad del sistema durante toda la jornada comercial, de 8:00 a 22:00 hs (3.1.6.2 y 3.4.4).

Múltiples idiomas

● El idioma por defecto del sistema es el Español, idioma en el que está desarrollada la interfaz.

● Los textos de la interfaz (menús, botones, etiquetas, mensajes y encabezados de reportes) se guardan en la base de datos para poder traducirlos (CUS01).

● El cambio de idioma se aplica a todos los formularios abiertos sin reiniciar el sistema, mediante el patrón Observer (CUS01).

● El cambio de idioma solo modifica los textos de la interfaz; no modifica los datos registrados (artículos, ventas, compras, caja).

Bitácora

● La bitácora amplía el registro automático de movimientos definido en 3.3.5 y el historial completo de movimientos incluido en el alcance (3.1.1, punto 24), registrando además las acciones que realizan los usuarios dentro del sistema.

● Cada evento registra fecha y hora, usuario, módulo, descripción y nivel de criticidad: Baja, Media o Alta (CUS02).

● Los registros no pueden modificarse ni eliminarse desde el sistema, en línea con la restricción de eliminación física de datos (3.3.2) y las buenas prácticas de seguridad de la norma ISO/IEC 27001 (3.4.2).

● La consulta y exportación a Excel de la bitácora (CUS02) está restringida a los roles con permiso, y su exportación sigue el mismo formato que los reportes del sistema (3.1.6.2).

Backup

● El backup se genera con las herramientas propias de SQL Server (archivo .bak), ya que el sistema depende de SQL Server como gestor de base de datos (3.4).

● Su objetivo es resguardar la información frente a las dependencias identificadas en 3.2.3: fallas en el suministro eléctrico o en el equipamiento informático del kiosco.

● Se recomienda realizar el backup al finalizar la jornada comercial y guardar una copia en un dispositivo externo a la computadora de caja.

● Solo el administrador puede realizar o restaurar un backup; la restauración requiere confirmar la operación con su contraseña.

Dígitos verificadores

● Responden al requisito de integridad y consistencia de los datos establecido en 3.4.1 y 3.4.3.

● Se calcula un Dígito Verificador Horizontal (DVH) por cada registro y un Dígito Verificador Vertical (DVV) por cada tabla protegida: Usuarios, Roles y Permisos, Artículos, Ventas, Compras, Caja y Bitácora.

● Los DVH y DVV se calculan automáticamente en cada alta o modificación de un registro protegido y se verifican en cada inicio de sesión (CU04).

● Si se detectan inconsistencias, solo el administrador puede ingresar al sistema para recalcular los dígitos verificadores o restaurar un backup (CU03).

Encriptado

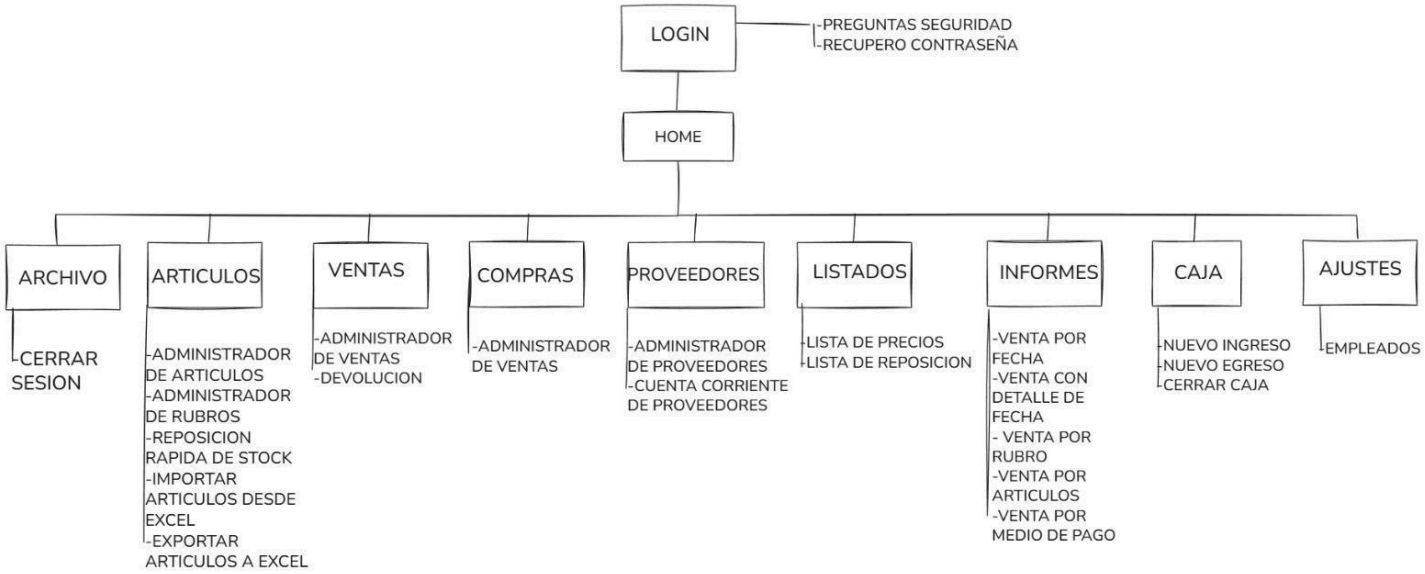
● Las contraseñas se almacenan únicamente como hash SHA256 (encriptado irreversible), según lo definido en 3.3.1, 3.3.7 y 3.4.2.

● La contraseña temporal generada en el alta del empleado debe reemplazarse obligatoriamente en el primer inicio de sesión.

● La recuperación de contraseña se realiza validando al usuario y sus preguntas de seguridad, y depende de la conexión a internet y del servicio de correo electrónico (3.2.3 y 3.4.5).

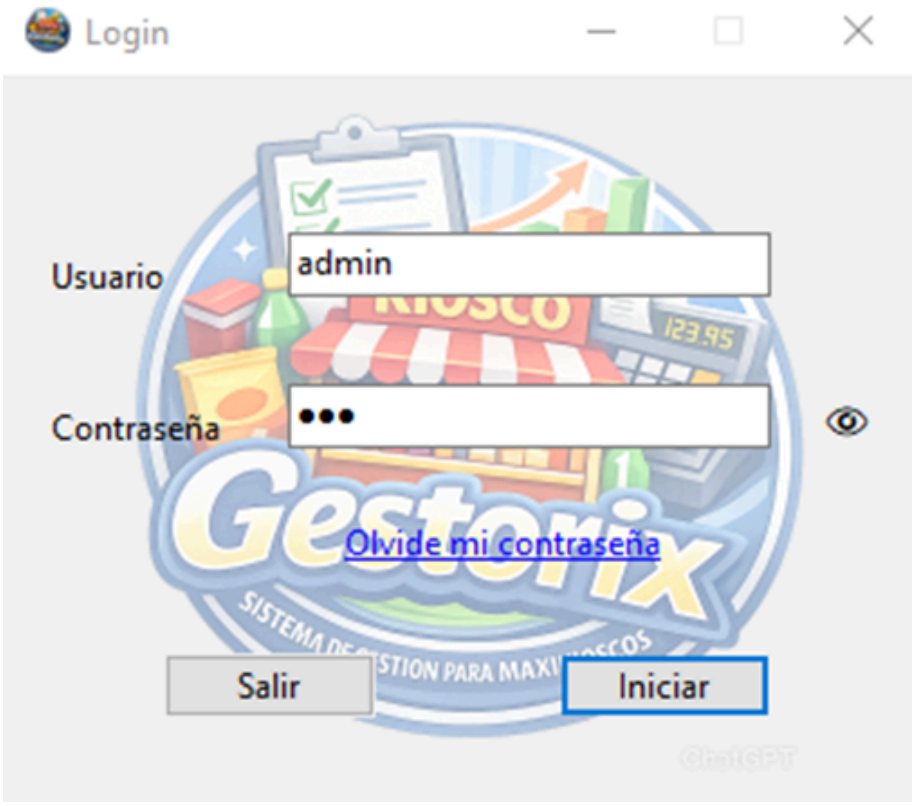
● La clase Encriptacion centraliza el cálculo del hash SHA256 y es utilizada por todos los formularios que manejan contraseñas.

3.5.10 Mapa de Navegación

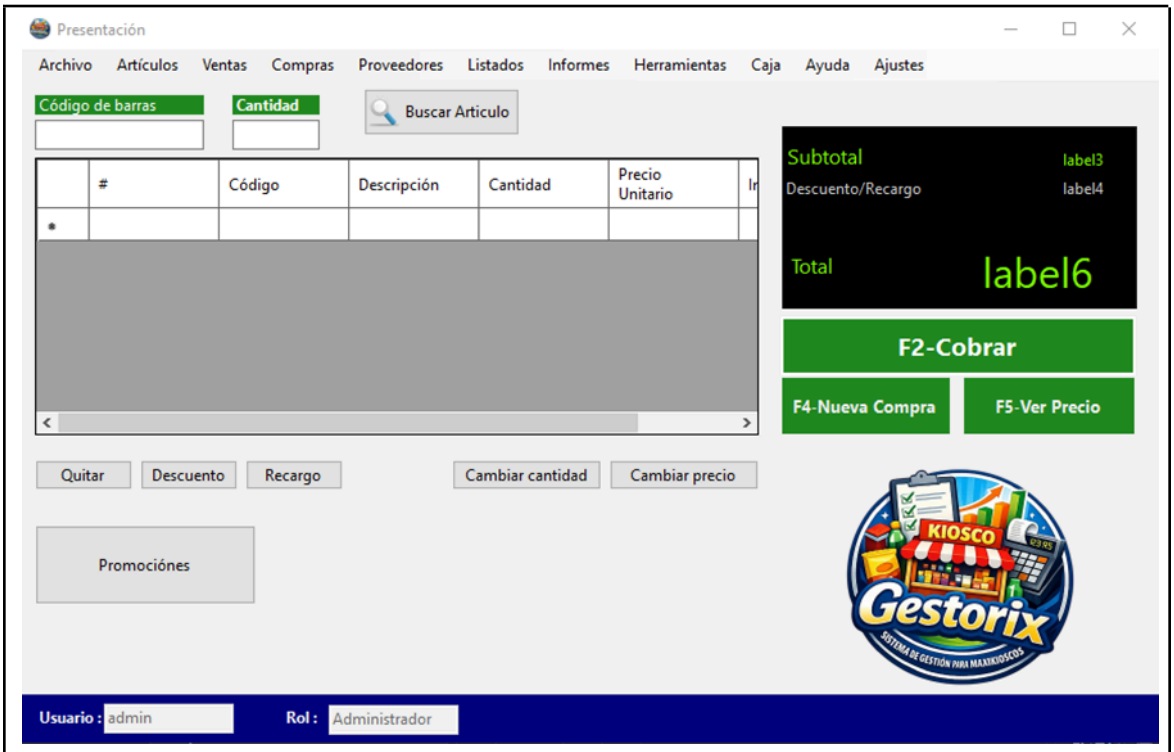


3.6 Prototipos de Interfaces del Usuario

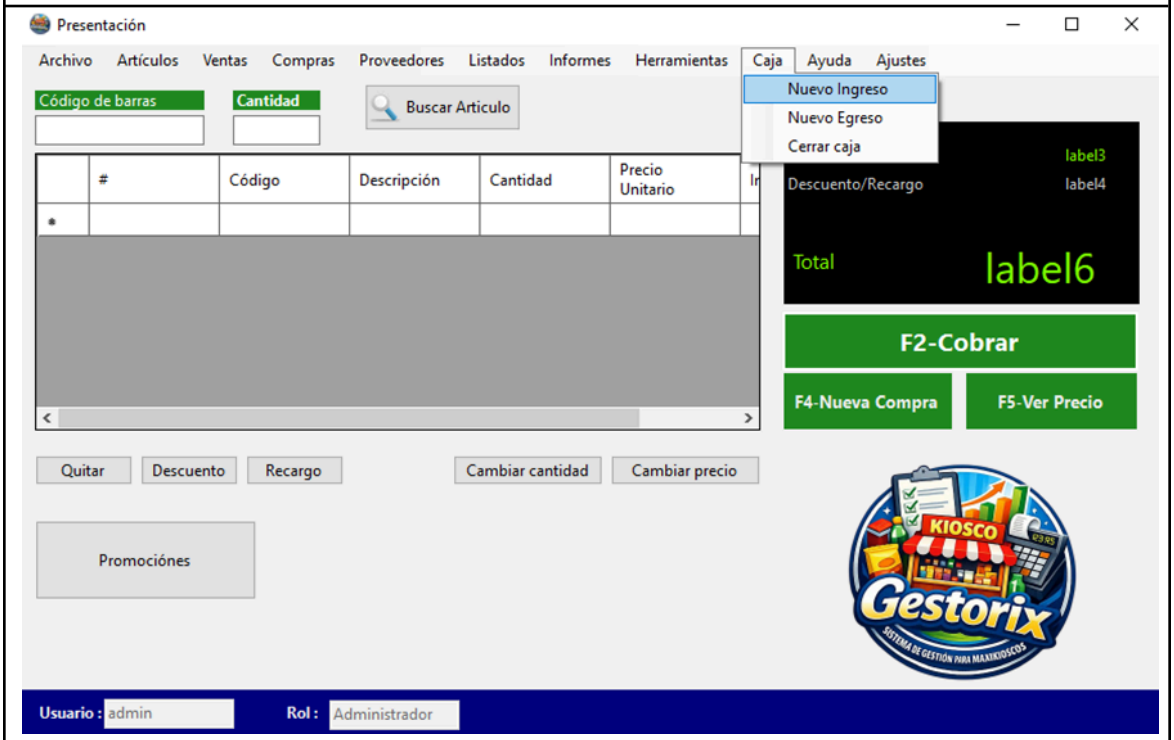
1. Ingreso a la app



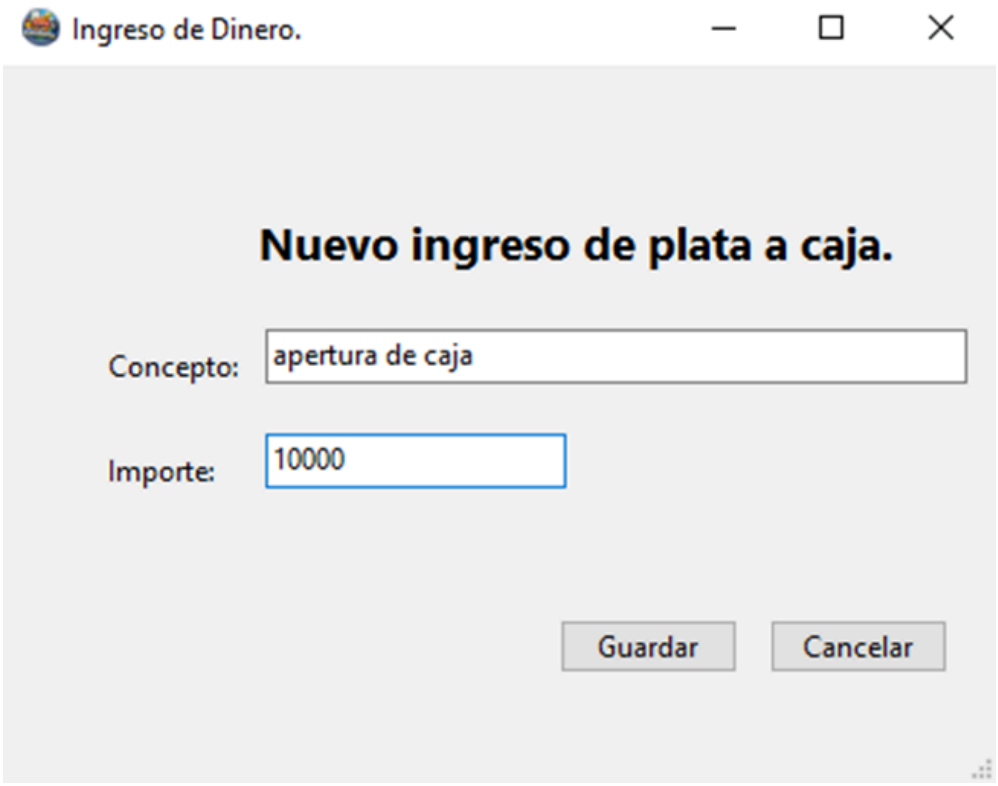
2. Una vez ingresado el usuario, se le mostrará la pantalla Principal.



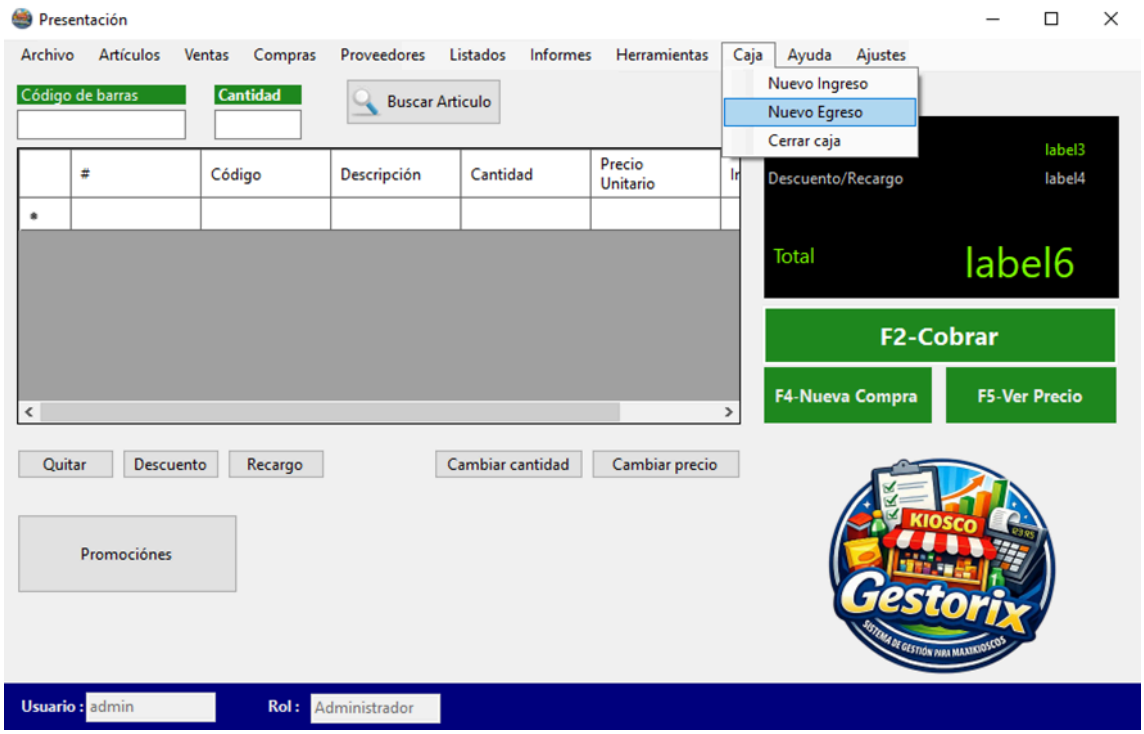
3. El usuario ya está dentro y deberá ingresar el monto con el cual abre **caja**.



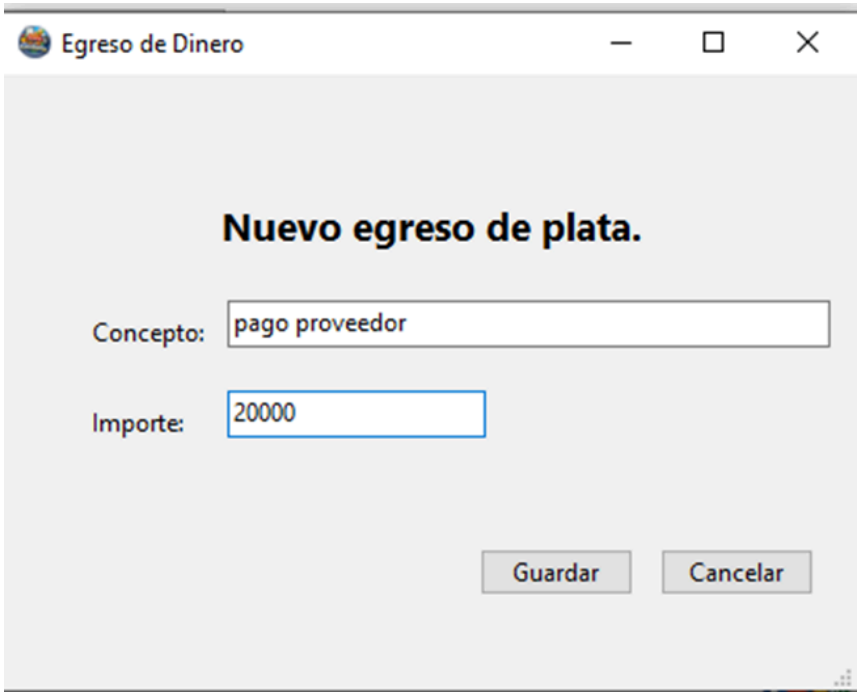
4. En la parte superior hay un menú de navegación donde deberá seleccionar el menú caja.



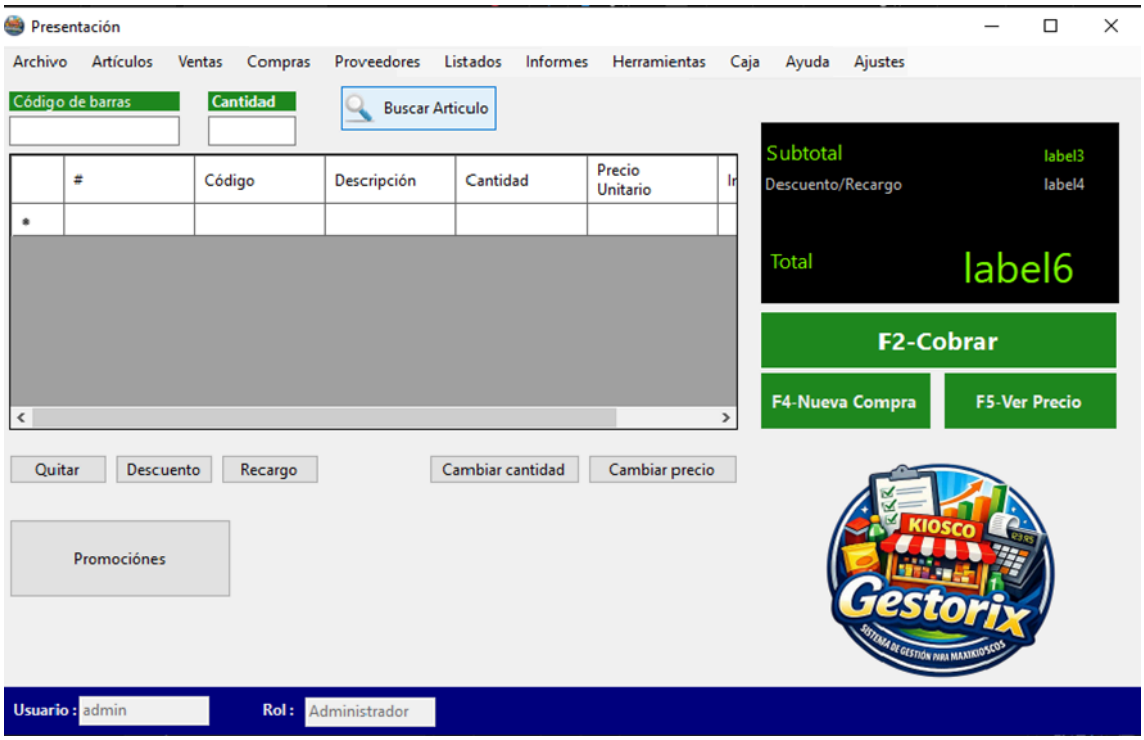
5. Una vez abierto debe agregar el motivo del ingreso de plata a la caja. Luego debe apretar el botón de guardar y lo va a devolver a la pantalla principal.
6. Para el egreso de plata, por pago a un proveedor u otro concepto deberá ir al menú y seleccionar Caja.



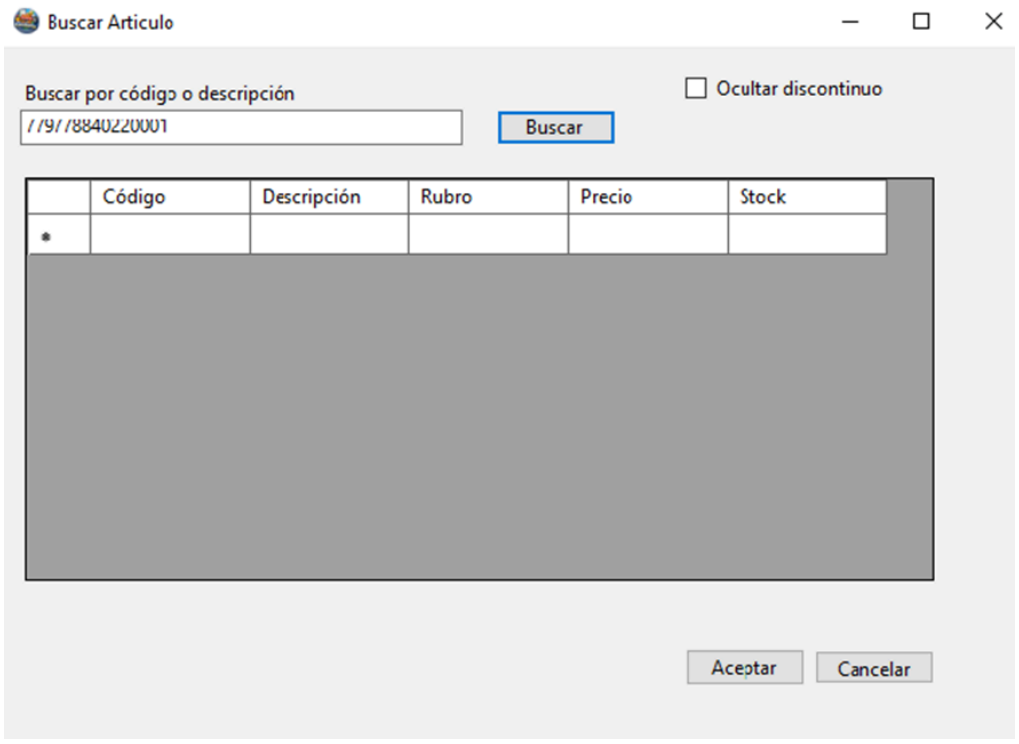
7. Aca debe seleccionar Nuevo Egreso.



8. Deberá ingresar el motivo por el cual se retiró la plata, después de llenar el concepto junto al importe, debe seleccionar Guardar.
9. Vuelve al menu principal. Debe de buscar articulos para empezar a registrar los pedidos.

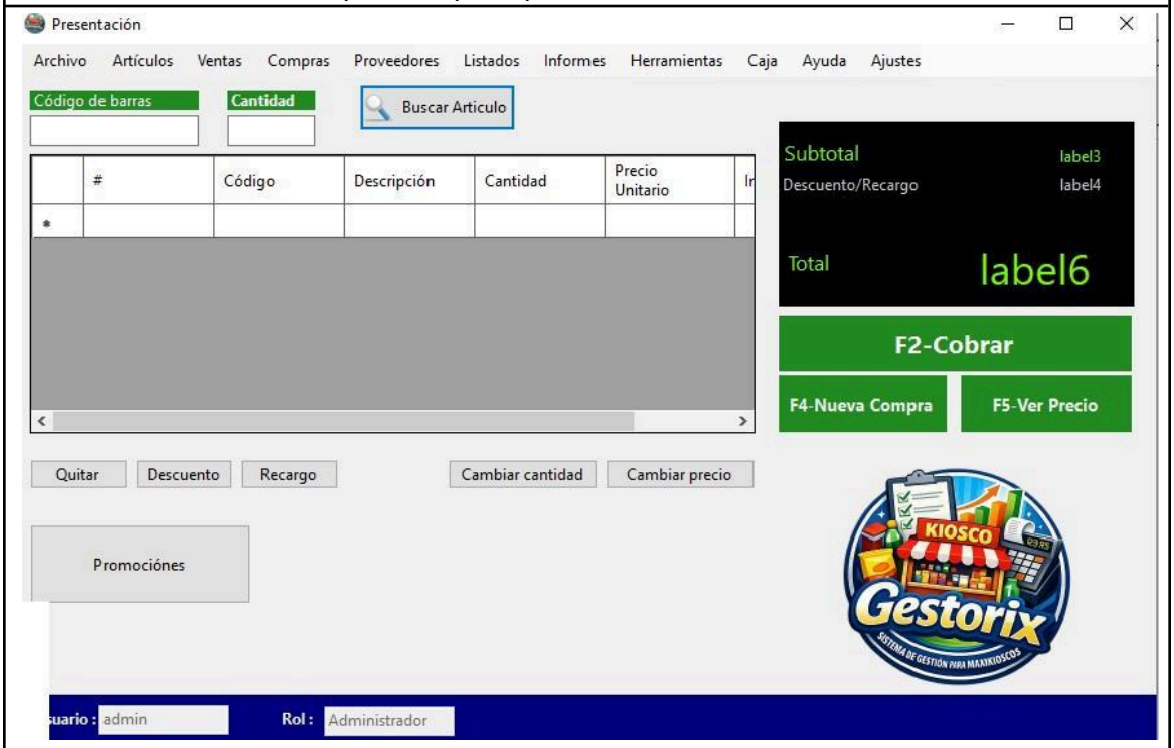


10. Selecciona el botón de **Buscar artículo**.

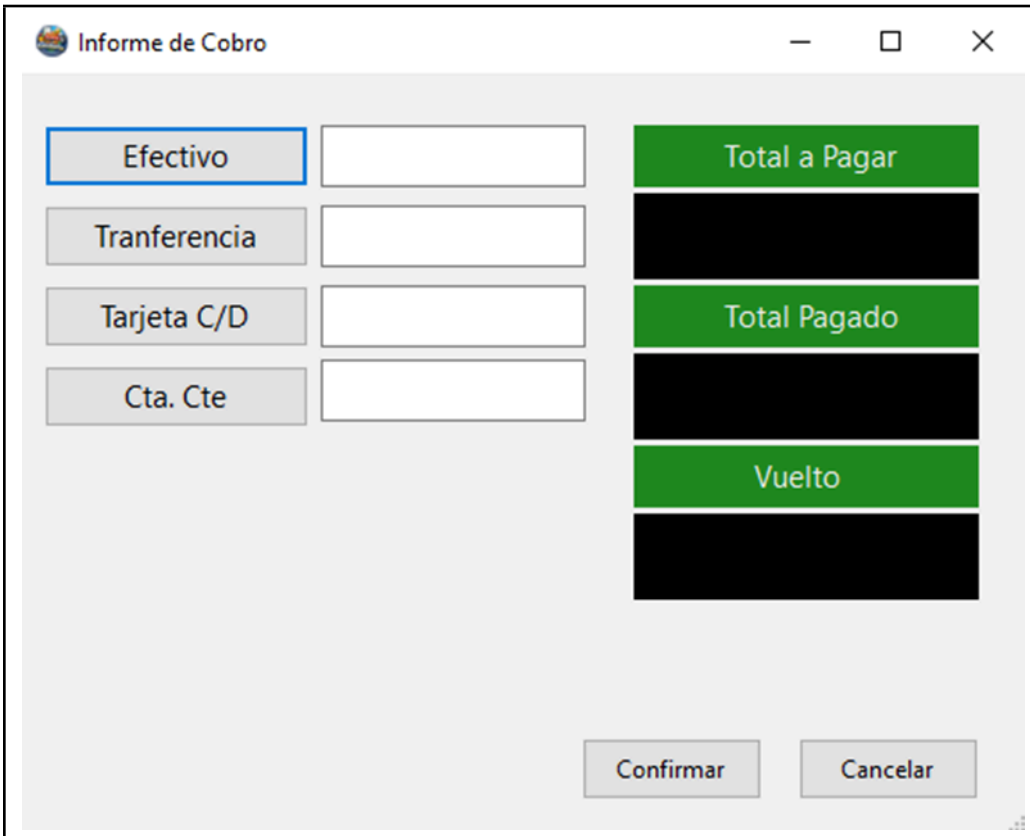


11. Ingresa el código, caso contrario ingresa el nombre o descripción, luego presiona el botón buscar y el o los resultados se mostrarán en las grillas.

12. Una vez seleccionado el producto presiona en botón Aceptar que lo devuelve a la pantalla principal.



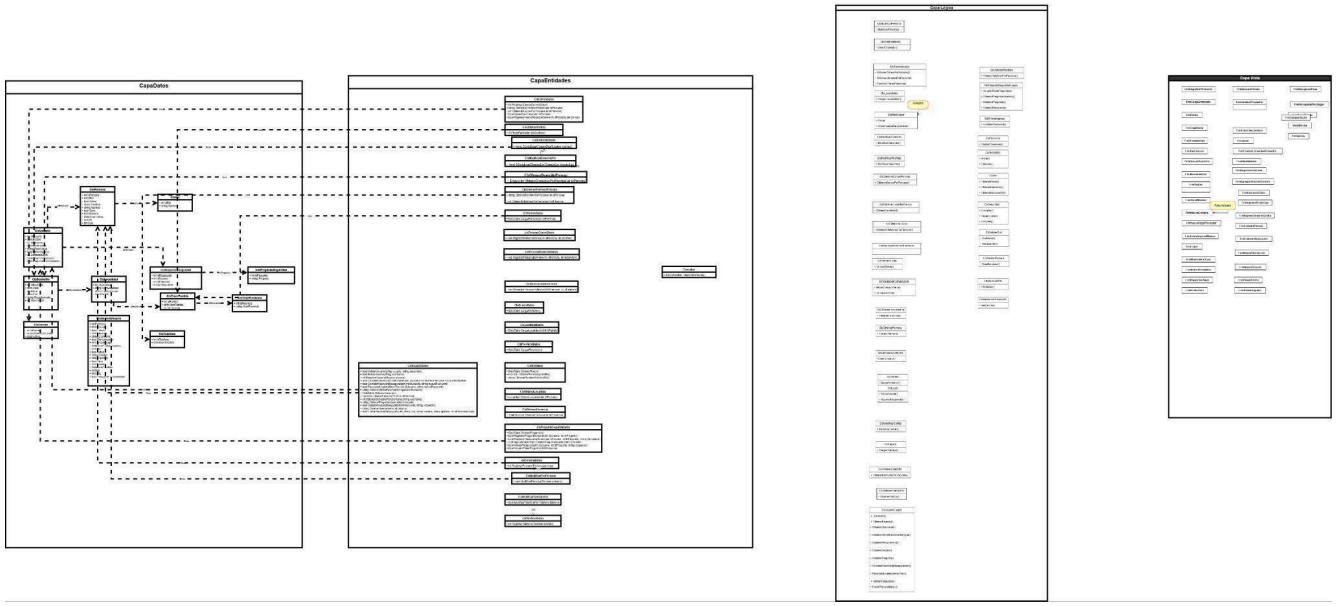
13. El pedido estará cargado en las grillas, para cerrar el pedido, debe seleccionar F2-Cobrar o simplemente oprimir la tecla F2.



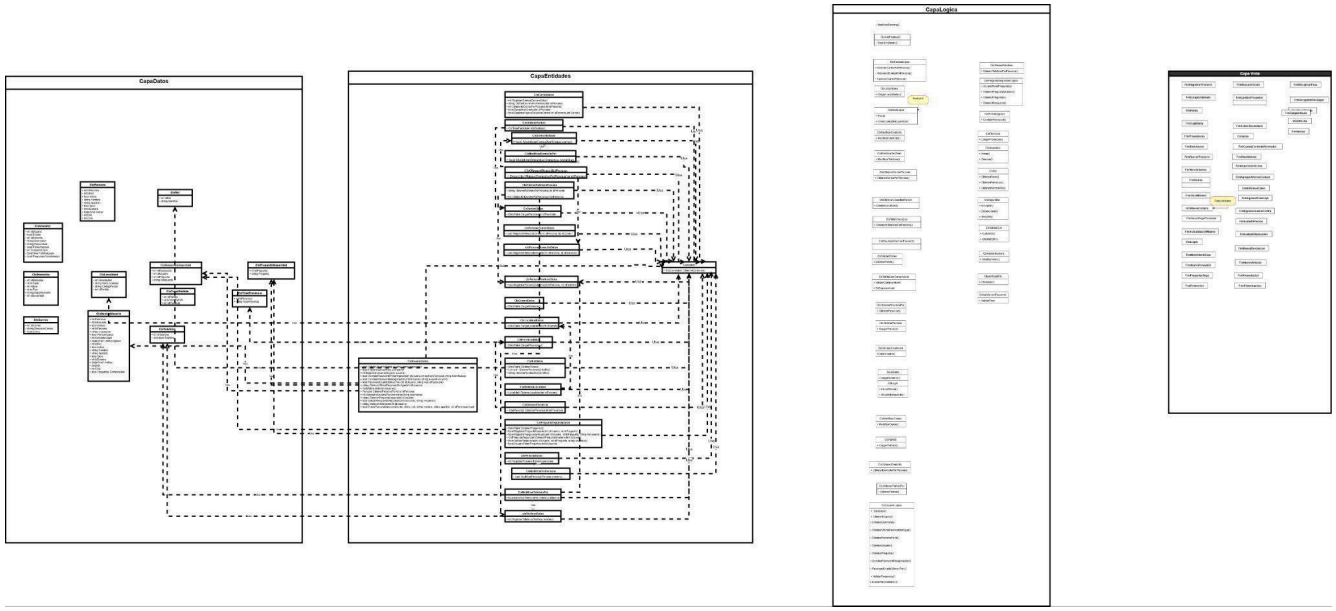
14. Aca debe ingresar cual es el ingreso de plata, ya sea por Efectivo, transferencia, Débito o Cta.Cte.
15. Presiona el botón de Confirmar, si está todo bien se cierra, y lo devuelve a la pantalla principal.

3.7 Diagrama Global de Clases Del Sistema

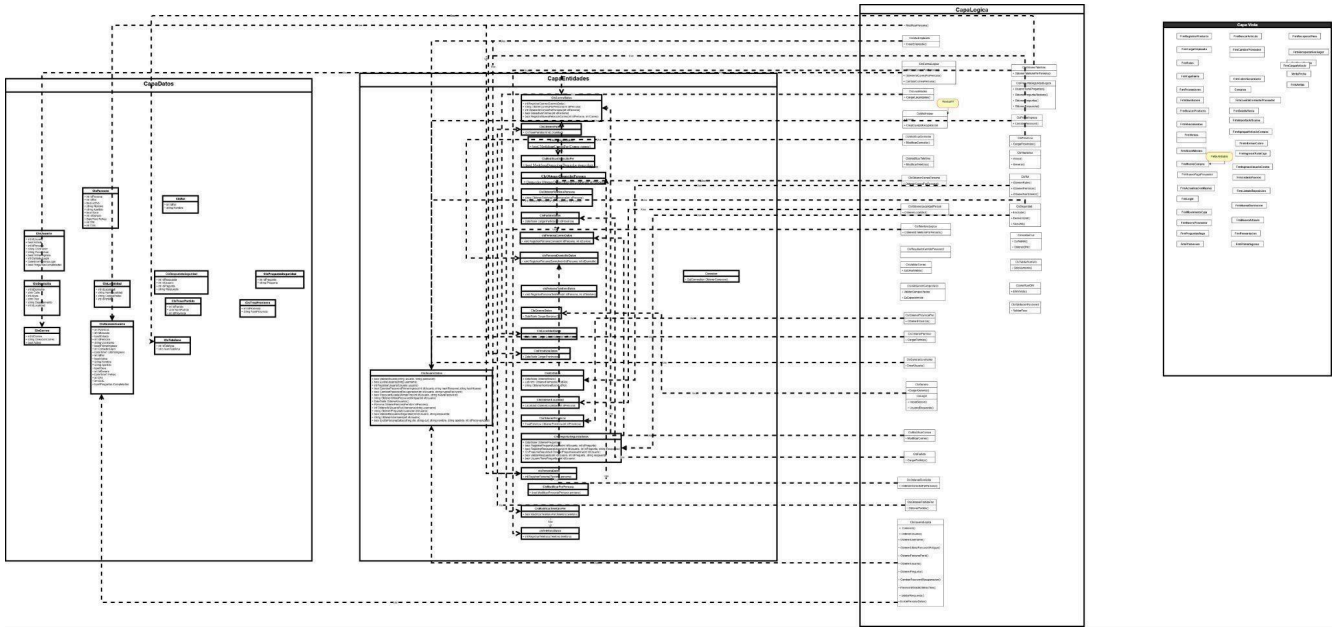
Capa Datos



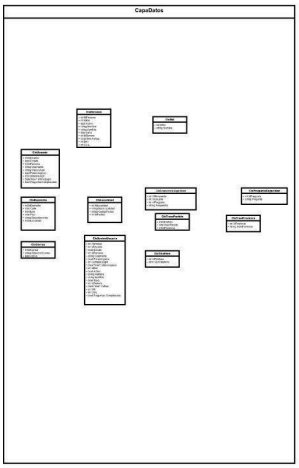
Capa Entidades

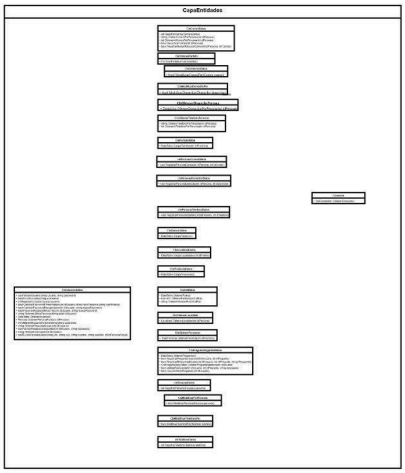


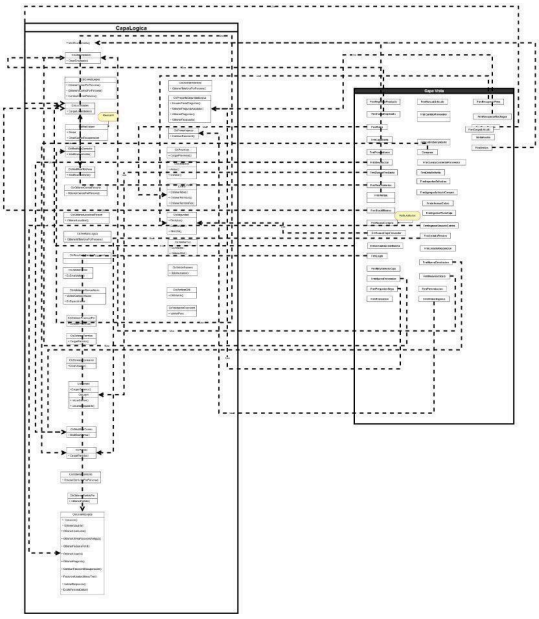
Capa Lógica



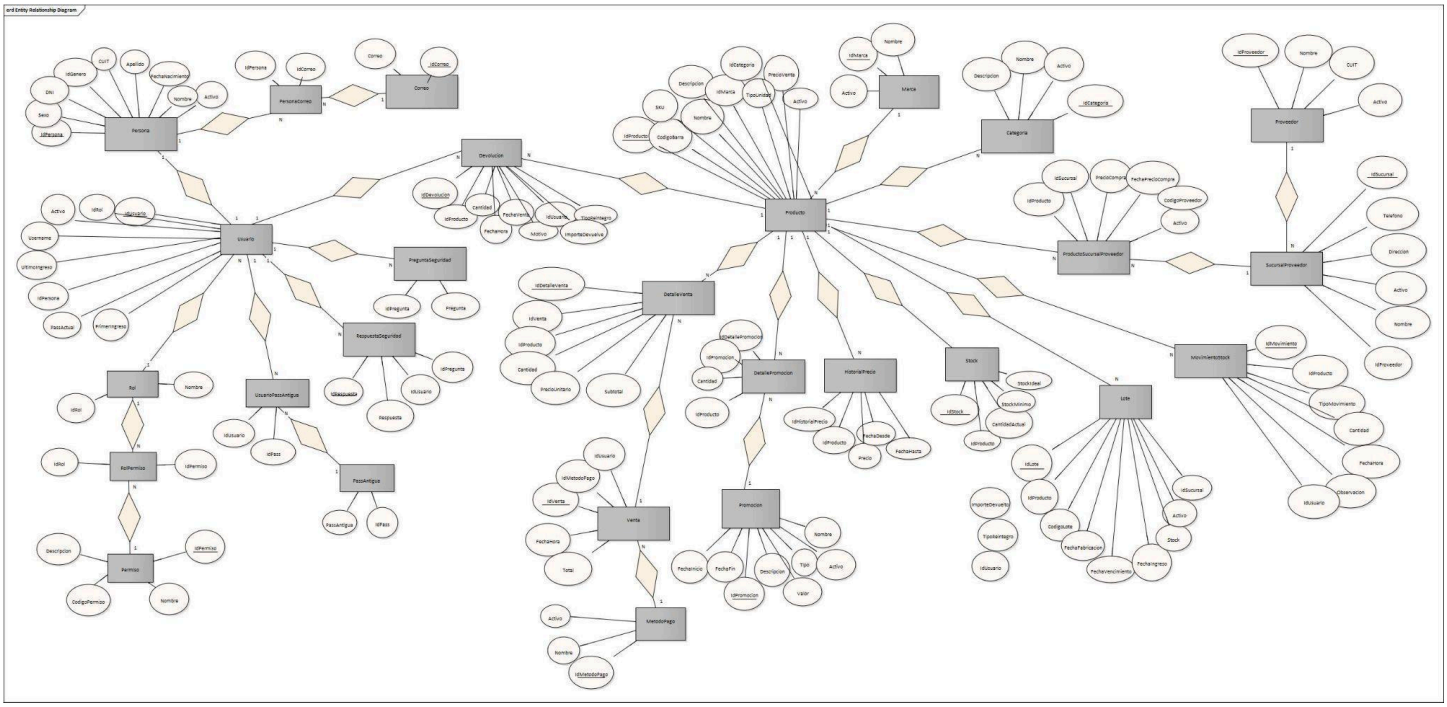
Capa Vista





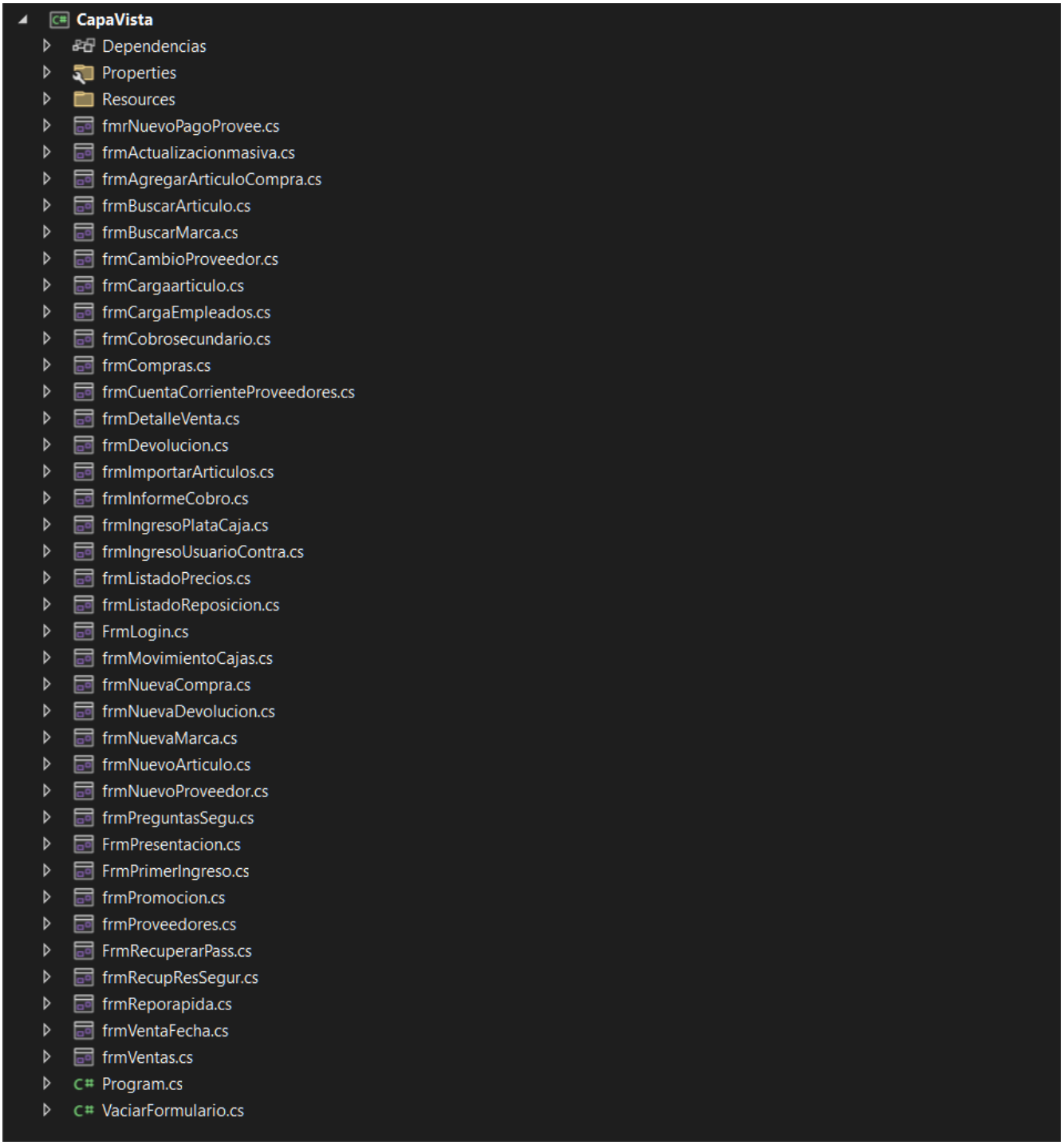


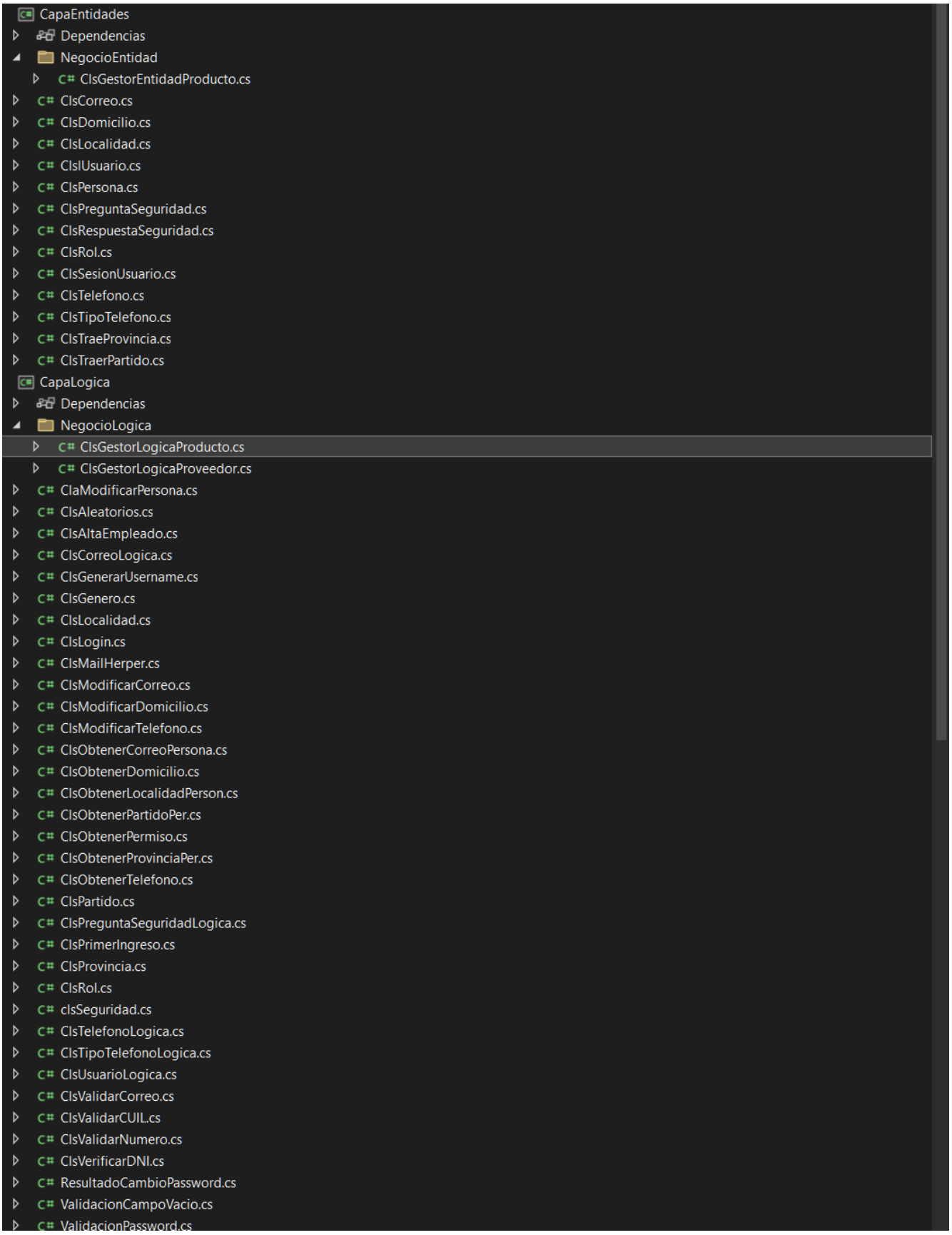
3.9 DER del Sistema

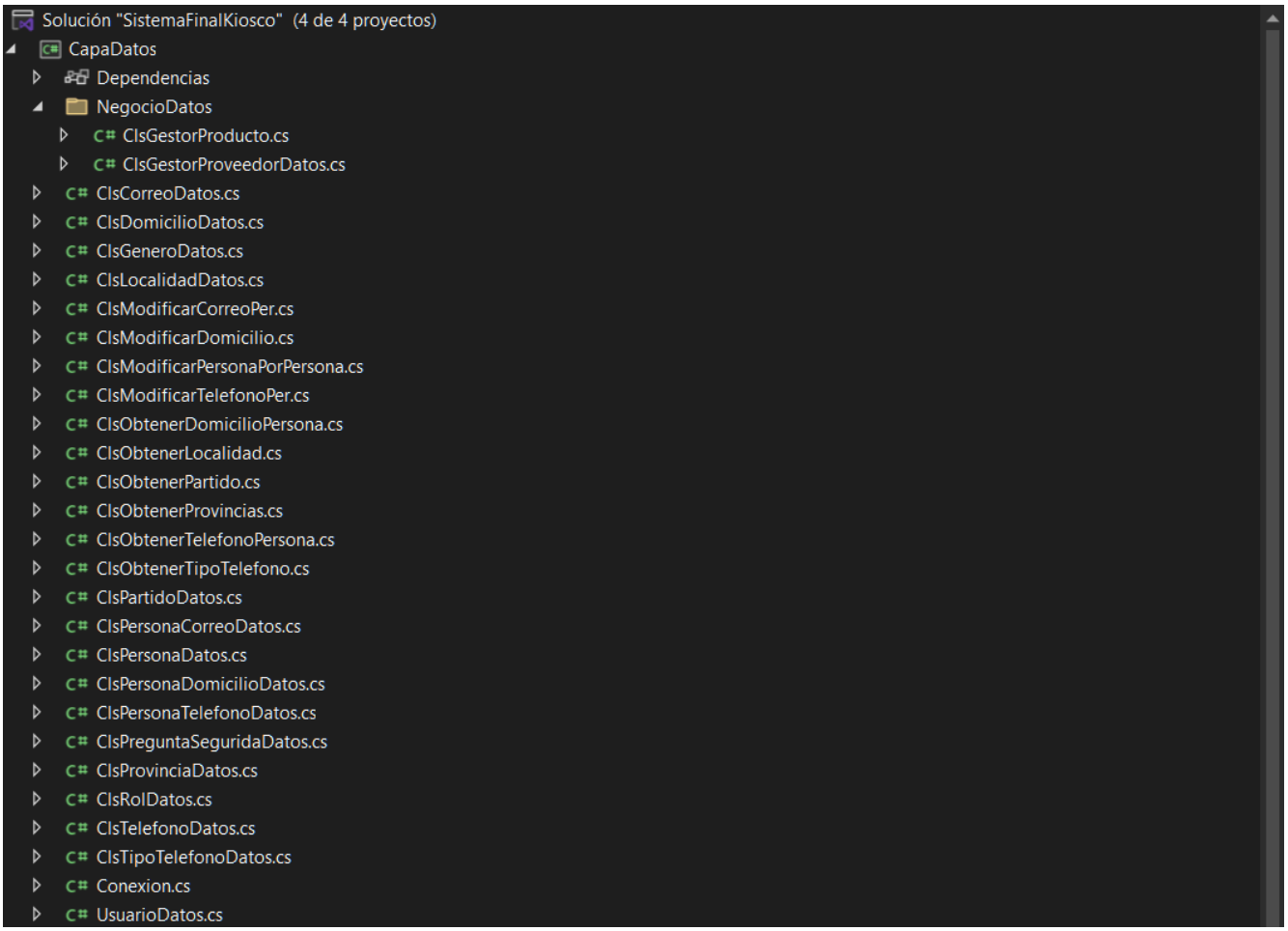


4.0 Estructura principal del software con interfaces

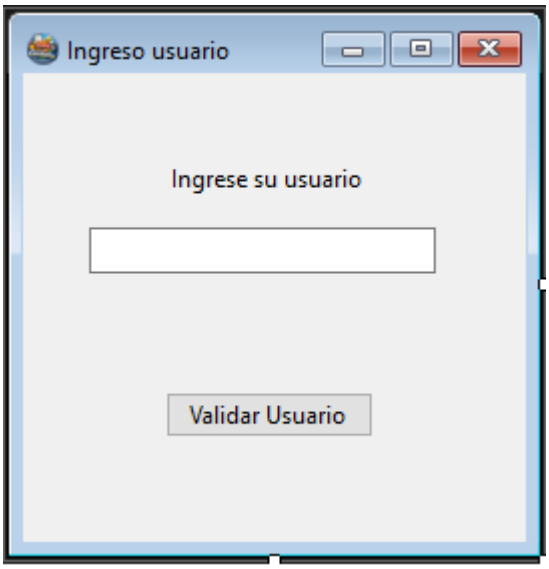
4.1 Estructura de capas del Software

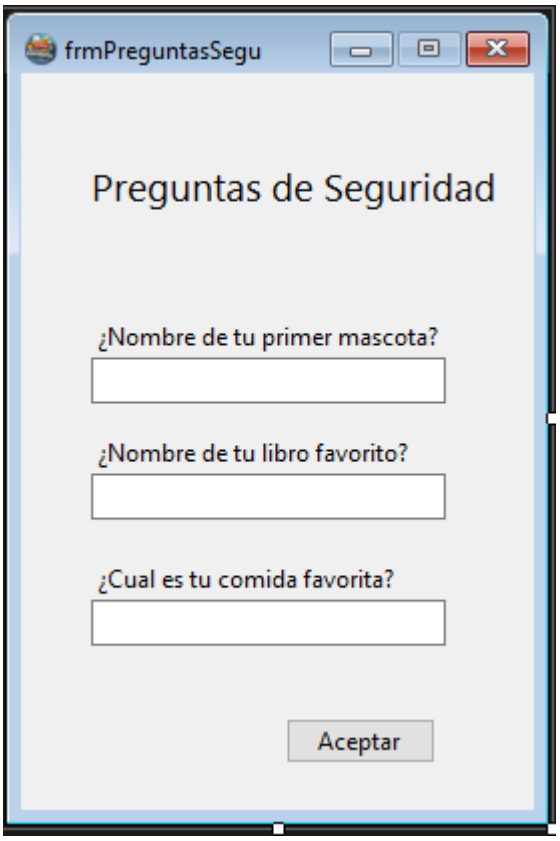


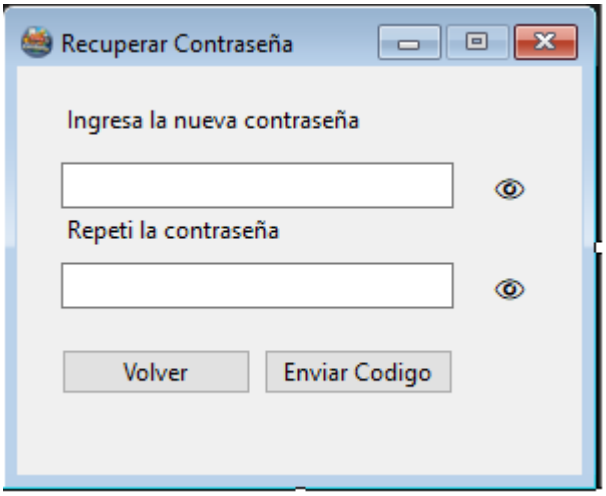

CapaEntidades
Dependencias
NegocioEntidad
ClsGestorEntidadProducto.cs
ClsCorreo.cs
ClsDomicilio.cs
ClsLocalidad.cs
ClsIUsuario.cs
ClsPersona.cs
ClsPreguntaSeguridad.cs
ClsRespuestaSeguridad.cs
ClsRol.cs
ClsSesionUsuario.cs
ClsTelefono.cs
ClsTipoTelefono.cs
ClsTraeProvincia.cs
ClsTraerPartido.cs
CapaLogica
Dependencias
NegocioLogica
ClsGestorLogicaProducto.cs
ClsGestorLogicaProveedor.cs
ClaModificarPersona.cs
ClsAleatorios.cs
ClsAltaEmpleado.cs
ClsCorreoLogica.cs
ClsGenerarUsername.cs
ClsGenero.cs
ClsLocalidad.cs
ClsLogin.cs
ClsMailHerper.cs
ClsModificarCorreo.cs
ClsModificarDomicilio.cs
ClsModificarTelefono.cs
ClsObtenerCorreoPersona.cs
ClsObtenerDomicilio.cs
ClsObtenerLocalidadPerson.cs
ClsObtenerPartidoPer.cs
ClsObtenerPermiso.cs
ClsObtenerProvinciaPer.cs
ClsObtenerTelefono.cs
ClsPartido.cs
ClsPreguntaSeguridadLogica.cs
ClsPrimerIngreso.cs
ClsProvincia.cs
ClsRol.cs
clsSeguridad.cs
ClsTelefonoLogica.cs
ClsTipoTelefonoLogica.cs
ClsUsuarioLogica.cs
ClsValidarCorreo.cs
ClsValidarCUIL.cs
ClsValidarNumero.cs
ClsVerificarDNI.cs
ResultadoCambioPassword.cs
ValidacionCampoVacio.cs
ValidacionPassword.cs


Solución "SistemaFinalKiosco" (4 de 4 proyectos)
CapaDatos
Dependencias
NegocioDatos
ClsGestorProducto.cs
ClsGestorProveedorDatos.cs
ClsCorreoDatos.cs
ClsDomicilioDatos.cs
ClsGeneroDatos.cs
ClsLocalidadDatos.cs
ClsModificarCorreoPer.cs
ClsModificarDomicilio.cs
ClsModificarPersonaPorPersona.cs
ClsModificarTelefonoPer.cs
ClsObtenerDomicilioPersona.cs
ClsObtenerLocalidad.cs
ClsObtenerPartido.cs
ClsObtenerProvincias.cs
ClsObtenerTelefonoPersona.cs
ClsObtenerTipoTelefono.cs
ClsPartidoDatos.cs
ClsPersonaCorreoDatos.cs
ClsPersonaDatos.cs
ClsPersonaDomicilioDatos.cs
ClsPersonaTelefonoDatos.cs
ClsPreguntaSeguridaDatos.cs
ClsProvinciaDatos.cs
ClsRolDatos.cs
ClsTelefonoDatos.cs
ClsTipoTelefonoDatos.cs
Conexion.cs
UsuarioDatos.cs

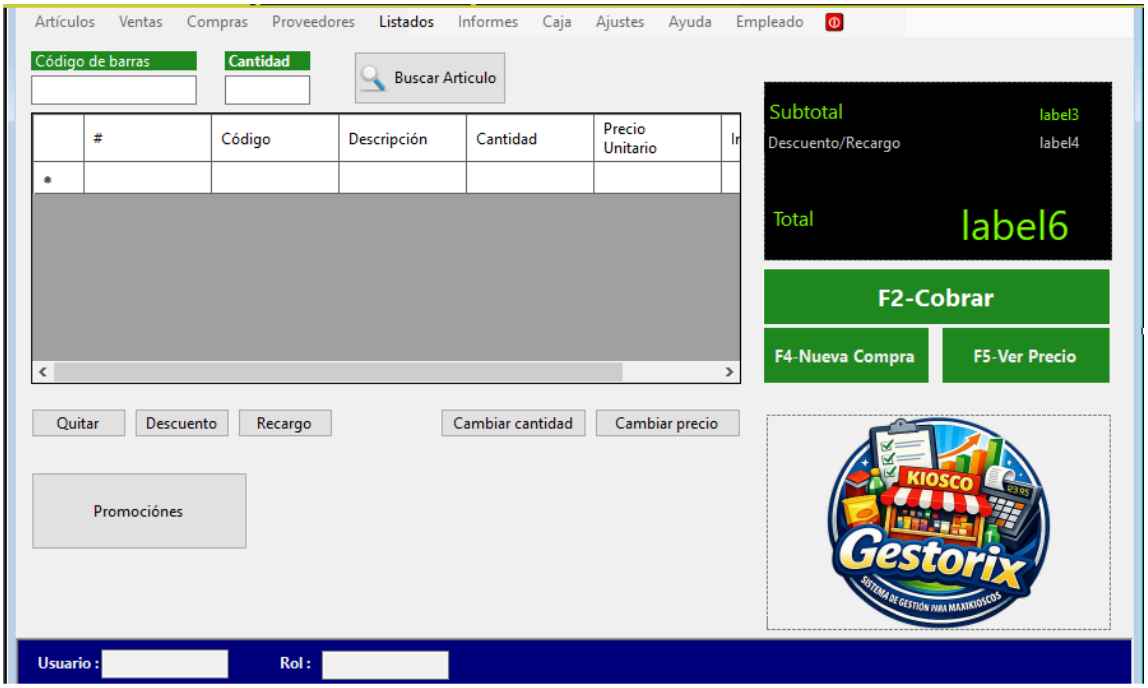
4.2 Estructura principal del Login del software con interfaces



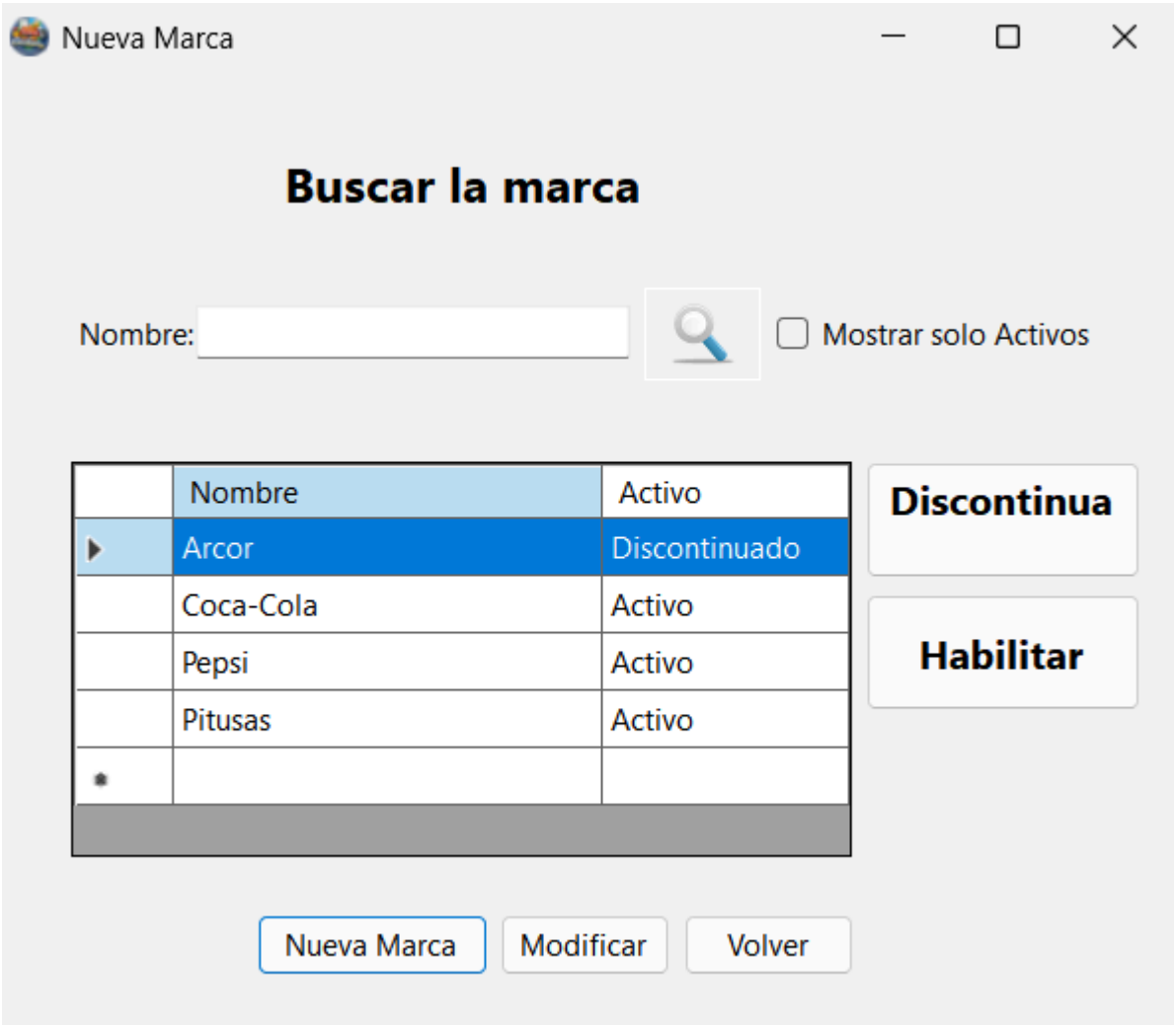

frmPreguntasSegu
Preguntas de Seguridad
¿Nombre de tu primer mascota?
¿Nombre de tu libro favorito?
¿Cual es tu comida favorita?
Aceptar

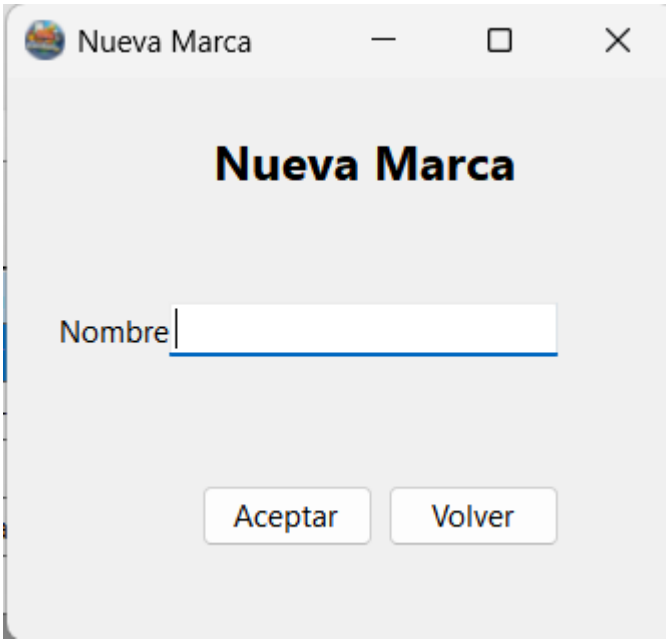

Recuperar Contraseña
Ingresa la nueva contraseña
Repeti la contraseña
Volver
Enviar Codigo

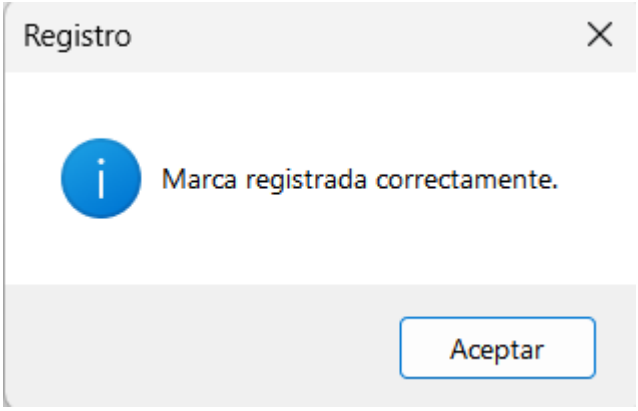
4.3 Estructura principal del menú principal del software con interfaces

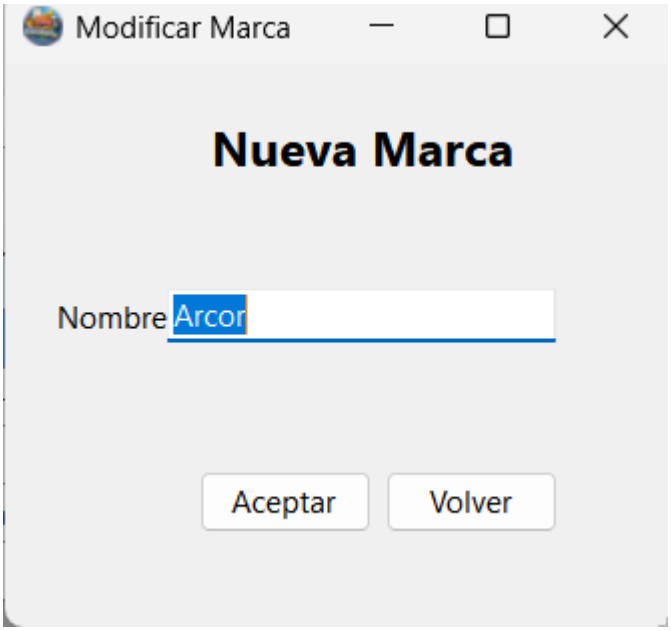


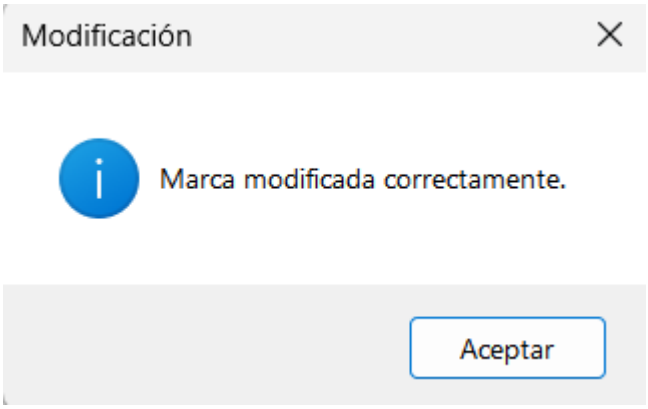
4.4 Estructura principal de un caso de uso de negocio del software con interfaces y código: ABM Marca




Nueva Marca
Nueva Marca
Nombre
Aceptar
Volver


Registro
Marca registrada correctamente.
Aceptar





Capa Vista // FrmBuscarMarca

```
using CapaLogica.NegocioLogica;
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows.Forms;
```

```
namespace CapaVista
{
    public partial class frmBuscarMarca : Form
    {
        public frmBuscarMarca()
        {
            InitializeComponent();

            dataGridView1.SelectionMode = DataGridViewSelectionMode.FullRowSelect;
            dataGridView1.MultiSelect = false;

            dataGridView1.Columns["dgvNombre"].AutoSizeMode = DataGridViewAutoSizeColumnMode.AllCells;

            dataGridView1.Columns["dgvNombre"].AutoSizeMode = DataGridViewAutoSizeColumnMode.Fill;

            this.Load += frmBuscarMarca_Load;
        }

        #region Metodos

        private void CargarMarcas()
        {
            ClsGestorLogicaProducto logica = new ClsGestorLogicaProducto();

            DataTable marcas;

            if (chkSoloActivos.Checked)
            {
                marcas = logica.CargarMarcasActivas();
            }
            else
            {
                marcas = logica.CargarTodasLasMarcas();
            }

            dataGridView1.Rows.Clear();

            foreach (DataRow fila in marcas.Rows)
            {
                string estado;

                if (Convert.ToBoolean(fila["Activo"]))
                {
                    estado = "Activo";
                }
                else
                {
                    estado = "Discontinuado";
                }

                int filaIndex = dataGridView1.Rows.Add(fila["Nombre"], estado);

                dataGridView1.Rows[filaIndex].Tag = fila["IdMarca"];
            }
        }

        #endregion
```

```
#region Eventos

private void frmBuscarMarca_Load(object sender, EventArgs e)
{
    CargarMarcas();
}

private void btnNuevaMarca_Click(object sender, EventArgs e)
{
    frmNuevaMarca nuevamarca = new frmNuevaMarca();

    nuevamarca.ShowDialog();

    CargarMarcas();
}

private void Modificar_Click(object sender, EventArgs e)
{
    if (dataGridView1.CurrentRow == null)
    {
        MessageBox.Show("Seleccione una marca para modificar.", "Modificar marca.",
            MessageBoxButtons.OK,
            MessageBoxIcon.Warning);

        return;
    }

    int idMarca = Convert.ToInt32(dataGridView1.CurrentRow.Tag);

    string nombre = dataGridView1.CurrentRow.Cells["dgvNombre"].Value.ToString();

    frmNuevaMarca modificarMarca = new frmNuevaMarca();

    modificarMarca.Text = "Modificar Marca";
    modificarMarca.IdMarca = idMarca;
    modificarMarca.NombreMarca = nombre;

    modificarMarca.ShowDialog();

    CargarMarcas();
}

private void btnBuscar_Click(object sender, EventArgs e)
{
    ClsGestorLogicaProducto logica = new ClsGestorLogicaProducto();

    string nombre = textBox1.Text.Trim();

    bool soloActivos = chkSoloActivos.Checked;

    DataTable marcas = logica.BuscarMarcas(nombre, soloActivos);

    dataGridView1.Rows.Clear();

    foreach (DataRow fila in marcas.Rows)
    {
        string estado;

        if (Convert.ToBoolean(fila["Activo"]))
```

```
            {
                estado = "Activo";
            }
            else
            {
                estado = "Discontinuado";
            }

            int filaIndex = dataGridView1.Rows.Add(fila["Nombre"],estado);

            dataGridView1.Rows[filaIndex].Tag = fila["IdMarca"];
        }
    }

    private void btnVolver_Click(object sender, EventArgs e)
    {
        this.Close();
    }
    private void btnDiscontinuar_Click(object sender, EventArgs e)
    {
        if (dataGridView1.CurrentRow == null)
        {
            MessageBox.Show("Seleccione una marca para discontinuar.", "Discontinuar marca",
                MessageBoxButtons.OK,
                MessageBoxIcon.Warning);

            return;
        }
        int idMarca = Convert.ToInt32(dataGridView1.CurrentRow.Tag);


        string nombreMarca = dataGridView1.CurrentRow.Cells["dgvNombre"].Value.ToString();

        DialogResult resultado = MessageBox.Show("¿Está seguro de discontinuar la marca \"" + nombreMarca + "\"?", "Discontinuar
marca",
            MessageBoxButtons.YesNo,
            MessageBoxIcon.Question);

        if (resultado != DialogResult.Yes)
        {
            return;
        }

        ClsGestorLogicaProducto logica = new ClsGestorLogicaProducto();

        bool resultadoDesactivar = logica.DesactivarMarca(idMarca);

        if (resultadoDesactivar)
        {
            MessageBox.Show("La marca fue discontinuada correctamente.", "Discontinuar marca",
                MessageBoxButtons.OK,
                MessageBoxIcon.Information);

            CargarMarcas();
        }
    }

    private void chkSoloActivos_CheckedChanged(object sender, EventArgs e)
    {
```

```
            CargarMarcas();
        }

        private void btnHabilitar_Click(object sender, EventArgs e)
        {
            if (dataGridView1.CurrentRow == null)
            {
                MessageBox.Show("Seleccione una marca para habilitar.", "Habilitar marca",
                    MessageBoxButtons.OK,
                    MessageBoxIcon.Warning);

                return;
            }

            int idMarca = Convert.ToInt32(dataGridView1.CurrentRow.Tag);

            string nombreMarca = dataGridView1.CurrentRow.Cells["dgvNombre"].Value.ToString();

            DialogResult resultado = MessageBox.Show("¿Está seguro de habilitar la marca \"" + nombreMarca + "\"?", "Habilitar marca",
                MessageBoxButtons.YesNo,
                MessageBoxIcon.Question);

            if (resultado != DialogResult.Yes)
            {
                return;
            }

            ClsGestorLogicaProducto logica = new ClsGestorLogicaProducto();

            bool resultadoHabilitar = logica.HabilitarMarca(idMarca);

            if (resultadoHabilitar)
            {
                MessageBox.Show("La marca fue habilitada correctamente.", "Habilitar marca",
                    MessageBoxButtons.OK,
                    MessageBoxIcon.Information);

                CargarMarcas();
            }
        }
    }

    #endregion


}
```

Capa Lógica // ClsGestorLogicaProducto

```
using CapaDatos.NegocioDatos;
using CapaEntidades.NegocioEntidad;
using System;
using System.Collections.Generic;
using System.Data;
using System.Linq;
using System.Text;
```

```
using System.Threading.Tasks;
using static CapaEntidades.NegocioEntidad.ClsGestorEntidadProducto;


public int RegistrarMarca(ClsMarca marca)
{
    return datos.RegistrarMarca(marca);
}

public DataTable CargarMarcasActivas()
{
    return datos.CargarMarcasActivas();
}

public DataTable CargarTodasLasMarcas()
{
    return datos.CargarTodasLasMarcas();
}

public bool ModificarMarca(ClsMarca marca)
{
    return datos.ModificarMarca(marca);
}
public bool DesactivarMarca(int idMarca)
{
    return datos.DesactivarMarca(idMarca);
}
public bool HabilitarMarca(int idMarca)
{
    return datos.HabilitarMarca(idMarca);
}
public DataTable BuscarMarcas(string nombre, bool soloActivos)
{
    return datos.BuscarMarcas(nombre, soloActivos);
}
```

Capa Entidades // ClsGestorProductoEntidad

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;



public class ClsMarca
{
    public int IdMarca { get; set; }
    public string Nombre { get; set; }
    public bool Activo { get; set; }

}
```

Capa Datos // ClsGestorProducto

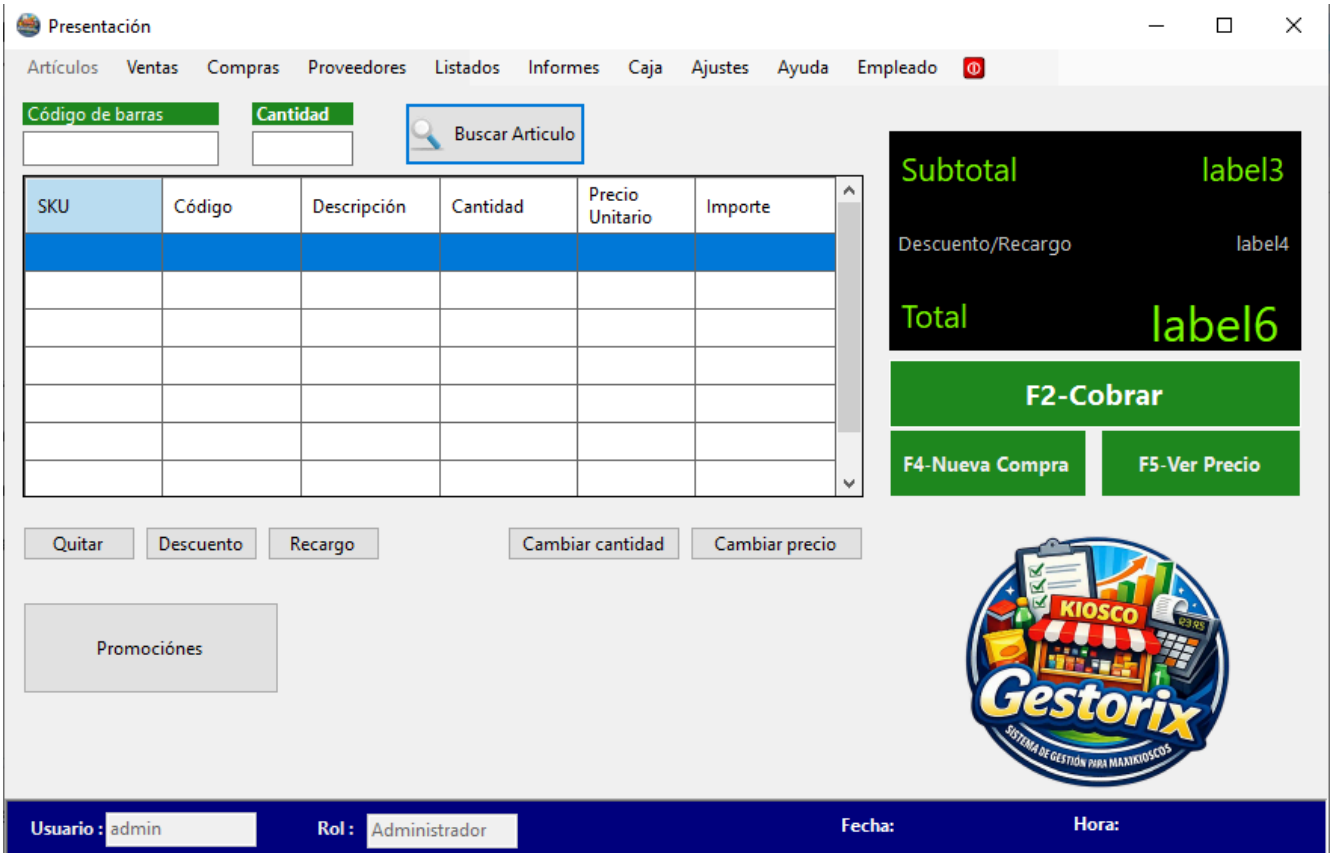
```
using System;
using System.Collections.Generic;
using System.Data;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

Conexion cn = new Conexion();

ublic int RegistrarMarca(ClsGestorEntidadProducto.ClsMarca marca)
      {
         using (SqlConnection conexion = cn.ObtenerConexion())
         {
            string consulta = @"
               INSERT INTO Marca
               (
                  Nombre,
                  Activo
               )
               Values
               (
                  @Nombre,
                  @Activo
               );

               SELECT SCOPE_IDENTITY();";

            using (SqlCommand cmd = new SqlCommand(consulta, conexion))
            {

               cmd.Parameters.AddWithValue("@Nombre", marca.Nombre);
               cmd.Parameters.AddWithValue("@Activo", marca.Activo);

               conexion.Open();

               return Convert.ToInt32(cmd.ExecuteScalar());
            }

         }
      }
      public DataTable CargarMarcasActivas()
      {
         using (SqlConnection conexion = cn.ObtenerConexion())
         {
            string consulta = @"
              SELECT
                  IdMarca,
                  Nombre,
                  Activo
               FROM Marca
               WHERE Activo = 1
               ORDER BY Nombre;";

            using (SqlCommand cmd = new SqlCommand(consulta, conexion))
            {
```

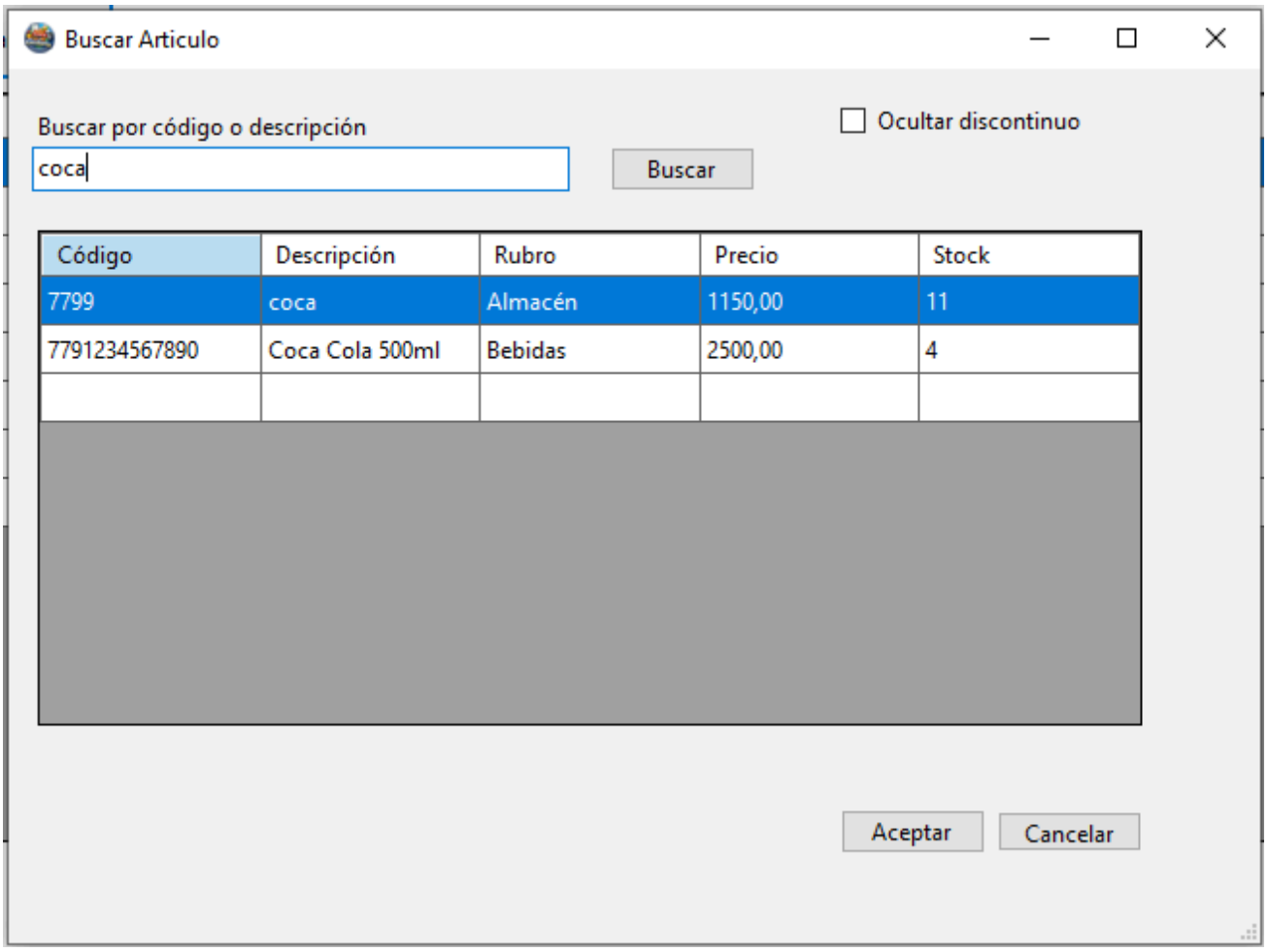
```
            using (SqlDataAdapter da = new SqlDataAdapter(cmd))
            {
                DataTable tabla = new DataTable();

                da.Fill(tabla);

                return tabla;
            }
        }

    }

}
public bool ModificarMarca(ClsGestorEntidadProducto.ClsMarca marca)
{
    using (SqlConnection conexion = cn.ObtenerConexion())
    {
        string consulta = @"
            UPDATE Marca
            SET Nombre = @Nombre
            WHERE IdMarca = @IdMarca";

        using (SqlCommand cmd = new SqlCommand(consulta, conexion))
        {
            cmd.Parameters.AddWithValue("@IdMarca", marca.IdMarca);
            cmd.Parameters.AddWithValue("@Nombre", marca.Nombre);

            conexion.Open();

            return cmd.ExecuteNonQuery() > 0;
        }
    }
}
public bool HabilitarMarca(int idMarca)
{
    using (SqlConnection conexion = cn.ObtenerConexion())
    {
        string consulta = @"
            UPDATE Marca
            SET Activo = 1
            WHERE IdMarca = @IdMarca
            AND Activo = 0";

        using (SqlCommand cmd = new SqlCommand(consulta, conexion))
        {
            cmd.Parameters.AddWithValue("@IdMarca", idMarca);

            conexion.Open();

            return cmd.ExecuteNonQuery() > 0;
        }
    }
}
public bool DesactivarMarca(int idMarca)
{
    using (SqlConnection conexion = cn.ObtenerConexion())
    {
        string consulta = @"
            UPDATE Marca
```

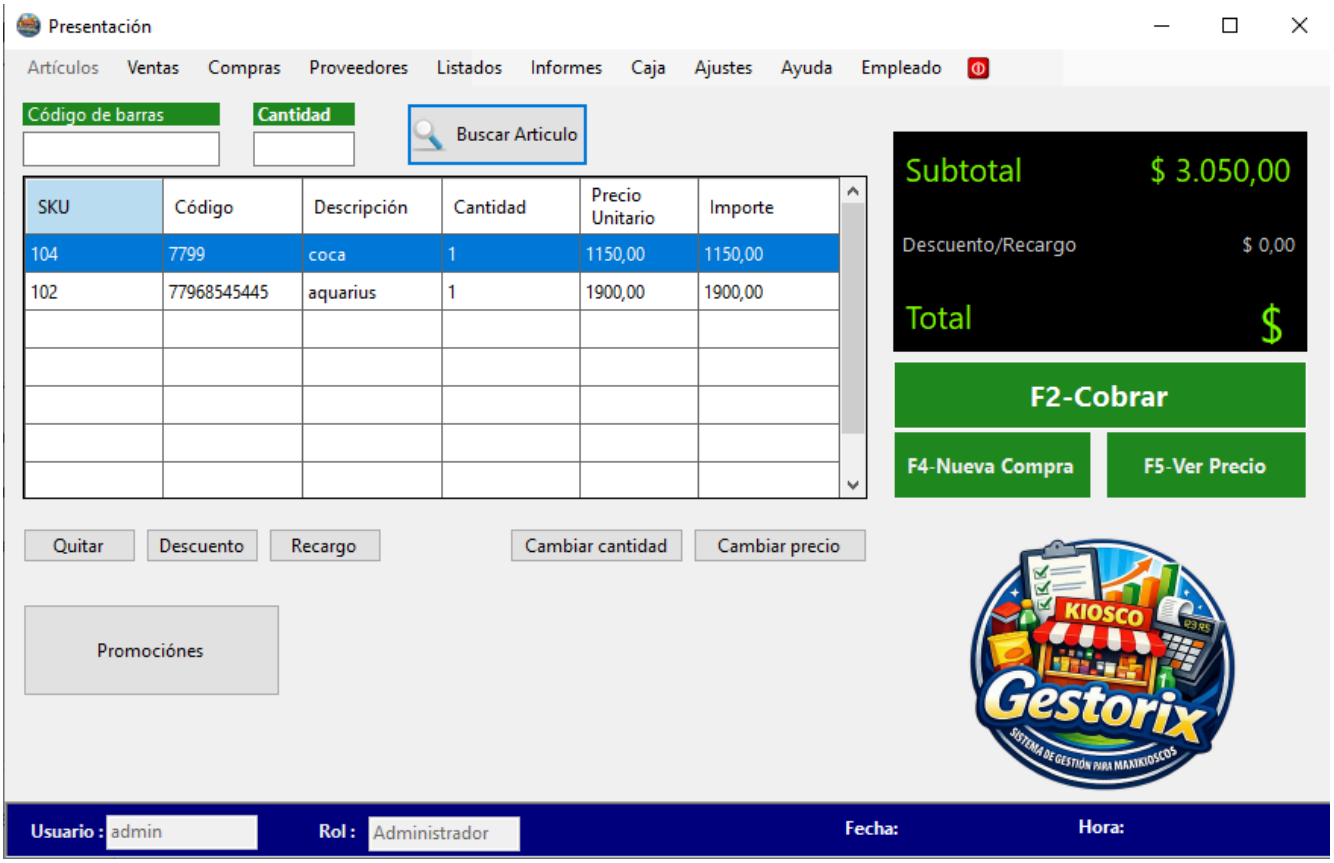
```
            SET Activo = 0
            WHERE IdMarca = @IdMarca
            AND Activo = 1";
        using (SqlCommand cmd = new SqlCommand(consulta, conexion))
        {
            cmd.Parameters.AddWithValue("@IdMarca", idMarca);

            conexion.Open();

            return cmd.ExecuteNonQuery() > 0;
        }
    }
}
public DataTable CargarTodasLasMarcas()
{
    using (SqlConnection conexion = cn.ObtenerConexion())
    {
        string consulta = @"
            SELECT
                IdMarca,
                Nombre,
                Activo
            FROM Marca
            ORDER BY Nombre;";

        using (SqlCommand cmd = new SqlCommand(consulta, conexion))
        {
            using (SqlDataAdapter da = new SqlDataAdapter(cmd))
            {
                DataTable tabla = new DataTable();

                da.Fill(tabla);

                return tabla;
            }
        }
    }
}
public DataTable BuscarMarcas(string nombre, bool soloActivos)
{
    using (SqlConnection conexion = cn.ObtenerConexion())
    {
        string consulta = @"
            SELECT
                IdMarca,
                Nombre,
                Activo
            FROM Marca
            WHERE Nombre LIKE @Nombre";

        if (soloActivos)
        {
            consulta += @"
                AND Activo = 1";
        }

        consulta += @"
            ORDER BY Nombre;";
```

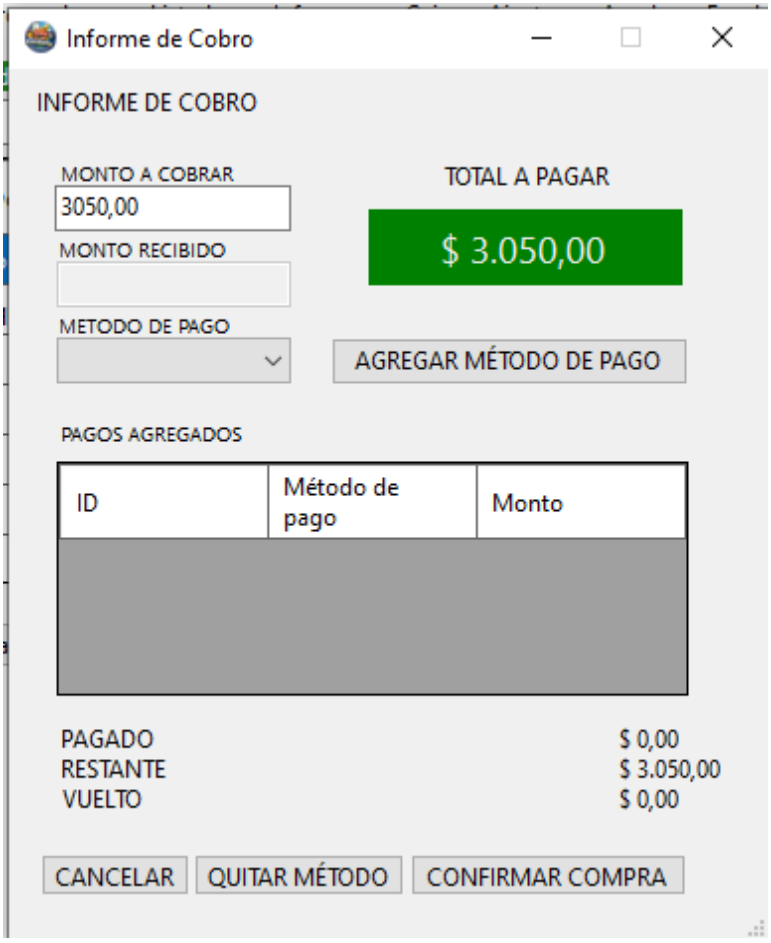
```
            using (SqlCommand cmd = new SqlCommand(consulta, conexion))
            {
                cmd.Parameters.AddWithValue("@Nombre", "%" + nombre + "%");

                using (SqlDataAdapter da = new SqlDataAdapter(cmd))
                {
                    DataTable tabla = new DataTable();

                    da.Fill(tabla);

                    return tabla;
                }
            }
        }
    }
```

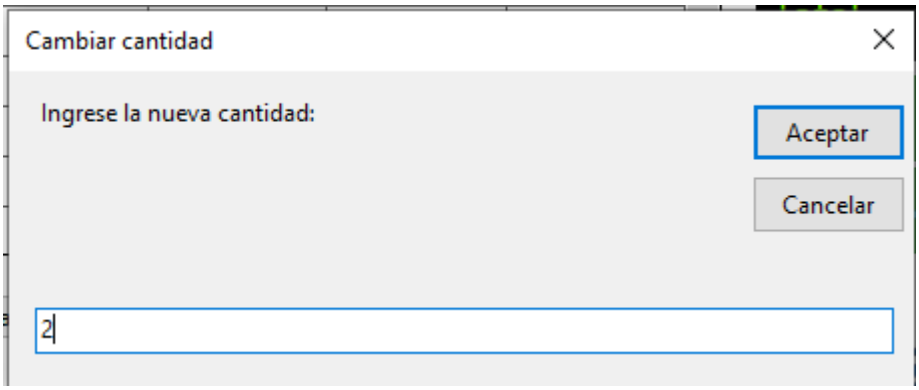
4.5 Estructura principal de un caso de uso de negocio del software con interfaces y código: Venta




Buscar Articulo
Buscar por código o descripción
coca
Buscar
Ocultar discontinuo
Código
Descripción
Rubro
Precio
Stock
7799
coca
Almacén
1150,00
11
7791234567890
Coca Cola 500ml
Bebidas
2500,00
4
Aceptar
Cancelar


Presentación
Artículos
Ventas
Compras
Proveedores
Listados
Informes
Caja
Ajustes
Ayuda
Empleado
Código de barras
Cantidad
Buscar Articulo
SKU
Código
Descripción
Cantidad
Precio Unitario
Importe
104
7799
coca
1
1150,00
1150,00
102
77968545445
aquarius
1
1900,00
1900,00
Subtotal
$ 3.050,00
Descuento/Recargo
$ 0,00
Total
$
F2-Cobrar
F4-Nueva Compra
F5-Ver Precio
Quitar
Descuento
Recargo
Cambiar cantidad
Cambiar precio
Promociónes
KIOSCO
Gestorix
SISTEMA DE GESTIÓN PARA MAXIKIOSCOS
Usuario :
admin
Rol :
Administrador
Fecha:
Hora:





Capa Vista // frmmenuPres

```
using CapaEntidades;
using CapaLogica;
using DocumentFormat.OpenXml.Office.Word;
using System;
using System.Collections.Generic;
```

```
using System.ComponentModel;
using System.Data;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows.Forms;
using static CapaEntidades.NegocioEntidad.ClsGestorEntidadProducto;
using CapaLogica.NegocioLogica;
using System.Data;


namespace CapaVista
{
    public partial class frmmenuPres : Form
    {
        public int IdRol { get; set; }

        private decimal subtotal = 0.00m;
        private decimal descuentoRecargo = 0.00m;
        private decimal total;

        public frmmenuPres()
        {
            InitializeComponent();

            dtgPresentacion.ReadOnly = true;
            dtgPresentacion.SelectionMode = DataGridViewSelectionMode.FullRowSelect;
            dtgPresentacion.MultiSelect = false;

            dtgPresentacion.AutoSizeColumnsMode = DataGridViewAutoSizeColumnsMode.Fill;

            dtgPresentacion.AllowUserToResizeColumns = false;
            dtgPresentacion.AllowUserToOrderColumns = false;
            dtgPresentacion.RowHeadersVisible = false;

            this.KeyPreview = true;

            this.Load += frmmenuPres_Load;
        }
        private void frmmenuPres_Load(object sender, EventArgs e)
        {
            txtUsuario.Text = SesionUsuario.Username;

            CapaLogica.ClsRol rol = new CapaLogica.ClsRol();

            txtRol.Text = rol.ObtenerNombreRol(SesionUsuario.IdRol);

            CargarBotones();

        }


        #region Metodo
        private void CargarBotones()
        {
            //Menu principal
            btnArt.Enabled = SesionUsuario.Permisos.Contains(100);
            btnVentas.Enabled = SesionUsuario.Permisos.Contains(200);
```

```
            btnCompras.Enabled = SesionUsuario.Permisos.Contains(300);
            btnProve.Enabled = SesionUsuario.Permisos.Contains(400);
            toolStripMenuItem1.Enabled = SesionUsuario.Permisos.Contains(500);
            btnInformes.Enabled = SesionUsuario.Permisos.Contains(600);
            btnCaja.Enabled = SesionUsuario.Permisos.Contains(700);
            btnAjuste.Enabled = SesionUsuario.Permisos.Contains(800);
            btnAyuda.Enabled = SesionUsuario.Permisos.Contains(900);
            btnEmpleado.Enabled = SesionUsuario.Permisos.Contains(1000);

            //Listados
            listaDePreciosToolStripMenuItem.Enabled = SesionUsuario.Permisos.Contains(510);
            listadoDeReposiciónToolStripMenuItem.Enabled = SesionUsuario.Permisos.Contains(520);

            // Articulos
            // Administrador de articulos
                menuitemAdmistradorarticulos.Enabled = SesionUsuario.Permisos.Contains(111) || SesionUsuario.Permisos.Contains(112) ||
SesionUsuario.Permisos.Contains(113);
            // Administrador de rubros
            menuitemAdministradoarticulos.Enabled = SesionUsuario.Permisos.Contains(120);
            // Reposicion rapida de stock
            meniitemReporapida.Enabled = SesionUsuario.Permisos.Contains(113);
            // Cargar de las marcas
            cargarDeLasMarcasToolStripMenuItem.Enabled = SesionUsuario.Permisos.Contains(130);
            // Importar articulos desde Excel
            menuitemImporexcel.Enabled = SesionUsuario.Permisos.Contains(140);
            // Exportar articulos a Excel
            menuitemExporexcel.Enabled = SesionUsuario.Permisos.Contains(150);

            //Ventas
            menuitemAdministradorventas.Enabled = SesionUsuario.Permisos.Contains(210);
            notaDeCreditoToolStripMenuItem.Enabled = SesionUsuario.Permisos.Contains(220);

            //Proveedores
            administradorDeProveedoresToolStripMenuItem.Enabled = SesionUsuario.Permisos.Contains(410);
            cuentaCorrienteDeProveedoresToolStripMenuItem.Enabled = SesionUsuario.Permisos.Contains(420);

            //Informes
            ventasPorDiaToolStripMenuItem.Enabled = SesionUsuario.Permisos.Contains(610);
            ventaToolStripMenuItem.Enabled = SesionUsuario.Permisos.Contains(620);
            ventaPorRubroToolStripMenuItem.Enabled = SesionUsuario.Permisos.Contains(630);
            ventaPorArticulosToolStripMenuItem.Enabled = SesionUsuario.Permisos.Contains(640);
            ventaPorMedioDePagoToolStripMenuItem.Enabled = SesionUsuario.Permisos.Contains(650);

            //Caja
            nuevoIngresoToolStripMenuItem.Enabled = SesionUsuario.Permisos.Contains(710);
            nuevoEgresoToolStripMenuItem.Enabled = SesionUsuario.Permisos.Contains(720);
            abrirCajaToolStripMenuItem.Enabled = SesionUsuario.Permisos.Contains(730);
            cerrarSeToolStripMenuItem.Enabled = SesionUsuario.Permisos.Contains(740);

            //Empleado
            permisosToolStripMenuItem.Enabled = SesionUsuario.Permisos.Contains(1270);
        }
        //Ingreso de plata
        public enum TipoMovimiento
        {
            Ingreso,
            Egreso
        }
        public enum TipoInforme
```

```
    {
        VentasPorFecha,
        VentasConDetallePorFecha,
        VentasPorRubro,
        VentasPorArticulo,
        VentaPorMedioPago
    }

    private void BloquearMenu()
    {
        // Menu superior
        btnArt.Enabled = false;
        btnVentas.Enabled = false;
        btnCompras.Enabled = false;
        btnProve.Enabled = false;
        toolStripMenuItem1.Enabled = false;
        btnInformes.Enabled = false;
        btnAjuste.Enabled = false;
        btnAyuda.Enabled = false;
        btnEmpleado.Enabled = false;

        // Caja queda disponible
        btnCaja.Enabled = true;

        nuevoIngresoToolStripMenuItem.Enabled = false;
        nuevoEgresoToolStripMenuItem.Enabled = false;
        cerrarSeToolStripMenuItem.Enabled = false;
        abrirCajaToolStripMenuItem.Enabled = true;

        // Botones del formulario
        btnCambiarcantidad.Enabled = false;
        btnCambiarprecio.Enabled = false;
        btnRecargo.Enabled = false;
        btnDescuento.Enabled = false;
        btnQuitar.Enabled = false;
        btnCobrar.Enabled = false;
        btnNuevaCompra.Enabled = false;
        btnVerPrecio.Enabled = false;
        btnBuscarArticulo.Enabled = false;
        btnPromocion.Enabled = false;

        // Entrada de datos
        txtCodigoBarra.Enabled = false;
        txtCantidad.Enabled = false;
        dtgPresentacion.Enabled = false;

        txtUsuario.Text = "";
        txtRol.Text = "";
    }


    private void AgregarProductoAGrilla(
int idProducto,
string sku,
string codigoBarra,
string descripcion,
decimal precio,
decimal cantidad,
decimal stockDisponible)
```

```
{
    // No permitir productos sin stock
    if (stockDisponible <= 0)
    {
        MessageBox.Show(
            "El producto no tiene stock disponible.",
            "Sin stock",
            MessageBoxButtons.OK,
            MessageBoxIcon.Warning);

        return;
    }

    // Buscar si el producto ya está cargado
    foreach (DataGridViewRow fila in dtgPresentacion.Rows)
    {
        if (fila.IsNewRow)
            continue;

        if (fila.Cells["ClnIdProducto"].Value != null &&
            Convert.ToInt32(fila.Cells["ClnIdProducto"].Value) == idProducto)
        {
            decimal cantidadActual =
                Convert.ToDecimal(fila.Cells["ClnCantidad"].Value);

            decimal cantidadFinal = cantidadActual + cantidad;

            // IMPORTANTE: validar antes de modificar la grilla
            if (cantidadFinal > stockDisponible)
            {
                MessageBox.Show(
                    $"Stock insuficiente.\n\n" +
                    $"Stock disponible: {stockDisponible}\n" +
                    $"Cantidad cargada: {cantidadActual}\n" +
                    $"Cantidad a agregar: {cantidad}\n" +
                    $"Cantidad resultante: {cantidadFinal}",
                    "Stock insuficiente",
                    MessageBoxButtons.OK,
                    MessageBoxIcon.Warning);

                return;
            }

            fila.Cells["ClnCantidad"].Value = cantidadFinal;
            fila.Cells["ClnImporte"].Value = cantidadFinal * precio;

            CalcularYMostrarMontos();
            return;
        }
    }

    // Producto nuevo: también validar antes de agregarlo
    if (cantidad > stockDisponible)
    {
        MessageBox.Show(
            $"Stock insuficiente.\n\n" +
            $"Stock disponible: {stockDisponible}\n" +
            $"Cantidad solicitada: {cantidad}",
            "Stock insuficiente",
```

```
            MessageBoxButtons.OK,
            MessageBoxIcon.Warning);

        return;
    }

    int nuevaFila = -1;

    for (int i = 0; i < dtgPresentacion.Rows.Count; i++)
    {
        if (dtgPresentacion.Rows[i].IsNewRow)
            continue;

        if (dtgPresentacion.Rows[i].Cells["ClnIdProducto"].Value == null)
        {
            nuevaFila = i;
            break;
        }
    }

    if (nuevaFila == -1)
    {
        nuevaFila = dtgPresentacion.Rows.Add();
    }

    dtgPresentacion.Rows[nuevaFila].Cells["ClnIdProducto"].Value = idProducto;
    dtgPresentacion.Rows[nuevaFila].Cells["ClnSKU"].Value = sku;
    dtgPresentacion.Rows[nuevaFila].Cells["ClnCodigo"].Value = codigoBarra;
    dtgPresentacion.Rows[nuevaFila].Cells["ClnDescripcion"].Value = descripcion;
    dtgPresentacion.Rows[nuevaFila].Cells["ClnCantidad"].Value = cantidad;
    dtgPresentacion.Rows[nuevaFila].Cells["ClnPrecioUnitario"].Value = precio;
    dtgPresentacion.Rows[nuevaFila].Cells["ClnImporte"].Value = cantidad * precio;

    CalcularYMostrarMontos();
}

private void IngresarProductoPorCodigo()
{
    string codigoBarra = txtCodigoBarra.Text.Trim();

    if (string.IsNullOrWhiteSpace(codigoBarra))
    {
        txtCodigoBarra.Focus();
        return;
    }

    // Si no se ingresa cantidad, por defecto es 1
    decimal cantidad = 1;

    if (!string.IsNullOrWhiteSpace(txtCantidad.Text))
    {
        if (!decimal.TryParse(txtCantidad.Text, out cantidad) || cantidad <= 0)
        {
            MessageBox.Show(
                "Ingrese una cantidad válida mayor a 0.",
                "Cantidad inválida",
                MessageBoxButtons.OK,
                MessageBoxIcon.Warning);
```

```
            txtCantidad.SelectAll();
            txtCantidad.Focus();
            return;
        }
    }

    ClsGestorLogicaProducto logicaProducto =
        new ClsGestorLogicaProducto();

    DataTable tablaProducto =
        logicaProducto.ObtenerProductoPorCodigoBarra(codigoBarra);

    // Verificamos que el código exista
    if (tablaProducto.Rows.Count == 0)
    {
        MessageBox.Show(
            "No se encontró un producto con ese código de barras.",
            "Producto no encontrado",
            MessageBoxButtons.OK,
            MessageBoxIcon.Warning);

        txtCodigoBarra.SelectAll();
        txtCodigoBarra.Focus();
        return;
    }

    DataRow producto = tablaProducto.Rows[0];

    int idProducto = Convert.ToInt32(producto["IdProducto"]);
    string sku = producto["SKU"].ToString();
    string codigo = producto["CodigoBarra"].ToString();
    string descripcion = producto["Descripcion"].ToString();
    decimal precio = Convert.ToDecimal(producto["PrecioVenta"]);

    // Si el producto no tiene registro de stock, consideramos stock 0
    decimal stockDisponible = producto["CantidadActual"] == DBNull.Value
        ? 0
        : Convert.ToDecimal(producto["CantidadActual"]);

    // Agregamos el producto.
    // AgregarProductoAGrilla se encarga también de validar
    // la cantidad acumulada contra el stock disponible.
    AgregarProductoAGrilla(
        idProducto,
        sku,
        codigo,
        descripcion,
        precio,
        cantidad,
        stockDisponible);

    // Limpiamos para dejar preparada la próxima carga
    txtCodigoBarra.Clear();
    txtCantidad.Clear();

    // El cajero queda listo para escanear el siguiente producto
    txtCodigoBarra.Focus();
}
#endregion
```

```
#region Eventos

private void txtCodigoBarra_KeyDown(object sender, KeyEventArgs e)
{
   if (e.KeyCode == Keys.Enter)
   {
      e.SuppressKeyPress = true;
      IngresarProductoPorCodigo();
   }
}
private void txtCantidad_KeyDown(object sender, KeyEventArgs e)
{
   if (e.KeyCode == Keys.Enter)
   {
      e.SuppressKeyPress = true;
      IngresarProductoPorCodigo();
   }
}
//Menu Item
private void menuitemImporexcel_Click(object sender, EventArgs e)
{
   frmImportarArticulos frmImportar = new frmImportarArticulos();
   frmImportar.ShowDialog();
}
private void cuentaCorrienteDeProveedoresToolStripMenuItem_Click(object sender, EventArgs e)
{
   frmCuentaCorrienteProveedores cuentacorr = new frmCuentaCorrienteProveedores();
   cuentacorr.ShowDialog();
}
//Informes
private void ventasPorDiaToolStripMenuItem_Click_1(object sender, EventArgs e)
{
   frmVentaFecha ventaFecha = new frmVentaFecha(TipoInforme.VentasPorFecha);
   ventaFecha.ShowDialog();
}
private void ventaToolStripMenuItem_Click(object sender, EventArgs e)
{
   frmVentaFecha ventaFecha = new frmVentaFecha(TipoInforme.VentasConDetallePorFecha);
   ventaFecha.ShowDialog();
}
private void ventaPorRubroToolStripMenuItem_Click(object sender, EventArgs e)
{
   frmVentaFecha ventaRubro = new frmVentaFecha(TipoInforme.VentasPorRubro);
   ventaRubro.ShowDialog();
}
private void ventaPorArticulosToolStripMenuItem_Click(object sender, EventArgs e)
{
   frmVentaFecha ventaArticulo = new frmVentaFecha(TipoInforme.VentasPorArticulo);
   ventaArticulo.ShowDialog();
}
private void ventaPorMedioDePagoToolStripMenuItem_Click(object sender, EventArgs e)
{
   frmVentaFecha ventraPago = new frmVentaFecha(TipoInforme.VentaPorMedioPago);
   ventraPago.ShowDialog();
}
//Caja
private void nuevoIngresoToolStripMenuItem_Click(object sender, EventArgs e)
{
```

```
        frmIngresoPlataCaja ingresoPlata = new frmIngresoPlataCaja(TipoMovimiento.Ingreso);

        ingresoPlata.ShowDialog();
    }
    private void nuevoEgresoToolStripMenuItem_Click(object sender, EventArgs e)
    {
        frmIngresoPlataCaja egresoPlata = new frmIngresoPlataCaja(TipoMovimiento.Egreso);

        egresoPlata.ShowDialog();
    }

    private void listaDePreciosToolStripMenuItem_Click(object sender, EventArgs e)
    {
        frmListadoPrecios listaprecio = new frmListadoPrecios();
        listaprecio.ShowDialog();
    }
    private void listadoDeReposiciónToolStripMenuItem_Click(object sender, EventArgs e)
    {
        frmListadoReposicion listarepo = new frmListadoReposicion();
        listarepo.ShowDialog();
    }
    private void menuitemAministradorcompras_Click(object sender, EventArgs e)
    {
        frmCompras compra = new frmCompras();
        compra.ShowDialog();
    }
    private void menuitemAdmistradorarticulos_Click(object sender, EventArgs e)
    {
        frmCargaarticulo cargaarticulo = new frmCargaarticulo();
        cargaarticulo.ShowDialog();
    }
    private void meniitemReporapida_Click(object sender, EventArgs e)
    {
        frmReporapida reporapida = new frmReporapida();
        reporapida.ShowDialog();
    }
    private void menuitemAdministradorventas_Click(object sender, EventArgs e)
    {
        frmVentas ventas = new frmVentas();
        ventas.Show();

    }
    private void notaDeCreditoToolStripMenuItem_Click(object sender, EventArgs e)
    {
        frmDevolucion devolucion = new frmDevolucion();
        devolucion.ShowDialog();
    }
    private void administradorDeProveedoresToolStripMenuItem_Click(object sender, EventArgs e)
    {
        frmProveedores proveedores = new frmProveedores();
        proveedores.ShowDialog();
    }

    //Cerrar sesion
    private void cerrarSeToolStripMenuItem_Click(object sender, EventArgs e)
    {
            DialogResult resultado = MessageBox.Show("Estas seguro de cerrar la sesión?", "Cerrar sesión", MessageBoxButtons.YesNo,
MessageBoxIcon.Question);
```

```
        if (resultado == DialogResult.Yes)
        {
            ClsLogin login = new ClsLogin();
            login.CerrarSesion();

            BloquearMenu();

        }
    }
    private void abrirCajaToolStripMenuItem_Click(object sender, EventArgs e)
    {
        FrmLogin login = Application.OpenForms.OfType<FrmLogin>().FirstOrDefault();
        /*aplication.openform coleccion de toos los formularios abierto
        oftype- frmlogin de todos esos formulario se queda con el que sea frmlogin
        es decir buscame el login que ya existe*/

        if (login != null)
        {
            login.LimpiarCampos();
        }


        this.Close();
    }
    private void tollClosed_Click(object sender, EventArgs e)
    {
            DialogResult resultado = MessageBox.Show("Estas seguro de cerrar la sesión?", "Cerrar sesión", MessageBoxButtons.YesNo,
MessageBoxIcon.Question);
        if (resultado == DialogResult.Yes)
        {
            Application.Exit();

        }
    }
    private void CalcularYMostrarMontos()
    {
        subtotal = 0.00m;

        foreach (DataGridViewRow fila in dtgPresentacion.Rows)
        {
            if (fila.IsNewRow)
                continue;

            if (fila.Cells["ClnImporte"].Value != null && decimal.TryParse(fila.Cells["ClnImporte"].Value.ToString(),out decimal importe))
            {
                subtotal += importe;
            }
        }

        total = subtotal + descuentoRecargo;

        lblSubtotalMonto.Text = $"$ {subtotal:N2}";
        lblDescuentoMonto.Text = $"$ {descuentoRecargo:N2}";
        lblTotalMonto.Text = $"$ {total:N2}";
    }


    private void btnConfig_ItemClicked(object sender, ToolStripItemClickedEventArgs e)
    {
```

```
}
private void permisosToolStripMenuItem_Click(object sender, EventArgs e)
{
    frmCargaEmpleados Carga = new frmCargaEmpleados();
    Carga.ShowDialog();
}
private void btnNuevaCompra_Click(object sender, EventArgs e)

{
    btnNuevaCompra.Enabled = false;
    frmCobroSecundario secundario = new frmCobroSecundario();

    secundario.FormClosed += (s, arg) =>
    {
        btnNuevaCompra.Enabled = true;
    };

    secundario.Show();

}
private void btnBuscarArticulo_Click(object sender, EventArgs e)
{
    frmBuscarArticulo buscarArticulo = new frmBuscarArticulo();

    if (buscarArticulo.ShowDialog() == DialogResult.OK)
    {
        ClsProducto producto = buscarArticulo.ProductoSeleccionado;

        if (producto != null)
        {
            int fila = -1;

            for (int i = 0; i < dtgPresentacion.Rows.Count; i++)
            {
                if (dtgPresentacion.Rows[i].IsNewRow)
                    continue;

                if (dtgPresentacion.Rows[i].Cells["ClnSKU"].Value == null ||
                    string.IsNullOrWhiteSpace(dtgPresentacion.Rows[i].Cells["ClnSKU"].Value.ToString()))
                {
                    fila = i;
                    break;
                }
            }

            if (fila == -1)
            {
                fila = dtgPresentacion.Rows.Add();
            }

            dtgPresentacion.Rows[fila].Cells["ClnIdProducto"].Value = producto.IdProducto;
            dtgPresentacion.Rows[fila].Cells["ClnSKU"].Value = producto.SKU;
            dtgPresentacion.Rows[fila].Cells["ClnCodigo"].Value = producto.CodigoBarra;
            dtgPresentacion.Rows[fila].Cells["ClnDescripcion"].Value = producto.Descripcion;
            dtgPresentacion.Rows[fila].Cells["ClnCantidad"].Value = 1;
            dtgPresentacion.Rows[fila].Cells["ClnPrecioUnitario"].Value = producto.PrecioVenta;
            dtgPresentacion.Rows[fila].Cells["ClnImporte"].Value = producto.PrecioVenta;
```

```
                CalcularYMostrarMontos();
            }
        }
    }
    private void EjecutarCobro()
    {
        // Solo permitir cobrar si el rol tiene acceso a ventas/caja (Rol 1 o Rol 2)
        if (IdRol == 1 || IdRol == 2)
        {
                        MessageBox.Show($"Abriendo módulo de cobro por: $ {total:N2}", "Procesar Pago", MessageBoxButtons.OK,
MessageBoxIcon.Information);
            // Aquí va tu lógica para abrir la ventana de pagos final o guardar en Base de Datos
        }
        else
        {
                MessageBox.Show("Tu usuario no tiene permisos para realizar cobros.", "Acceso Denegado", MessageBoxButtons.OK,
MessageBoxIcon.Warning);
        }
    }//revisar esto


    private void btnCobrar_Click_1(object sender, EventArgs e)
    {
        if (total <= 0)
        {
            MessageBox.Show(
                "Debe agregar al menos un producto para realizar el cobro.",
                "Cobrar venta",
                MessageBoxButtons.OK,
                MessageBoxIcon.Warning);

            txtCodigoBarra.Focus();
            return;
        }

        frmInformeCobro cobro =
  new frmInformeCobro(total, dtgPresentacion);

        if (cobro.ShowDialog() == DialogResult.OK)
        {
            LimpiarVenta();
        }
    }

    private void LimpiarVenta()
    {
        // Eliminamos todos los productos del carrito
        dtgPresentacion.Rows.Clear();

        // Reiniciamos los valores de la venta
        subtotal = 0.00m;
        descuentoRecargo = 0.00m;
        total = 0.00m;

        // Actualizamos los montos visuales
        CalcularYMostrarMontos();

        // Volvemos a crear las filas vacías visuales
        RellenarFilas();
```

```
    // Dejamos listo para la siguiente venta
    txtCodigoBarra.Clear();
    txtCodigoBarra.Focus();
}
private void RellenarFilas()
{
    int filasVisibles = dtgPresentacion.Height / dtgPresentacion.RowTemplate.Height;
    while (dtgPresentacion.Rows.Count < filasVisibles)
    {
        dtgPresentacion.Rows.Add();
    }
}

private void menuitemAdministradoarticulos_Click(object sender, EventArgs e)
{

}
private void btnPromocion_Click(object sender, EventArgs e)
{
    frmPromocion promo = new frmPromocion();
    promo.ShowDialog();
}
private void cargarDeLasMarcasToolStripMenuItem_Click(object sender, EventArgs e)
{
    frmBuscarMarca marca = new frmBuscarMarca();
    marca.ShowDialog();
}
private void label2_Click(object sender, EventArgs e)
{

}
private void frmmenuPres_Load_1(object sender, EventArgs e)
{
    RellenarFilas();
}
private void tmrFechaHora_Tick(object sender, EventArgs e)
{
    lblFecha.Text = "Fecha: " + DateTime.Now.ToString("dd/MM/yyyy");
    lblHora.Text = "Hora: " + DateTime.Now.ToString("HH:mm:ss");
}
#endregion
```

```
private void btnCambiarcantidad_Click(object sender, EventArgs e)
{
    if (dtgPresentacion.CurrentRow == null || dtgPresentacion.CurrentRow.Cells["ClnIdProducto"].Value == null)
    {
```

```
            MessageBox.Show("Seleccione un producto para cambiar la cantidad.","Cambiar cantidad",
                MessageBoxButtons.OK, MessageBoxIcon.Warning);

            return;
        }

        DataGridViewRow fila = dtgPresentacion.CurrentRow;

        int idProducto = Convert.ToInt32(fila.Cells["ClnIdProducto"].Value);
        decimal cantidadActual = Convert.ToDecimal(fila.Cells["ClnCantidad"].Value);
                    string entrada = Microsoft.VisualBasic.Interaction.InputBox("Ingrese la nueva cantidad:","Cambiar cantidad",
cantidadActual.ToString());

        if (string.IsNullOrWhiteSpace(entrada))
            return;

        if (!decimal.TryParse(entrada, out decimal nuevaCantidad) || nuevaCantidad <= 0)
        {
            MessageBox.Show("Ingrese una cantidad válida mayor a 0.","Cantidad inválida",
                MessageBoxButtons.OK, MessageBoxIcon.Warning);

            return;
        }

        // Consultamos el stock real del producto
        ClsGestorLogicaProducto logicaProducto = new ClsGestorLogicaProducto();

        DataTable tablaStock = logicaProducto.ObtenerStockPorProducto(idProducto);

        decimal stockDisponible = 0;

        if (tablaStock.Rows.Count > 0 && tablaStock.Rows[0]["CantidadActual"] != DBNull.Value)
        {
            stockDisponible = Convert.ToDecimal(tablaStock.Rows[0]["CantidadActual"]);
        }

        // No permitir superar el stock
        if (nuevaCantidad > stockDisponible)
        {
            MessageBox.Show($"Stock insuficiente.\n\n" +$"Stock disponible: {stockDisponible}\n" +
                $"Cantidad solicitada: {nuevaCantidad}","Stock insuficiente",
                MessageBoxButtons.OK, MessageBoxIcon.Warning);

            return;
        }

        decimal precioUnitario = Convert.ToDecimal(fila.Cells["ClnPrecioUnitario"].Value);

        fila.Cells["ClnCantidad"].Value = nuevaCantidad;
        fila.Cells["ClnImporte"].Value = nuevaCantidad * precioUnitario;

        CalcularYMostrarMontos();

        txtCodigoBarra.Focus();
    }

    private void btnQuitar_Click(object sender, EventArgs e)
    {
        if (dtgPresentacion.CurrentRow == null || dtgPresentacion.CurrentRow.Cells["ClnIdProducto"].Value == null)
```

```
        {
            MessageBox.Show("Seleccione un producto para quitar.","Quitar producto",
                MessageBoxButtons.OK, MessageBoxIcon.Warning);

            return;
        }

        DataGridViewRow fila = dtgPresentacion.CurrentRow;

        foreach (DataGridViewCell celda in fila.Cells)
        {
            celda.Value = null;
        }

        CalcularYMostrarMontos();

        txtCodigoBarra.Focus();
    }

  }

}
```

Capa Vista // frmInformeCobro

```
using CapaEntidades;
using CapaEntidades.NegocioEntidad;
using CapaLogica.NegocioLogica;
using System;
using System.Collections.Generic;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows.Forms;

namespace CapaVista
{
  public partial class frmInformeCobro : Form
  {
    private readonly decimal totalVenta;
    private decimal totalPagado = 0;
    private decimal vueltoVenta = 0;
    private readonly DataGridView carritoVenta;
    public frmInformeCobro(decimal total, DataGridView carrito)
    {
      InitializeComponent();

      totalVenta = total;
      carritoVenta = carrito;

      lblTotal.Text = $"$ {totalVenta:N2}";
      lblPagado.Text = "$ 0,00";
      lblRestante.Text = $"$ {totalVenta:N2}";
      lblVuelto.Text = "$ 0,00";
```

```
    btnConfirmar.Visible = true;
    btnConfirmar.BringToFront();

    CargarMetodosPago();

    txtMonto.Text = totalVenta.ToString("0.00");
}

private void CargarMetodosPago()
{
    ClsGestorVentaLogica gestorVenta = new ClsGestorVentaLogica();

    DataTable tabla = gestorVenta.ObtenerMetodosPago();

    cmbMetodoPago.DataSource = tabla;
    cmbMetodoPago.DisplayMember = "Nombre";
    cmbMetodoPago.ValueMember = "IdMetodoPago";
    cmbMetodoPago.SelectedIndex = -1;
}

private void CalcularPagos()
{
    totalPagado = 0;

    foreach (DataGridViewRow fila in dtgPagos.Rows)
    {
        if (fila.IsNewRow)
            continue;

        if (fila.Cells["ClnMontoPago"].Value != null && decimal.TryParse( fila.Cells["ClnMontoPago"].Value.ToString(),out decimal monto))
        {
            totalPagado += monto;
        }
    }

    decimal restante = totalVenta - totalPagado;

    lblPagado.Text = $"$ {totalPagado:N2}";
    lblRestante.Text = $"$ {Math.Max(restante, 0):N2}";

    btnConfirmar.Enabled = restante == 0;
}

private void btnAgregarPago_Click(object sender, EventArgs e)
{
    if (cmbMetodoPago.SelectedIndex == -1)
    {
        MessageBox.Show(
            "Seleccione un método de pago.",
            "Método de pago",
            MessageBoxButtons.OK,
            MessageBoxIcon.Warning);

        cmbMetodoPago.Focus();
        return;
    }

    if (!decimal.TryParse(txtMonto.Text, out decimal monto) || monto <= 0)
```

```
{
    MessageBox.Show(
        "Ingrese un monto válido mayor a 0.",
        "Monto inválido",
        MessageBoxButtons.OK,
        MessageBoxIcon.Warning);

    txtMonto.SelectAll();
    txtMonto.Focus();
    return;
}

decimal restante = totalVenta - totalPagado;

if (monto > restante)
{
    MessageBox.Show(
        $"El monto ingresado supera el saldo restante.\n\n" +
        $"Restante: $ {restante:N2}",
        "Monto inválido",
        MessageBoxButtons.OK,
        MessageBoxIcon.Warning);

    txtMonto.SelectAll();
    txtMonto.Focus();
    return;
}

int idMetodoPago = Convert.ToInt32(cmbMetodoPago.SelectedValue);

string metodoPago = cmbMetodoPago.Text;

bool esEfectivo = metodoPago.Equals("Efectivo", StringComparison.OrdinalIgnoreCase);

if (esEfectivo)
{
    decimal montoRecibido;

    if (string.IsNullOrWhiteSpace(txtMontoRecibido.Text))
    {
        montoRecibido = monto;
    }
    else
    {
        if (!decimal.TryParse(txtMontoRecibido.Text,out montoRecibido) || montoRecibido <= 0)
        {
            MessageBox.Show(
                "Ingrese un monto recibido válido.",
                "Monto recibido",
                MessageBoxButtons.OK,
                MessageBoxIcon.Warning);

            txtMontoRecibido.SelectAll();
            txtMontoRecibido.Focus();
            return;
        }

        if (montoRecibido < monto)
        {
```

```
                MessageBox.Show(
                    $"El monto recibido no alcanza.\n\n" +
                    $"Monto a aplicar: $ {monto:N2}\n" +
                    $"Monto recibido: $ {montoRecibido:N2}",
                    "Efectivo insuficiente",
                    MessageBoxButtons.OK,
                    MessageBoxIcon.Warning);

                txtMontoRecibido.SelectAll();
                txtMontoRecibido.Focus();
                return;
            }
        }

        vueltoVenta = montoRecibido - monto;
    }

    dtgPagos.Rows.Add(idMetodoPago,metodoPago,monto);

    CalcularPagos();

    txtMonto.Clear();
    txtMontoRecibido.Clear();

    cmbMetodoPago.SelectedIndex = -1;

    lblVuelto.Text = $"$ {vueltoVenta:N2}";

    cmbMetodoPago.Focus();
}
private void txtMonto_TextChanged(object sender, EventArgs e)
{
    CalcularVuelto();
}

private void txtMontoRecibido_TextChanged(object sender, EventArgs e)
{
    CalcularVuelto();
}
private void cmbMetodoPago_KeyDown(object sender, KeyEventArgs e)
{
    if (e.KeyCode == Keys.Enter &&
        cmbMetodoPago.SelectedIndex != -1)
    {
        e.SuppressKeyPress = true;

        btnAgregarPago.PerformClick();
    }
}

private void btnConfirmar_Click(object sender, EventArgs e)
{
    try
    {

        if (totalPagado != totalVenta)
        {
            MessageBox.Show(
                "La venta todavía no está completamente pagada.",
```

```
        "Confirmar venta",
        MessageBoxButtons.OK,
        MessageBoxIcon.Warning);

    return;
}

List<DetalleVentaEntidad> detalles = new List<DetalleVentaEntidad>();

foreach (DataGridViewRow fila in carritoVenta.Rows)
{
    if (fila.IsNewRow || fila.Cells["ClnIdProducto"].Value == null)
    {
        continue;
    }

    int idProducto = Convert.ToInt32(fila.Cells["ClnIdProducto"].Value);
    decimal cantidad = Convert.ToDecimal(fila.Cells["ClnCantidad"].Value);
    decimal precioUnitario = Convert.ToDecimal(fila.Cells["ClnPrecioUnitario"].Value);
    decimal subtotal = Convert.ToDecimal(fila.Cells["ClnImporte"].Value);

    detalles.Add(new DetalleVentaEntidad
        {
            IdProducto = idProducto,
            Cantidad = cantidad,
            PrecioUnitario = precioUnitario,
            Subtotal = subtotal
        });
}
List<PagoVentaEntidad> pagos = new List<PagoVentaEntidad>();

foreach (DataGridViewRow fila in dtgPagos.Rows)
{
    if (fila.IsNewRow || fila.Cells["ClnIdMetodoPago"].Value == null)
    {
        continue;
    }

    int idMetodoPago = Convert.ToInt32(fila.Cells["ClnIdMetodoPago"].Value);

    decimal monto = Convert.ToDecimal(fila.Cells["ClnMontoPago"].Value);

    pagos.Add(
        new PagoVentaEntidad
        {
            IdMetodoPago = idMetodoPago,
            Monto = monto
        });
}

ClsGestorVentaLogica gestorVenta = new ClsGestorVentaLogica();

int idVenta = gestorVenta.RegistrarVenta(
    totalVenta,
    SesionUsuario.IdUsuario,
    detalles,
    pagos);

MessageBox.Show(
```

```
            $"Venta #{idVenta} registrada correctamente.",
            "Venta registrada",
            MessageBoxButtons.OK,
            MessageBoxIcon.Information);

        DialogResult = DialogResult.OK;

        Close();
    }
    catch (Exception ex)
    {
        MessageBox.Show(
            $"No se pudo registrar la venta.\n\n{ex.Message}",
            "Error",
            MessageBoxButtons.OK,
            MessageBoxIcon.Error);
    }
}
private void cmbMetodoPago_SelectedIndexChanged(object sender, EventArgs e)
{
    if (cmbMetodoPago.SelectedIndex == -1)
    {
        txtMontoRecibido.Clear();
        txtMontoRecibido.Enabled = false;
        return;
    }

    decimal restante = totalVenta - totalPagado;

    txtMonto.Text = restante.ToString("0.00");

    string metodo = cmbMetodoPago.Text;

    bool esEfectivo = metodo.Equals( "Efectivo",StringComparison.OrdinalIgnoreCase);

    txtMontoRecibido.Enabled = esEfectivo;

    if (esEfectivo)
    {
        txtMontoRecibido.Clear();
        txtMonto.SelectAll();
        txtMonto.Focus();
    }
    else
    {
        txtMontoRecibido.Clear();
    }
}

private void CalcularVuelto()
{
    if (cmbMetodoPago.SelectedIndex == -1 || !cmbMetodoPago.Text.Equals("Efectivo", StringComparison.OrdinalIgnoreCase))
    {
        return;
    }

    if (!decimal.TryParse(
        txtMonto.Text,
        out decimal montoEfectivo))
```

```
        {
            lblVuelto.Text = "$ 0,00";
            return;
        }

        if (string.IsNullOrWhiteSpace(txtMontoRecibido.Text))
        {
            lblVuelto.Text = "$ 0,00";
            return;
        }

        if (!decimal.TryParse(
            txtMontoRecibido.Text,
            out decimal recibido))
        {
            lblVuelto.Text = "$ 0,00";
            return;
        }

        decimal vuelto = recibido - montoEfectivo;

        lblVuelto.Text = vuelto >= 0? $"$ {vuelto:N2}" : "$ 0,00";
    }

    private void btnQuitarPago_Click(object sender, EventArgs e)
    {
        if (dtgPagos.CurrentRow == null || dtgPagos.CurrentRow.IsNewRow)
        {
            MessageBox.Show("Seleccione un método de pago para quitar.","Quitar pago",
                MessageBoxButtons.OK, MessageBoxIcon.Warning);

            return;
        }

        dtgPagos.Rows.Remove(dtgPagos.CurrentRow);

        CalcularPagos();

        decimal restante = totalVenta - totalPagado;

        txtMonto.Text = restante.ToString("0.00");

        vueltoVenta = 0;
        lblVuelto.Text = "$ 0,00";

        txtMontoRecibido.Clear();

        cmbMetodoPago.SelectedIndex = -1;
        cmbMetodoPago.Focus();
    }

    private void btnCancelar_Click(object sender, EventArgs e)
    {
        this.Close();
    }
  }
}
```

Capa Lógica // ClsGestorLogicaProducto

```
using CapaDatos.NegocioDatos;
using CapaEntidades.NegocioEntidad;
using System;
using System.Collections.Generic;
using System.Data;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using static CapaEntidades.NegocioEntidad.ClsGestorEntidadProducto;

namespace CapaLogica.NegocioLogica
{
    public class ClsGestorLogicaProducto
    {
        ClsGestorProducto datos = new ClsGestorProducto();

        public int RegistrarProducto(ClsProducto producto)
        {
            return datos.RegistrarProducto(producto);
        }
        public int ObtenerProximoSKU()
        {
            return datos.ObtenerProximoSKU();
        }
        public bool ExisteCodigoBarra(string codigoBarra, int idProductoExcluir = 0)
        {
            return datos.ExisteCodigoBarra(codigoBarra, idProductoExcluir);
        }
        public DataTable ObtenerProductoPorCodigoBarra(string codigoBarra)
        {
            return datos.ObtenerProductoPorCodigoBarra(codigoBarra);
        }
        public DataTable CargarIVAsActivos()
        {
            return datos.CargarIVAsActivos();
        }
        public DataTable CargarUnidades()
        {
            return datos.CargarUnidades();
        }
        public int RegistrarMarca(ClsMarca marca)
        {
            return datos.RegistrarMarca(marca);
        }
        public DataTable CargarMarcasActivas()
        {
            return datos.CargarMarcasActivas();
        }
        public DataTable CargarTodasLasMarcas()
        {
            return datos.CargarTodasLasMarcas();
        }
        public bool ModificarMarca(ClsMarca marca)
        {
            return datos.ModificarMarca(marca);
        }
        public bool DesactivarMarca(int idMarca)
```

```
{
    return datos.DesactivarMarca(idMarca);
}
public bool HabilitarMarca(int idMarca)
{
    return datos.HabilitarMarca(idMarca);
}
public DataTable BuscarMarcas(string nombre, bool soloActivos)
{
    return datos.BuscarMarcas(nombre, soloActivos);
}
public DataTable CargarCategoriasActivas()
{
    return datos.CargarCategoriasActivas();
}
public void RegistrarStock(int idProducto,decimal cantidadActual,decimal stockMinimo, decimal stockIdeal)
{
    datos.RegistrarStock(idProducto,cantidadActual,stockMinimo,stockIdeal);
}
public bool RegistrarStock(ClsStock stock)
{
    return datos.RegistrarStock(stock);
}
public DataTable ObtenerProveedoresPorProducto(int idProducto)
{
    return datos.ObtenerProveedoresPorProducto(idProducto);
}
public DataTable ObtenerProductoPorId(int idProducto)
{
    return datos.ObtenerProductoPorId(idProducto);
}
public bool ModificarProducto(ClsProducto producto)
{
    return datos.ModificarProducto(producto);
}
public DataTable CargarArticulos()
{
    return datos.CargarArticulos();
}
public DataTable BuscarArticulos(string texto, bool ocultarDiscontinuos)
{
    return datos.BuscarArticulos(texto, ocultarDiscontinuos);
}
public DataTable ObtenerStockPorProducto(int idProducto)
{
    return datos.ObtenerStockPorProducto(idProducto);
}
public void RegistrarProductoSucursalProveedor(ClsProductoSucursalProveedor relacion)
{
    datos.RegistrarProductoSucursalProveedor(relacion);
}
public bool RegistrarLote(ClsLote lote)
{
    return datos.RegistrarLote(lote);
}
public bool ModificarLote(ClsLote lote)
{
    return datos.ModificarLote(lote);
}
```

```
        public bool ModificarProductoSucursalProveedor(ClsProductoSucursalProveedor relacion)
        {
            return datos.ModificarProductoSucursalProveedor(relacion);
        }


    }
}
```

Capa Datos // ClsGestorVentaDatos

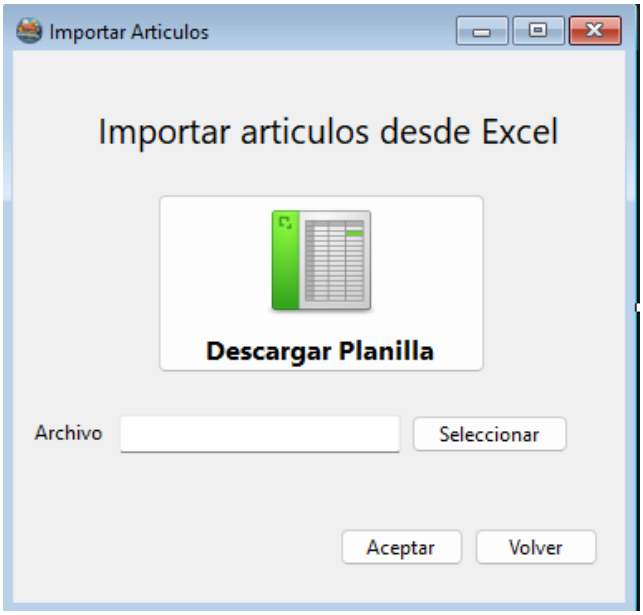
```
using Microsoft.Data.SqlClient;
using SistemaKiosco;
using System;
using System.Collections.Generic;
using System.Data;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using CapaEntidades.NegocioEntidad;
using System.Collections.Generic;

namespace CapaDatos.NegocioDatos
{
    public class ClsGestorVentaDatos
    {
        Conexion cn = new Conexion();

        public DataTable ObtenerMetodosPago()
        {
            using (SqlConnection conexion = cn.ObtenerConexion())
            {
                string consulta = @"
                    SELECT
                        IdMetodoPago,
                        Nombre
                    FROM MetodoPago
                    WHERE Activo = 1
                    ORDER BY Nombre;";

                using (SqlCommand cmd = new SqlCommand(consulta, conexion))
                {
                    using (SqlDataAdapter adaptador = new SqlDataAdapter(cmd))
                    {
                        DataTable tabla = new DataTable();

                        adaptador.Fill(tabla);

                        return tabla;
                    }
                }
            }
        }

        public int RegistrarVenta(decimal total,int idUsuario,List<DetalleVentaEntidad> detalles,List<PagoVentaEntidad> pagos)
        {
            using (SqlConnection conexion = cn.ObtenerConexion())
            {
```

```
conexion.Open();

SqlTransaction transaccion = conexion.BeginTransaction();

try
{


   string sqlVenta = @"
      INSERT INTO Venta
         (FechaHora, Total, IdUsuario)
      VALUES
         (@FechaHora, @Total, @IdUsuario);

      SELECT CAST(SCOPE_IDENTITY() AS INT);";

   int idVenta;

   using (SqlCommand cmdVenta = new SqlCommand(sqlVenta, conexion, transaccion))
   {
      cmdVenta.Parameters.AddWithValue("@FechaHora",DateTime.Now);
      cmdVenta.Parameters.AddWithValue("@Total",total);
      cmdVenta.Parameters.AddWithValue("@IdUsuario",idUsuario);
      idVenta = Convert.ToInt32(cmdVenta.ExecuteScalar());
   }



   foreach (DetalleVentaEntidad detalle in detalles)
   {
      string sqlDetalle = @"
         INSERT INTO DetalleVenta
            (
               IdVenta,
               IdProducto,
               Cantidad,
               PrecioUnitario,
               Subtotal
            )
         VALUES
            (
               @IdVenta,
               @IdProducto,
               @Cantidad,
               @PrecioUnitario,
               @Subtotal
            );";

      using (SqlCommand cmdDetalle =new SqlCommand(sqlDetalle,conexion,transaccion))
      {
         cmdDetalle.Parameters.AddWithValue("@IdVenta",idVenta);
         cmdDetalle.Parameters.AddWithValue("@IdProducto",detalle.IdProducto);
         cmdDetalle.Parameters.AddWithValue("@Cantidad",detalle.Cantidad);
         cmdDetalle.Parameters.AddWithValue("@PrecioUnitario",detalle.PrecioUnitario);
         cmdDetalle.Parameters.AddWithValue("@Subtotal",detalle.Subtotal);
         cmdDetalle.ExecuteNonQuery();
      }
```

```
    string sqlStock = @"
        UPDATE Stock
        SET CantidadActual =
            CantidadActual - @Cantidad
        WHERE IdProducto = @IdProducto
          AND CantidadActual >= @Cantidad;";

    int filasAfectadas;

    using (SqlCommand cmdStock = new SqlCommand(sqlStock,conexion,transaccion))
    {
        cmdStock.Parameters.AddWithValue( "@Cantidad",detalle.Cantidad);
        cmdStock.Parameters.AddWithValue("@IdProducto",detalle.IdProducto);
        filasAfectadas = cmdStock.ExecuteNonQuery();
    }


    if (filasAfectadas == 0)
    {
        throw new Exception($"Stock insuficiente para el producto " +$"{detalle.IdProducto}.");
    }




    string sqlMovimiento = @"
        INSERT INTO MovimientoStock
            (
                IdProducto,
                TipoMovimiento,
                Cantidad,
                FechaHora,
                Observacion,
                IdUsuario
            )
        VALUES
            (
                @IdProducto,
                @TipoMovimiento,
                @Cantidad,
                @FechaHora,
                @Observacion,
                @IdUsuario
            );";

    using (SqlCommand cmdMovimiento = new SqlCommand(sqlMovimiento,conexion,transaccion))
    {
        cmdMovimiento.Parameters.AddWithValue("@IdProducto",detalle.IdProducto);
        cmdMovimiento.Parameters.AddWithValue("@TipoMovimiento","EGRESO");
        cmdMovimiento.Parameters.AddWithValue("@Cantidad",detalle.Cantidad);
        cmdMovimiento.Parameters.AddWithValue("@FechaHora",DateTime.Now);
        cmdMovimiento.Parameters.AddWithValue("@Observacion",$"Venta #{idVenta}");
        cmdMovimiento.Parameters.AddWithValue("@IdUsuario",idUsuario);
        cmdMovimiento.ExecuteNonQuery();
    }
}
```

```
                foreach (PagoVentaEntidad pago in pagos)
                {
                    string sqlPago = @"
                        INSERT INTO VentaMetodoPago
                            (
                                IdVenta,
                                IdMetodoPago,
                                Monto
                            )
                        VALUES
                            (
                                @IdVenta,
                                @IdMetodoPago,
                                @Monto
                            );";

                    using (SqlCommand cmdPago = new SqlCommand(sqlPago,conexion, transaccion))
                    {
                        cmdPago.Parameters.AddWithValue("@IdVenta",idVenta);
                        cmdPago.Parameters.AddWithValue("@IdMetodoPago",pago.IdMetodoPago);
                        cmdPago.Parameters.AddWithValue("@Monto",pago.Monto);
                        cmdPago.ExecuteNonQuery();
                    }
                }



                transaccion.Commit();

                return idVenta;
            }
            catch
            {
                transaccion.Rollback();
                throw;
            }
        }
    }
  }
}
```

4.6 Estructura principal de un caso de uso de negocio del software con interfaces y código: Importar y Exportar Excel



Capa Vista // frmExportarArchivos

```
using CapaLogica;
using CapaLogica.NegocioLogica;
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows.Forms;
using ClosedXML.Excel;
using System.IO;

namespace CapaVista
{
    public partial class frmExportarArchivos : Form
    {
        private ClsGestorLogicaProducto logicaProducto;
        private ClsGestorLogicaProveedor logicaProveedor;

        public frmExportarArchivos()
        {
            InitializeComponent();
```

```
    logicaProducto = new ClsGestorLogicaProducto();
    logicaProveedor = new ClsGestorLogicaProveedor();
}

#region Metodos
private void frmExportarArchivos_Load_1(object sender, EventArgs e)
{
    CargarProveedores();
    CargarRubros();
    CargarMarcas();
    CargarEstados();
}

private void CargarProveedores()
{
    DataTable tablaProveedores = logicaProveedor.ObtenerProveedores();

    cmbProveedor.DataSource = tablaProveedores;
    cmbProveedor.DisplayMember = "RazonSocial";
    cmbProveedor.ValueMember = "IdProveedor";

    cmbProveedor.SelectedIndex = -1;
}
private void CargarRubros()
{
    DataTable tablaRubros = logicaProducto.CargarCategoriasActivas();

    cmbRubro.DataSource = tablaRubros;
    cmbRubro.DisplayMember = "Nombre";
    cmbRubro.ValueMember = "IdCategoria";

    cmbRubro.SelectedIndex = -1;
}
private void CargarMarcas()
{
    DataTable tablaMarcas = logicaProducto.CargarMarcasActivas();

    DataRow filaTodos = tablaMarcas.NewRow();
    filaTodos["IdMarca"] = 0;
    filaTodos["Nombre"] = "Todos";

    tablaMarcas.Rows.InsertAt(filaTodos, 0);

    cmbMarca.DataSource = tablaMarcas;
    cmbMarca.DisplayMember = "Nombre";
    cmbMarca.ValueMember = "IdMarca";

    cmbMarca.SelectedIndex = 0;
}
private void CargarSucursales(int idProveedor)
{
    if (idProveedor <= 0)
    {
        cmbSucursal.DataSource = null;
        return;
    }

    DataTable tablaSucursales = logicaProveedor.ObtenerSucursalesPorProveedor(idProveedor);
```

```
    cmbSucursal.DataSource = tablaSucursales;
    cmbSucursal.DisplayMember = "Sucursal";
    cmbSucursal.ValueMember = "IdSucursal";

    cmbSucursal.SelectedIndex = -1;
}
private void CargarEstados()
{
    cmbEstado.Items.Clear();

    cmbEstado.Items.Add("Todos");
    cmbEstado.Items.Add("Activo");
    cmbEstado.Items.Add("Inactivo");

    cmbEstado.SelectedIndex = 0;
}
private DataTable ObtenerArticulosFiltrados()
{
    int idProveedor = 0;
    int idRubro = 0;
    int idMarca = 0;
    int idSucursal = 0;
    int estado = -1;

    if (cmbProveedor.SelectedValue != null)
    {
        int.TryParse(cmbProveedor.SelectedValue.ToString(), out idProveedor);
    }

    if (cmbRubro.SelectedValue != null)
    {
        int.TryParse(cmbRubro.SelectedValue.ToString(), out idRubro);
    }

    if (cmbMarca.SelectedValue != null)
    {
        int.TryParse(cmbMarca.SelectedValue.ToString(), out idMarca);
    }

    if (cmbSucursal.SelectedValue != null)
    {
        int.TryParse(cmbSucursal.SelectedValue.ToString(), out idSucursal);
    }

    if (cmbEstado.SelectedIndex == 1)
    {
        estado = 1;
    }
    else if (cmbEstado.SelectedIndex == 2)
    {
        estado = 0;
    }

    return logicaProducto.CargarArticulosParaExportar(idProveedor, idRubro, idMarca, idSucursal, estado,
        chkArticulosStock.Checked, chkArticulosVencimiento.Checked, chkArticuloLote.Checked);
}
private void CrearExcel(DataTable tabla, string rutaArchivo)
{
    using (XLWorkbook libro = new XLWorkbook())
```

```
{
    IXLWorksheet hoja = libro.Worksheets.Add("Articulos");

    string[] encabezados =
    {
        "Código de Barra",
        "Descripción",
        "Rubro",
        "Marca",
        "Proveedor",
        "Sucursal",
        "Precio Costo",
        "Precio Venta",
        "IVA",
        "Alicuota",
        "Cantidad Actual",
        "Stock Mínimo",
        "Stock Ideal",
        "Fecha Vencimiento",
        "Estado"
    };

    for (int columna = 0; columna < encabezados.Length; columna++)
    {
        hoja.Cell(1, columna + 1).Value = encabezados[columna];
    }

    for (int fila = 0; fila < tabla.Rows.Count; fila++)
    {
        DataRow registro = tabla.Rows[fila];

        hoja.Cell(fila + 2, 1).Value = registro["CodigoBarra"]?.ToString() ?? "";
        hoja.Cell(fila + 2, 2).Value = registro["Descripcion"]?.ToString() ?? "";
        hoja.Cell(fila + 2, 3).Value = registro["Rubro"]?.ToString() ?? "";
        hoja.Cell(fila + 2, 4).Value = registro["Marca"]?.ToString() ?? "";
        hoja.Cell(fila + 2, 5).Value = registro["Proveedor"]?.ToString() ?? "";
        hoja.Cell(fila + 2, 6).Value = registro["Sucursal"]?.ToString() ?? "";
        hoja.Cell(fila + 2, 7).Value = registro["PrecioCosto"] == DBNull.Value ? "" : Convert.ToDecimal(registro["PrecioCosto"]);
        hoja.Cell(fila + 2, 8).Value = registro["PrecioVenta"] == DBNull.Value ? "" : Convert.ToDecimal(registro["PrecioVenta"]);
        hoja.Cell(fila + 2, 9).Value = registro["IVA"]?.ToString() ?? "";
        hoja.Cell(fila + 2, 10).Value = registro["Alicuota"] == DBNull.Value ? "" : Convert.ToDecimal(registro["Alicuota"]);
        hoja.Cell(fila + 2, 11).Value = registro["CantidadActual"] == DBNull.Value ? 0 : Convert.ToDecimal(registro["CantidadActual"]);
        hoja.Cell(fila + 2, 12).Value = registro["StockMinimo"] == DBNull.Value ? 0: Convert.ToDecimal(registro["StockMinimo"]);
        hoja.Cell(fila + 2, 13).Value = registro["StockIdeal"] == DBNull.Value ? 0 : Convert.ToDecimal(registro["StockIdeal"]);


        if (registro["FechaVencimiento"] != DBNull.Value)
        {
            hoja.Cell(fila + 2, 13).Value = Convert.ToDateTime(registro["FechaVencimiento"]);
            hoja.Cell(fila + 2, 13).Style.DateFormat.Format = "dd/MM/yyyy";
        }
        else
        {
            hoja.Cell(fila + 2, 13).Value = "";
        }

        hoja.Cell(fila + 2, 14).Value = Convert.ToBoolean(registro["Activo"]) ? "Activo" : "Inactivo";
    }
```

```
        hoja.RangeUsed().SetAutoFilter();
        hoja.Columns().AdjustToContents();

        libro.SaveAs(rutaArchivo);


    }
}

#endregion


#region Eventos
private void cmbProveedor_SelectedIndexChanged(object sender, EventArgs e)
{
    if (cmbProveedor.SelectedValue == null)
    {
        cmbSucursal.DataSource = null;
        return;
    }

    if (!int.TryParse(cmbProveedor.SelectedValue.ToString(), out int idProveedor))
    {
        cmbSucursal.DataSource = null;
        return;
    }

    CargarSucursales(idProveedor);
}
private void btnExportar_Click(object sender, EventArgs e)
{
    DataTable tabla = ObtenerArticulosFiltrados();

    if (tabla.Rows.Count == 0)
    {
        MessageBox.Show("No se encontraron artículos con los filtros seleccionados.",
            "Exportar artículos", MessageBoxButtons.OK, MessageBoxIcon.Information);

        return;
    }

    using (SaveFileDialog guardar = new SaveFileDialog())
    {
        guardar.Title = "Guardar archivo de artículos";
        guardar.Filter = "Archivo Excel (*.xlsx)|*.xlsx";
        guardar.DefaultExt = "xlsx";
        guardar.AddExtension = true;
        guardar.FileName = "Articulos.xlsx";

        if (guardar.ShowDialog() != DialogResult.OK)
        {
            return;
        }


        CrearExcel(tabla, guardar.FileName);

        MessageBox.Show($"El archivo se exportó correctamente.\n\nArtículos exportados: {tabla.Rows.Count}",
            "Exportación completa", MessageBoxButtons.OK, MessageBoxIcon.Information);
```

```
        }

    }

    #endregion
    private void label5_Click(object sender, EventArgs e)
    {

    }



}
}
```

Capa Vista // frmImportarArticulos

```
using CapaEntidades.NegocioEntidad;
using CapaLogica.NegocioLogica;
using ClosedXML.Excel;
using Microsoft.Data.SqlClient;
using SistemaKiosco;
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows.Forms;
using static CapaEntidades.NegocioEntidad.ClsGestorEntidadProducto;

namespace CapaVista
{
    public partial class frmImportarArticulos : Form
    {
        public frmImportarArticulos()
        {
            InitializeComponent();
        }

        #region Metodos
        private void ImportarArticulosDesdeExcel(string rutaArchivo)
        {
            try
            {
                ClsGestorLogicaProducto logica = new ClsGestorLogicaProducto();

                using (XLWorkbook libro = new XLWorkbook(rutaArchivo))
                {
                    //Verifica las hojas de los articulos
                    if (!libro.Worksheets.Contains("ARTICULOS"))
                    {
                        MessageBox.Show("El archivo no contiene la hoja 'ARTICULOS'.", "Importación de artículos",
                            MessageBoxButtons.OK, MessageBoxIcon.Error);
```

```
            return;
        }

        IXLWorksheet hoja = libro.Worksheet("ARTICULOS");

        int ultimaFila = hoja.LastRowUsed()?.RowNumber() ?? 0;

        if (ultimaFila < 3)
        {
            MessageBox.Show("No se encontraron artículos para importar.","Importación de artículos",
                MessageBoxButtons.OK, MessageBoxIcon.Warning);

            return;
        }

        //Listas temporalres
        List<string> errores = new List<string>();
        List<ClsGestorEntidadProducto.ClsProducto> productos = new List<ClsGestorEntidadProducto.ClsProducto>();
        List<ClsGestorEntidadProducto.ClsProductoSucursalProveedor> relaciones = new
List<ClsGestorEntidadProducto.ClsProductoSucursalProveedor>();
        List<(string CodigoBarra, string Proveedor, string Sucursal, decimal PrecioCompra)> proveedoresCostos = new List<(string
CodigoBarra, string Proveedor, string Sucursal, decimal PrecioCompra)>();
        Dictionary<string, DateTime?> fechasVencimiento = new Dictionary<string, DateTime?>();

        int cantidadArticulos = 0;

        //Leer articuos
        for (int fila = 3; fila <= ultimaFila; fila++)
        {
            string codigoBarra = hoja.Cell(fila, 1).GetString().Trim();
            string nombre = hoja.Cell(fila, 2).GetString().Trim();
            string descripcion = hoja.Cell(fila, 3).GetString().Trim();
            string marca = hoja.Cell(fila, 4).GetString().Trim();
            string categoria = hoja.Cell(fila, 5).GetString().Trim();
            string unidad = hoja.Cell(fila, 6).GetString().Trim();
            string ivaTexto = hoja.Cell(fila, 7).GetString().Trim();


            //Fecha vencimineto
            DateTime? fechaVencimiento = null;

            IXLCell celdaFecha = hoja.Cell(fila, 8);

            if (!celdaFecha.IsEmpty())
            {
                //Reconoce fecha como celda
                if (celdaFecha.DataType == XLDataType.DateTime)
                {
                    fechaVencimiento = celdaFecha.GetDateTime().Date;
                }
                else
                {
                    string fechaTexto = celdaFecha.GetString().Trim();

                    //fecha escritacomo dd/mm/aaaa
                    if (DateTime.TryParseExact(fechaTexto,new string[]
                        {
                            "dd/MM/yyyy",
                            "d/M/yyyy",
```

```
                        "dd/M/yyyy",
                        "d/MM/yyyy"
                    },
                    System.Globalization.CultureInfo.InvariantCulture,
                    System.Globalization.DateTimeStyles.None,
                    out DateTime fechaExacta))
                {
                    fechaVencimiento = fechaExacta.Date;
                }

                //Fecha como ocho digitos
                if (!fechaVencimiento.HasValue)
                {
                    string numeroTexto = fechaTexto.Replace("/", "").Replace("-", "").Trim();

                    if (numeroTexto.All(char.IsDigit))
                    {

                        numeroTexto = numeroTexto.PadLeft(8, '0');

                        if (numeroTexto.Length == 8 && DateTime.TryParseExact( numeroTexto,
                                "ddMMyyyy", System.Globalization.CultureInfo.InvariantCulture, System.Globalization.DateTimeStyles.None,
out DateTime fechaNumerica))
                        {
                            fechaVencimiento = fechaNumerica.Date;
                        }
                    }
                }

                if (!fechaVencimiento.HasValue && DateTime.TryParse(fechaTexto,
                        System.Globalization.CultureInfo.CurrentCulture, System.Globalization.DateTimeStyles.None, out DateTime
fechaConvertida))
                {
                    fechaVencimiento =
                        fechaConvertida.Date;
                }

                //La fecha no es valida
                if (!fechaVencimiento.HasValue)
                {
                    errores.Add( $"Fila {fila}: la Fecha Vencimiento '{fechaTexto}' no tiene un formato válido. Use dd/MM/yyyy o 8 dígitos,
por ejemplo 25092026.");

                    continue;
                }
            }
        }

        //Fila vacia
        if (string.IsNullOrWhiteSpace(codigoBarra) && string.IsNullOrWhiteSpace(nombre) &&
string.IsNullOrWhiteSpace(descripcion) && string.IsNullOrWhiteSpace(marca) &&
            string.IsNullOrWhiteSpace(categoria) && string.IsNullOrWhiteSpace(unidad) && string.IsNullOrWhiteSpace(ivaTexto) &&
            fechaVencimiento == null)
        {
            continue;
        }

        //Codigo de barra
        if (!string.IsNullOrWhiteSpace(codigoBarra))
```

```
{
    if (logica.ExisteCodigoBarra(codigoBarra))
    {
        errores.Add($"Fila {fila}: el código de barra '{codigoBarra}' ya existe en la base de datos.");

        continue;
    }
}

//Campos obligatorio
if (string.IsNullOrWhiteSpace(nombre))
{
    errores.Add($"Fila {fila}: el campo 'Nombre' es obligatorio.");

    continue;
}

if (string.IsNullOrWhiteSpace(marca))
{
    errores.Add($"Fila {fila}: el campo 'Marca' es obligatorio.");

    continue;
}

int idMarca = logica.ObtenerIdMarcaPorNombre(marca);

if (idMarca == 0)
{
    errores.Add($"Fila {fila}: la marca '{marca}' no existe o no está activa.");

    continue;
}

if (string.IsNullOrWhiteSpace(categoria))
{
    errores.Add($"Fila {fila}: el campo 'Categoría' es obligatorio.");

    continue;
}

int idCategoria = logica.ObtenerIdCategoriaPorNombre(categoria);

if (idCategoria == 0)
{
    errores.Add($"Fila {fila}: la categoría '{categoria}' no existe o no está activa.");

    continue;
}

if (string.IsNullOrWhiteSpace(unidad))
{
    errores.Add($"Fila {fila}: el campo 'Unidad' es obligatorio.");

    continue;
}

int idUnidad = logica.ObtenerIdUnidadPorNombre(unidad);

if (idUnidad == 0)
```

```
                {
                    errores.Add($"Fila {fila}: la unidad '{unidad}' no existe.");

                    continue;
                }

                if (string.IsNullOrWhiteSpace(ivaTexto))
                {
                    errores.Add($"Fila {fila}: el campo 'IVA' es obligatorio.");

                    continue;
                }

                decimal alicuota;

                if (!decimal.TryParse(ivaTexto, System.Globalization.NumberStyles.Number,
System.Globalization.CultureInfo.CurrentCulture, out alicuota))
                {
                    errores.Add($"Fila {fila}: el IVA '{ivaTexto}' no tiene un formato válido.");

                    continue;
                }

                int idIVA = logica.ObtenerIdIVAPorAlicuota(alicuota);

                if (idIVA == 0)
                {
                    errores.Add($"Fila {fila}: no existe un IVA activo con alícuota {alicuota}.");

                    continue;
                }

                //Crear Producto
                ClsGestorEntidadProducto.ClsProducto producto = new ClsGestorEntidadProducto.ClsProducto();

                producto.CodigoBarra = codigoBarra;
                producto.Nombre = nombre;
                producto.Descripcion = descripcion;
                producto.IdMarca = idMarca;
                producto.IdCategoria = idCategoria;
                producto.IdUnidad = idUnidad;
                producto.IdIVA = idIVA;
                producto.Activo = true;
                producto.PrecioVenta = 0;

                productos.Add(producto);

                if (!string.IsNullOrWhiteSpace(codigoBarra))
                {
                    fechasVencimiento[codigoBarra] = fechaVencimiento;
                }

                cantidadArticulos++;
            }

            //Validar articulo
            if (cantidadArticulos == 0 && errores.Count == 0)
            {
                MessageBox.Show("No se encontraron artículos para importar.","Importación de artículos",
```

```
                MessageBoxButtons.OK, MessageBoxIcon.Warning);

            return;
        }

        if (errores.Count > 0)
        {
            string mensaje = "Se encontraron los siguientes errores:\n\n";

            mensaje += string.Join(Environment.NewLine, errores);

            mensaje += "\n\nNo se importó ningún artículo. Corrija el archivo y vuelva a intentarlo.";

            MessageBox.Show(mensaje,"Errores de validación", MessageBoxButtons.OK, MessageBoxIcon.Error);

            return;
        }

        //Verificacion proveedores costo
        if (!libro.Worksheets.Contains("PROVEEDORES_COSTOS"))
        {
            MessageBox.Show("El archivo no contiene la hoja 'PROVEEDORES_COSTOS'.", "Importación de artículos",
                MessageBoxButtons.OK, MessageBoxIcon.Error);

            return;
        }

        IXLWorksheet hojaProveedoresCostos = libro.Worksheet("PROVEEDORES_COSTOS");

        int ultimaFilaProveedores = hojaProveedoresCostos.LastRowUsed()?.RowNumber() ?? 0;

        //Leer proveedores costo
        if (ultimaFilaProveedores >= 3)
        {
            for (int fila = 3; fila <= ultimaFilaProveedores; fila++)
            {
                string codigoBarra = hojaProveedoresCostos.Cell(fila, 1).GetString().Trim();
                string proveedor = hojaProveedoresCostos.Cell(fila, 2).GetString().Trim();
                string sucursal = hojaProveedoresCostos.Cell(fila, 3).GetString().Trim();
                string precioTexto = hojaProveedoresCostos.Cell(fila, 4).GetString().Trim();

                // Fila vacía
                if (string.IsNullOrWhiteSpace(codigoBarra) && string.IsNullOrWhiteSpace(proveedor) &&
string.IsNullOrWhiteSpace(sucursal) && string.IsNullOrWhiteSpace(precioTexto))
                {
                    continue;
                }

                if (string.IsNullOrWhiteSpace(codigoBarra))
                {
                    errores.Add($"PROVEEDORES_COSTOS - Fila {fila}: el Código de Barra es obligatorio.");

                    continue;
                }

                if (string.IsNullOrWhiteSpace(proveedor))
                {
                    errores.Add($"PROVEEDORES_COSTOS - Fila {fila}: el Proveedor es obligatorio.");
```

```
                    continue;
                }

                if (string.IsNullOrWhiteSpace(sucursal))
                {
                    errores.Add($"PROVEEDORES_COSTOS - Fila {fila}: la Sucursal es obligatoria.");

                    continue;
                }

                if (string.IsNullOrWhiteSpace(precioTexto))
                {
                    errores.Add($"PROVEEDORES_COSTOS - Fila {fila}: el Precio de Compra es obligatorio.");

                    continue;
                }

                decimal precioCompra;

                if (!decimal.TryParse(precioTexto, System.Globalization.NumberStyles.Number,
System.Globalization.CultureInfo.CurrentCulture, out precioCompra))
                {
                    errores.Add($"PROVEEDORES_COSTOS - Fila {fila}: el Precio de Compra '{precioTexto}' no tiene un formato válido.");

                    continue;
                }

                if (precioCompra < 0)
                {
                    errores.Add( $"PROVEEDORES_COSTOS - Fila {fila}: el Precio de Compra no puede ser negativo.");

                    continue;
                }

                proveedoresCostos.Add((codigoBarra, proveedor, sucursal, precioCompra));
            }
        }

        //Validar codigo de barra
        foreach (var proveedorCosto in proveedoresCostos)
        {
            bool existeArticuloEnImportacion = productos.Any(p => p.CodigoBarra == proveedorCosto.CodigoBarra);

            if (!existeArticuloEnImportacion)
            {
                errores.Add($"PROVEEDORES_COSTOS: el código de barra '{proveedorCosto.CodigoBarra}' " + $"no corresponde a
ningún artículo de la hoja ARTICULOS.");
            }
        }

        //Validar proveedores y sucursales
        if (errores.Count == 0 && proveedoresCostos.Count > 0)
        {
            Conexion conexionValidacion = new Conexion();

            using (SqlConnection conexion = conexionValidacion.ObtenerConexion())
            {
                conexion.Open();
```

```
                using (SqlTransaction transaccion = conexion.BeginTransaction())
                {
                    try
                    {
                        foreach (var proveedorCosto in proveedoresCostos)
                        {
                            int idSucursal = logica.ObtenerIdSucursalImportacion( proveedorCosto.Proveedor,  proveedorCosto.Sucursal,
conexion, transaccion);

                            if (idSucursal == 0)
                            {
                                errores.Add($"PROVEEDORES_COSTOS: el proveedor '{proveedorCosto.Proveedor}' " +
                                    $"no tiene una sucursal activa llamada '{proveedorCosto.Sucursal}'.");
                            }
                        }

                        transaccion.Rollback();
                    }
                    catch
                    {
                        try
                        {
                            transaccion.Rollback();
                        }
                        catch
                        {
                        }

                        throw;
                    }
                }
            }
        }

        if (errores.Count > 0)
        {
            string mensaje = "Se encontraron los siguientes errores:\n\n";

            mensaje += string.Join(Environment.NewLine,errores);
            mensaje += "\n\nNo se importó ningún artículo. Corrija el archivo y vuelva a intentarlo.";

            MessageBox.Show(mensaje,"Errores de validación",MessageBoxButtons.OK, MessageBoxIcon.Error);

            return;
        }

        //Guardartodo en una transaccion
        Conexion cn = new Conexion();

        using (SqlConnection conexion = cn.ObtenerConexion())
        {
            conexion.Open();

            using (SqlTransaction transaccion = conexion.BeginTransaction())
            {
                try
                {

                    //Obtener siguiente sku
```

```
                int siguienteSKU = logica.ObtenerProximoSKUImportacion(conexion, transaccion);

                //Registrar producto
                foreach (ClsGestorEntidadProducto.ClsProducto producto in productos)
                {
                    producto.SKU = siguienteSKU.ToString();

                    int idProducto = logica.RegistrarProductoImportacion(producto, conexion, transaccion);

                    producto.IdProducto = idProducto;

                    siguienteSKU++;
                }

                //Registrarproveedore costo y lote
                foreach (var proveedorCosto in proveedoresCostos)
                {
                    ClsGestorEntidadProducto.ClsProducto producto = productos.FirstOrDefault(p => p.CodigoBarra ==
proveedorCosto.CodigoBarra);

                    if (producto == null)
                    {
                        throw new Exception($"No se encontró el artículo con código de barra '{proveedorCosto.CodigoBarra}'.");
                    }

                    int idSucursal = logica.ObtenerIdSucursalImportacion(proveedorCosto.Proveedor,proveedorCosto.Sucursal,conexion,
transaccion);

                    if (idSucursal == 0)
                    {
                        throw new Exception($"No se encontró la sucursal '{proveedorCosto.Sucursal}' " +
                            $"del proveedor '{proveedorCosto.Proveedor}'.");
                    }

                    //Relacion sucursal proveedor
                    ClsGestorEntidadProducto.ClsProductoSucursalProveedor relacion = new
ClsGestorEntidadProducto.ClsProductoSucursalProveedor();

                    relacion.IdProducto = producto.IdProducto;
                    relacion.IdSucursal = idSucursal;
                    relacion.PrecioCompra = proveedorCosto.PrecioCompra;
                    relacion.FechaPrecioCompra = DateTime.Now;
                    relacion.CodigoProveedor = null;
                    relacion.Activo = true;
                    logica.RegistrarProductoSucursalProveedorImportacion(relacion, conexion,transaccion);
                    relaciones.Add(relacion);

                    //Lote
                    DateTime? fechaVencimiento = null;

                    if (!string.IsNullOrWhiteSpace(proveedorCosto.CodigoBarra))
                    {
                        fechasVencimiento.TryGetValue(proveedorCosto.CodigoBarra, out fechaVencimiento);
                    }

                    if (fechaVencimiento.HasValue)
                    {
                        ClsLote lote = new ClsLote();
```

```
                        lote.IdProducto = producto.IdProducto;
                        lote.CodigoLote = "IMP-" + producto.SKU;
                        lote.FechaFabricacion = DateTime.Now.Date;
                        lote.FechaVencimiento = fechaVencimiento.Value.Date;
                        lote.FechaIngreso = DateTime.Now;
                        lote.Stock = 0;
                        lote.Activo = true;
                        lote.IdSucursal = idSucursal;
                        logica.RegistrarLoteImportacion(lote, conexion, transaccion);
                    }
                }

                //Confirmar todo
                transaccion.Commit();

                MessageBox.Show($"Importación realizada correctamente.\n\n" +$"Artículos importados: {productos.Count}\n" +
                    $"Relaciones proveedor/sucursal: {relaciones.Count}", "Importación de artículos",
                    MessageBoxButtons.OK, MessageBoxIcon.Information);
            }
            catch (Exception ex)
            {
                try
                {
                    transaccion.Rollback();
                }
                catch
                {
                }

                MessageBox.Show("No se pudo completar la importación.\n\n" +
                    "No se guardó ningún artículo, proveedor, costo ni lote.\n\n" +ex.Message, "Error de importación",
                    MessageBoxButtons.OK, MessageBoxIcon.Error);
            }
        }
    }
  }
  catch (Exception ex)
  {
    MessageBox.Show("Ocurrió un error al leer o procesar el archivo de Excel.\n\n" +ex.Message, "Importación de artículos",
        MessageBoxButtons.OK, MessageBoxIcon.Error);
  }
}
private void GenerarPlantillaImportacion()
{
  try
  {
    using (XLWorkbook libro = new XLWorkbook())
    {
      //Colores
      XLColor colorTitulo = XLColor.FromHtml("#1F4E78");
      XLColor colorEncabezado = XLColor.FromHtml("#4472C4");
      XLColor colorCarga = XLColor.FromHtml("#F3F8FC");
      XLColor colorCargaAlterna = XLColor.FromHtml("#EAF2F8");
      XLColor colorObligatorio = XLColor.FromHtml("#FCE4D6");
      XLColor colorBorde = XLColor.FromHtml("#B4C7E7");
      XLColor colorInstrucciones = XLColor.FromHtml("#FFF2CC");

      //Constantes de filas
```

```
int filaEncabezado = 2;
int primeraFilaDatos = 3;
int ultimaFilaDatos = 502;

//Hoja de los articulos
IXLWorksheet hojaArticulos = libro.Worksheets.Add("ARTICULOS");

hojaArticulos.Cell(1, 1).Value = "PLANTILLA DE IMPORTACIÓN DE ARTÍCULOS";
hojaArticulos.Range("A1:H1").Merge();

//Encabezado
hojaArticulos.Cell(filaEncabezado, 1).Value = "Código de Barra";
hojaArticulos.Cell(filaEncabezado, 2).Value = "Nombre (*)";
hojaArticulos.Cell(filaEncabezado, 3).Value = "Descripción";
hojaArticulos.Cell(filaEncabezado, 4).Value = "Marca (*)";
hojaArticulos.Cell(filaEncabezado, 5).Value = "Categoría (*)";
hojaArticulos.Cell(filaEncabezado, 6).Value = "Unidad (*)";
hojaArticulos.Cell(filaEncabezado, 7).Value = "IVA (*)";
hojaArticulos.Cell(filaEncabezado, 8).Value = "Fecha Vencimiento";

//Estilo del titulo
hojaArticulos.Range("A1:H1").Style.Fill.SetBackgroundColor(colorTitulo);
hojaArticulos.Range("A1:H1").Style.Font.Bold = true;
hojaArticulos.Range("A1:H1").Style.Font.FontColor = XLColor.White;
hojaArticulos.Range("A1:H1").Style.Font.FontSize = 16;
hojaArticulos.Range("A1:H1").Style.Alignment.Horizontal = XLAlignmentHorizontalValues.Center;
hojaArticulos.Range("A1:H1").Style.Alignment.Vertical = XLAlignmentVerticalValues.Center;
hojaArticulos.Row(1).Height = 30;

//Estilo del encabedo
hojaArticulos.Range("A2:H2").Style.Fill.SetBackgroundColor(colorEncabezado);
hojaArticulos.Range("A2:H2").Style.Font.Bold = true;
hojaArticulos.Range("A2:H2").Style.Font.FontColor = XLColor.White;
hojaArticulos.Range("A2:H2").Style.Alignment.Horizontal = XLAlignmentHorizontalValues.Center;
hojaArticulos.Range("A2:H2").Style.Alignment.Vertical = XLAlignmentVerticalValues.Center;
hojaArticulos.Range("A2:H2").Style.Alignment.WrapText = true;
hojaArticulos.Row(2).Height = 30;

//Bordes
hojaArticulos.Range($"A2:H{ultimaFilaDatos}").Style.Border.OutsideBorder = XLBorderStyleValues.Thin;
hojaArticulos.Range($"A2:H{ultimaFilaDatos}").Style.Border.InsideBorder = XLBorderStyleValues.Thin;
hojaArticulos.Range($"A2:H{ultimaFilaDatos}").Style.Border.OutsideBorderColor = colorBorde;
hojaArticulos.Range($"A2:H{ultimaFilaDatos}").Style.Border.InsideBorderColor = colorBorde;

//Colo de las filas
hojaArticulos.Range($"A{primeraFilaDatos}:H{ultimaFilaDatos}").Style.Fill.SetBackgroundColor(colorCarga);

for (int fila = primeraFilaDatos; fila <= ultimaFilaDatos; fila++)
{
   if (fila % 2 == 0)
   {
      hojaArticulos.Range($"A{fila}:H{fila}").Style.Fill.SetBackgroundColor(colorCargaAlterna);
   }
}

//Campo oblogatorios
hojaArticulos.Range($"B{primeraFilaDatos}:B{ultimaFilaDatos}").Style.Fill.SetBackgroundColor(colorObligatorio);
hojaArticulos.Range($"D{primeraFilaDatos}:G{ultimaFilaDatos}").Style.Fill.SetBackgroundColor(colorObligatorio);
```

```
            //Codigo de barras
            hojaArticulos.Range($"A{primeraFilaDatos}:A{ultimaFilaDatos}").Style.NumberFormat.Format = "@";
            hojaArticulos.Range($"A{primeraFilaDatos}:A{ultimaFilaDatos}").Style.Alignment.Horizontal =
XLAlignmentHorizontalValues.Center;

            //Fecha
            IXLRange rangoFechas = hojaArticulos.Range($"H{primeraFilaDatos}:H{ultimaFilaDatos}");

            rangoFechas.Style.NumberFormat.Format = "00\\/00\\/0000";
            rangoFechas.Style.Alignment.Horizontal = XLAlignmentHorizontalValues.Center;

            //Alineacion
            hojaArticulos.Range($"D{primeraFilaDatos}:G{ultimaFilaDatos}").Style.Alignment.Horizontal =
XLAlignmentHorizontalValues.Center;

            //Anchos
            hojaArticulos.Column(1).Width = 18;
            hojaArticulos.Column(2).Width = 28;
            hojaArticulos.Column(3).Width = 35;
            hojaArticulos.Column(4).Width = 22;
            hojaArticulos.Column(5).Width = 22;
            hojaArticulos.Column(6).Width = 18;
            hojaArticulos.Column(7).Width = 15;
            hojaArticulos.Column(8).Width = 20;

            //Hojas auxiliares
            IXLWorksheet hojaMarcas = libro.Worksheets.Add("_MARCAS");
            IXLWorksheet hojaCategorias = libro.Worksheets.Add("_CATEGORIAS");
            IXLWorksheet hojaUnidades = libro.Worksheets.Add("_UNIDADES");
            IXLWorksheet hojaIVA = libro.Worksheets.Add("_IVA");
            IXLWorksheet hojaProveedores = libro.Worksheets.Add("_PROVEEDORES");
            IXLWorksheet hojaSucursales = libro.Worksheets.Add("_SUCURSALES");

            //Marcas
            DataTable tablaMarcas = new ClsGestorLogicaProducto().CargarMarcasActivas();

            hojaMarcas.Cell(1, 1).Value = "Marca";

            int filaMarca = 2;

            foreach (DataRow fila in tablaMarcas.Rows)
            {
                hojaMarcas.Cell(filaMarca, 1).Value = fila["Nombre"].ToString();

                filaMarca++;
            }

            //Categorias
            DataTable tablaCategorias = new ClsGestorLogicaProducto().CargarCategoriasActivas();

            hojaCategorias.Cell(1, 1).Value = "Categoría";

            int filaCategoria = 2;

            foreach (DataRow fila in tablaCategorias.Rows)
            {
                hojaCategorias.Cell(filaCategoria, 1).Value = fila["Nombre"].ToString();

                filaCategoria++;
```

```
}

//Unidades
DataTable tablaUnidades = new ClsGestorLogicaProducto().CargarUnidades();

hojaUnidades.Cell(1, 1).Value = "Unidad";

int filaUnidad = 2;

foreach (DataRow fila in tablaUnidades.Rows)
{
   hojaUnidades.Cell(filaUnidad, 1).Value = fila["Nombre"].ToString();

   filaUnidad++;
}

//Iva
DataTable tablaIVA = new ClsGestorLogicaProducto().CargarIVAsActivos();

hojaIVA.Cell(1, 1).Value = "Alicuota";

int filaIVA = 2;

foreach (DataRow fila in tablaIVA.Rows)
{
   hojaIVA.Cell(filaIVA, 1).Value = Convert.ToDecimal(fila["Alicuota"]);

   filaIVA++;
}

//Proveedores
ClsGestorLogicaProveedor logicaProveedor = new ClsGestorLogicaProveedor();
DataTable tablaProveedores = logicaProveedor.ObtenerProveedores();

hojaProveedores.Cell(1, 1).Value = "IdProveedor";
hojaProveedores.Cell(1, 2).Value = "Proveedor";

int filaProveedor = 2;

foreach (DataRow fila in tablaProveedores.Rows)
{
   bool proveedorActivo = false;

   if (fila["Estado"] != DBNull.Value)
   {
      string estado = fila["Estado"].ToString().Trim();

      proveedorActivo = estado.Equals("Activo", StringComparison.OrdinalIgnoreCase) ||
      estado.Equals("1", StringComparison.OrdinalIgnoreCase) || estado.Equals("True", StringComparison.OrdinalIgnoreCase);
   }

   if (proveedorActivo)
   {
      int idProveedor;

      if (!int.TryParse(fila["IdProveedor"].ToString(), out idProveedor))
      {
         continue;
      }
```

```
            hojaProveedores.Cell(filaProveedor, 1).Value = idProveedor;
            hojaProveedores.Cell(filaProveedor, 2).Value = fila["RazonSocial"].ToString();

            filaProveedor++;
        }
    }

    //Sucursales
    hojaSucursales.Cell(1, 1).Value = "IdProveedor";
    hojaSucursales.Cell(1, 2).Value = "Proveedor";
    hojaSucursales.Cell(1, 3).Value = "IdSucursal";
    hojaSucursales.Cell(1, 4).Value = "Sucursal";

    int filaSucursalAuxiliar = 2;

    for (int fila = 2; fila < filaProveedor; fila++)
    {
        string valorIdProveedor = hojaProveedores.Cell(fila, 1).GetString().Trim();

        if (!int.TryParse(valorIdProveedor, out int idProveedor))
        {
            continue;
        }

        string nombreProveedor = hojaProveedores.Cell(fila, 2).GetString().Trim();

        if (idProveedor <= 0 || string.IsNullOrWhiteSpace(nombreProveedor))
        {
            continue;
        }

        DataTable tablaSucursales = logicaProveedor.ObtenerSucursalesPorProveedor(idProveedor);

        if (tablaSucursales == null || tablaSucursales.Rows.Count == 0)
        {
            continue;
        }

        foreach (DataRow sucursal in tablaSucursales.Rows)
        {
            if (!tablaSucursales.Columns.Contains("Sucursal"))
            {
                continue;
            }

            if (sucursal["Sucursal"] == DBNull.Value)
            {
                continue;
            }

            string nombreSucursal = sucursal["Sucursal"].ToString().Trim();

            if (string.IsNullOrWhiteSpace(nombreSucursal))
            {
                continue;
            }

            int idSucursal;
```

```
        if (!int.TryParse(sucursal["IdSucursal"].ToString(), out idSucursal))
        {
            continue;
        }

        hojaSucursales.Cell(filaSucursalAuxiliar, 1).Value = idProveedor;
        hojaSucursales.Cell(filaSucursalAuxiliar, 2).Value = nombreProveedor;
        hojaSucursales.Cell(filaSucursalAuxiliar, 3).Value = idSucursal;
        hojaSucursales.Cell(filaSucursalAuxiliar, 4).Value = nombreSucursal;

        filaSucursalAuxiliar++;
    }
}

//Rango nombrados
if (filaMarca > 2)
{
    hojaMarcas.Range($"A2:A{filaMarca - 1}").AddToNamed("ListaMarcas");
}

if (filaCategoria > 2)
{
    hojaCategorias.Range($"A2:A{filaCategoria - 1}").AddToNamed("ListaCategorias");
}

if (filaUnidad > 2)
{
    hojaUnidades.Range($"A2:A{filaUnidad - 1}").AddToNamed("ListaUnidades");
}

if (filaIVA > 2)
{
    hojaIVA.Range($"A2:A{filaIVA - 1}").AddToNamed("ListaIVA");
}

hojaArticulos.Range($"A{primeraFilaDatos}:A{ultimaFilaDatos}").AddToNamed("ListaCodigosBarra");

if (filaProveedor > 2)
{
    hojaProveedores.Range($"B2:B{filaProveedor - 1}").AddToNamed("ListaProveedores");
}

//Validacionde los los articulos
if (filaMarca > 2)
{
    hojaArticulos.Range($"D{primeraFilaDatos}:D{ultimaFilaDatos}").CreateDataValidation().List("=ListaMarcas");
}

if (filaCategoria > 2)
{
    hojaArticulos.Range($"E{primeraFilaDatos}:E{ultimaFilaDatos}").CreateDataValidation().List("=ListaCategorias");
}

if (filaUnidad > 2)
{
    hojaArticulos.Range($"F{primeraFilaDatos}:F{ultimaFilaDatos}").CreateDataValidation().List("=ListaUnidades");
}
```

```
if (filaIVA > 2)
{
    hojaArticulos.Range($"G{primeraFilaDatos}:G{ultimaFilaDatos}").CreateDataValidation().List("=ListaIVA");
}

//Validacion de la fecha
var validacionFecha = rangoFechas.CreateDataValidation();

validacionFecha.IgnoreBlanks = true;

//Hoja de los proveedores costo

IXLWorksheet hojaProveedoresCostos = libro.Worksheets.Add("PROVEEDORES_COSTOS");

hojaProveedoresCostos.Cell(1, 1).Value = "PROVEEDORES Y COSTOS";
hojaProveedoresCostos.Range("A1:D1").Merge();

//Encabezados
hojaProveedoresCostos.Cell(2, 1).Value = "Código de Barra";
hojaProveedoresCostos.Cell(2, 2).Value = "Proveedor (*)";
hojaProveedoresCostos.Cell(2, 3).Value = "Sucursal (*)";
hojaProveedoresCostos.Cell(2, 4).Value = "Precio Compra (*)";

//Titulo pagina 2


hojaProveedoresCostos.Range("A1:D1").Style.Fill.SetBackgroundColor(colorTitulo);
hojaProveedoresCostos.Range("A1:D1").Style.Font.Bold = true;
hojaProveedoresCostos.Range("A1:D1").Style.Font.FontColor = XLColor.White;
hojaProveedoresCostos.Range("A1:D1").Style.Font.FontSize = 16;
hojaProveedoresCostos.Range("A1:D1").Style.Alignment.Horizontal = XLAlignmentHorizontalValues.Center;
hojaProveedoresCostos.Range("A1:D1").Style.Alignment.Vertical = XLAlignmentVerticalValues.Center;
hojaProveedoresCostos.Row(1).Height = 30;

//Encabezados
hojaProveedoresCostos.Range("A2:D2").Style.Fill.SetBackgroundColor(colorEncabezado);
hojaProveedoresCostos.Range("A2:D2").Style.Font.Bold = true;
hojaProveedoresCostos.Range("A2:D2").Style.Font.FontColor = XLColor.White;
hojaProveedoresCostos.Range("A2:D2").Style.Alignment.Horizontal = XLAlignmentHorizontalValues.Center;
hojaProveedoresCostos.Range("A2:D2").Style.Alignment.Vertical = XLAlignmentVerticalValues.Center;
hojaProveedoresCostos.Range("A2:D2").Style.Alignment.WrapText = true;
hojaProveedoresCostos.Row(2).Height = 30;

//Borde
hojaProveedoresCostos.Range($"A2:D{ultimaFilaDatos}").Style.Border.OutsideBorder = XLBorderStyleValues.Thin;
hojaProveedoresCostos.Range($"A2:D{ultimaFilaDatos}").Style.Border.InsideBorder = XLBorderStyleValues.Thin;
hojaProveedoresCostos.Range($"A2:D{ultimaFilaDatos}").Style.Border.OutsideBorderColor = colorBorde;
hojaProveedoresCostos.Range($"A2:D{ultimaFilaDatos}").Style.Border.InsideBorderColor = colorBorde;

//Colo de las filad
hojaProveedoresCostos.Range($"A{primeraFilaDatos}:D{ultimaFilaDatos}").Style.Fill.SetBackgroundColor(colorCarga);

for (int fila = primeraFilaDatos; fila <= ultimaFilaDatos; fila++)
{
    if (fila % 2 == 0)
    {
        hojaProveedoresCostos.Range($"A{fila}:D{fila}").Style.Fill.SetBackgroundColor(colorCargaAlterna);
    }
}
```

```
            //Obligatorios
            hojaProveedoresCostos.Range($"B{primeraFilaDatos}:D{ultimaFilaDatos}").Style.Fill.SetBackgroundColor(colorObligatorio);

            //Codigo de barra
            hojaProveedoresCostos.Range($"A{primeraFilaDatos}:A{ultimaFilaDatos}").Style.NumberFormat.Format = "@";
            hojaProveedoresCostos.Range($"A{primeraFilaDatos}:A{ultimaFilaDatos}").Style.Alignment.Horizontal =
XLAlignmentHorizontalValues.Center;

            //Precio
            hojaProveedoresCostos.Range($"D{primeraFilaDatos}:D{ultimaFilaDatos}").Style.NumberFormat.Format = "#,##0.00";
            hojaProveedoresCostos.Range($"D{primeraFilaDatos}:D{ultimaFilaDatos}").Style.Alignment.Horizontal =
XLAlignmentHorizontalValues.Right;

            //Sucursal
            hojaProveedoresCostos.Range($"C{primeraFilaDatos}:C{ultimaFilaDatos}").Style.Alignment.Horizontal =
XLAlignmentHorizontalValues.Center;

            //Anchos
            hojaProveedoresCostos.Column(1).Width = 20;
            hojaProveedoresCostos.Column(2).Width = 30;
            hojaProveedoresCostos.Column(3).Width = 30;
            hojaProveedoresCostos.Column(4).Width = 20;

            //Validacioon del codgigo de barras
            var validacionCodigo = hojaProveedoresCostos.Range($"A{primeraFilaDatos}:A{ultimaFilaDatos}").CreateDataValidation();

            validacionCodigo.List("=ListaCodigosBarra");
            validacionCodigo.IgnoreBlanks = true;

            //Validacion de los proveedores
            if (filaProveedor > 2)
            {
                hojaProveedoresCostos.Range($"B{primeraFilaDatos}:B{ultimaFilaDatos}").CreateDataValidation().List("=ListaProveedores");
            }

            //Rango de sucursales
            int ultimaFilaSucursales = 0;

            if (hojaSucursales.LastRowUsed() != null)
            {
                ultimaFilaSucursales = hojaSucursales.LastRowUsed().RowNumber();
            }

            for (int fila = 2; fila < filaProveedor; fila++)
            {
                string valorIdProveedor = hojaProveedores.Cell(fila, 1).GetString().Trim();

                if (!int.TryParse(valorIdProveedor, out int idProveedor))
                {
                    continue;
                }

                string nombreRango = "Proveedor_" + idProveedor;

                List<string> sucursales = new List<string>();

                for (int filaS = 2; filaS <= ultimaFilaSucursales; filaS++)
                {
```

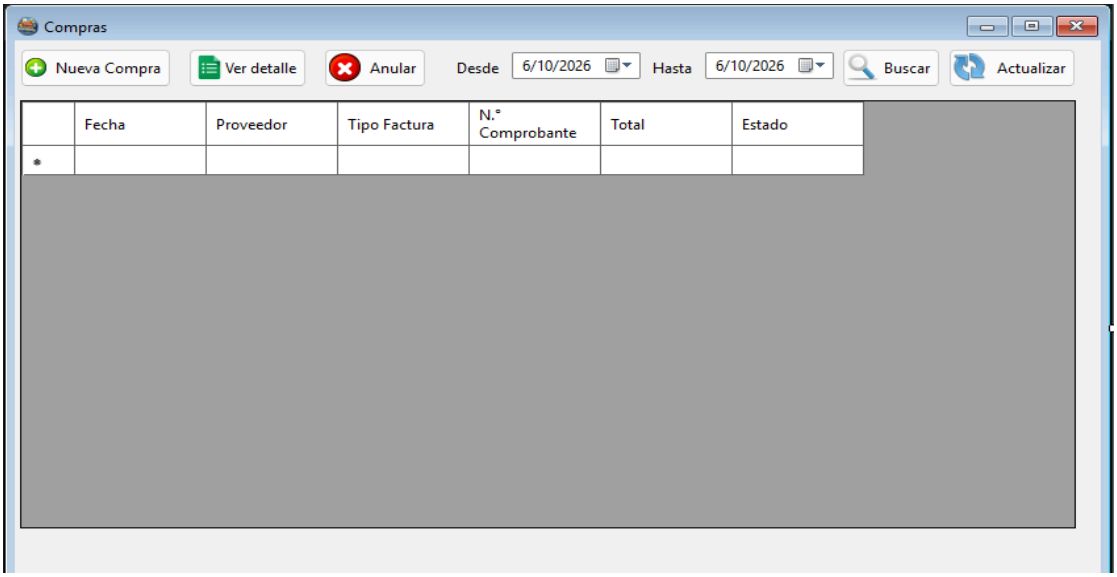
```
                string valorIdProveedorSucursal = hojaSucursales.Cell(filaS, 1).GetString().Trim();

                if (!int.TryParse(valorIdProveedorSucursal, out int idProveedorSucursal))
                {
                    continue;
                }

                if (idProveedorSucursal == idProveedor)
                {
                    string nombreSucursal = hojaSucursales.Cell(filaS, 4).GetString().Trim();

                    if (!string.IsNullOrWhiteSpace(nombreSucursal))
                    {
                        sucursales.Add(nombreSucursal);
                    }
                }
            }

            if (sucursales.Count > 0)
            {
                int columna = 5 + idProveedor;

                for (int i = 0; i < sucursales.Count; i++)
                {
                    hojaSucursales.Cell(i + 2, columna).Value = sucursales[i];
                }

                IXLRange rangoSucursal = hojaSucursales.Range(hojaSucursales.Cell(2, columna), hojaSucursales.Cell(
sucursales.Count + 1, columna));

                libro.NamedRanges.Add(nombreRango, rangoSucursal);
            }
        }

        //Sucursal dependiente
        if (filaProveedor > 2)
        {
            string ultimaFilaProveedor = (filaProveedor - 1).ToString();

            for (int fila = primeraFilaDatos; fila <= ultimaFilaDatos; fila++)
            {
                string formulaSucursal = "=INDIRECT(" +
                    "\"Proveedor_\"&" +
                    "INDEX(" +
                    "_PROVEEDORES!A2:A" +
                    ultimaFilaProveedor +
                    ",MATCH(B" +
                    fila +
                    ",_PROVEEDORES!B2:B" +
                    ultimaFilaProveedor +
                    ",0)" +
                    ")" +
                    ")";

                var validacionSucursal = hojaProveedoresCostos.Cell(fila, 3).CreateDataValidation();
                validacionSucursal.List(formulaSucursal);
                validacionSucursal.IgnoreBlanks = true;
                validacionSucursal.InCellDropdown = true;
            }
```

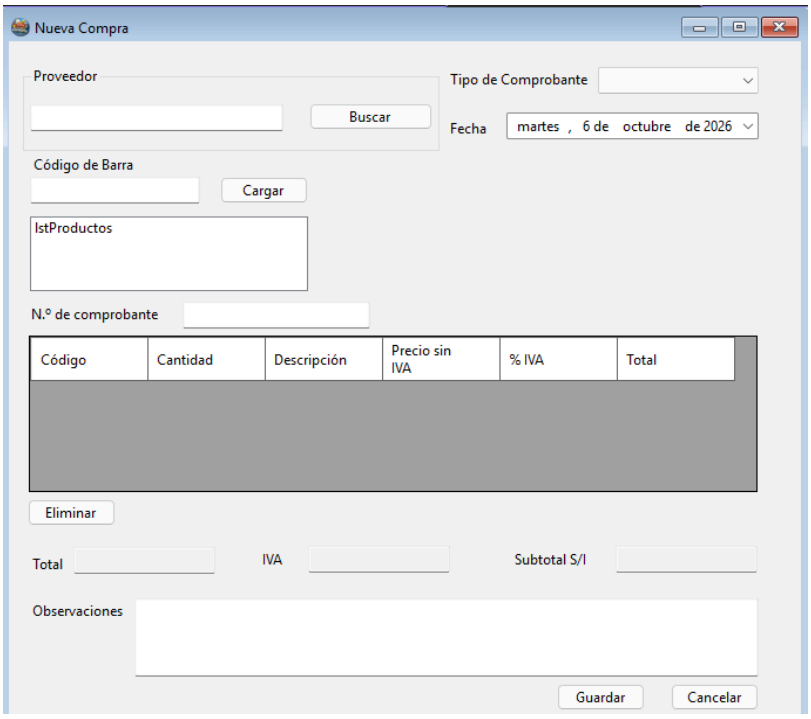
```
        }

        IXLWorksheet hojaInstrucciones = libro.Worksheets.Add("INSTRUCCIONES");

        hojaInstrucciones.Cell(1, 1).Value = "INSTRUCCIONES PARA IMPORTAR ARTÍCULOS";
        hojaInstrucciones.Range("A1:D1").Merge();

        //Instrucciones
        hojaInstrucciones.Cell(2, 1).Value = "IMPORTACIÓN DE ARTÍCULOS";
        hojaInstrucciones.Range("A2:D2").Merge();
        hojaInstrucciones.Cell(3, 1).Value = "1. Complete la hoja ARTICULOS.";
        hojaInstrucciones.Cell(4, 1).Value = "2. Los campos marcados con (*) son obligatorios.";
        hojaInstrucciones.Cell(5, 1).Value = "3. Seleccione Marca, Categoría, Unidad e IVA mediante los desplegables.";
        hojaInstrucciones.Cell(6, 1).Value = "4. La Fecha Vencimiento puede cargarse como dd/MM/yyyy o como 8 dígitos, por ejemplo
25092026.";
        hojaInstrucciones.Cell(7, 1).Value = "5. En PROVEEDORES_COSTOS puede cargar más de una fila para el mismo artículo.";
        hojaInstrucciones.Cell(8, 1).Value = "6. Seleccione primero el Proveedor y luego la Sucursal correspondiente.";
        hojaInstrucciones.Cell(9, 1).Value = "7. La Sucursal se filtra automáticamente según el Proveedor seleccionado.";
        hojaInstrucciones.Cell(10, 1).Value = "8. El Código de Barra de PROVEEDORES_COSTOS se obtiene de los códigos cargados
en ARTICULOS.";
        hojaInstrucciones.Cell(11, 1).Value = "9. El Precio de Compra debe ser mayor o igual a cero.";
        hojaInstrucciones.Cell(13, 1).Value = "IMPORTANTE";
        hojaInstrucciones.Range("A13:D13").Merge();
        hojaInstrucciones.Cell(14, 1).Value = "No modifique ni elimine las hojas auxiliares ocultas.";
        hojaInstrucciones.Cell(15, 1).Value = "Los campos con (*) son necesarios para realizar la importación.";
        hojaInstrucciones.Column(1).Width = 100;

        //Estilo instrucciones
        hojaInstrucciones.Range("A1:D1").Style.Fill.SetBackgroundColor(colorTitulo);
        hojaInstrucciones.Range("A1:D1").Style.Font.Bold = true;
        hojaInstrucciones.Range("A1:D1").Style.Font.FontColor = XLColor.White;
        hojaInstrucciones.Range("A1:D1").Style.Font.FontSize = 16;
        hojaInstrucciones.Range("A1:D1").Style.Alignment.Horizontal = XLAlignmentHorizontalValues.Center;
        hojaInstrucciones.Range("A2:D2").Style.Fill.SetBackgroundColor(colorEncabezado);
        hojaInstrucciones.Range("A2:D2").Style.Font.Bold = true;
        hojaInstrucciones.Range("A2:D2").Style.Font.FontColor = XLColor.White;
        hojaInstrucciones.Range("A2:D2").Style.Font.FontSize = 13;
        hojaInstrucciones.Range("A13:D13").Style.Fill.SetBackgroundColor(colorObligatorio);
        hojaInstrucciones.Range("A13:D13").Style.Font.Bold = true;
        hojaInstrucciones.Range("A14:D15").Style.Fill.SetBackgroundColor(colorInstrucciones);
        hojaInstrucciones.Range("A3:A15").Style.Alignment.WrapText = true;

        //Filtros
        hojaArticulos.Range($"A2:H{ultimaFilaDatos}").SetAutoFilter();
        hojaProveedoresCostos.Range($"A2:D{ultimaFilaDatos}").SetAutoFilter();

        //Congelar los encabezados
        hojaArticulos.SheetView.FreezeRows(2);
        hojaProveedoresCostos.SheetView.FreezeRows(2);

        //Oculta las hojas auxiliares
        hojaMarcas.Visibility = XLWorksheetVisibility.VeryHidden;
        hojaCategorias.Visibility = XLWorksheetVisibility.VeryHidden;
        hojaUnidades.Visibility = XLWorksheetVisibility.VeryHidden;
        hojaIVA.Visibility = XLWorksheetVisibility.VeryHidden;
        hojaProveedores.Visibility = XLWorksheetVisibility.VeryHidden;
        hojaSucursales.Visibility = XLWorksheetVisibility.VeryHidden;

        //Guardar
```

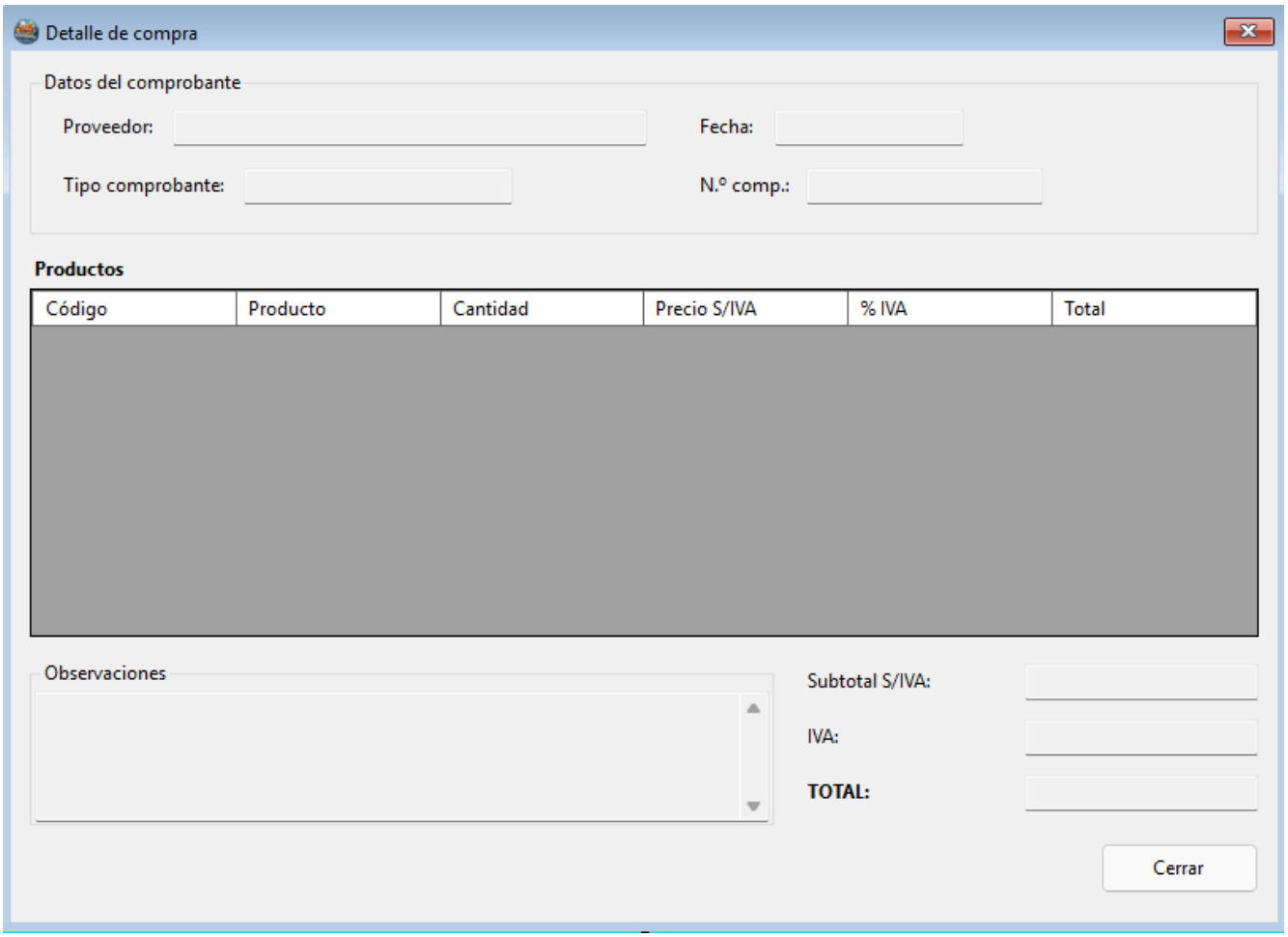
```
            SaveFileDialog guardar = new SaveFileDialog();

            guardar.Title = "Guardar plantilla de importación";
            guardar.Filter = "Archivo de Excel (*.xlsx)|*.xlsx";
            guardar.FileName = "Plantilla_Importacion_Articulos.xlsx";
            guardar.InitialDirectory = Environment.GetFolderPath(Environment.SpecialFolder.Desktop);

            if (guardar.ShowDialog() == DialogResult.OK)
            {
                libro.SaveAs(guardar.FileName);

                MessageBox.Show("La plantilla fue generada correctamente.","Plantilla de importación",
                    MessageBoxButtons.OK, MessageBoxIcon.Information);
            }
        }
    }
    catch (Exception ex)
    {
        MessageBox.Show("No se pudo generar la plantilla.\n\n" + ex.Message, "Plantilla de importación",
            MessageBoxButtons.OK, MessageBoxIcon.Error);
    }
}


#endregion

#region Eventos
private void btnSeleccionar_Click(object sender, EventArgs e)
{
    OpenFileDialog abrirArchivo = new OpenFileDialog();

    abrirArchivo.Title = "Seleccionar plantilla de artículos";
    abrirArchivo.Filter = "Archivo de Excel (*.xlsx)|*.xlsx";
    abrirArchivo.InitialDirectory = Environment.GetFolderPath(Environment.SpecialFolder.Desktop);
    abrirArchivo.Multiselect = false;

    if (abrirArchivo.ShowDialog() == DialogResult.OK)
    {
        txtTraerArchivo.Text = abrirArchivo.FileName;
    }
}
private void btnDescargarPlanilla_Click(object sender, EventArgs e)
{
    GenerarPlantillaImportacion();
}
private void btnAceptar_Click(object sender, EventArgs e)
{
    if (string.IsNullOrWhiteSpace(txtTraerArchivo.Text))
    {
        MessageBox.Show("Debe seleccionar un archivo de Excel.", "Importación de artículos",
            MessageBoxButtons.OK, MessageBoxIcon.Warning);

        return;
    }

    if (!File.Exists(txtTraerArchivo.Text))
    {
        MessageBox.Show("El archivo seleccionado no existe.", "Importación de artículos",
            MessageBoxButtons.OK, MessageBoxIcon.Warning);
```

```
                return;
            }

            ImportarArticulosDesdeExcel(txtTraerArchivo.Text);
        }
        #endregion

    }
}
```

4.7 Estructura principal de un caso de uso de negocio del software con interfaces y código: Compras




Nueva Compra
Proveedor
Buscar
Tipo de Comprobante
Fecha
martes , 6 de octubre de 2026
Código de Barra
Cargar
lstProductos
N.º de comprobante
Código
Cantidad
Descripción
Precio sin IVA
% IVA
Total
Eliminar
Total
IVA
Subtotal S/I
Observaciones
Guardar
Cancelar



Capa Vista // frmCompras

```
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows.Forms;
using CapaLogica.NegocioLogica;


namespace CapaVista
{
    public partial class frmCompras : Form
    {
        private ClsCompraLogica logica = new ClsCompraLogica();

        public frmCompras()
        {
            InitializeComponent();
```

```
        ConfigurarGrid();
        CargarCompras();

    }

    #region Metodos
    private void ConfigurarGrid()
    {
        dataGridView1.AutoGenerateColumns = false;
        dataGridView1.ReadOnly = true;
        dataGridView1.SelectionMode = DataGridViewSelectionMode.FullRowSelect;
        dataGridView1.MultiSelect = false;
        dataGridView1.AllowUserToAddRows = false;
        dataGridView1.AllowUserToResizeColumns = false;
        dataGridView1.AllowUserToOrderColumns = false;
        dataGridView1.RowHeadersVisible = false;

        dataGridView1.AutoSizeColumnsMode = DataGridViewAutoSizeColumnsMode.Fill;

        clnFecha.Width = 100;
        clnProveedor.Width = 220;
        clnTipoFactura.Width = 130;
        clnNCoprobante.Width = 150;
        clnTotal.Width = 120;
        clnEstado.Width = 100;
    }
    private void CargarCompras()
    {
        try
        {
            DateTime desde = dateTimePicker1.Value.Date;
            DateTime hasta = dateTimePicker2.Value.Date;

            DataTable tabla = logica.ObtenerCompras(desde, hasta);

            dataGridView1.Rows.Clear();

            foreach (DataRow fila in tabla.Rows)
            {
                int indice = dataGridView1.Rows.Add();

                dataGridView1.Rows[indice].Cells["clnFecha"].Value = Convert.ToDateTime(fila["Fecha"]).ToString("dd/MM/yyyy");
                dataGridView1.Rows[indice].Cells["clnProveedor"].Value = fila["Proveedor"].ToString();
                dataGridView1.Rows[indice].Cells["clnTipoFactura"].Value = fila["TipoComprobante"].ToString();
                dataGridView1.Rows[indice].Cells["clnNCoprobante"].Value = fila["NumeroComprobante"].ToString();
dataGridView1.Rows[indice].Cells["clnTotal"].Value = Convert.ToDecimal(fila["Total"]).ToString("N2");
                dataGridView1.Rows[indice].Cells["clnEstado"].Value = fila["Estado"].ToString();
                dataGridView1.Rows[indice].Tag = Convert.ToInt32(fila["IdCompra"]);
            }
        }
        catch (Exception ex)
        {
            MessageBox.Show("No se pudieron cargar las compras.\n\n" + ex.Message, "Error",
                MessageBoxButtons.OK, MessageBoxIcon.Error);
        }
    }
```

```
#endregion



#region Eventos
private void btnNuevacompra_Click(object sender, EventArgs e)
{
    frmNuevaCompra Nuevacomp = new frmNuevaCompra(false);
    Nuevacomp.ShowDialog();

    CargarCompras();
}
private void btnDetalle_Click(object sender, EventArgs e)
{
    if (dataGridView1.SelectedRows.Count == 0)
    {
        MessageBox.Show("Seleccione una compra para ver el detalle.", "Aviso",
            MessageBoxButtons.OK, MessageBoxIcon.Warning);

        return;
    }

    DataGridViewRow fila = dataGridView1.SelectedRows[0];

    if (fila.Tag == null)
    {
        MessageBox.Show("No se pudo obtener el identificador de la compra.", "Error",
            MessageBoxButtons.OK, MessageBoxIcon.Error);

        return;
    }

    int idCompra = Convert.ToInt32(fila.Tag);

    using (frmDetalleCompra formulario = new frmDetalleCompra(idCompra))
    {
        formulario.ShowDialog();
    }

}
private void button5_Click(object sender, EventArgs e)
{
    CargarCompras();
}
private void btnBuscar_Click(object sender, EventArgs e)
{
    if (dateTimePicker1.Value.Date > dateTimePicker2.Value.Date)
    {
        MessageBox.Show("La fecha Desde no puede ser posterior a la fecha Hasta.", "Aviso",
            MessageBoxButtons.OK, MessageBoxIcon.Warning);

        return;
    }

    CargarCompras();
}
```

```
#endregion


private void btnAnular_Click(object sender, EventArgs e)
{
    if (dataGridView1.SelectedRows.Count == 0)
    {
        MessageBox.Show("Seleccione una compra para anular.", "Aviso",
            MessageBoxButtons.OK, MessageBoxIcon.Warning);

        return;
    }

    DataGridViewRow fila = dataGridView1.SelectedRows[0];

    if (fila.Tag == null)
    {
        MessageBox.Show("No se pudo obtener el identificador de la compra.", "Error",
            MessageBoxButtons.OK, MessageBoxIcon.Error);

        return;
    }

    string estado = fila.Cells["clnEstado"].Value?.ToString();

    if (estado == "Anulada")
    {
        MessageBox.Show("La compra seleccionada ya se encuentra anulada.", "Aviso",
            MessageBoxButtons.OK, MessageBoxIcon.Information);

        return;
    }

    int idCompra = Convert.ToInt32(fila.Tag);

    string motivo = Microsoft.VisualBasic.Interaction.InputBox("Ingrese el motivo de la anulación:", "Anular compra", "");

    if (string.IsNullOrWhiteSpace(motivo))
    {
        MessageBox.Show("Debe ingresar un motivo para anular la compra.", "Aviso",
            MessageBoxButtons.OK, MessageBoxIcon.Warning);

        return;
    }

    DialogResult respuesta = MessageBox.Show("¿Está seguro de anular esta compra?\n\nMotivo: " + motivo, "Confirmar anulación",
        MessageBoxButtons.YesNo, MessageBoxIcon.Question);

    if (respuesta != DialogResult.Yes)
    {
        return;
    }

    try
    {
        logica.AnularCompra(idCompra, motivo);

        MessageBox.Show("La compra fue anulada correctamente.", "Compra anulada",
            MessageBoxButtons.OK, MessageBoxIcon.Information);
```

```
            CargarCompras();
        }
        catch (Exception ex)
        {
            MessageBox.Show("No se pudo anular la compra.\n\n" + ex.Message, "Error",
                MessageBoxButtons.OK, MessageBoxIcon.Error);
        }
    }
  }
}
```

Capa Vista // frmNuevaCompra

```
using CapaEntidades.NegocioEntidades;
using CapaLogica.NegocioLogica;
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows.Forms;

namespace CapaVista
{
    public partial class frmNuevaCompra : Form
    {
        private ClsCompraLogica logica = new ClsCompraLogica();
        private ClsGestorLogicaProveedor logicaProveedor = new ClsGestorLogicaProveedor();
        private int idProveedor = 0;
        private DataTable productosEncontrados;

        public frmNuevaCompra(bool mostrarChk)
        {
            InitializeComponent();

            dataGridView1.CellEndEdit += dataGridView1_CellEndEdit_1;
            txtCodigobarra.TextChanged += txtCodigobarra_TextChanged;
            txtCodigobarra.KeyDown += txtCodigobarra_KeyDown;

            ConfigurarAutocompletadoProveedores();
        }



        #region Metodos
        private void CalcularTotales()
        {
            decimal subtotalSinIVA = 0;
            decimal ivaTotal = 0;

            foreach (DataGridViewRow fila in dataGridView1.Rows)
            {
                if (fila.IsNewRow)
                {
                    continue;
```

```
            }

            if (fila.Cells["Column2"].Value == null || fila.Cells["Column4"].Value == null || fila.Cells["Column5"].Value == null)
            {
                continue;
            }


            decimal cantidad = Convert.ToDecimal(fila.Cells["Column2"].Value);
            decimal precio = Convert.ToDecimal(fila.Cells["Column4"].Value);
            decimal porcentajeIVA = Convert.ToDecimal(fila.Cells["Column5"].Value);

            decimal subtotal = cantidad * precio;
            decimal iva = subtotal * porcentajeIVA / 100;

            subtotalSinIVA += subtotal;
            ivaTotal += iva;
        }

        decimal total = subtotalSinIVA + ivaTotal;

        txtSubsinImpuesto.Text = subtotalSinIVA.ToString("N2");
        txtIva.Text = ivaTotal.ToString("N2");
        txtTotal.Text = total.ToString("N2");
    }
    private void ConfigurarAutocompletadoProveedores()
    {
        DataTable tabla = logicaProveedor.BuscarProveedor("");
        AutoCompleteStringCollection proveedores = new AutoCompleteStringCollection();

        foreach (DataRow fila in tabla.Rows)
        {
            proveedores.Add(fila["Nombre"].ToString());
        }

        txtProveedor.AutoCompleteMode = AutoCompleteMode.SuggestAppend;
        txtProveedor.AutoCompleteSource = AutoCompleteSource.CustomSource;
        txtProveedor.AutoCompleteCustomSource = proveedores;
    }
    #endregion



    #region Eventos
    private void chkActualizarprecio_CheckedChanged(object sender, EventArgs e)
    {

    }
    private void btnCargarProducto_Click(object sender, EventArgs e)
    {
        if (productosEncontrados == null || lstProductos.SelectedIndex < 0 || lstProductos.SelectedIndex >=
productosEncontrados.Rows.Count)
        {
            MessageBox.Show("Seleccione un producto de la lista.", "Aviso",
                MessageBoxButtons.OK, MessageBoxIcon.Warning);

            txtCodigobarra.Focus();
            return;
        }
```

```
    DataRow producto = productosEncontrados.Rows[lstProductos.SelectedIndex];

    int idProducto = Convert.ToInt32(producto["IdProducto"]);
    string codigoBarra = producto["CodigoBarra"].ToString();
    string nombre = producto["Nombre"].ToString();
    decimal precio = Convert.ToDecimal(producto["PrecioVenta"]);
    decimal iva = Convert.ToDecimal(producto["Alicuota"]);

    int indice = dataGridView1.Rows.Add();

    DataGridViewRow fila = dataGridView1.Rows[indice];

    fila.Cells["Column1"].Value = codigoBarra;
    fila.Cells["Column2"].Value = 1;
    fila.Cells["Column3"].Value = nombre;
    fila.Cells["Column4"].Value = precio;
    fila.Cells["Column5"].Value = iva;
    fila.Cells["Column6"].Value = precio * (1 + iva / 100);

    fila.Tag = idProducto;

    CalcularTotales();

    txtCodigobarra.Clear();
    lstProductos.Items.Clear();
    lstProductos.Visible = false;
    productosEncontrados = null;

    txtCodigobarra.Focus();
}
private void dataGridView1_CellContentClick(object sender, DataGridViewCellEventArgs e)
{

}
private void dataGridView1_CellEndEdit_1(object sender, DataGridViewCellEventArgs e)
{
    if (e.RowIndex < 0)
    {
        return;
    }

    DataGridViewRow fila = dataGridView1.Rows[e.RowIndex];

    if (fila.Cells["Column2"].Value == null || fila.Cells["Column4"].Value == null || fila.Cells["Column5"].Value == null)
    {
        return;
    }


    if (!decimal.TryParse(fila.Cells["Column2"].Value.ToString(), out decimal cantidad) || cantidad <= 0)
    {
        cantidad = 1;
        fila.Cells["Column2"].Value = 1;
    }

    if (!decimal.TryParse(fila.Cells["Column4"].Value.ToString(), out decimal precio) || precio < 0)
    {
        precio = 0;
```

```
            fila.Cells["Column4"].Value = 0;
        }

        if (!decimal.TryParse(fila.Cells["Column5"].Value.ToString(), out decimal iva))
        {
            iva = 0;
        }

        decimal subtotal = cantidad * precio;
        decimal total = subtotal + (subtotal * iva / 100);

        fila.Cells["Column6"].Value = total;

        CalcularTotales();
    }
    private void txtSubsinImpuesto_TextChanged(object sender, EventArgs e)
    {

    }
    private void btnEliminarCompra_Click(object sender, EventArgs e)
    {
        if (dataGridView1.SelectedRows.Count == 0)
        {
            MessageBox.Show("Seleccione un producto para eliminar.", "Aviso",
                MessageBoxButtons.OK, MessageBoxIcon.Warning);

            return;
        }

        DialogResult respuesta = MessageBox.Show("¿Está seguro de eliminar el producto seleccionado?", "Confirmar eliminación",
            MessageBoxButtons.YesNo, MessageBoxIcon.Question);

        if (respuesta != DialogResult.Yes)
        {
            return;
        }

        dataGridView1.Rows.RemoveAt(dataGridView1.SelectedRows[0].Index);

        CalcularTotales();
    }
    private void btnGuardar_Click(object sender, EventArgs e)
    {
        if (idProveedor == 0)
        {
            MessageBox.Show("Debe seleccionar un proveedor.", "Aviso",
                MessageBoxButtons.OK, MessageBoxIcon.Warning);

            txtProveedor.Focus();
            return;
        }

        if (string.IsNullOrWhiteSpace(comboBox1.Text))
        {
            MessageBox.Show("Debe seleccionar un tipo de comprobante.", "Aviso",
                MessageBoxButtons.OK, MessageBoxIcon.Warning);

            comboBox1.Focus();
            return;
```

```
}

if (string.IsNullOrWhiteSpace(txtNumeroComprobante.Text))
{
    MessageBox.Show("Debe ingresar el número de comprobante.", "Aviso",
        MessageBoxButtons.OK, MessageBoxIcon.Warning);

    txtNumeroComprobante.Focus();
    return;
}

if (dataGridView1.Rows.Count == 0)
{
    MessageBox.Show("Debe agregar al menos un producto a la compra.", "Aviso",
        MessageBoxButtons.OK, MessageBoxIcon.Warning);

    txtCodigobarra.Focus();
    return;
}

try
{
    ClsCompraEntidad compra = new ClsCompraEntidad();

    compra.IdProveedor = idProveedor;
    compra.Fecha = dateTimePicker1.Value;
    compra.TipoComprobante = comboBox1.Text.Trim();
    compra.NumeroComprobante = txtNumeroComprobante.Text.Trim();
    compra.SubtotalSinIVA = Convert.ToDecimal(txtSubsinImpuesto.Text);
    compra.IVA = Convert.ToDecimal(txtIva.Text);
    compra.Total = Convert.ToDecimal(txtTotal.Text);
    compra.Observaciones = txtObservaciones.Text.Trim();
    compra.Detalles = new List<ClsDetalleCompraEntidad>();

    foreach (DataGridViewRow fila in dataGridView1.Rows)
    {
        if (fila.IsNewRow)
        {
            continue;
        }

        if (fila.Tag == null)
        {
            MessageBox.Show("Uno de los productos no tiene un identificador válido.", "Error",
                MessageBoxButtons.OK, MessageBoxIcon.Error);

            return;
        }

        ClsDetalleCompraEntidad detalle = new ClsDetalleCompraEntidad();

        detalle.IdProducto = Convert.ToInt32(fila.Tag);
        detalle.Cantidad = Convert.ToDecimal(fila.Cells["Column2"].Value);
        detalle.PrecioSinIVA = Convert.ToDecimal(fila.Cells["Column4"].Value);
        detalle.PorcentajeIVA = Convert.ToDecimal(fila.Cells["Column5"].Value);
        detalle.Total = Convert.ToDecimal(fila.Cells["Column6"].Value);

        compra.Detalles.Add(detalle);
    }
```

```
            int idCompra = logica.GuardarCompra(compra);

            MessageBox.Show("La compra se guardó correctamente.\n\nN.º de compra: " + idCompra,
                "Compra guardada", MessageBoxButtons.OK, MessageBoxIcon.Information);

            this.DialogResult = DialogResult.OK;
            this.Close();
        }
        catch (Exception ex)
        {
            MessageBox.Show("No se pudo guardar la compra.\n\n" + ex.Message, "Error",
                MessageBoxButtons.OK, MessageBoxIcon.Error);
        }
    }
    private void btnBuscar_Click(object sender, EventArgs e)
    {
        string texto = txtProveedor.Text.Trim();

        if (string.IsNullOrWhiteSpace(texto))
        {
            MessageBox.Show("Ingrese el nombre o CUIT del proveedor.", "Aviso",
                MessageBoxButtons.OK, MessageBoxIcon.Warning);

            txtProveedor.Focus();
            return;
        }

        DataTable tabla = logicaProveedor.BuscarProveedor(texto);

        if (tabla.Rows.Count == 0)
        {
            MessageBox.Show("No se encontró ningún proveedor activo.", "Proveedor no encontrado",
                MessageBoxButtons.OK, MessageBoxIcon.Warning);

            txtProveedor.Focus();
            return;
        }

        if (tabla.Rows.Count > 1)
        {
            MessageBox.Show("Se encontraron varios proveedores. Ingrese un nombre o CUIT más específico.",
                "Varios proveedores", MessageBoxButtons.OK, MessageBoxIcon.Information);

            return;
        }

        idProveedor = Convert.ToInt32(tabla.Rows[0]["IdProveedor"]);
        txtProveedor.Text = tabla.Rows[0]["Nombre"].ToString();


    }
    private void txtCodigobarra_TextChanged(object sender, EventArgs e)
    {
        string texto = txtCodigobarra.Text.Trim();

        lstProductos.Items.Clear();
        lstProductos.Visible = false;
```

```
    if (string.IsNullOrWhiteSpace(texto))
    {
        productosEncontrados = null;
        return;
    }

    productosEncontrados = logica.BuscarProducto(texto);

    if (productosEncontrados.Rows.Count == 0)
    {
        return;
    }

    foreach (DataRow fila in productosEncontrados.Rows)
    {
        lstProductos.Items.Add(fila["CodigoBarra"].ToString() + " - " + fila["Nombre"].ToString());
    }

    lstProductos.Visible = true;
    lstProductos.BringToFront();

    if (lstProductos.Items.Count > 0)
    {
        lstProductos.SelectedIndex = 0;
    }

    txtCodigobarra.Focus();
    txtCodigobarra.SelectionStart = txtCodigobarra.Text.Length;
}
private void txtCodigobarra_KeyDown(object sender, KeyEventArgs e)
{
    if (!lstProductos.Visible || lstProductos.Items.Count == 0)
    {
        return;
    }

    if (e.KeyCode == Keys.Down)
    {
        e.SuppressKeyPress = true;

        if (lstProductos.SelectedIndex < lstProductos.Items.Count - 1)
        {
            lstProductos.SelectedIndex++;
        }
    }
    else if (e.KeyCode == Keys.Up)
    {
        e.SuppressKeyPress = true;

        if (lstProductos.SelectedIndex > 0)
        {
            lstProductos.SelectedIndex--;
        }
    }
    else if (e.KeyCode == Keys.Enter)
    {
        e.SuppressKeyPress = true;
        btnCargarProducto_Click(sender, e);
    }
```

```
        else if (e.KeyCode == Keys.Escape)
        {
            e.SuppressKeyPress = true;

            lstProductos.Items.Clear();
            lstProductos.Visible = false;
            productosEncontrados = null;
        }
    }
    private void lstProductos_DoubleClick(object sender, EventArgs e)
    {
        btnCargarProducto_Click(sender, e);
    }

    #endregion

}
}
```

Capa Vista // frmDetalleCompra

```
using System;
using System.Data;
using System.Windows.Forms;
using CapaLogica.NegocioLogica;

namespace CapaVista
{
    public partial class frmDetalleCompra : Form
    {
        private ClsCompraLogica logica = new ClsCompraLogica();

        private int idCompra;

        public frmDetalleCompra(int idCompra)
        {
            InitializeComponent();

            this.idCompra = idCompra;

            CargarDetalle();

        }

        #region Metodos
        private void CargarDetalle()
        {
            try
```

```
        {
            DataTable tabla = logica.ObtenerDetalleCompra(idCompra);

            if (tabla.Rows.Count == 0)
            {
                MessageBox.Show("No se encontró el detalle de la compra.", "Aviso",
                    MessageBoxButtons.OK, MessageBoxIcon.Warning);

                Close();
                return;
            }

            DataRow primeraFila = tabla.Rows[0];

            txtProveedor.Text = primeraFila["Proveedor"].ToString();
            txtFecha.Text = Convert.ToDateTime(primeraFila["Fecha"]).ToString("dd/MM/yyyy");
            txtTipoComprobante.Text = primeraFila["TipoComprobante"].ToString();
            txtNumeroComprobante.Text = primeraFila["NumeroComprobante"].ToString();
            txtSubtotal.Text = Convert.ToDecimal(primeraFila["SubtotalSinIVA"]).ToString("N2");
            txtIVA.Text = Convert.ToDecimal(primeraFila["IVA"]).ToString("N2");
            txtTotal.Text = Convert.ToDecimal(primeraFila["Total"]).ToString("N2");
            txtObservaciones.Text = primeraFila["Observaciones"] == DBNull.Value ? "" : primeraFila["Observaciones"].ToString();
            dgvDetalle.Rows.Clear();

            foreach (DataRow fila in tabla.Rows)
            {
                int indice = dgvDetalle.Rows.Add();

                dgvDetalle.Rows[indice].Cells["clnCodigo"].Value = fila["CodigoBarra"].ToString();
                dgvDetalle.Rows[indice].Cells["clnProducto"].Value = fila["Producto"].ToString();
                dgvDetalle.Rows[indice].Cells["clnCantidad"].Value = Convert.ToDecimal(fila["Cantidad"]).ToString("N3");
                dgvDetalle.Rows[indice].Cells["clnPrecio"].Value = Convert.ToDecimal(fila["PrecioSinIVA"]).ToString("N2");
                dgvDetalle.Rows[indice].Cells["clnIVA"].Value = Convert.ToDecimal(fila["PorcentajeIVA"]).ToString("N2");
                dgvDetalle.Rows[indice].Cells["clnTotal"].Value = Convert.ToDecimal(fila["TotalDetalle"]).ToString("N2");
            }
        }
        catch (Exception ex)
        {
            MessageBox.Show("No se pudo cargar el detalle de la compra.\n\n" + ex.Message, "Error",
                MessageBoxButtons.OK, MessageBoxIcon.Error);
        }
    }

    #endregion


    #region Eventos
    private void btnCerrar_Click(object sender, EventArgs e)
    {
        Close();
    }
    #endregion


}
}
```

Capa Lógica // ClsCompraLogica

```
using CapaDatos.NegocioDatos;
using CapaEntidades.NegocioEntidades;
using System;
using System.Collections.Generic;
using System.Data;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace CapaLogica.NegocioLogica
{
    public class ClsCompraLogica
    {
        private ClsCompraDatos datos = new ClsCompraDatos();

        public DataTable BuscarProducto(string texto)
        {
            return datos.BuscarProducto(texto);
        }
        public int GuardarCompra(ClsCompraEntidad compra)
        {
            return datos.GuardarCompra(compra);
        }
        public DataTable ObtenerCompras(DateTime desde, DateTime hasta)
        {
            return datos.ObtenerCompras(desde, hasta);
        }
        public DataTable ObtenerDetalleCompra(int idCompra)
        {
            return datos.ObtenerDetalleCompra(idCompra);
        }
        public void AnularCompra(int idCompra, string motivo)
        {
            datos.AnularCompra(idCompra, motivo);
        }
    }
}
```

Capa Datos // ClsCompraDatos

```
using CapaEntidades.NegocioEntidades;
using Microsoft.Data.SqlClient;
using SistemaKiosco;
using System;
using System.Collections.Generic;
using System.Data;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace CapaDatos.NegocioDatos
{
    public class ClsCompraDatos
    {
        private Conexion cn = new Conexion();
```

```
        public DataTable BuscarProducto(string texto)
        {
            using (SqlConnection conexion = cn.ObtenerConexion())
            {
                using (SqlCommand cmd = new SqlCommand("sp_BuscarProducto", conexion))
                {
                    cmd.CommandType = CommandType.StoredProcedure;

                    cmd.Parameters.AddWithValue("@Texto", texto);

                    using (SqlDataAdapter adaptador = new SqlDataAdapter(cmd))
                    {
                        DataTable tabla = new DataTable();
                        adaptador.Fill(tabla);
                        return tabla;
                    }
                }
            }
        }
        public int GuardarCompra(ClsCompraEntidad compra)
        {
            using (SqlConnection conexion = cn.ObtenerConexion())
            {
                conexion.Open();

                using (SqlCommand cmd = new SqlCommand("sp_GuardarCompra", conexion))
                {
                    cmd.CommandType = CommandType.StoredProcedure;
                    cmd.Parameters.AddWithValue("@IdProveedor", compra.IdProveedor);
                    cmd.Parameters.AddWithValue("@Fecha", compra.Fecha);
                    cmd.Parameters.AddWithValue("@TipoComprobante", compra.TipoComprobante);
                    cmd.Parameters.AddWithValue("@NumeroComprobante", compra.NumeroComprobante);
                    cmd.Parameters.AddWithValue("@SubtotalSinIVA", compra.SubtotalSinIVA);
                    cmd.Parameters.AddWithValue("@IVA", compra.IVA);
                    cmd.Parameters.AddWithValue("@Total", compra.Total);
                    cmd.Parameters.AddWithValue("@Observaciones", string.IsNullOrWhiteSpace(compra.Observaciones) ? (object)DBNull.Value
: compra.Observaciones);

                    DataTable tablaDetalles = new DataTable();

                    tablaDetalles.Columns.Add("IdProducto", typeof(int));
                    tablaDetalles.Columns.Add("Cantidad", typeof(decimal));
                    tablaDetalles.Columns.Add("PrecioSinIVA", typeof(decimal));
                    tablaDetalles.Columns.Add("PorcentajeIVA", typeof(decimal));
                    tablaDetalles.Columns.Add("Total", typeof(decimal));

                    foreach (ClsDetalleCompraEntidad detalle in compra.Detalles)
                    {
                        tablaDetalles.Rows.Add(detalle.IdProducto, detalle.Cantidad, detalle.PrecioSinIVA,detalle.PorcentajeIVA, detalle.Total);
                    }

                    SqlParameter parametroDetalles = cmd.Parameters.AddWithValue("@Detalles", tablaDetalles);

                    parametroDetalles.SqlDbType = SqlDbType.Structured;
                    parametroDetalles.TypeName = "dbo.TipoDetalleCompra";

                    return Convert.ToInt32(cmd.ExecuteScalar());
                }
            }
```

```
    }
    public DataTable ObtenerCompras(DateTime desde, DateTime hasta)
    {
        using (SqlConnection conexion = cn.ObtenerConexion())
        {
            using (SqlCommand cmd = new SqlCommand("sp_ObtenerCompras", conexion))
            {
                cmd.CommandType = CommandType.StoredProcedure;
                cmd.Parameters.AddWithValue("@Desde", desde.Date);
                cmd.Parameters.AddWithValue("@Hasta", hasta.Date);

                using (SqlDataAdapter adaptador = new SqlDataAdapter(cmd))
                {
                    DataTable tabla = new DataTable();
                    adaptador.Fill(tabla);
                    return tabla;
                }
            }
        }
    }
    public DataTable ObtenerDetalleCompra(int idCompra)
    {
        using (SqlConnection conexion = cn.ObtenerConexion())
        {
            using (SqlCommand cmd = new SqlCommand("sp_ObtenerDetalleCompra", conexion))
            {
                cmd.CommandType = CommandType.StoredProcedure;
                cmd.Parameters.AddWithValue("@IdCompra", idCompra);

                using (SqlDataAdapter adaptador = new SqlDataAdapter(cmd))
                {
                    DataTable tabla = new DataTable();
                    adaptador.Fill(tabla);
                    return tabla;
                }
            }
        }
    }
    public void AnularCompra(int idCompra, string motivo)
    {
        using (SqlConnection conexion = cn.ObtenerConexion())
        {
            conexion.Open();

            using (SqlCommand cmd = new SqlCommand("sp_AnularCompra", conexion))
            {
                cmd.CommandType = CommandType.StoredProcedure;
                cmd.Parameters.AddWithValue("@IdCompra", idCompra);
                cmd.Parameters.AddWithValue("@Motivo", motivo);

                cmd.ExecuteNonQuery();
            }
        }
    }

  }
}
```

5.0 Roles y Permisos

Gestorix funciona con un sistema de roles personalizables por parte del cliente, permitiendo crear, modificar y eliminar roles en base a los siguientes permisos:

ID_Permiso	CodigoPermiso	Permiso	SubPermiso	Descripción
100	1.0.0	Artículos	Total	Permite usar todas las funciones de la sección Artículos.
110	1.1.0		Gestión de Artículos	Permite gestionar el Alta, Baja, Modificación y Carga de Artículos.
111	1.1.1		Alta de Artículos	Permite gestionar el Alta de nuevos Artículos.
112	1.1.2		Baja/Modificación de Artículos	Permite gestionar la Baja y Modificación de los Artículos.
113	1.1.3		Carga de Artículos	Permite Cargar Artículos Existentes al Stock.
120	1.2.0		Gestión de Rubros	Permite administrar la Alta, Baja y Modificación de los Rubros/Categorías de los Productos.
121	1.2.1		Alta de Rubros	Permite administrar la Alta de los Rubros/Categorías de los Productos.
122	1.2.2		Baja/Modificación de Rubros	Permite administrar la Baja y Modificación de los Rubros/Categorías de los Productos.
130	1.3.0		Gestión de Marcas	Permite administrar la Alta, Baja y Modificación de las Marcas de los Productos.
131	1.3.1		Alta de Marcas	Permite administrar la Alta de las Marcas de los Productos.
132	1.3.2		Baja/Modificación de Marcas	Permite administrar la Baja y Modificación de las Marcas de los Productos.
140	1.4.0		Importar Articulos desde Excel	Permite importar artículos desde archivos Excel externos al sistema.
150	1.5.0		Exportar Artículos a Excel	Permite exportar información de artículos a archivos Excel.
200	2.0.0	Ventas	Total	Permite usar todas las funciones de la sección Ventas.
210	2.1.0		Realizar Ventas	Permite realizar ventas.
220	2.2.0		Devoluciones	Permite realizar Devoluciones y ver las realizadas previamente.
221	2.2.1		Realizar Devoluciones	Permite realizar Devoluciones.
222	2.2.2		Historial Devoluciones	Permite acceder al Historial de Devoluciones.

230	2.3.0		Crear Promociones	Permite crear nuevas Promociones.
240	2.4.0		Baja Promociones	Permite dar de Baja Promociones.
300	3.0.0	Compras	Total	Permite usar todas las funciones de la sección Compras.
400	4.0.0	Proveedores	Total	Permite usar todas las funciones de la sección Proveedores.
410	4.1.0		Gestión de Proveedores	Permite administrar la Alta, Baja y Modificación de los Proveedores.
411	4.1.1		Alta de Proveedores	Permite administrar la Alta de Proveedores.
412	4.1.2		Baja/ Modificación de Proveedores	Permite administrar la Baja y Modificación de Proveedores.
420	4.2.0		Cuenta Corriente de Proveedores	Permite generar un próximo pago a Proveedores.
500	5.0.0	Listados	Total	Permite usar todas las funciones de la sección Listados.
510	5.1.0		Listado de Precios	Permite Visualizar la Lista de Precios.
520	5.2.0		Listado de Reposición	Permite Visualizar el Listado de Reposición.
600	6.0.0	Informes	Total	Permite usar todas las funciones de la sección Informes.
610	6.1.0		Ventas por Fecha	Permite Realizar, Visualizar y Exportar Informes de Ventas por Fecha.
620	6.2.0		Ventas con Detalle de Fecha	Permite Realizar, Visualizar y Exportar Informes de Ventas por Detalle de Fecha.
630	6.3.0		Venta por Rubro	Permite Realizar, Visualizar y Exportar Informes de Ventas por Rubro.
640	6.4.0		Venta por Artículos	Permite Realizar, Visualizar y Exportar Informes de Ventas por Artículos.
650	6.5.0		Venta por Medio de Pago	Permite Realizar, Visualizar y Exportar Informes de Ventas por Medio de Pago.
700	7.0.0	Caja	Total	Permite usar todas las funciones de la sección Caja.
710	7.1.0		Nuevo Ingreso	Permite realizar un Nuevo Ingreso Manualmente.
720	7.2.0		Nuevo Egreso	Permite realizar un Nuevo Egreso Manualmente.
730	7.3.0		Abrir Caja	Permite Abrir la Caja.

740	7.4.0		Cerrar Caja	Permite Cerrar la Caja.
800	8.0.0	Ajustes	Total	Permite Acceder a los ajustes del Sistema.
810	8.1.0		Gestión de Idiomas	Permite agregar idiomas y administrar sus traducciones.
820	8.2.0		Bitácora	Permite consultar y exportar la bitácora del sistema.
830	8.3.0		Realizar Backup	Permite generar copias de seguridad de la base de datos.
840	8.4.0		Restaurar Backup	Permite restaurar la base de datos desde un backup.
850	8.5.0		Gestión de Dígitos Verificadores	Permite recalcular los dígitos verificadores ante inconsistencias.
900	9.0.0	Ayuda	Total	Permite Acceder a la sección de Ayuda.
1000	10.0.0	Empleado	Total	Permite usar todas las funciones de la sección Empleado.
1010	10.1.0		Alta Empleado	Permite realizar una alta en el sistema de un nuevo empleado.
1020	10.2.0		Modificación/Baja Empleado	Permite modificar los datos o dar de baja al empleado.
1030	10.3.0		Modificación de Datos Propios	Permite modificar datos propios por parte del empleado.
1100	11.0.0	Alarmas	Total	Permite usar todas las funciones de la sección Alarmas.
1110	11.1.0		Visualizar Alarma	Permite Visualizar las Alarmas.
1120	11.2.0		Dar de Baja Alarma	Permite Dar de Baja una Alarma.
1200	12.0.0	Datos Sensibles	Total	Permite usar todas las funciones de la sección Datos Sensibles.
1210	12.1.0		Datos Sensibles Articulo	Permite modificar datos sensibles de los Artículos.
1220	12.2.0		Datos Sensibles Rubro	Permite modificar datos sensibles de los Rubros/Categorías.
1230	12.3.0		Datos Sensibles Marca	Permite modificar datos sensibles de la Marcas.
1240	12.4.0		Datos Sensibles Proveedor	Permite modificar datos sensibles de los Proveedores.
1250	12.5.0		Datos Sensibles	Permite modificar datos sensibles de los Empleados.

			Empleados	
1260	12.6.0		Datos Sensibles Promociones	Permite modificar datos sensibles de las Promociones.
1270	12.7.0		Roles y Permisos	Permite gestionar los Roles y Permisos.

Funcionando con una lógica en cascada, si se le otorga un permiso X.0.0, se le otorgara por defecto sus concatenados X.1.0, X.2.0, etc. lo mismo pasaría con un X.1.0 que otorga el X.1.1, X.1.2.
Si bien le damos la opción al cliente de otorgar permisos sobre datos que normalmente solo deberían tener acceso los dueños/administradores de la empresa, también nos aseguramos por medio advertencias tanto en la guía como en el programa al intentar activarlos que no es recomendable por la seguridad y guardado de datos.

Roles Base

Para este proyecto, Gestorix entrega 4 roles principales ya creados, Administrador, Empleado General, Empleado Ventas y Empleado Compras.

Administrador

Tiene acceso a todos los permisos, solo debería ser utilizado por el Dueño del Maxikiosco MTK.

1.0.0 - Artículos (Total)
2.0.0 - Ventas (Total)
3.0.0 - Compras (Total)
4.0.0 - Proveedores (Total)
5.0.0 - Listados (Total)
6.0.0 - Informes (Total)
7.0.0 - Caja (Total)
8.0.0 - Ajustes (Total)
9.0.0 - Ayuda (Total)
10.0.0 - Empleado (Total)
11.0.0 - Alarmas (Total)
12.0.0 - Datos Sensibles (Total)

Empleado General

Rol pensado para el Empleado normal del Maxikiosco MTK, donde los trabajadores son pocos y no hay una distribución clásica de tareas, todos tienen que poder encargarse de todo si lo requiere en el ámbito de las tareas normales de Compra y Venta.

1.1.0 - Gestión de Artículos

1.2.1 - Alta de Rubros
1.3.0 - Gestión de Marcas
1.4.0 - Importar Artículos desde Excel
2.0.0 - Ventas (Total)

3.0.0 - Compras (Total)
4.2.0 - Cuenta Corriente de Proveedores
5.1.0 - Listado de Precios
7.0.0 - Caja (Total)
9.0.0 - Ayuda (Total)
10.3.0 - Modificación de Datos Propios
11.0.0 - Alarmas (Total)

Empleado Compras

Rol planeado para el clásico empleado del ámbito de Compras.

1.1.0 - Gestión de Artículos
1.2.1 - Alta de Rubros
1.3.0 - Gestión de Marcas
1.4.0 - Importar Artículos desde Excel
3.0.0 - Compras (Total)
4.2.0 - Cuenta Corriente de Proveedores
5.2.0 - Listado de Reposición
7.1.0 - Nuevo Ingreso
7.2.0 - Nuevo Egreso
9.0.0 - Ayuda (Total)
10.3.0 - Modificación de Datos Propios
11.0.0 - Alarmas (Total)

Empleado Ventas

Rol planeado para el clásico empleado del ámbito de Ventas.

2.0.0 - Ventas (Total)
5.1.0 - Listado de Precios
7.0.0 - Caja (Total)
9.0.0 - Ayuda (Total)
10.3.0 - Modificación de Datos Propios
11.1.0 - Visualizar Alarma